simplydata

simple words for data science

The manual: lessons, every word, the ideas behind them, and solutions

```
import simplydata as sdata

table = sdata.table(sdata.example("marks"))
table.compare_averages("marks", for_each="class")
chart = table.chart()
chart.boxes(x="class", y="marks")
chart.save("marks.png")
```

Version 0.1.0

Open source (MIT licence): https://github.com/bioforkgithub/simplydata

Made from the code by docs/make_manual.py; every example in it was run to make its chart.

Contents

I Getting started 1

Welcome 2

Install and first steps 4

How the words are built 5

The practice tables 6

II Lessons 9

1 Your first chart 10

2 Titles and labels 13

3 Colours, groups and legends 16

4 Different chart types 21

5 Statistics: averages, error bars and fit lines 26

6 Axes and reference lines 31

7 Several charts together 36

8 Hard tricks, and your own program 41

9 Tables: open one and look at it 48

10 Tables: choose rows and columns, and undo 52

11 Tables: clean the data and add new columns 57

12 Tables: summaries, and from a table to a chart 64

13 Numbers: lists and grids of numbers 72

14 Statistical tests: real difference, or chance? 76

15 A whole analysis: from a question to a figure 81

16 Models: explain one column from several others 86

17 Patterns across many columns 90

18 Survival, and planning a study **95**

19 Fitting curves, finding the best value and solving **99**

20 Distributions, chances and values between points **104**

III Dictionary of words **109**

How to read the dictionary **110**

Chart words **111**
Start, show and save . . . 111
Chart types . . . 113
Statistics . . . 121
Lines and shading . . . 129
Words on the chart . . . 131
Axes . . . 135
Colours and look . . . 140
Order of groups . . . 146
Several charts . . . 148

Table words **151**
Tables: start and look . . . 151
Choose columns . . . 154
Choose rows . . . 157
Empty cells . . . 163
Fix columns . . . 165
Sort and new columns . . . 169
Dates and text . . . 172
Row after row . . . 174
Summaries . . . 176
Statistical tests . . . 182
Fitting . . . 188
Finding and solving . . . 189
Values between points . . . 192
Distributions . . . 193
Models . . . 200
Patterns across columns . . . 201
Survival . . . 205
Planning a study . . . 207
Combine and reshape . . . 209
Undo and finish . . . 210

Number words **214**
Numbers: make and look . . . 214
Numbers: maths in words . . . 217
Numbers: choose, sort and fill . . . 221
Numbers: summaries . . . 226
Grids of numbers . . . 228
Numbers: undo and finish . . . 235

Help words **238**

Help . . . 238

IV Data science ideas 240

Describing numbers **241**
Average or median? . . . 241
Spread: range, percentiles and standard deviation . . . 241
The shape of the values: distributions . . . 242

How sure are we? **242**
Standard deviation or standard error? . . . 242
Confidence intervals . . . 243
Why averages behave well: the central limit theorem . . . 243

Testing: real difference, or chance? **243**
The null hypothesis and the p-value . . . 243
Two kinds of mistake, and power . . . 244
Many tests at once . . . 244
Effect sizes: how big is the difference? . . . 244

Links between columns **245**
Correlation . . . 245
Correlation is not cause . . . 245
Fitting a line or a formula . . . 245

The shape of a table **246**
Tidy data: long and wide tables . . . 246
Missing values and imputation . . . 246

Which test when? **247**
Writing it up . . . 248

V Solutions 249

Solutions to the tasks **250**
Lesson 1: Your first chart . . . 250
Lesson 2: Titles and labels . . . 252
Lesson 3: Colours, groups and legends . . . 255
Lesson 4: Different chart types . . . 258
Lesson 5: Statistics: averages, error bars and fit lines . . . 260
Lesson 6: Axes and reference lines . . . 262
Lesson 7: Several charts together . . . 264
Lesson 8: Hard tricks, and your own program . . . 266
Lesson 9: Tables: open one and look at it . . . 268
Lesson 10: Tables: choose rows and columns, and undo . . . 269
Lesson 11: Tables: clean the data and add new columns . . . 271
Lesson 12: Tables: summaries, and from a table to a chart . . . 273
Lesson 13: Numbers: lists and grids of numbers . . . 274
Lesson 14: Statistical tests: real difference, or chance? . . . 275
Lesson 15: A whole analysis: from a question to a figure . . . 277
Lesson 16: Models: explain one column from several others . . . 279

Lesson 17: Patterns across many columns 281
Lesson 18: Survival, and planning a study 282
Lesson 19: Fitting curves, finding the best value and solving 283
Lesson 20: Distributions, chances and values between points 285

Index of words **287**

PART I

Getting started

Welcome

simplydata gives you one small vocabulary of plain English words for working with data: reading a table, cleaning it, summing it up, testing a difference and drawing a chart. Underneath, the work is done by the libraries that scientists use every day: pandas and Polars for tables, NumPy for numbers, SciPy for statistics, and matplotlib and seaborn for charts. You learn the words; simplydata picks the right call in the right library, with the right settings.

```
import simplydata as sdata

table = sdata.table(sdata.example("marks"))   # or your own: sdata.table("marks.xlsx")
table.keep_rows("marks", above=50)
table.average_of("marks", for_each="class")   # says the answer in a sentence
table.save("good_marks.xlsx")

chart = table.chart()                         # or sdata.chart(table)
chart.bars(x="class", y="marks", error="standard error")
chart.title_top("Good marks in each class")
chart.save("marks.png")
```

The program reads like a list of instructions, one step on each line. A table, a chart and a list of numbers are separate objects, as in pandas, NumPy and matplotlib: one cleaned table can feed many charts, and several charts can share one page.

Who this manual is for

Everyone who wants to learn data science with Python: school pupils, students, teachers and researchers. You do not need to know Python first. Every word is simple enough for a child to read, and every result names the technical term (“Welch’s t-test”, “linear regression”) so that you can write it up in a report or a paper.

How this manual is built

Part II: Lessons

Eighteen short lessons that teach the whole of simplydata in a day. Each lesson shows code you can run, says what every line does, shows the real chart and output it makes, and ends with tasks to try yourself.

Part III: Dictionary of words

Every word, with what it does, its options, an example you can run and the library it uses underneath.

Part IV: Data science ideas

The ideas behind the numbers in plain words: average and median, standard deviation and standard error, confidence intervals, p-values, effect sizes, and a guide to which statistical test to use when.

Part V: Solutions

A worked solution for every “Try it yourself” task.

Everything in this manual is made from simplydata's own code by `docs/make_manual.py`: the examples were run to make the charts and the printed output you see, so they are exactly what you will get.

Install and first steps

Install

```
pip install simplydata           # coming soon
pip install simplydata[fast]     # adds Polars, the fast engine for big data
```

Where to type the code

Any place that runs Python works. In **Jupyter**, **VS Code** or **Spyder**, a chart appears by itself when you write `chart` on the last line of a cell. In a script, `chart.show()` opens it in a window, and `chart.save("name.png")` always saves it as a file.

simplydata teaches itself

```
import simplydata as sdata
sdata.lesson()                      # lists the lessons
sdata.lesson(1)                     # the first lesson
sdata.lesson(1, solutions=True)     # the first lesson, with worked solutions to its tasks
sdata.words()                       # every word, with a short example
sdata.help("bars")                  # one word: what it does, its options and an example
sdata.example()                     # the made-up tables you can practise with
```

Your own data

`sdata.table("my_file.xlsx")` and `sdata.chart("my_file.xlsx")` read Excel, CSV, TSV, Parquet and JSON files. They also take a pandas or Polars table you already have, or a small table typed by hand: `sdata.table({"name": ["Asha", "Ben"], "height": [144, 152]})`. Your file is never changed: only `table.save(...)` writes a file.

When something goes wrong

simplydata explains its errors in plain words. Misspell a column and it says which columns there are and which one you probably meant. Ask for a word that does not exist and `sdata.help` suggests the nearest one. A missing word can be requested on GitHub, at https://github.com/bioforkgithub/simplydata/issues.

How the words are built

A few rules make the words easy to guess. If you know one word of a pattern, you can usually guess its siblings.

Pattern	Examples
verb + thing changes a table or numbers	`keep_rows`, `remove_columns`, `fill_empty_cells`, `add_column`, `keep_numbers`
chart types are nouns on a chart	`chart.bars(...)`, `chart.histogram(...)`, `chart.dots(...)`
thing + where places something on a chart	`title_top`, `label_y`, `line_at_y`, `legend_bottom`, `shade_between_x`
measure + _of sums up one column; **relation + _between** links two; **compare_ + measure** tests	`average_of`, `median_of`, `correlation_between`, `compare_averages`
show_ / hide_ + thing looks without changing	`show_first_rows`, `show_grid`, `hide_legend`
as_ + library hands the data on; **save** writes a file	`as_pandas`, `as_polars`, `as_numpy`; `table.save("clean.xlsx")`

- **One option name, one meaning, everywhere:** `for_each=` (groups), `colour=` (one colour, or a colour per group), `between=[low, high]`, `from_column=`, `with_value=`, `biggest_first=`, `per_row=`, `label=`, `fast=`.
- **Measure names are the same wherever they appear:** `"average"`, `"median"`, `"total"`, `"count"`, `"smallest"`, `"biggest"`, `"standard deviation"`, `"standard error"`, `"confidence interval"`.
- **Axes are x and y.** Anything that belongs to one axis ends in `_x` or `_y`: `label_y`, `range_x`, `log_scale_y`.
- **The word names the idea**, and the printed answer names the technical term, so you can follow (and write) the Methods section of a paper.
- **Order does not matter** on a chart: after the chart type, write the other steps in any order. Both `colour` and `color`, `grey` and `gray` work.
- **Every change can be taken back**: tables and numbers remember each step, so `undo()`, `steps()` and `start_again()` always work.

The practice tables

All the examples use small made-up tables that come with simplydata, so they run anywhere without a file. Every number in them is invented. Get one with `sdata.example("marks")`, and list them all with `sdata.example()`.

marks

Test marks of 18 made-up children in three classes. `sdata.example("marks")`

Column	What it holds
`name`	the child's first name
`class`	A, B or C
`subject`	Maths, Science or English
`hours`	hours spent studying for the test
`marks`	marks in the test, out of 100
`height`	the child's height in cm

weather

Weather in three made-up cities, month by month. `sdata.example("weather")`

Column	What it holds
`month`	Jan, Feb ... Dec
`month_number`	1 to 12
`city`	Sunnyvale, Rainbridge or Frostby
`temperature`	average temperature in °C
`rain`	rain that month in mm
`wind_direction`	where the wind mostly came from, in degrees (0 to 360)
`wind_speed`	average wind speed in km/h

fruit

Fruit sold by two made-up shops in one week. One row is one sale. `sdata.example("fruit")`

Column	What it holds
`day`	Mon to Fri
`shop`	Corner shop or Market stall
`fruit`	apple, banana, cherry, grape, orange or pear
`count`	how many pieces of fruit were sold
`price`	price of one piece (made-up money)

growth

Population of four made-up countries, every two years from 2000 to 2024. `sdata.example("growth")`

Column	What it holds
`year`	2000, 2002 ... 2024
`country`	Applia, Bananaland, Cherria or Dateland
`population`	people, in millions
`forest`	how much of the land is forest, in %

shop

Daily sales of two made-up shops from January to March 2026. Some cells are empty. `sdata.example("shop")`

Column	What it holds
`date`	every day from 2026-01-01 to 2026-03-31 (real dates)
`shop`	Northgate or Southside
`sales`	money taken that day (made-up money); empty on days the till broke
`customers`	how many people bought something; empty on days nobody counted

school

Made-up pupils in 12 schools: study, sleep and test marks. Some sleep cells are empty. `sdata.example("school")`

Column	What it holds
`school`	School A to School L (15 pupils in each)
`pupil`	P001 to P180
`hours`	hours spent studying for the test
`sleep`	hours of sleep the night before; empty when the pupil did not say
`breakfast`	yes or no: did the pupil eat breakfast that morning
`marks`	marks in the test, out of 100
`passed`	yes (50 marks or more) or no
`days_absent`	days the pupil missed school that term

meadow

Plants counted in 30 made-up plots of three fields, half of them grazed by sheep. `sdata.example("meadow")`

Column	What it holds
`plot`	a short name for the plot, such as NG1
`field`	North field, South field or River field
`grazed`	yes or no: do sheep graze the plot
`clover`	how many clover plants in the plot
`daisy`	how many daisies
`buttercup`	how many buttercups
`dandelion`	how many dandelions
`nettle`	how many nettles
`thistle`	how many thistles
`moss`	how many patches of moss
`fern`	how many ferns
`orchid`	how many orchids
`yarrow`	how many yarrow plants

trial

120 made-up patients in a trial of a new medicine, followed for up to 3 years. `sdata.example("trial")`

Column	What it holds
`patient`	T001 to T120
`treatment`	new medicine or usual care
`age`	age in years when the patient joined
`days`	days from joining until the patient died or the follow-up ended
`died`	yes (died on that day) or no (still alive when the follow-up ended)

island

A map of a made-up island with two hills, on a 15 x 15 grid. `sdata.example("island")`

Column	What it holds
`east`	how far east, 0 to 14 (km)
`north`	how far north, 0 to 14 (km)
`height`	height of the ground in m (below 0 is under the sea)

PART II

Lessons

LESSON 1

Your first chart

In this lesson you will learn

- how to get a table to practise with
- how to start a chart and draw bars
- how to save your chart as a picture

Example

A simplydata program is made of small steps, one step on each line, and each step says what it does.

```
import simplydata as sdata
sdata.example()
marks = sdata.example("marks")
print(marks)
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.save("lesson1.png")
```

What each line does

1. Load simplydata. From now on we call it by its short name, `"sdata"`.
2. List the made-up tables you can practise with, and their columns.
3. Get the marks table: test marks of made-up children. We call it marks.
4. Look at the table. Each row is one child in one test. The names at the top (name, class, subject ...) are the columns.
5. Start a new, empty chart that uses the marks table. We call it chart, and every next line starts with that name.
6. Draw bars: one bar for each name (along x, the bottom), as tall as the marks (up y, the side). Each child did three tests, so the bar shows the average: add the three marks up and divide by 3.
7. Save the chart as a picture called lesson1.png, in the folder you are working in. Open it to see your chart!

What it prints

```
Example tables you can use (all made up, for practice):

  sdata.example("marks")
      Test marks of 18 made-up children in three classes.
      Columns: name, class, subject, hours, marks, height

  sdata.example("weather")
      Weather in three made-up cities, month by month.
      Columns: month, month_number, city, temperature, rain, wind_direction, wind_speed

  sdata.example("fruit")
```

```
    Fruit sold by two made-up shops in one week. One row is one sale.
    Columns: day, shop, fruit, count, price

sdata.example("growth")
    Population of four made-up countries, every two years from 2000 to 2024.
    Columns: year, country, population, forest

sdata.example("shop")
    Daily sales of two made-up shops from January to March 2026. Some cells are empty.
    Columns: date, shop, sales, customers

sdata.example("school")
    Made-up pupils in 12 schools: study, sleep and test marks. Some sleep cells are empty.
    Columns: school, pupil, hours, sleep, breakfast, marks, passed, days_absent

sdata.example("meadow")
    Plants counted in 30 made-up plots of three fields, half of them grazed by sheep.
    Columns: plot, field, grazed, clover, daisy, buttercup, dandelion, nettle, thistle, moss, fern, orchid
   ↪ , yarrow

sdata.example("trial")
    120 made-up patients in a trial of a new medicine, followed for up to 3 years.
    Columns: patient, treatment, age, days, died

sdata.example("island")
    A map of a made-up island with two hills, on a 15 x 15 grid.
    Columns: east, north, height


... (56 more lines)
```

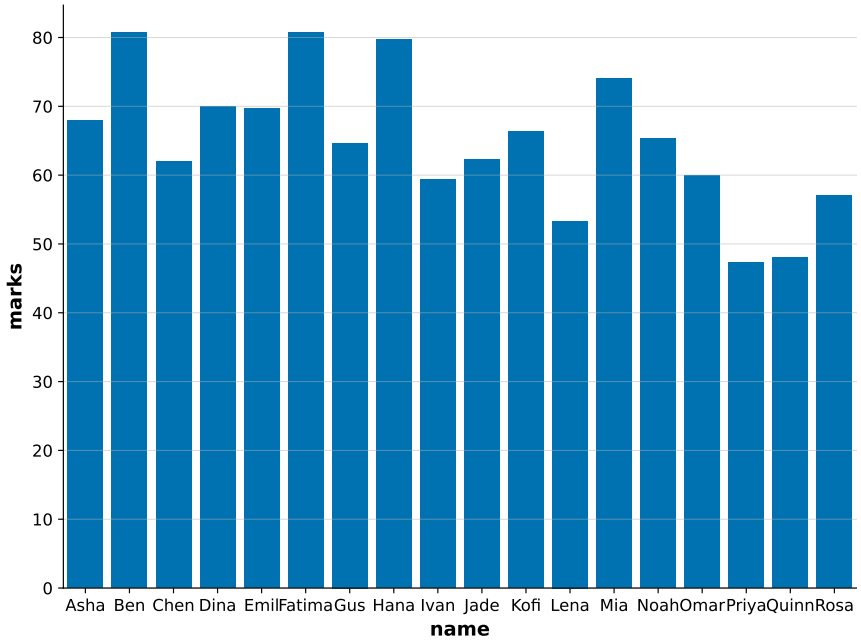


Figure 1.1: Lesson 1, example: `lesson1.png`.

Good to know

- Chart types are nouns on a chart: `chart.bars`, `chart.lines`, `chart.dots` … The chart is the thing, and bars is what it shows.

- For your own data, write `chart = sdata.chart("my_marks.xlsx")`. Excel, CSV, TSV, Parquet and JSON files all work.
- `chart.save("lesson1.pdf")` saves a PDF instead: the end of the name decides the type.
- In Jupyter, VS Code or Spyder, write `chart` on the last line of a cell and the chart appears by itself. In a script, `chart.show()` opens it in a window.
- Are the names under the bars bumping into each other? Lesson 2 shows how to turn them.

Try it yourself

1. Change `x="name"` to `x="subject"`. What does each bar show now?
2. Save the chart as `"my_first_chart.png"` and also as `"my_first_chart.pdf"`.
3. Make a chart of the `"fruit"` table: bars with `x="fruit"` and `y="count"`.

Worked solutions: page 250, or type `sdata.lesson(1, solutions=True)`.

Words in this lesson: `sdata.example()`, `sdata.chart()`, `chart.bars()`, `chart.save()`, `chart.show()`.

LESSON 2

Titles and labels

In this lesson you will learn

- how to put a title on top
- how to name the x-axis and the y-axis
- how to add a small caption and change the size of the text

Example 1

A chart is easier to read when it says what it shows. First draw, then add the words.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.title_top("Our class marks")
chart.label_x("Student")
chart.label_y("Average mark")
chart.rotate_labels_x(45)
chart.caption_bottom("Made-up data from sdata.example")
chart.font_size(14)
chart.save("lesson2.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. Draw one bar for each child.
5. Write a title on top of the chart.
6. Write a name under the x-axis (the bottom).
7. Write a name beside the y-axis (the side).
8. Turn the names under the bars by 45 degrees, so long names do not bump into each other.
9. Write a small line under the whole chart.
10. Make all the text a bit bigger (normal is 12).
11. Save it.

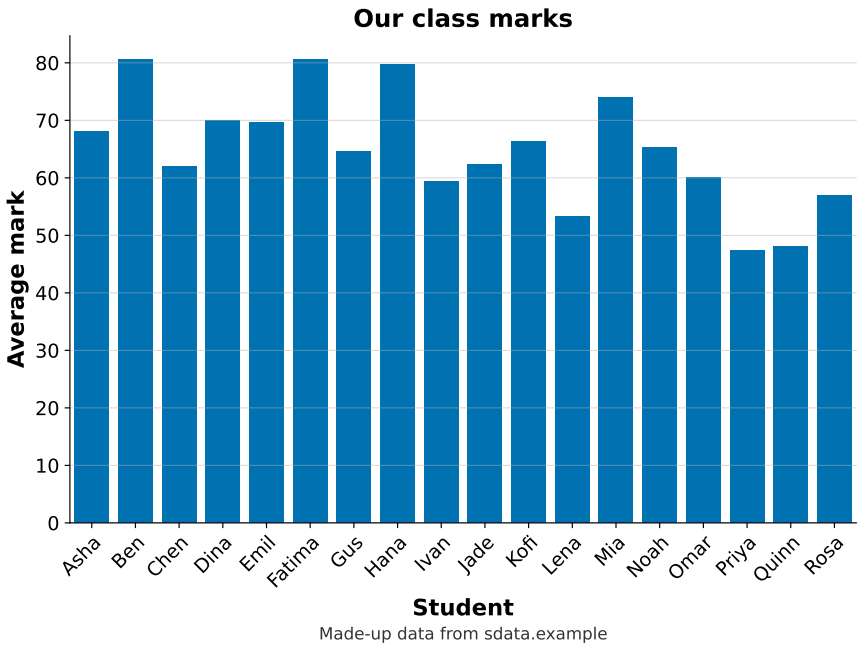


Figure 2.1: Lesson 2, example 1: `lesson2.png`.

Example 2

Words can also go next to the dots: a name for each dot, where it covers nothing.

```
import simplydata as sdata
kids = {"name": ["Asha", "Ben", "Chen", "Dina", "Eli"], "hours": [2.0, 5.5, 3.1, 7.2, 4.4], "
    ↪ marks": [48, 71, 55, 88, 62]}
chart = sdata.chart(kids)
chart.dots(x="hours", y="marks")
chart.names_on_dots("name")
chart.label_x("Hours of study")
chart.save("lesson2_names.png")
```

What each line does

1. Load simplydata.
2. A small table typed by hand: each column name, then its values in a list.
3. Start a chart of it.
4. One dot for each child.
5. Write each name next to its dot, where it covers no other name or dot. A name with no free place is left out, and a line says so.
6. Name the x-axis.
7. Save it.

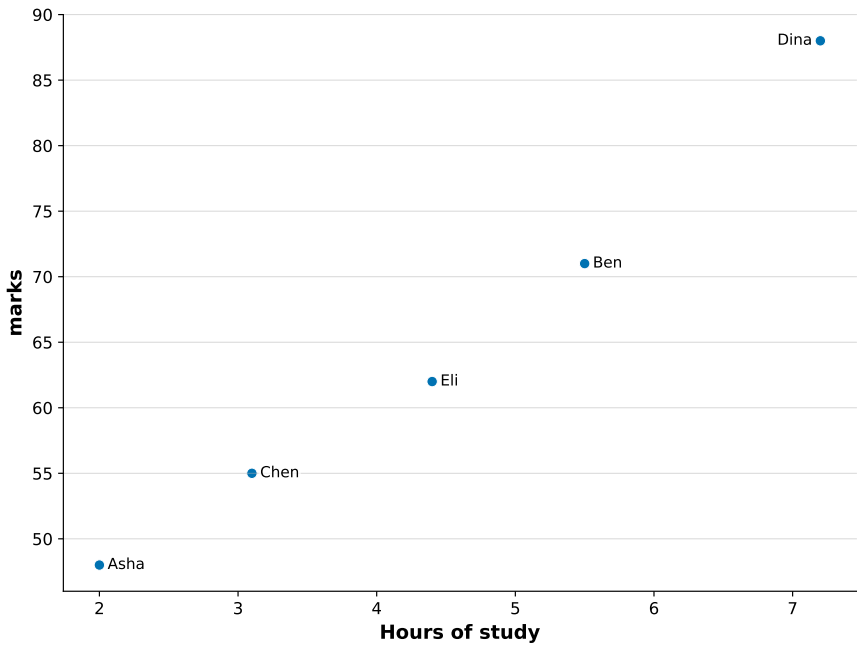


Figure 2.2: Lesson 2, example 2: `lesson2_names.png`.

Good to know

- After the chart type, the order of words such as `title_top` and `label_y` does not matter: the chart is drawn when you save or show it.
- Words that belong to one axis end in _x or _y: `label_x`, `label_y`, `rotate_labels_x`.

Try it yourself

1. Give the chart your own title, and label the y-axis `"Marks out of 100"`.
2. Try `chart.font_size(20)`. Then try 9. Which is easier to read?
3. Take away the line `chart.rotate_labels_x(45)`. What happens to the names?

Worked solutions: page 252, or type `sdata.lesson(2, solutions=True)`.

Words in this lesson: `chart.title_top()`, `chart.label_x()`, `chart.label_y()`, `chart.rotate_labels_x()`, `chart.caption_bottom()`, `chart.font_size()`, `chart.names_on_dots()`.

LESSON 3

Colours, groups and legends

In this lesson you will learn

- how to give each group its own colour
- how to choose your own colours
- how to move the legend (the box that says which colour is which)
- how to change the whole look with a style

Example 1

`colour=` inside a chart type splits the data into groups.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="subject", y="marks", colour="class")
chart.colours("orange", "teal", "purple")
chart.legend_bottom()
chart.title_top("Marks in each subject")
chart.save("lesson3_groups.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. For each subject, draw one bar per class, each class in its own colour.
5. Choose the colours yourself, in order: class A orange, B teal, C purple.
6. Put the legend under the chart.
7. Add a title.
8. Save it.

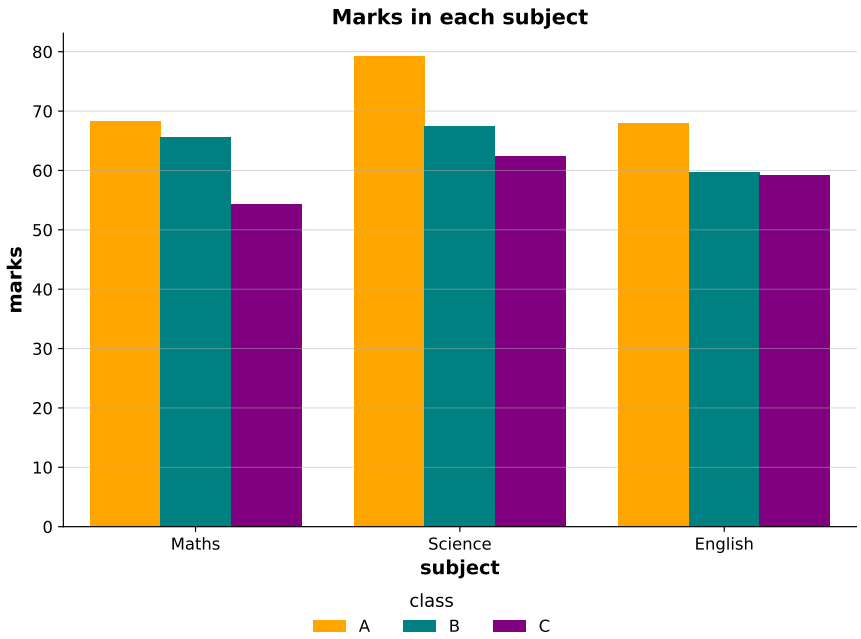


Figure 3.1: Lesson 3, example 1: `lesson3_groups.png`.

Example 2

Without groups, one colour is used for everything. You can choose it.

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="rain")
chart.colour("blue")
chart.style("school")
chart.save("lesson3_look.png")
```

What each line does

1. Load simplydata.
2. Get the weather table: three made-up cities.
3. Start a chart.
4. Draw a line of rain through the months. With three cities in each month, the line shows their average.
5. Make the line blue.
6. Change the whole look. Try `"paper"`, `"school"`, `"presentation"` and `"dark"`.
7. Save it.



Figure 3.2: Lesson 3, example 2: `lesson3_look.png`.

Example 3

Groups come in the order they first appear in the table. You can choose another order, and give the legend a name.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks", colour="subject")
chart.order_x("C", "B", "A")
chart.order_legend("Science", "Maths", "English")
chart.label_legend("Subject")
chart.save("lesson3_order.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. For each class, one bar per subject.
5. Put the classes along the x-axis in this order.
6. Put the subjects in this order: in the legend and side by side in each class.
7. The name above the legend (normally the column name).
8. Save it.

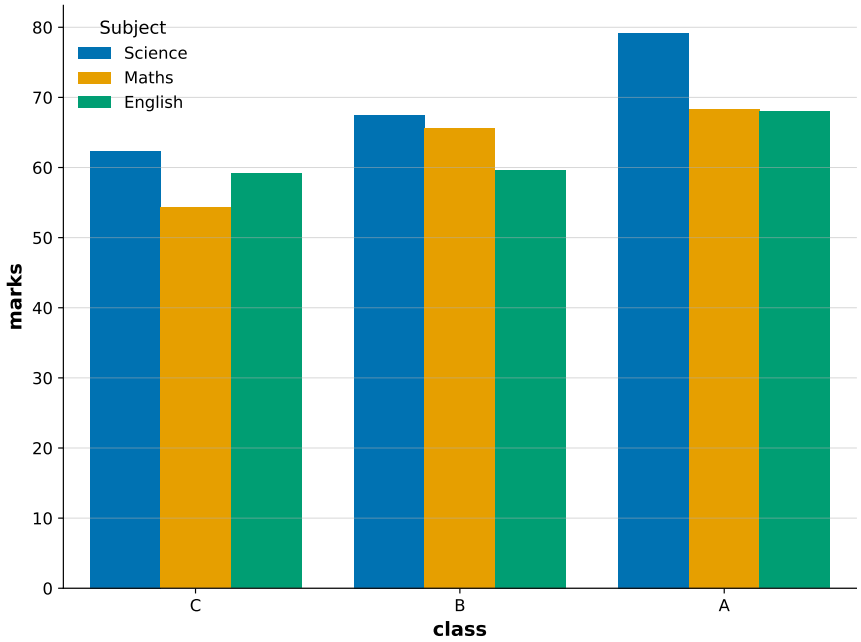


Figure 3.3: Lesson 3, example 3: `lesson3_order.png`.

Example 4

Numbers can have colours too: a colour scale. And dots that overlap are easier to see when they let the others show through.

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.heatmap(x="month", y="city", value="temperature")
chart.colour_scale("blue to red", middle=10)
chart.save("lesson3_colour_scale.png")
marks = sdata.example("marks")
dots = sdata.chart(marks)
dots.dots(x="hours", y="marks", colour="class", transparency=0.5)
dots.save("lesson3_transparency.png")
```

What each line does

1. Load simplydata.
2. Get the weather table.
3. Start a chart.
4. One square per city and month, coloured by temperature.
5. Blue below 10 °C, red above, white at 10. `middle=` makes a scale that goes two ways.
6. Save it.
7. Get the marks table.
8. A second chart.
9. `transparency=0.5` makes each dot half see-through, so dots on top of each other show.
10. Save it.

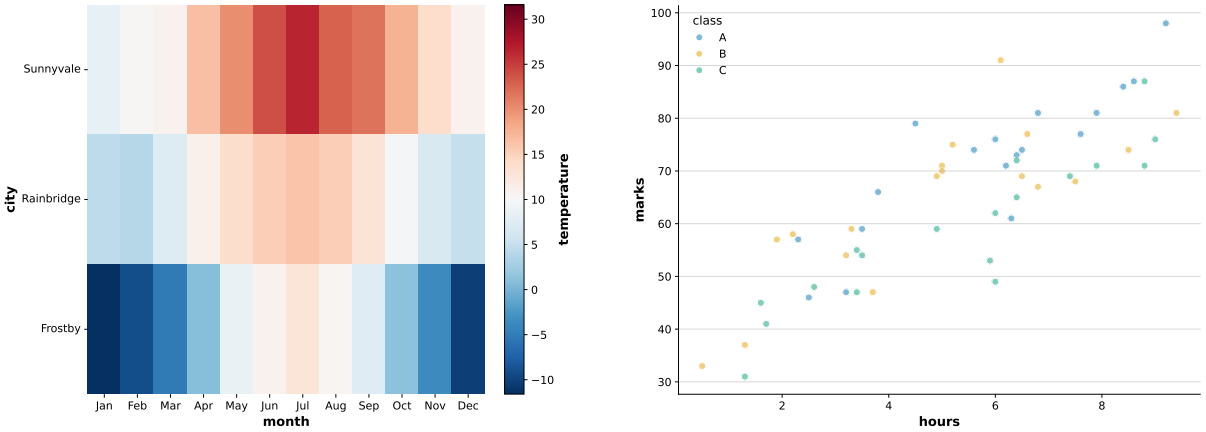


Figure 3.4: Lesson 3, example 4: `lesson3_colour_scale.png` and `lesson3_transparency.png`.

Good to know

- `colour="class"` (a column) gives each group a colour. `colour="red"` (a colour) makes that step red. color works too.
- The legend can go on any side: `legend_top`, `legend_bottom`, `legend_left`, `legend_right`. `hide_legend` takes it away and `show_legend` brings it back.
- Month and weekday names (`"Jan"`, `"Monday"`) come in calendar order by themselves, also in the small charts of `one_chart_per` and the frames of animate. `order_x(biggest_first=True)` puts the biggest bar first, and `order_y` orders the groups up the y-axis.

Try it yourself

1. In the weather chart, add `colour="city"` to `chart.lines(...)`. How many lines do you get?
2. Move the legend to the top with `chart.legend_top()`.
3. Try `chart.style("dark")` and choose three colours that look good on it.

Worked solutions: page 255, or type `sdata.lesson(3, solutions=True)`.

Words in this lesson: `chart.colours()`, `chart.colour()`, `chart.colour_scale()`, `chart.legend_bottom()`, `chart.legend_top()`, `chart.hide_legend()`, `chart.style()`, `chart.lines()`, `chart.order_x()`, `chart.order_legend()`, `chart.label_legend()`.

LESSON 4

Different chart types

In this lesson you will learn

- lines, dots, histograms, boxes and pies
- which chart suits which question
- how to put two chart types on top of each other

Example 1

Lines show how something changes, for example over time.

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature", colour="city", dots=True)
chart.save("lesson4_lines.png")
```

What each line does

1. Load simplydata.
2. Get the weather table.
3. Start a chart.
4. One line per city. `dots=True` also marks each month with a dot.
5. Save it.

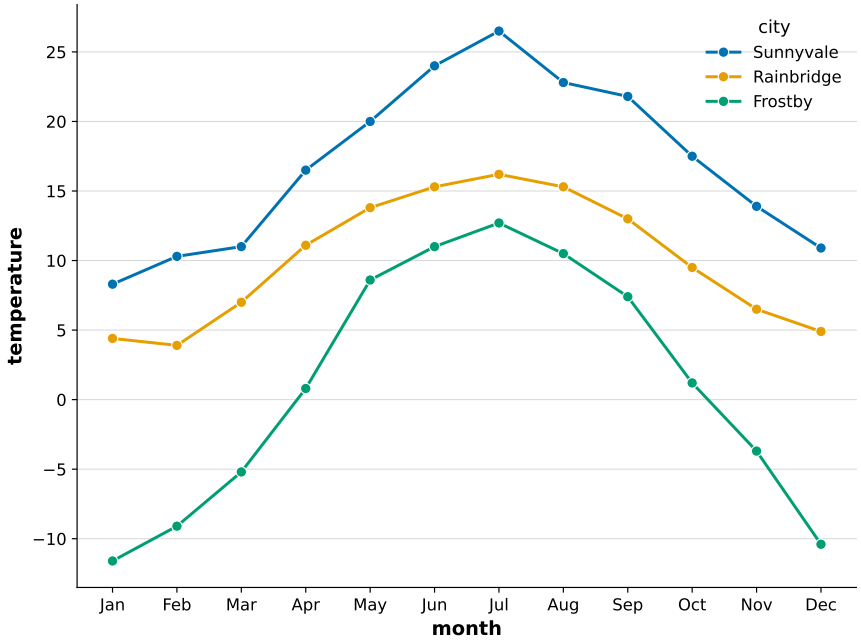


Figure 4.1: Lesson 4, example 1: `lesson4_lines.png`.

Example 2

Dots show if two things go together. This is also called a scatter chart.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks", colour="class")
chart.save("lesson4_dots.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. One dot for each test: how long the child studied, and the mark they got.
5. Save it. Do more hours give more marks?

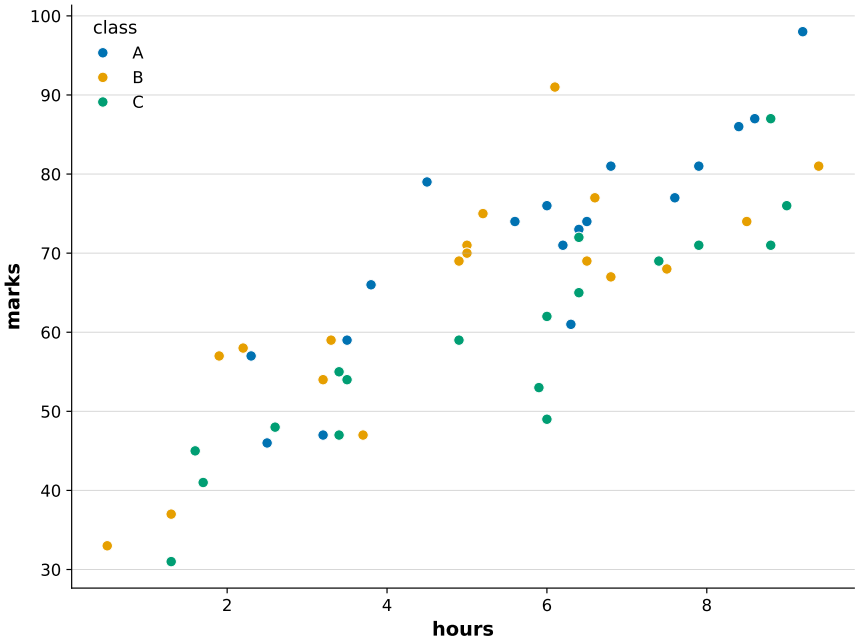


Figure 4.2: Lesson 4, example 2: `lesson4_dots.png`.

Example 3

A histogram counts how many values fall in each range, like 40 to 50, 50 to 60 ... Its shape is called the distribution of the values.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.histogram(x="marks", bins=8)
chart.save("lesson4_histogram.png")
```

What each line does

1 Load simplydata.
2 Get the marks table.
3 Start a chart.
4 Cut the marks into 8 ranges (bins) and draw a bar for how many marks are in each.
5 Save it.

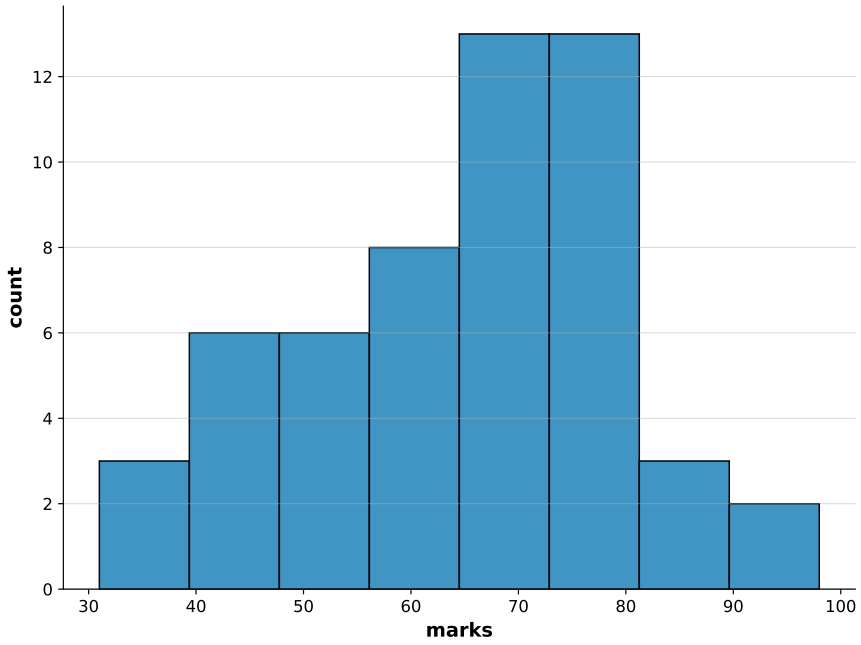


Figure 4.3: Lesson 4, example 3: `lesson4_histogram.png`.

Example 4

Boxes show how spread out the values are. Two chart types on one chart are drawn one on top of the other.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.boxes(x="class", y="marks")
chart.all_dots(x="class", y="marks")
chart.save("lesson4_boxes.png")
```

What each line does

1 Load simplydata.
2 Get the marks table.
3 Start a chart.
4 One box per class. The line in the box is the median: the middle mark when they are put in order. The box holds the middle half of the marks (the interquartile range).
5 Also show every single mark as a dot, on top of the boxes.
6 Save it.

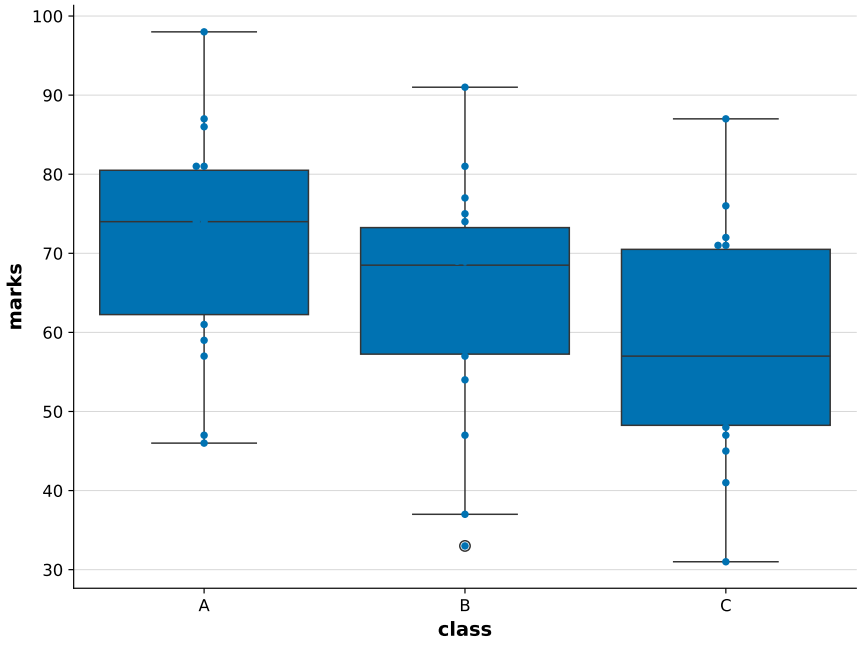


Figure 4.4: Lesson 4, example 4: `lesson4_boxes.png`.

Example 5

A pie shows how a whole is shared out.

```
import simplydata as sdata
fruit = sdata.example("fruit")
chart = sdata.chart(fruit)
chart.pie(labels="fruit", values="count")
chart.save("lesson4_pie.png")
```

What each line does

1. Load simplydata.
2. Get the fruit table: what two shops sold in a week.
3. Start a chart.
4. One slice per fruit, as big as the number sold.
5. Save it.

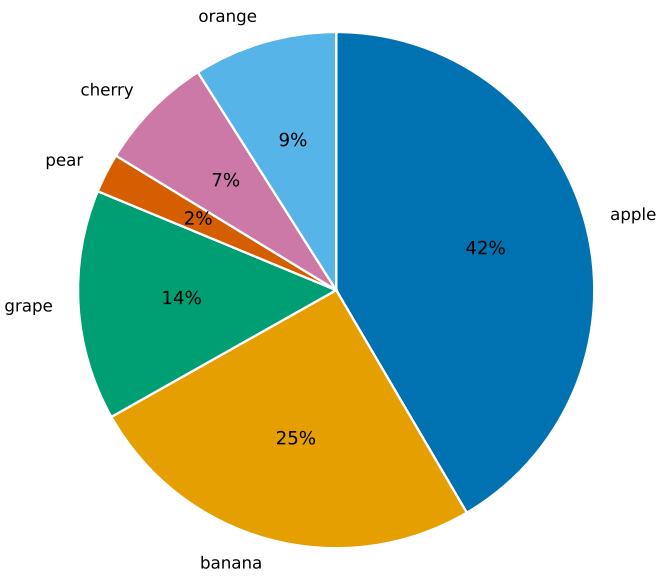


Figure 4.5: Lesson 4, example 5: `lesson4_pie.png`.

Good to know

- Chart types on the same chart are layers of one chart. Lesson 7 shows how to make several charts and put them on one page.
- There are more chart types: area, violins, heatmap, polar, contour, surface and `dots_3d`. Type `sdata.words()` to see them all and `sdata.help("violins")` to learn one.

Try it yourself

1. Draw `chart.violins(x="class", y="marks")` instead of boxes. What is different?
2. Draw a histogram of the weather `"temperature"` column with `bins=6`.
3. Make a chart of `sdata.example("growth")` and draw `chart.area(x="year", y="population", colour="country")`.

Worked solutions: page 258, or type `sdata.lesson(4, solutions=True)`.

Words in this lesson: `chart.lines()`, `chart.dots()`, `chart.histogram()`, `chart.boxes()`, `chart.all_dots()`, `chart.pie()`, `chart.violins()`, `chart.area()`.

LESSON 5

Statistics: averages, error bars and fit lines

In this lesson you will learn

- what a bar shows when a group has many values: the average or the median
- how to add error bars, and the difference between standard deviation and standard error
- how to draw the straight line that fits some dots best

Example 1

The average (also called the mean) is all the values added up, divided by how many there are. Error bars show how far the values spread around it.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks", show="average", error="standard deviation")
chart.numbers_on_bars()
chart.save("lesson5_average.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. One bar per class, as tall as the average mark. The error bars show the standard deviation: how far marks usually are from the average. A line is printed that says which error bars were drawn.
5. Write the average on each bar.
6. Save it.

What it prints

```
Error bars: the standard deviation of the values (how spread out they are around the average).
```

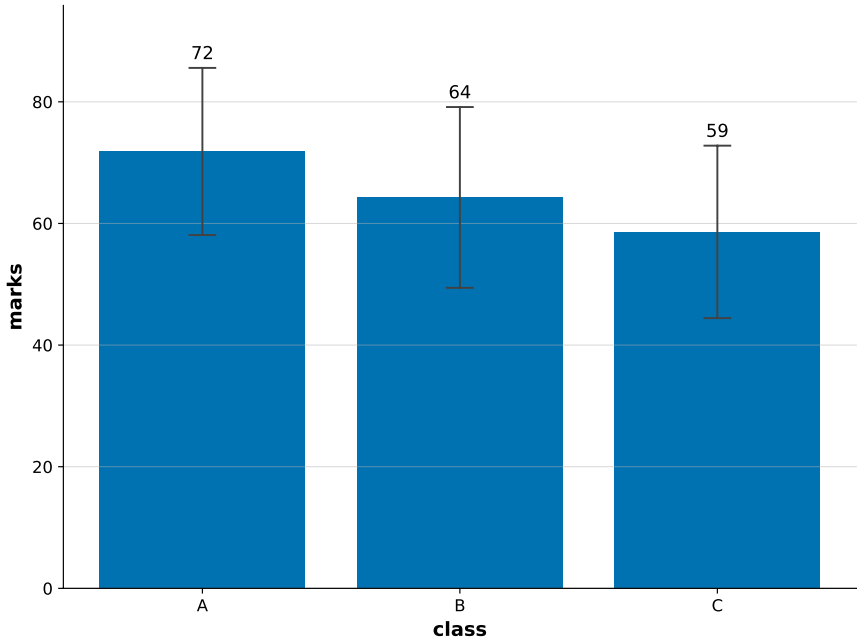


Figure 5.1: Lesson 5, example 1: `lesson5_average.png`.

Example 2

A fit line is the straight line that goes as close as possible to all the dots.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.fit_line(confidence_band=True, show_equation=True)
chart.save("lesson5_fit.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. One dot per test: hours of study and the mark.
5. Add the best straight line (also called linear regression). `confidence_band=True` shades where the true line very likely is (the 95 % confidence interval). `show_equation=True` writes its equation and R^2 (1 means the dots sit right on the line, 0 means no pattern at all).
6. Save it.

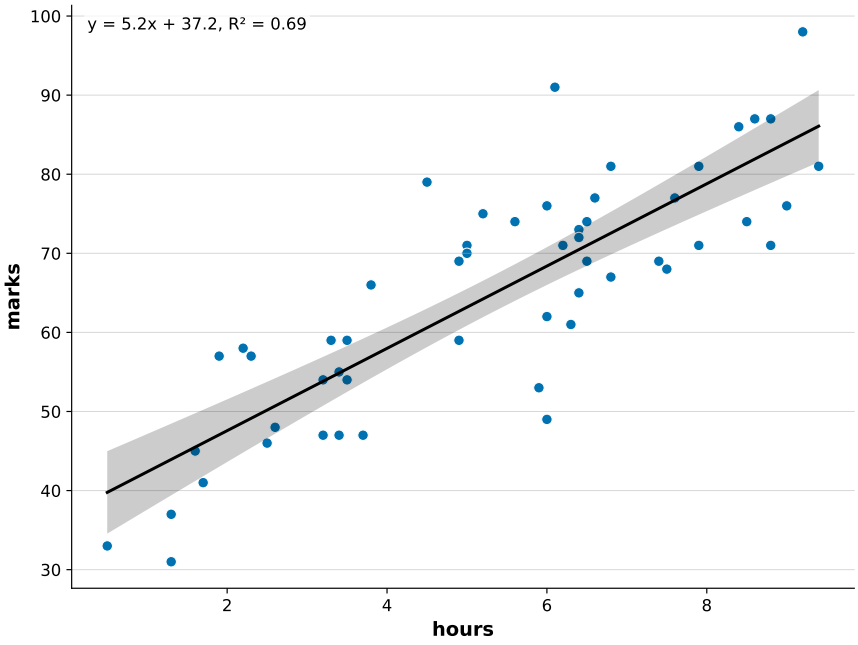


Figure 5.2: Lesson 5, example 2: `lesson5_fit.png`.

Example 3

A density curve is a smooth histogram: high where many values are.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.density(x="marks", colour="class")
chart.save("lesson5_density.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. One smooth curve for each class (a kernel density estimate).
5. Save it.

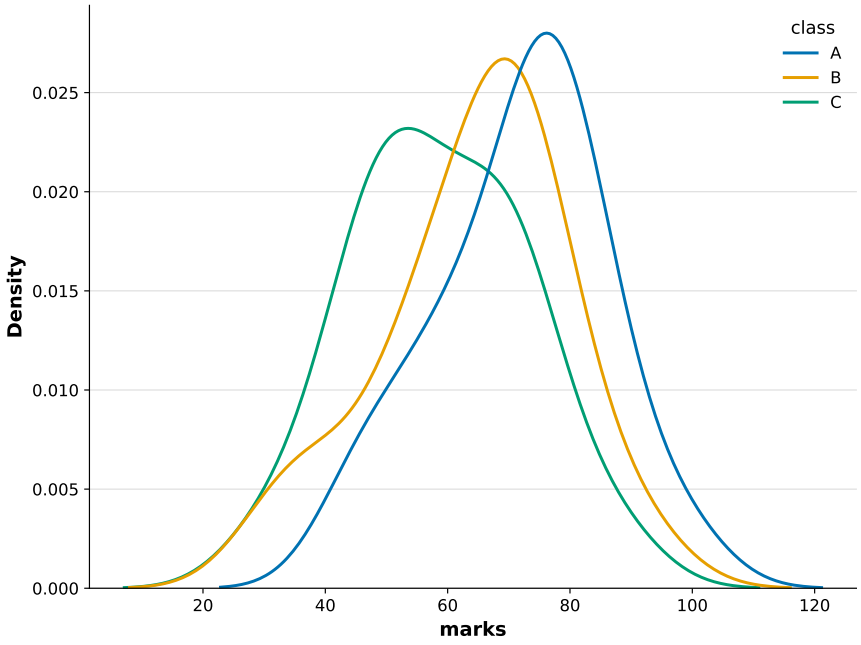


Figure 5.3: Lesson 5, example 3: `lesson5_density.png`.

Example 4

Two more ways to see the shape of values: are they bell-shaped, and what share lies below each value?

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.qq_dots("marks")
chart.save("lesson5_qq.png")
shares = sdata.chart(marks)
shares.cumulative_share(x="marks", colour="class")
shares.save("lesson5_cumulative.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. A Q-Q plot: if the marks follow a bell curve (normal), the dots lie close to the straight line.
5. Save it.
6. A second chart.
7. For each mark, the share of children with that mark or less (an ECDF). The class whose line is further right did better.
8. Save it.

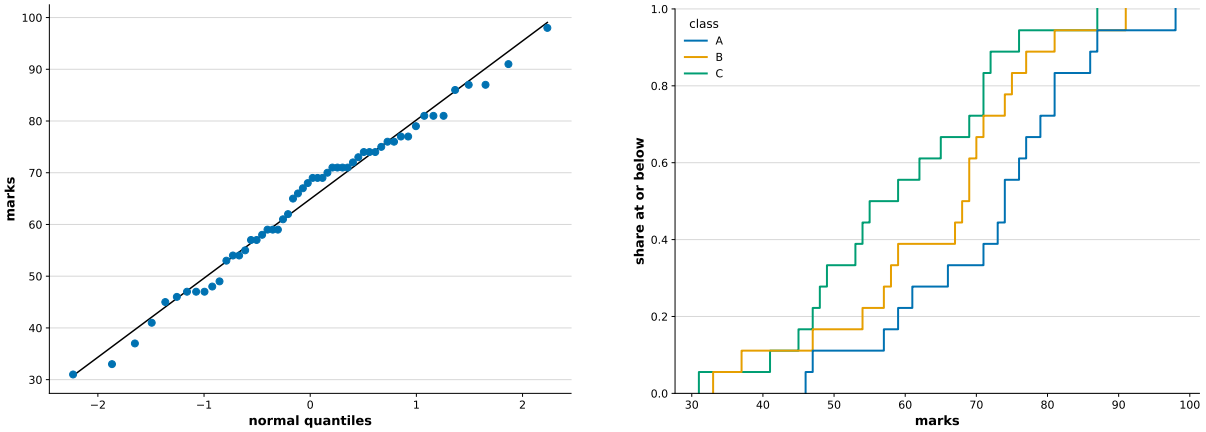


Figure 5.4: Lesson 5, example 4: `lesson5_qq.png` and `lesson5_cumulative.png`.

Good to know

- Average or median? The median is the middle value when the values are put in order. One very big or very small value pulls the average towards it, but hardly moves the median, so the median is safer when a few values are far from the rest (outliers).
- `show=` can be `"average"`, `"median"`, `"total"` (all added up), `"count"` (how many rows), `"smallest"`, `"biggest"` or `"standard deviation"`: the same names as the table words `average_of`, `median_of` and so on.
- Standard deviation or standard error? The standard deviation describes the values: how spread out they are. The standard error describes the average: how sure we are of it. It gets smaller as you collect more values, even when the values spread just as much.
- `error="confidence interval"` draws the 95 % confidence interval: the range the true average is very likely to be in. The short forms `"sd"`, `"se"` and `"ci"` work too. Always say in your caption which error bars you drew.
- `fit_curve(degree=2)` fits a curve with one bend; `smooth_line()` follows the dots freely.

Try it yourself

1. Change `show="average"` to `show="median"`. Do the bars change much?
2. Change `error="standard deviation"` to `error="standard error"`. Are the error bars bigger or smaller? Why?
3. Make a chart of the weather table, draw dots with `x="month_number"`, `y="temperature"` and add `chart.fit_curve(degree=2)`.

Worked solutions: page 260, or type `sdata.lesson(5, solutions=True)`.

Words in this lesson: `show=`, `error=`, `chart.numbers_on_bars()`, `chart.fit_line()`, `chart.density()`, `chart.fit_curve()`, `chart.smooth_line()`, `chart.qq_dots()`, `chart.cumulative_share()`.

LESSON 6

Axes and reference lines

In this lesson you will learn

- how to choose where an axis starts and stops
- how to draw a line at a value, like a pass mark, and shade a band
- how to choose the tick marks, and what a log scale is

Example 1

The axes are the two edges with numbers: x along the bottom, y up the side.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.range_y(0, 100)
chart.ticks_y(0, 25, 50, 75, 100)
chart.line_at_y(50, label="pass mark")
chart.shade_between_y(40, 60)
chart.rotate_labels_x()
chart.save("lesson6_marks.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. One bar for each child.
5. Make the y-axis go from 0 to 100, the lowest and highest mark you can get.
6. Put the numbers on the y-axis at these places.
7. Draw a dashed line across at 50, with a name.
8. Shade the band between 40 and 60.
9. Turn the names so they fit.
10. Save it.

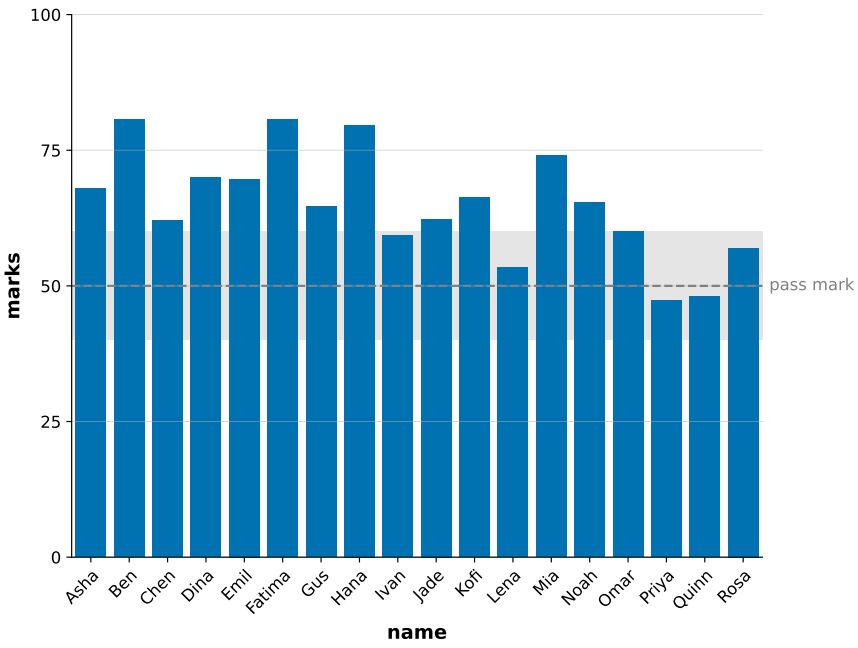


Figure 6.1: Lesson 6, example 1: `lesson6_marks.png`.

Example 2

On a log scale 1, 10, 100 and 1000 are the same distance apart. It helps when some numbers are much bigger than others.

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.lines(x="year", y="population", colour="country")
chart.log_scale_y()
chart.line_at_x(2010, label="2010")
chart.label_y("People (millions)")
chart.save("lesson6_log.png")
```

What each line does

1. Load simplydata.
2. Get the growth table: people in four made-up countries.
3. Start a chart.
4. One line per country.
5. Use a log scale on the y-axis. A country that grows by the same share each year now gets a straight line.
6. Draw an up-and-down line at the year 2010.
7. Name the y-axis.
8. Save it.

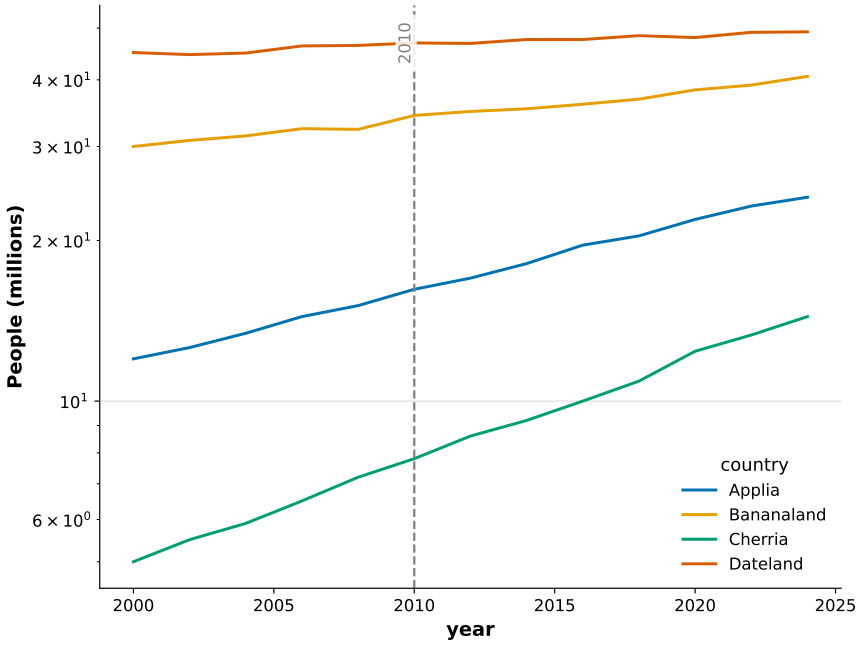


Figure 6.2: Lesson 6, example 2: `lesson6_log.png`.

Example 3

A band shades the space between a low and a high value at each x: a range, or the uncertainty around a line.

```
import simplydata as sdata
days = {"day": [1, 2, 3, 4, 5], "lowest": [3, 5, 4, 6, 8], "average": [6, 8, 7, 9, 11], "
    ↪ highest": [9, 11, 10, 12, 14]}
chart = sdata.chart(days)
chart.band(x="day", between=["lowest", "highest"], label="lowest to highest")
chart.lines(x="day", y="average")
chart.label_y("Temperature (°C)")
chart.save("lesson6_band.png")
```

What each line does

1. Load simplydata.
2. A small table: the lowest, average and highest temperature on five days.
3. Start a chart of it.
4. Shade from the lowest to the highest value of each day. `label=` names it in the legend.
5. The average as a line through the band.
6. Name the y-axis.
7. Save it.

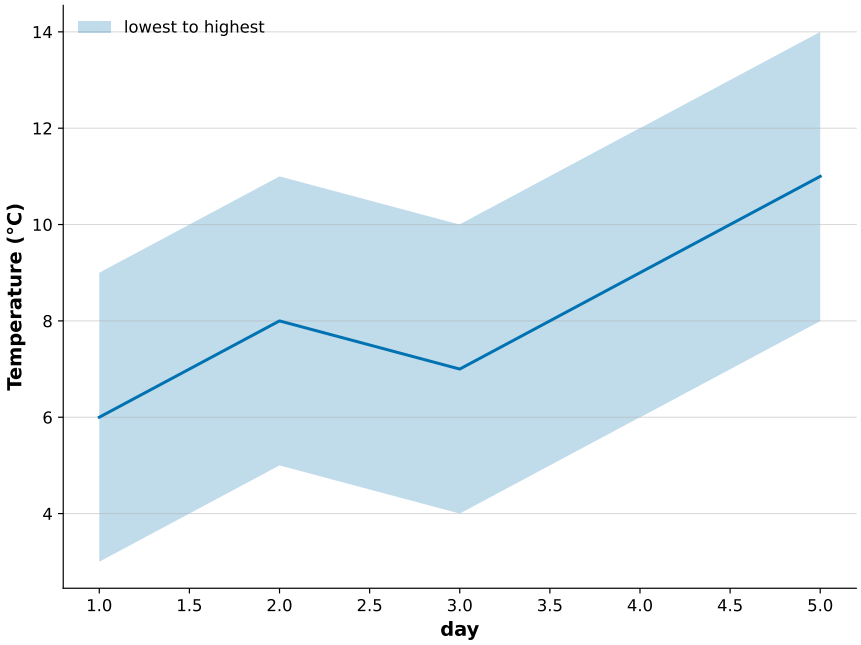


Figure 6.3: Lesson 6, example 3: `lesson6_band.png`.

Example 4

Axes can say what their numbers mean, and a 1:1 line shows where two columns agree.

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.lines(x="year", y="forest", colour="country")
chart.tick_format_y("percent")
chart.save("lesson6_percent.png")
guesses = {"measured": [12, 18, 25, 31, 40], "predicted": [14, 17, 27, 29, 43]}
check = sdata.chart(guesses)
check.dots(x="measured", y="predicted")
check.line_with_slope(1, through=(0, 0), label="1:1")
check.save("lesson6_one_to_one.png")
```

What each line does

1. Load simplydata.
2. Get the growth table.
3. Start a chart.
4. The forest of each country, in %.
5. Write the y numbers as 40 %, 50 % … (`"thousands"`, `"scientific"` and dates such as `"Jan 2026"` work too).
6. Save it.
7. Five measured values and what a model predicted for them.
8. A second chart.
9. One dot per value.
10. The 1:1 line: dots on it were predicted exactly; above it, too high.
11. Save it.

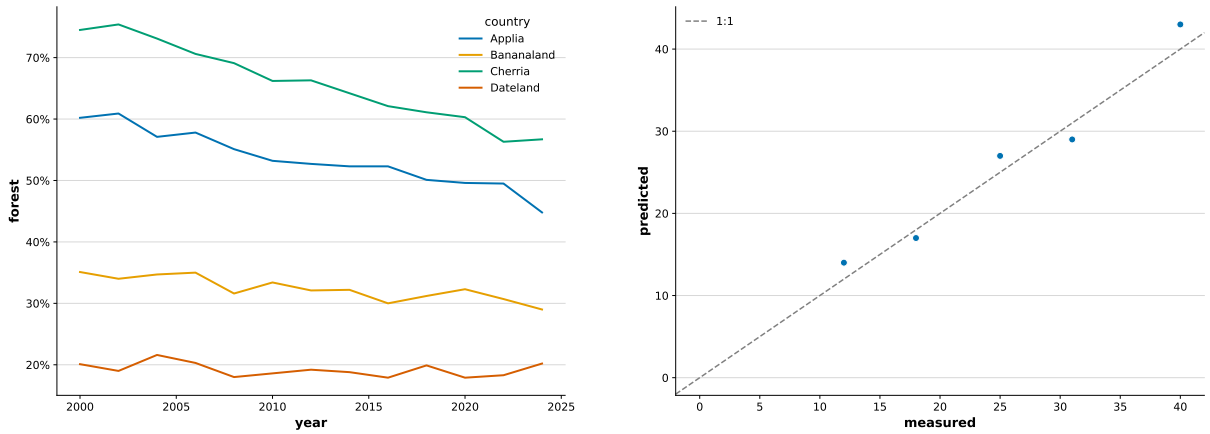


Figure 6.4: Lesson 6, example 4: `lesson6_percent.png` and `lesson6_one_to_one.png`.

Good to know

- Every axis word ends in _x or _y: `range_x`, `ticks_x`, `log_scale_x`, `line_at_x`, `shade_between_x`, `flip_x` ...
- `range_y(0, None)` fixes only the bottom and lets the top choose itself.
- Bars should start at 0: a bar is read by its length, so a y-axis that starts at 40 makes small differences look big.

Try it yourself

1. Move the pass mark to 60 and shade between 0 and 40 instead.
2. Take away `chart.log_scale_y()`. Which countries look different?
3. Try `chart.flip_y()` on any chart. What happens?

Worked solutions: page 262, or type `sdata.lesson(6, solutions=True)`.

Words in this lesson: `chart.range_y()`, `chart.ticks_y()`, `chart.line_at_y()`, `chart.shade_between_y()`, `chart.log_scale_y()`, `chart.line_at_x()`, `chart.flip_y()`, `chart.band()`, `chart.tick_format_y()`, `chart.line_with_slope()`.

LESSON 7

Several charts together

In this lesson you will learn

- how to make one small chart for each group
- how to put different charts on one page: side by side, one above the other, or in a grid
- how to write A, B, C on the charts, as figures in papers have
- how to compare every number column with every other

Example 1

`one_chart_per` draws the same chart once for each group.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.fit_line()
chart.one_chart_per("class")
chart.save("lesson7_one_chart_per.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. Dots of hours and marks.
5. Add a fit line.
6. Make one small chart for each class, with the same axes so they are easy to compare.
7. Save it.

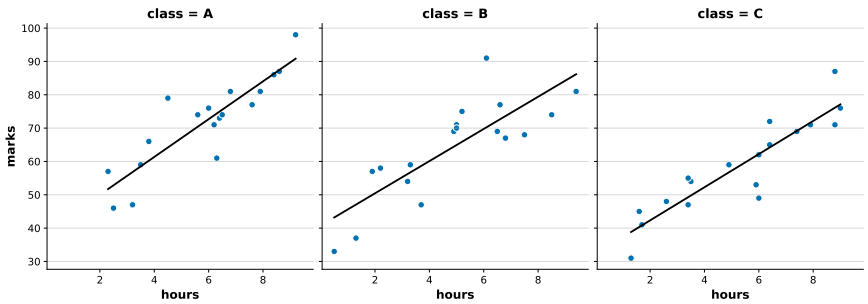


Figure 7.1: Lesson 7, example 1: `lesson7_one_chart_per.png`.

Example 2

Each chart is an object of its own, so different charts can share one page.

```
import simplydata as sdata
weather = sdata.example("weather")
a = sdata.chart(weather).lines(x="month", y="temperature", colour="city")
b = sdata.chart(weather).bars(x="month", y="rain", colour="city")
page = sdata.one_above_other(a, b)
page.title_top("A year of weather")
page.save("lesson7_page.png")
```

What each line does

1. Load simplydata.
2. Get the weather table.
3. Make a first chart, called a. Steps can also be joined with dots on one line.
4. Make a second chart, called b.
5. Put a above b on one page.
6. Give the whole page a title.
7. Save the page.

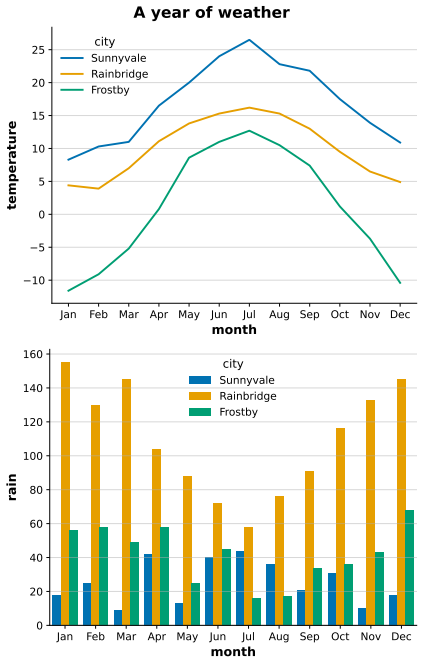


Figure 7.2: Lesson 7, example 2: `lesson7_page.png`.

Example 3

A grid holds many charts in rows and columns. `letters=True` writes A, B, C and D on them, as figures in papers have.

```
import simplydata as sdata
marks = sdata.example("marks")
a = sdata.chart(marks).bars(x="class", y="marks").title_top("Average")
b = sdata.chart(marks).histogram(x="marks").title_top("All marks")
c = sdata.chart(marks).dots(x="height", y="marks").title_top("Height")
d = sdata.chart(marks).boxes(x="subject", y="marks").title_top("Subjects")
page = sdata.grid(a, b, c, d, per_row=2, letters=True)
page.save("lesson7_grid.png")
```

What each line does

1 Load simplydata.
2 Get the marks table.
3 Chart a.
4 Chart b.
5 Chart c.
6 Chart d.
7 All four in a grid, 2 charts per row, with a letter on each.
8 Save it.

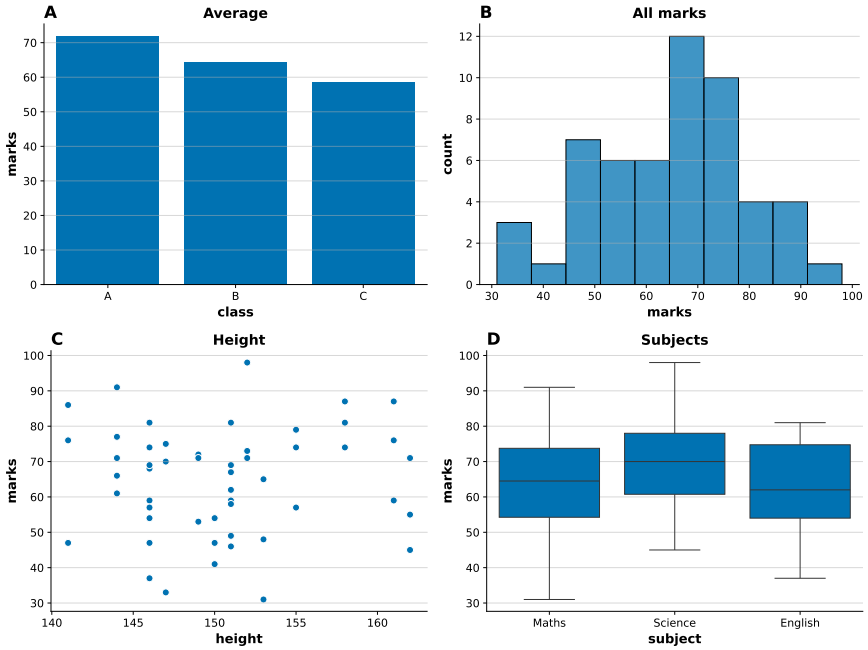


Figure 7.3: Lesson 7, example 3: `lesson7_grid.png`.

Example 4

Charts from different tables can share a page too.

```
import simplydata as sdata
rain = sdata.chart(sdata.example("weather")).lines(x="month", y="rain", colour="city")
forest = sdata.chart(sdata.example("growth")).lines(x="year", y="forest", colour="country")
page = sdata.side_by_side(rain, forest, letters=True)
page.save("lesson7_two_tables.png")
```

What each line does

1 Load simplydata.
2 A chart of the weather table, called rain.
3 A chart of the growth table, called forest.
4 Put the two charts next to each other, with A and B.
5 Save the page.

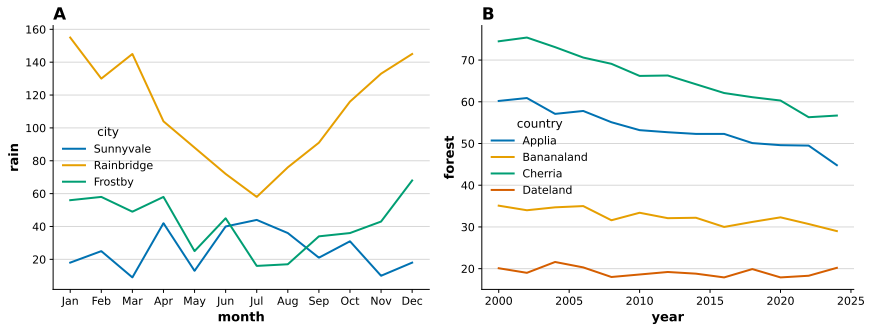


Figure 7.4: Lesson 7, example 4: `lesson7_two_tables.png`.

Example 5

`sdata.pairs` draws every number column against every other, in one go.

```
import simplydata as sdata
pairs = sdata.pairs(sdata.example("marks"), colour="class")
pairs.save("lesson7_pairs.png")
```

What each line does

1. Load simplydata.
2. Hours, marks and height, each against each other. The middle line shows how each column is spread out.
3. Save it. Which two columns go together?

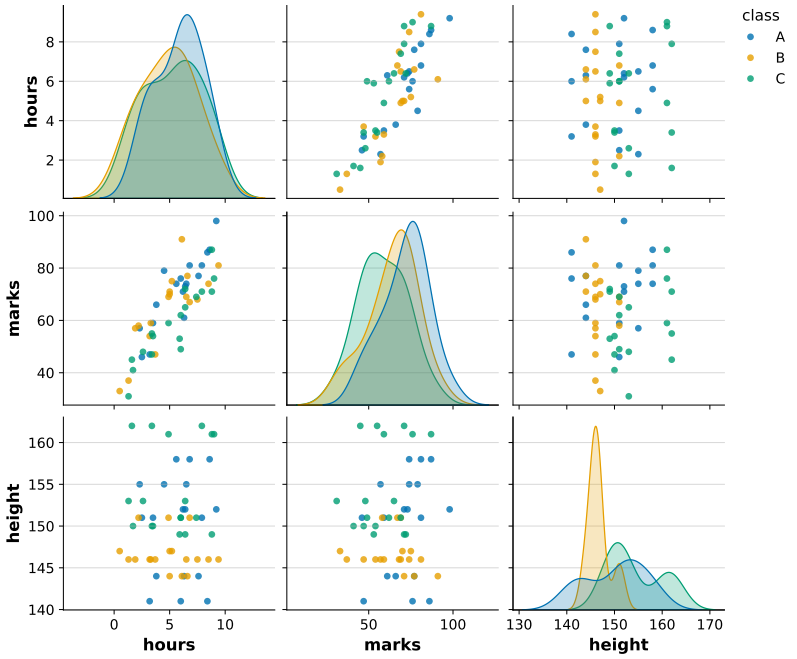


Figure 7.5: Lesson 7, example 5: `lesson7_pairs.png`.

Good to know

- `same_y=True` gives all the charts on a page the same y-axis, so heights can be compared fairly: `sdata.side_by_side(a, b, same_y=True)`.

- `sdata.side_by_side`, `sdata.one_above_other` and `sdata.grid` all take `letters=True`.
- `chart.one_chart_per(down="class", across="subject")` makes a table of small charts: the classes go down, the subjects go across. `per_row=3` puts 3 small charts in a row.

Try it yourself

1. Change `chart.one_chart_per("class")` to `chart.one_chart_per("subject")`.
2. Use `sdata.side_by_side(a, b)` instead of `sdata.one_above_other(a, b)` for the weather charts.
3. Make `sdata.pairs` of `sdata.example("weather")` with `colour="city"`.

Worked solutions: page 264, or type `sdata.lesson(7, solutions=True)`.

Words in this lesson: `chart.one_chart_per()`, `sdata.one_above_other()`, `sdata.grid()`, `sdata.side_by_side()`, `sdata.pairs()`, `chart.fit_line()`, `chart.histogram()`.

LESSON 8

Hard tricks, and your own program

In this lesson you will learn

- a second y-axis, a zoom box and a moving chart
- a heatmap that sorts itself
- how to write your own program from start to end

Example 1

A second y-axis shows a column measured in other units on the same chart.

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature")
chart.second_y("rain", drawn_as="bars", label="Rain (mm)")
chart.label_y("Temperature (°C)")
chart.save("lesson8_second_y.png")
```

What each line does

1. Load simplydata.
2. Get the weather table.
3. Start a chart.
4. A line of temperature (average of the three cities).
5. Add rain, drawn as bars, with its own y-axis on the right.
6. Name the left y-axis.
7. Save it.

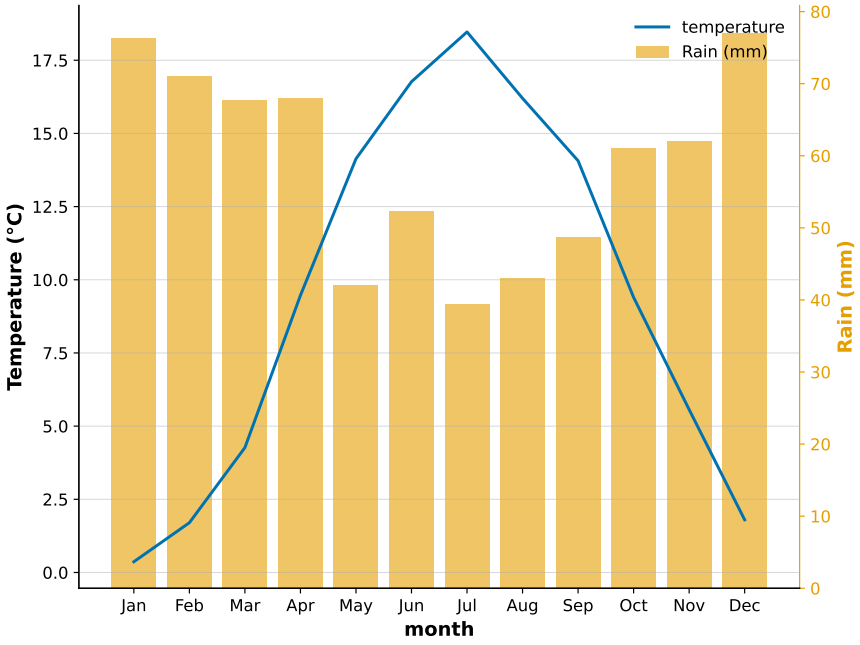


Figure 8.1: Lesson 8, example 1: `lesson8_second_y.png`.

Example 2

A zoom box shows a small part of the chart close up.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.zoom_box(x_between=[2, 4], y_between=[40, 60], where="top left")
chart.save("lesson8_zoom.png")
```

What each line does

1. Load simplydata.
2. Get the marks table.
3. Start a chart.
4. Dots of hours and marks.
5. Show the part between 2 and 4 hours and between 40 and 60 marks, bigger, in a box at the top left.
6. Save it.

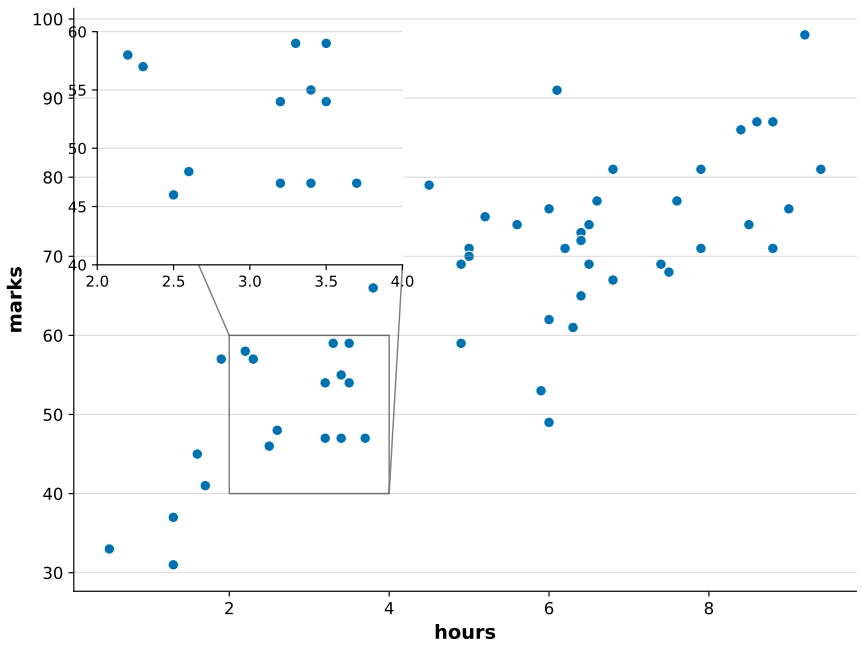


Figure 8.2: Lesson 8, example 2: `lesson8_zoom.png`.

Example 3

A heatmap colours a grid of numbers. `cluster=True` puts rows that are alike next to each other.

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.heatmap(x="month", y="city", value="rain", numbers=True, cluster=True)
chart.label_colour_bar("Rain (mm)")
chart.size(width=12, height=4)
chart.save("lesson8_heatmap.png")
```

What each line does

1. Load simplydata.
2. Get the weather table.
3. Start a chart.
4. One square per city and month, coloured by rain, with the number written in. `cluster=True` sorts the rows and columns and draws a tree of how alike they are (hierarchical clustering).
5. Name the colour bar.
6. Make the chart wide and low.
7. Save it.

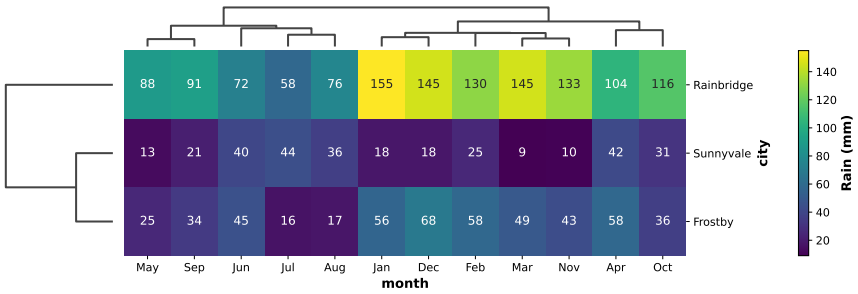


Figure 8.3: Lesson 8, example 3: `lesson8_heatmap.png`.

Example 4

An animation is a chart that moves: one picture for each value of a column.

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.bars(x="country", y="population")
chart.range_y(0, 50)
chart.animate(over="year")
chart.save("lesson8_growth.gif")
```

What each line does

1. Load simplydata.
2. Get the growth table.
3. Start a chart.
4. One bar per country.
5. Keep the y-axis still, from 0 to 50.
6. Make one picture for each year.
7. Save it as a `.gif`, a moving picture. Open it in a web browser to watch.

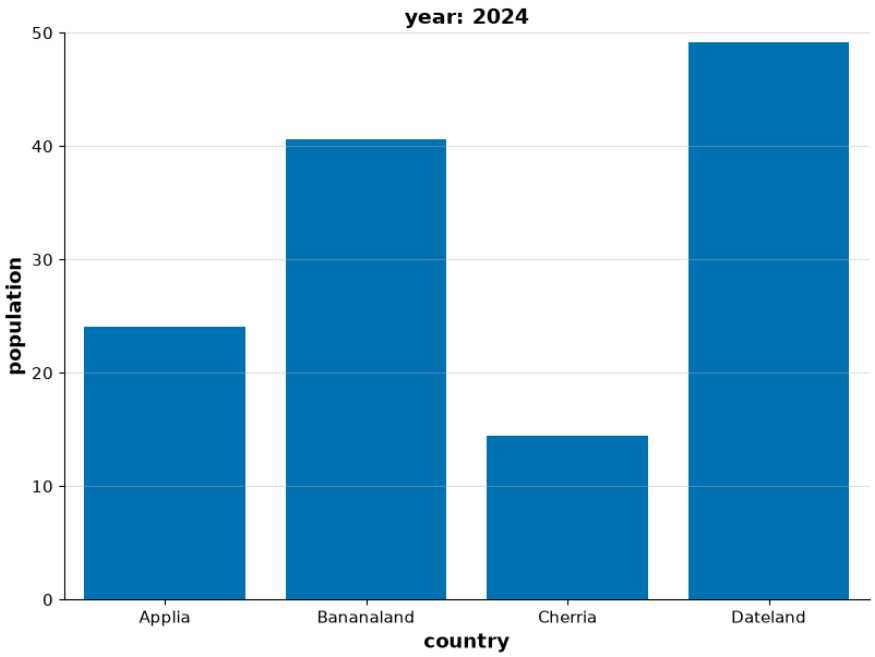


Figure 8.4: Lesson 8, example 4: `lesson8_growth.gif` (the last picture of the animation).

Example 5

Your own program. Every chart follows the same plan: get the data, look, start, draw, words, look, save.

```
import simplydata as sdata
island = sdata.example("island")
print(island)
chart = sdata.chart(island)
chart.contour(x="east", y="north", z="height")
chart.title_top("Treasure Island")
chart.label_x("East (km)")
chart.label_y("North (km)")
chart.label_colour_bar("Height (m)")
chart.note("X marks the spot", x=10, y=4, arrow=True)
chart.style("presentation")
chart.save("lesson8_my_program.png")
```

What each line does

1. Load simplydata.
2. Get the data. For your own file, skip this line and write chart = `sdata.chart("my_file.xlsx")` below.
3. Look at the data before you draw it.
4. Start a chart.
5. Draw: a map where each colour band is one height.
6. Words: a title ...
7. ... and the axis names ...
9. ... and the name of the colour bar.
10. A note with an arrow.
11. Look: big and clear for showing to others.
12. Save.

What it prints

```
     east  north  height
0       0      0      -6
1       0      1      -5
2       0      2      -4
3       0      3      -3
4       0      4      -1
..    ...    ...     ...
220    14     10      -2
221    14     11      -3
222    14     12      -4
223    14     13      -5
224    14     14      -6

[225 rows x 3 columns]
```

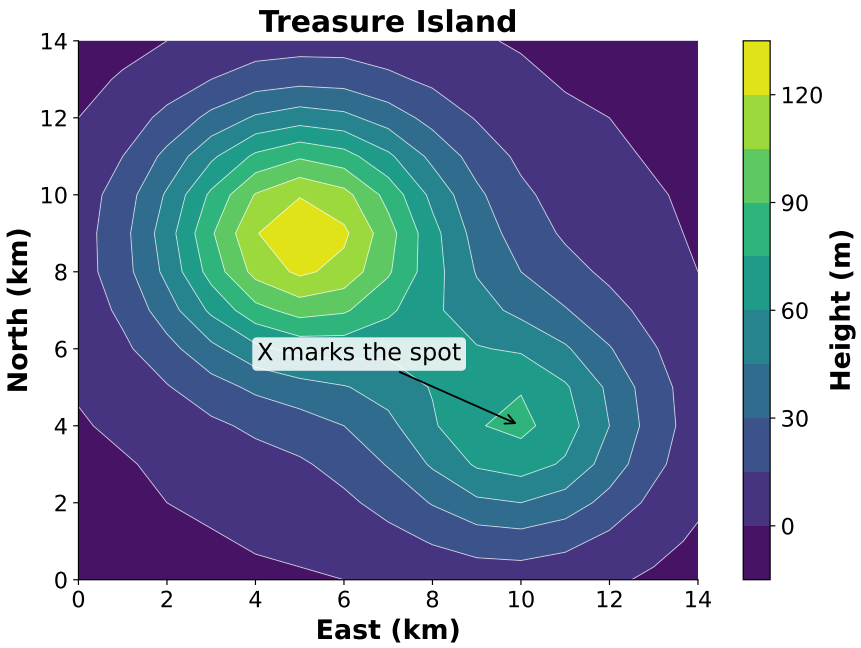


Figure 8.5: Lesson 8, example 5: `lesson8_my_program.png`.

Good to know

- Those are all the charts! `sdata.words()` lists every word and `sdata.help("word")` explains any of them. Lessons 9 to 12 teach tables: how to look at your data, clean it and sum it up before you draw it.
- Your own data: save it from Excel as `.xlsx` or `.csv`, then chart = `sdata.chart("my_file.xlsx")`. If a column name is spelt wrong, simplydata tells you the right one.

Try it yourself

1. Write your own program, from an empty file, for a question you care about.
2. Use any example table, or your own. Give it a title, axis names and a note.
3. Make a second chart of the same table, put the two charts side by side and save the page as a PDF.

Worked solutions: page 266, or type `sdata.lesson(8, solutions=True)`.

Words in this lesson: `chart.second_y()`, `chart.zoom_box()`, `chart.heatmap()`, `chart.label_colour_bar()`, `chart.size()`, `chart.animate()`, `chart.contour()`, `chart.note()`, `chart.style()`, `sdata.words()`, `sdata.help()`.

LESSON 9

Tables: open one and look at it

In this lesson you will learn

- how to turn data into a simplydata table
- how to look at the first and last rows, and the names of the columns
- how to describe every column and count what is in it

Example 1

Before you draw, look at your data. `sdata.table` gives you plain words for that.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
print(table)
table.show_first_rows(5)
table.show_last_rows(3)
print(table.column_names())
table.describe_columns()
table.count_rows()
```

What each line does

1. Load simplydata.
2. Make a table from the practice marks. For your own file, write `sdata.table("my_file.xlsx")`.
3. Print the table. The first line says how many rows and columns it has. Then come the first 10 rows.
4. Show only the first 5 rows.
5. Show the last 3 rows.
6. Print the names of the columns.
7. Describe every column: is it numbers or text, how many cells are empty, and the smallest, biggest and average number.
8. Count the rows, and say the answer in a sentence. One row is one child in one test.

What it prints

```
simplydata table: 54 rows, 6 columns
    name class  subject  hours  marks  height
1   Asha     A    Maths    6.3     61     144
2   Asha     A  Science    3.8     66     144
3   Asha     A  English    7.6     77     144
4    Ben     A    Maths    6.4     73     152
5    Ben     A  Science    9.2     98     152
6    Ben     A  English    6.2     71     152
7   Chen     A    Maths    7.9     81     151
8   Chen     A  Science    3.5     59     151
9   Chen     A  English    2.5     46     151
10  Dina     A    Maths    6.5     74     155
   name class  subject  hours  marks  height
```

```
1  Asha     A    Maths    6.3     61     144
2  Asha     A  Science    3.8     66     144
3  Asha     A  English    7.6     77     144
4   Ben     A    Maths    6.4     73     152
5   Ben     A  Science    9.2     98     152
    name class  subject  hours  marks  height
52  Rosa     C    Maths    3.4     55     162
53  Rosa     C  Science    1.6     45     162
54  Rosa     C  English    7.9     71     162
['name', 'class', 'subject', 'hours', 'marks', 'height']
The table has 54 rows and 6 columns.

 column   kind  empty cells smallest biggest average different values
   name   text            0                                       18
  class   text            0                                        3
subject   text            0                                        3
  hours number            0      0.5     9.4    5.33
  marks number            0       31      98   64.91
 height number            0      141     162  150.39
The table has 54 rows.
```

Example 2

`count_each` counts how often each different value comes up.

```
import simplydata as sdata
table = sdata.table(sdata.example("fruit"))
table.show_first_rows()
table.count_each("fruit")
table.count_each("shop")
```

What each line does

1. Load simplydata.
2. Make a table of fruit sold by two shops.
3. Show the first rows. With nothing in the brackets you get 5.
4. For each fruit: in how many rows it comes up. It says the answer in a sentence, and gives back a small table that also has the percent of all the rows.
5. The same for each shop.

What it prints

```
   day          shop   fruit  count  price
1  Mon   Corner shop   apple     39   0.44
2  Mon  Market stall  banana     22   0.23
3  Mon  Market stall   grape     13   0.05
4  Mon  Market stall    pear      6   0.41
5  Tue   Corner shop   apple     39   0.44
How many rows have each "fruit" (6 different values): apple 9, banana 8, grape 5, cherry 4, orange 4, pear
    ↪ 3.
How many rows have each "shop" (2 different values): Market stall 19, Corner shop 14.
```

Example 3

`sdata.table` can read many files at once. Here we first save two small files, then read them back together.

```
import simplydata as sdata
weather = sdata.table(sdata.example("weather"))
weather.keep_first_rows(12)
weather.save("lesson9_part1.csv")
weather.start_again()
weather.keep_last_rows(12)
weather.save("lesson9_part2.csv")
both = sdata.table("lesson9_part*.csv")
both.count_each("file")
both.show(rows=3)
```

What each line does

1. Load simplydata.
2. Make a table of the weather.
3. Keep the first 12 rows: one city, a whole year.
4. Save them as a CSV file.
5. Back to the whole table.
6. Keep the last 12 rows: another city.
7. Save them as a second file.
8. Read every file whose name fits the pattern (* stands for any text) and stack them. A new column, `"file"`, says where each row came from.
9. 12 rows from each file.
10. Show the first 3 rows with every column. `both.show()` shows the whole table.

What it prints

```
Kept 12 of 36 rows.
Kept 12 of 36 rows.
Read 2 files and stacked them: 24 rows. The column "file" says which file each row came from.
How many rows have each "file" (2 different values): lesson9_part1.csv 12, lesson9_part2.csv 12.
                file month  month_number       city  temperature  rain  wind_direction  wind_speed
1  lesson9_part1.csv   Jan             1  Sunnyvale          8.3    18             221        21.0
2  lesson9_part1.csv   Feb             2  Sunnyvale         10.3    25             191         8.7
3  lesson9_part1.csv   Mar             3  Sunnyvale         11.0     9             224         6.7
(the first 3 rows of 24; table.show(rows=24) shows them all)
```

Good to know

- Looking never changes the table. Only steps that say what they change (`keep_rows`, `sort_by` ...) do, and lesson 10 shows them.
- In Jupyter, VS Code or Spyder, write table on the last line of a cell to see it as a nice table.
- `sdata.table` reads the same files as `sdata.chart`: `.xlsx`, `.csv`, `.tsv`, `.parquet` and `.json`. For one sheet of an Excel file: `sdata.table("results.xlsx", sheet="2026")`.
- The row numbers on the left start at 1, the way people count: the last row of a table of 54 rows is row 54. Lesson 10 uses these numbers to choose rows by their place.
- A folder works too: `sdata.table("data/")` reads every table file in it. A web address works like a file name: `sdata.table("https://example.org/data.csv")`.
- Files written in another way: `separator=";"`, `decimal_mark=","` (3,5 is three and a half), `thousands_mark="."`, `empty_values=["NA", "-"]`, `skip_rows=2` and `encoding="latin-1"`.

Try it yourself

1. Make a table of `sdata.example("weather")`. How many rows does it have?
2. Describe its columns. Which column has the biggest numbers?
3. Count each `"city"`. Does every city have the same number of rows?

Worked solutions: page 268, or type `sdata.lesson(9, solutions=True)`.

Words in this lesson: `sdata.table()`, `table.show_first_rows()`, `table.show_last_rows()`, `table.column_names()`, `table.describe_columns()`, `table.count_rows()`, `table.count_each()`, `table.show()`.

LESSON 10

Tables: choose rows and columns, and undo

In this lesson you will learn

- how to keep only the columns and rows you want
- how to sort the rows
- how to take a step back, and start again
- how to save your table

Example 1

A table is changed in small steps, like a chart. Each step says what it did.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_columns("name", "subject", "marks")
table.keep_rows("subject", equal_to="Maths")
table.keep_rows("marks", above=60)
table.sort_by("marks", biggest_first=True)
table.show_first_rows(10)
table.steps()
table.undo()
table.start_again()
print(table)
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. Keep only these three columns, in this order.
4. Keep only the rows where the subject is Maths. simplydata says how many rows it kept.
5. Of those, keep only marks above 60.
6. Put the rows in order, the biggest mark first.
7. Show the first 10 rows: the best Maths marks.
8. List every step you did, in order.
9. Take back the last step (the sorting).
10. Go back to the whole table, as it was at the start.
11. All 54 rows and 6 columns are back.

What it prints

```
Kept 18 of 54 rows.
Kept 11 of 18 rows.
      name subject  marks
1     Hana   Maths     91
2     Chen   Maths     81
3     Jade   Maths     81
4     Dina   Maths     74
```

```
5   Fatima   Maths     74
6      Ben   Maths     73
7     Noah   Maths     72
8     Kofi   Maths     68
9      Gus   Maths     67
10    Omar   Maths     62
1. keep columns name, subject, marks
2. keep rows where subject is "Maths"
3. keep rows where marks is above 60
4. sort by marks (biggest first)
Undone: sort by marks (biggest first)
simplydata table: 54 rows, 6 columns
    name class  subject  hours  marks  height
1   Asha     A     Maths    6.3     61     144
2   Asha     A  Science    3.8     66     144
3   Asha     A  English    7.6     77     144
4    Ben     A     Maths    6.4     73     152
5    Ben     A  Science    9.2     98     152
6    Ben     A  English    6.2     71     152
7   Chen     A     Maths    7.9     81     151
8   Chen     A  Science    3.5     59     151
9   Chen     A  English    2.5     46     151
10  Dina     A     Maths    6.5     74     155
```

Example 2

`keep_rows` and `remove_rows` know many conditions. Then save what you made.

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.keep_rows("month_number", between=[6, 8])
table.remove_rows("city", equal_to="Frostby")
table.remove_columns("wind_direction", "wind_speed")
table.rename_column("temperature", to="temp")
print(table)
table.save("lesson10_summer.xlsx")
```

What each line does

1. Load simplydata.
2. Make a table of the weather.
3. Keep the summer months: 6, 7 and 8. `between=` keeps both ends.
4. Remove the rows of the city Frostby.
5. Remove two columns we do not need.
6. Give the column `"temperature"` the shorter name `"temp"`.
7. Look at what is left: 6 rows and 5 columns.
8. Save the table as an Excel file. A name ending in `.csv` saves a CSV file.

What it prints

```
Kept 9 of 36 rows.
Removed 3 of 9 rows. 6 rows left.
simplydata table: 6 rows, 5 columns
  month  month_number        city  temp  rain
1   Jun             6   Sunnyvale  24.0    40
2   Jul             7   Sunnyvale  26.5    44
3   Aug             8   Sunnyvale  22.8    36
4   Jun             6  Rainbridge  15.3    72
```

```
5   Jul            7  Rainbridge  16.2    58
6   Aug            8  Rainbridge  15.3    76
```

Example 3

A condition can also be written out, with and, or, not and brackets. And `keep_biggest_rows` keeps the top rows of each group.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks > 60 and subject == 'Maths'")
table.undo()
table.remove_rows("hours < 3 or class in ['B', 'C']")
table.start_again()
table.keep_biggest_rows(2, by="marks", for_each="subject")
table.show()
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. Keep the rows where both are true: a mark above 60, in Maths. Text goes in quotes, and == means `"is equal to"`.
4. Back to all the rows.
5. Remove the rows where at least one is true: under 3 hours of study, or class B or C.
6. Back to the whole table.
7. Keep the 2 best marks in each subject: 6 rows.
8. Show the whole (small) table.

What it prints

```
Kept 11 of 54 rows.
Undone: keep rows where marks > 60 and subject == 'Maths'
Removed 38 of 54 rows. 16 rows left.
Kept the 2 rows with the biggest marks for each subject: 6 of 54 rows.
Note: 3 rows had the same marks as the last row kept and were left out; rows that tie are kept in table
    ↪ order.
     name class  subject  hours  marks  height
1  Fatima     A  English    6.8     81     158
2    Asha     A  English    7.6     77     144
3    Hana     B    Maths    6.1     91     144
4    Chen     A    Maths    7.9     81     151
5     Ben     A  Science    9.2     98     152
6  Fatima     A  Science    8.6     87     158
```

Example 4

Rows and columns can also be chosen by their place in the table. Positions count from 1, the numbers `table.show()` prints, and both ends are included.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.show_last_rows(3)
table.keep_rows_by_position(10, to=20)
table.keep_rows_by_position(1, step=2)
```

```
table.keep_columns_by_position(1, to=3)
table.show()
table.start_again()
print(table.cell_at(row=3, column="marks"))
table.reverse_rows()
table.show_first_rows(3)
```

What each line does

1 Load simplydata.
2 Make a table of the marks.
3 The last three rows are numbered 52, 53 and 54: rows count from 1.
4 Keep rows 10 to 20. Both ends are included, so that is 11 rows.
5 Of those, keep every 2nd row: the 1st, 3rd, 5th ... 6 rows.
6 Keep the first three columns. It says which columns they were.
7 6 rows and 3 columns, numbered again from 1.
8 Back to the whole table.
9 One cell: the mark in row 3. `cell_at` only looks; the table does not change.
10 Turn the table upside down: the last row comes first.
11 Rosa's marks are on top now.

What it prints

```
    name class  subject  hours  marks  height
52  Rosa     C    Maths    3.4     55     162
53  Rosa     C  Science    1.6     45     162
54  Rosa     C  English    7.9     71     162
Kept rows 10 to 20 (11 of 54 rows).
Kept every 2nd row from row 1 to row 11: rows 1, 3, 5, 7, 9, 11 (6 of 11 rows).
Kept columns 1 to 3: name, class, subject.
     name class  subject
1    Dina     A    Maths
2    Dina     A  English
3    Emil     A  Science
4  Fatima     A    Maths
5  Fatima     A  English
6     Gus     B  Science
77
Reversed the order of the 54 rows: the last row is now first.
   name class  subject  hours  marks  height
1  Rosa     C  English    7.9     71     162
2  Rosa     C  Science    1.6     45     162
3  Rosa     C    Maths    3.4     55     162
```

Good to know

- Conditions for `keep_rows` and `remove_rows`: above, below, `at_least`, `at_most`, `equal_to`, `not_equal_to`, between, `one_of`, contains, `starts_with`, `ends_with` and `is_empty`.
- `one_of=` takes a list: `table.keep_rows("subject", one_of=["Maths", "Science"])`.
- `undo()` takes back one step at a time. `start_again()` is a step too, so `undo()` can take it back.
- Your file is never changed: only `table.save(...)` writes a file.
- Written conditions use == != < <= > >=, and, or, not, brackets, in [...] and is empty. An empty cell never meets a condition, and not does not change that: `"not`

`(marks > 50)"` leaves out empty marks too. To find them, write `"marks is empty"`. A note says how many rows had an empty cell.

- Dates go in quotes as year-month-day: `table.keep_rows("date > '2026-03-01'")`, or `table.keep_rows("date", after="2026-03-01")`.
- `keep_rows` chooses rows by their values, `keep_rows_by_position` by their place. Positions count from 1 and both ends are included, so rows 10 to 20 are 11 rows. `to="last"` goes to the end, and a list such as [1, 5, 9] keeps those rows in that order.
- Coming from pandas or NumPy, which count from 0 and leave out the end: df.iloc[9:20] is `table.keep_rows_by_position(10, to=20)`, df.iloc[::2] is `keep_rows_by_position(1, step=2)`, df.iloc[[0, 4]] is `keep_rows_by_position([1, 5])`, df.iloc[:, 0:3] is `keep_columns_by_position(1, to=3)`, df.loc[:, "hours":"height"] is `keep_columns("hours", to="height")`, `df.select_dtypes("number")` is `keep_columns(kind="number")`, df.at[2, "marks"] is `cell_at(row=3, column="marks")` and df.iloc[::-1] is `reverse_rows()`. For numbers, a[2:8] is `nums.keep_numbers_by_position(3, to=8)`, and for a grid, g[1:5, 0:3] is `grid.keep_rows_by_position(2, to=5)`.keep_columns_by_position(1, `to=3`).

Try it yourself

1. In the marks table, keep only class `"B"` and sort by `"hours"`. Who studied the longest?
2. Undo the sorting, then sort by name instead.
3. Keep 5 random rows with `table.keep_random_rows(5, seed=1)` and save them as `"five.csv"`.
4. With one written condition, keep the rows with marks below 50 or hours above 8. How many rows are there?
5. From memory, without looking back: start again, keep rows 5 to 9, then remove the last row. How many rows are left, and whose name is in row 1 now?

Worked solutions: page 269, or type `sdata.lesson(10, solutions=True)`.

Words in this lesson: `table.keep_columns()`, `table.keep_rows()`, `table.sort_by()`, `table.steps()`, `table.undo()`, `table.start_again()`, `table.remove_rows()`, `table.remove_columns()`, `table.rename_column()`, `table.save()`, `table.keep_random_rows()`, `table.keep_biggest_rows()`, `table.keep_rows_by_position()`, `table.remove_rows_by_position()`, `table.keep_columns_by_position()`, `table.cell_at()`, `table.reverse_rows()`.

LESSON 11

Tables: clean the data and add new columns

In this lesson you will learn

- how to find and fill empty cells
- how to fix columns: numbers written as text, extra spaces, wrong values
- how to add new columns, in plain words or like a sum in maths
- what long and wide tables are

Example 1

Real data often has empty cells. The practice tables have none, so we make some first.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.replace_values("marks", [61, 73], with_value=None)
table.count_empty_cells()
table.fill_empty_cells("marks", with_the="average")
table.undo()
table.remove_rows_with_empty_cells("marks")
print(table)
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. Make every mark of 61 or 73 empty. `with_value=None` means an empty cell.
4. Show how many empty cells each column has.
5. Fill the empty marks with the average of the other marks.
6. Changed your mind? Take it back: the cells are empty again.
7. This time remove the rows with an empty cell in marks.
8. Some rows fewer than at the start.

What it prints

```
Replaced 2 cells in marks.
Empty cells in each column:
  name     0
  class    0
  subject  0
  hours    0
  marks    2
  height   0
Total: 2 cells empty.
Filled 2 cells in marks.
Undone: fill empty cells in marks with the average
Removed 2 of 54 rows. 52 rows left.
simplydata table: 52 rows, 6 columns
    name class  subject  hours  marks  height
```

```
1   Asha     A  Science    3.8   66.0     144
2   Asha     A  English    7.6   77.0     144
3    Ben     A  Science    9.2   98.0     152
4    Ben     A  English    6.2   71.0     152
5   Chen     A    Maths    7.9   81.0     151
6   Chen     A  Science    3.5   59.0     151
7   Chen     A  English    2.5   46.0     151
8   Dina     A    Maths    6.5   74.0     155
9   Dina     A  Science    4.5   79.0     155
10  Dina     A  English    2.3   57.0     155
```

Example 2

Data typed by people is often untidy. These words fix whole columns at once.

```
import simplydata as sdata
table = sdata.table({"name": ["  Asha", "ben ", "Chen"], "height": ["144", "152", "unknown"
    ↪ ]})
table.remove_extra_spaces("name")
table.replace_values("name", "ben", with_value="Ben")
table.make_numbers("height")
table.describe_columns()
```

What each line does

1. Load simplydata.
2. Make a small table by hand: each column name, then its values in a list.
3. Take away the spaces before and after the names.
4. Write `"ben"` with a capital letter.
5. Turn the heights from text into numbers. `"unknown"` is not a number, so that cell becomes empty, and simplydata tells you.
6. Now height is a column of numbers.

What it prints

```
Replaced 1 cell in name.
1 cell could not become numbers and is now empty, for example: "unknown"
height is now numbers.
The table has 3 rows and 2 columns.

column   kind  empty cells smallest biggest average different values
  name   text            0                                        3
height number            1     144     152     148
```

Example 3

A new column can be worked out from other columns, in plain words.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_column("marks_per_hour", by_dividing=["marks", "hours"])
table.add_group_column("grade", from_column="marks", cut_at=[40, 60, 80], names=["D", "C", "B
    ↪ ", "A"])
table.show_first_rows()
table.count_each("grade")
```

What each line does

1 Load simplydata.
2 Make a table of the marks.
3 A new column called `"marks_per_hour"`: the marks divided by the hours.
4 A grade for each mark: below 40 is D, 40 to 59 is C, 60 to 79 is B, 80 and more is A.
5 Look at the two new columns on the right.
6 How many of each grade?

What it prints

```
   name class  subject  hours  marks  height  marks_per_hour grade
1  Asha     A    Maths    6.3     61     144        9.682540     B
2  Asha     A  Science    3.8     66     144       17.368421     B
3  Asha     A  English    7.6     77     144       10.131579     B
4   Ben     A    Maths    6.4     73     152       11.406250     B
5   Ben     A  Science    9.2     98     152       10.652174     A
How many rows have each "grade" (4 different values): B 25, C 18, A 8, D 3.
```

Example 4

Or write the new column like a sum in maths. Here each subject first gets its own column: the table goes from long to wide.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_columns("name", "class", "subject", "marks")
table.turn_rows_into_columns(names_from="subject", values_from="marks")
table.add_column("total", "Maths + Science + English")
table.add_column("average", "total / 3")
table.add_rank_column("place", from_column="total")
table.sort_by("place")
table.show_first_rows()
```

What each line does

1 Load simplydata.
2 Make a table of the marks.
3 Keep the columns we need.
4 Make one column for each subject: Maths, Science and English. Each child now has one row.
5 A new column `"total"`: the three marks added up.
6 The average of the three marks.
7 Each child's place: 1 for the biggest total.
8 Put the rows in order of place.
9 Who came first?

What it prints

```
     name class  Maths  Science  English  total    average  place
1     Ben     A     73       98       71    242  80.666667      1
2  Fatima     A     74       87       81    242  80.666667      1
3    Hana     B     91       71       77    239  79.666667      3
4     Mia     C     59       87       76    222  74.000000      4
5    Dina     A     74       79       57    210  70.000000      5
```

Example 5

The shop table has real dates and some empty cells: days when the till broke. Look where they are, then fill them.

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.count_empty_cells()
look = table.chart()
look.empty_cells()
look.save("lesson11_empty_cells.png")
table.add_is_empty_column("sales_was_empty", from_column="sales")
table.fill_empty_cells("sales", with_the="line between neighbours", sorted_by="date",
    ↪ for_each="shop")
table.add_date_part_column("weekday", from_column="date", part="weekday name")
table.add_rolling_column("sales_7_days", from_column="sales", window=7, for_each="shop",
    ↪ sorted_by="date")
table.show(rows=8)
```

What each line does

1. Load simplydata.
2. Daily sales of two shops, January to March 2026.
3. 10 empty sales and 5 empty customer counts.
4. A chart of the table, to see where the empty cells are.
5. One strip per column and one line per row: empty cells are coloured, and each column says how many it has.
6. Save it.
7. Before filling: a True/False column that remembers which sales were empty.
8. Fill each gap on the straight line between the day before and the day after, in date order, for each shop on its own. A gap at the very start or end has a neighbour on one side only, so it stays empty, and a note says so.
9. A new column with the weekday of each date.
10. The average of each day and the 6 days before it: a 7-day moving average, which smooths out the busy weekends.
11. Look at the first 8 rows.

What it prints

```
Empty cells in each column:
  date       0
  shop       0
  sales      10
  customers  5
Total: 15 cells empty.
"sales_was_empty" is True in 10 rows (where sales is empty) and False in the other 170 rows.
Filled 8 cells in sales on the line between neighbours in order of date, for each shop on its own.
Note: 2 cells stayed empty: they are at the start or end of a group, with a filled neighbour on one side
    ↪ only.
Each "sales_7_days" is the average of sales in this row and the 6 rows before it (separately for each shop),
    ↪  in order of date.
        date       shop  sales  customers  sales_was_empty    weekday  sales_7_days
1 2026-01-01  Northgate  398.0       56.0            False   Thursday           NaN
2 2026-01-02  Northgate  387.0       51.0            False     Friday           NaN
3 2026-01-03  Northgate  605.0       88.0            False   Saturday           NaN
4 2026-01-04  Northgate  592.0       83.0            False     Sunday           NaN
5 2026-01-05  Northgate  407.0       59.0            False     Monday           NaN
```

```
6 2026-01-06  Northgate  462.0       64.0            False    Tuesday          NaN
7 2026-01-07  Northgate  425.0       61.0            False  Wednesday   468.000000
8 2026-01-08  Northgate  418.0       62.0            False   Thursday   470.857143
(the first 8 rows of 180; table.show(rows=180) shows them all)
```

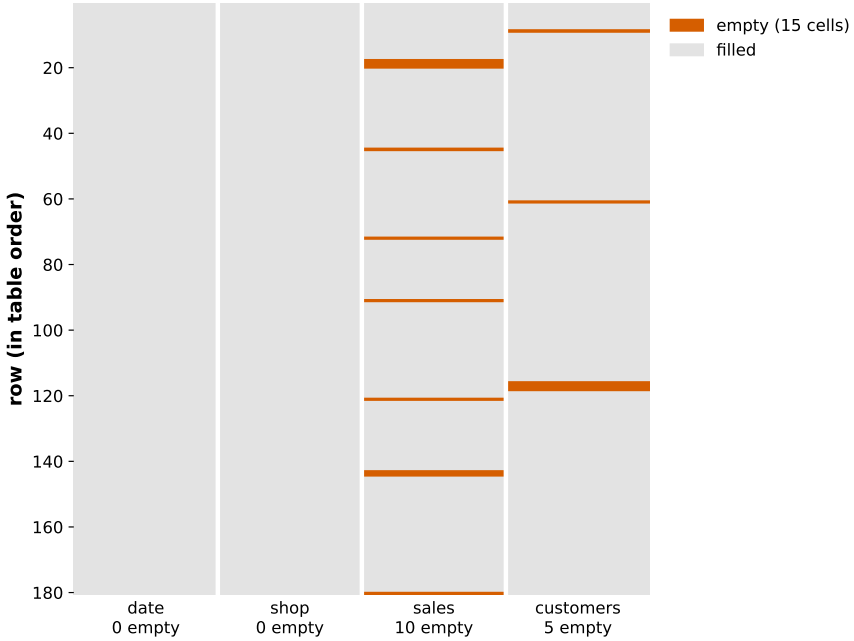


Figure 11.1: Lesson 11, example 5: `lesson11_empty_cells.png`.

Example 6

Names and dates typed in by people often need taking apart.

```
import simplydata as sdata
table = sdata.table({"name": ["Asha Rao", "Ben Lee", "Chen Wu"], "born": ["04/30/2014", "
    ↪ 12/25/2013", "07/03/2014"], "height": [144.37, 152.81, 139.5]})
table.split_column("name", separator=" ", into=["first_name", "family_name"])
table.add_text_part_column("initial", from_column="first_name", first_characters=1)
table.replace_text("family_name", "Lee", with_value="Li")
table.make_dates("born", order="month day year")
table.add_date_part_column("month_born", from_column="born", part="month name")
table.change_column("height", "height / 100")
table.round_to("height", 2)
table.show()
```

What each line does

1. Load simplydata.
2. A small table typed by hand. The birthdays are written month first, as in the USA.
3. Split each name at the space into two new columns.
4. The first letter of each first name.
5. Replace a piece of text inside the cells.
6. Turn the birthdays into dates. `order=` says how they are written, so 07/03/2014 is read as 3 July and not as 7 March.
7. The month of each birthday, as a word.

8 Change the heights from centimetres to metres, in the same column.
9 Round them to 2 decimal places.
10 Look at the whole table.

What it prints

```
Split "name" at " " into first_name, family_name. The column "name" is replaced by them.
Replaced "Lee" in 1 cell of family_name.
born is now dates (read as month day year).
Rounded height to 2 decimal places.
  first_name family_name       born  height initial month_born
1       Asha         Rao 2014-04-30    1.44       A      April
2        Ben          Li 2013-12-25    1.53       B   December
3       Chen          Wu 2014-07-03    1.40       C       July
```

Good to know

- Formulas know more maths than + - * /: sqrt, log, exp, sin, cos, tan, pi, and special functions such as erf, gamma, lgamma, beta and besselJ: `table.add_column("z", formula="erf(marks / 100)")`.
- Long or wide? A long table has one row for each measurement: the marks table has a row for each child in each subject. A wide table puts the measurements of one child side by side, one column per subject. Charts and tests like long tables (also called tidy data); sums across columns, like the total above, need wide ones.
- Filling empty cells is called imputation. Filling with the average makes the values look less spread out than they really are, so say in your report how you filled them. `with_the="median"`, `"value above"` or `"value below"` and `with_value=0` work too.
- `remove_rows_with_empty_cells()` with nothing in the brackets removes every row that has an empty cell in any column.
- More fixing words: `make_text`, `make_dates`, `make_lowercase` and `make_uppercase`.
- Plain words for new columns: `by_adding=`, `by_subtracting=`, `by_multiplying=` and `by_dividing=`, or `from_column=` with `plus=`, `minus=`, `times=` or `divide_by=`.
- In a maths-style column, a column name with spaces goes in backticks: `"‘max temp‘ - 2"`.
- `with_the="line between neighbours"` is linear interpolation. It suits values that change smoothly, like sales from day to day; it cannot know about a sudden jump. Keep a True/False column of what you filled, and say in your report how you filled it.
- Charts say it too: when a chart leaves out rows because of empty cells, one line tells you how many.
- Row after row: `add_previous_value_column` gives the value of the row before, and `add_change_column` the change since then (`in_percent=True` for a percent).

Try it yourself

1. In the first example, fill the empty marks with the median instead of the average.
2. Add a column `"height_m"` with the height in metres: `from_column="height"`, `divide_by=100`.
3. In the last example, add a column `"best_two"` with the formula `"total - English"`.
4. In the shop example, add a column `"change"` with the change in sales from the day before, `for_each="shop"` and `sorted_by="date"` (`add_change_column`).

> Worked solutions: page 271, or type `sdata.lesson(11, solutions=True)`.

Words in this lesson: `table.replace_values()`, `table.count_empty_cells()`, `table.fill_empty_cells()`, `table.remove_rows_with_empty_cells()`, `table.remove_extra_spaces()`, `table.make_numbers()`, `table.add_column()`, `table.add_group_column()`, `table.add_rank_column()`, `table.turn_rows_into_columns()`, `table.add_is_empty_column()`, `chart.empty_cells()`, `table.add_date_part_column()`, `table.add_rolling_column()`, `table.split_column()`, `table.add_text_part_column()`, `table.replace_text()`, `table.make_dates()`, `table.change_column()`, `table.round_to()`, `table.add_change_column()`.

LESSON 12

Tables: summaries, and from a table to a chart

In this lesson you will learn

- how to work out averages, totals and counts, for each group
- how to tell if two columns go together, and why that is not the same as a cause
- how to draw a table, and which rows a chart uses
- how to join two tables and turn columns into rows

Example 1

A summary turns many rows into a few numbers. It never changes your table, and it says its answer in a sentence.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.average_of("marks")
table.average_of("marks", for_each="class")
table.biggest_of("marks", for_each="subject")
table.correlation_between("hours", "marks")
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. The average of all the marks: just one number.
4. One average for each class. `for_each=` also gives back a new, small table.
5. The best mark in each subject.
6. Do more hours go with more marks? The answer is a number r from -1 to 1 (Pearson's correlation), explained in words.

What it prints

```
The average of "marks" is 64.91 (54 values).
The average of "marks" for each "class": A 71.83, B 64.28, C 58.61.
The biggest of "marks" for each "subject": English 81, Maths 91, Science 98.
The correlation between "hours" and "marks" is r = 0.83 (Pearson, 54 rows): very strong, going up, and very
    ↪ unlikely to be chance (p < 0.0001).
```

Example 2

A summary for each group is a table too, so you can sort it and draw it.

```
import simplydata as sdata
table = sdata.table(sdata.example("fruit"))
totals = table.total_of("count", for_each="fruit")
totals.sort_by("total of count", biggest_first=True)
```

```
chart = totals.chart()
chart.bars(x="fruit", y="total of count")
chart.numbers_on_bars()
chart.save("lesson12_fruit.png")
```

What each line does

1. Load simplydata.
2. Make a table of fruit sales.
3. A new table called totals: one row per fruit, with a column `"total of count"`. table itself does not change.
4. Best seller first.
5. Start a chart from the summary table.
6. One bar per fruit.
7. Write the total on each bar.
8. Save the chart.

What it prints

```
The total of "count" for each "fruit": apple 337, banana 205, cherry 59, grape 117, orange 73, pear 20.
```

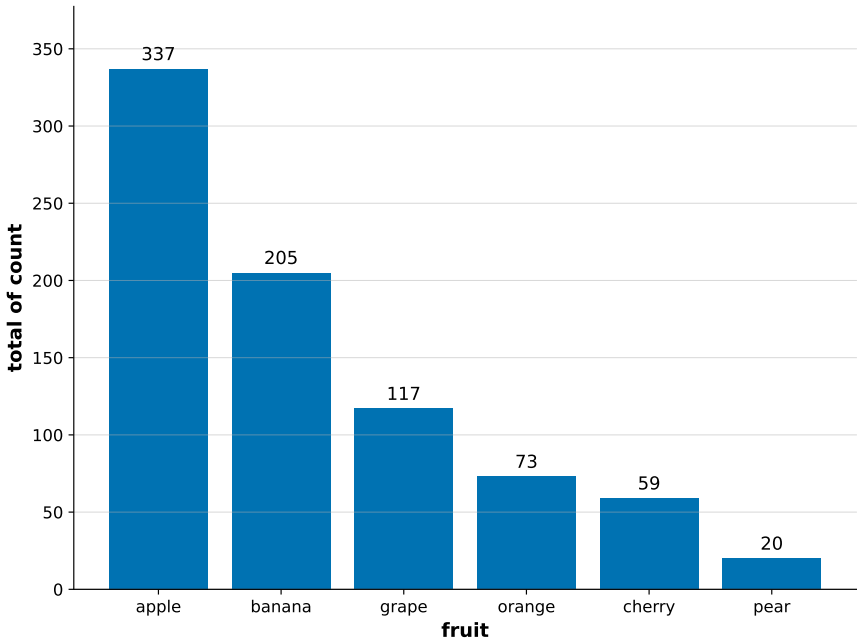


Figure 12.1: Lesson 12, example 2: `lesson12_fruit.png`.

Example 3

A chart uses the table as it was when you made the chart. Table steps after it are for the next chart.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("class", equal_to="A")
all_tests = table.chart()
all_tests.dots(x="hours", y="marks")
table.keep_rows("subject", equal_to="Maths")
maths = table.chart()
```

```
maths.dots(x="hours", y="marks")
page = sdata.side_by_side(all_tests, maths, same_y=True)
page.save("lesson12_maths.png")
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. Keep class A.
4. A chart of the table as it is now: every test of class A. `sdata.chart(table)` does the same.
5. Draw hours against marks.
6. A table step after the chart. The chart `all_tests` does not change.
7. A second chart, of the table as it is now: only the Maths tests of class A.
8. Draw the same thing again.
9. Both charts on one page, with the same y-axis, so you can compare them.
10. Save the page.

What it prints

```
Kept 18 of 54 rows.
Kept 6 of 18 rows.
```

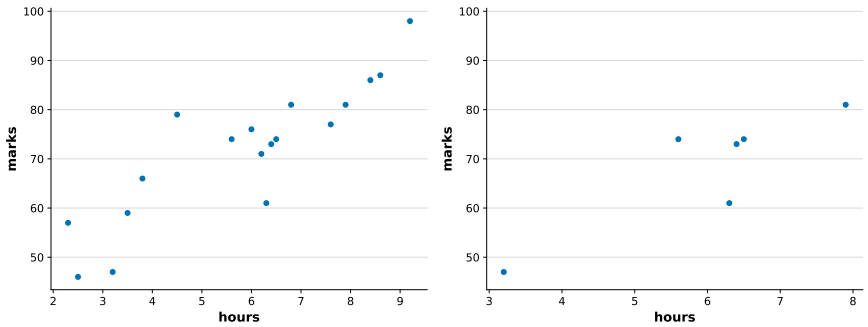


Figure 12.2: Lesson 12, example 3: `lesson12_maths.png`.

Example 4

Two tables can be joined: their rows are matched on a column they share.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
ages = {"name": ["Asha", "Ben", "Chen"], "age": [12, 13, 12]}
table.join_with(ages, using="name")
table.show_first_rows()
table.average_of("marks", for_each="age")
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. A small table of ages, typed by hand: each column name, then its values.
4. Bring in the age, matching the rows on the name. Only children found in both tables stay.
5. Look: there is a new column, age.

6 The average mark at each age.

What it prints

```
9 of 54 rows in this table found a match in the other table.
   name class  subject  hours  marks  height  age
1  Asha     A    Maths    6.3     61     144   12
2  Asha     A  Science    3.8     66     144   12
3  Asha     A  English    7.6     77     144   12
4   Ben     A    Maths    6.4     73     152   13
5   Ben     A  Science    9.2     98     152   13
The average of "marks" for each "age": 12 65, 13 80.67.
```

Example 5

Turning columns into rows makes a wide table long: the shape most charts like.

```
import simplydata as sdata
table = sdata.table({"name": ["Asha", "Ben", "Chen"], "test_1": [61, 73, 81], "test_2": [70,
    ↪ 68, 90]})
table.turn_columns_into_rows(["test_1", "test_2"], names_to="test", values_to="mark")
table.show_first_rows(6)
chart = table.chart()
chart.bars(x="name", y="mark", colour="test")
chart.save("lesson12_tests.png")
```

What each line does

1 Load simplydata.
2 A small, wide table: two tests, one column for each.
3 Put the two tests under each other. The new column `"test"` says which test, and `"mark"` holds the mark. Each child now has two rows.
4 Look at the new, long shape.
5 Start a chart from it.
6 Both tests side by side for each child.
7 Save it.

What it prints

```
    name    test  mark
1  Asha  test_1    61
2   Ben  test_1    73
3  Chen  test_1    81
4  Asha  test_2    70
5   Ben  test_2    68
6  Chen  test_2    90
```

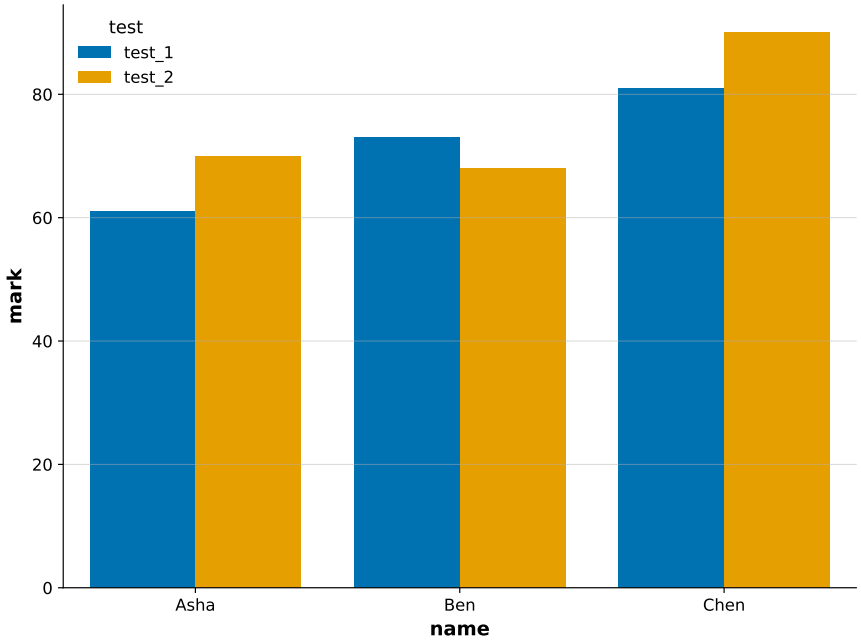


Figure 12.3: Lesson 12, example 5: `lesson12_tests.png`.

Example 6

`summary_of` gives several measures at once, and `every=` sums up dates by week, month or year.

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.summary_of("sales", for_each="shop", show=["average", "median", "standard deviation", "
    ↪ count", "empty cells"])
table.total_of("sales", for_each="date", every="month")
table.percent_of("sales", above=500, for_each="shop")
```

What each line does

1. Load simplydata.
2. Daily sales of two shops.
3. Five measures of each shop's sales in one tidy table. count counts the days with a number; empty cells counts the others.
4. The total sales of each month: `every=` puts the days of one month together.
5. On what percent of the days did each shop take more than 500? Each percent comes with its 95 % confidence interval.

What it prints

```
Note: 10 empty cells in "sales" were skipped.
Summary of "sales" for each "shop":
  Northgate: average 531.46, median 506, standard deviation 113.27, count 85, empty cells 5
  Southside: average 373.59, median 353, standard deviation 83.33, count 85, empty cells 5
Note: 10 empty cells in "sales" were skipped.
The total of "sales" for each month of "date": 2026-01 24013, 2026-02 23527, 2026-03 29389.
Note: in the new table each month is shown by its first day.
Note: 10 rows with an empty cell in "sales" were left out of the percent.
Percent of rows where "sales" is above 500, for each "shop": Northgate 52.9 % (45 of 85), Southside 9.4 % (8
    ↪  of 85). (The new table has exact 95 % confidence intervals.)
```

Example 7

A cross table puts one group down the side and another across the top. A correlation table compares many columns at once.

```
import simplydata as sdata
fruit = sdata.table(sdata.example("fruit"))
fruit.count_rows(for_each="shop", across="fruit")
fruit.count_different_values("fruit", for_each="shop")
marks = sdata.table(sdata.example("marks"))
marks.average_of("marks", for_each="class", across="subject")
marks.correlation_between("hours", "marks", "height")
```

What each line does

1. Load simplydata.
2. A table of fruit sales.
3. How many sales of each fruit in each shop: one row per shop, one column per fruit.
4. How many different fruits each shop sold.
5. A table of the marks.
6. The average mark of each class in each subject.
7. Every pair of the three columns at once: a correlation table, with the strongest pairs in words.

What it prints

```
Rows for each "shop" (rows) and each "fruit" (columns):
  Corner shop: apple 5, banana 3, cherry 2, grape 1, orange 2, pear 1
  Market stall: apple 4, banana 5, cherry 2, grape 4, orange 2, pear 2
Different values of "fruit" for each "shop": Corner shop 6, Market stall 6.
The average of "marks" for each "class" (rows) and each "subject" (columns): a new table with 3 rows and 3
    ↪ columns of numbers.
  A: English 68, Maths 68.33, Science 79.17
  B: English 59.67, Maths 65.67, Science 67.5
  C: English 59.17, Maths 54.33, Science 62.33
Correlations between 3 columns (Pearson, 3 pairs; each pair uses the rows where both columns have a number).
    ↪  The strongest:
  "hours" and "marks": r = 0.83 (54 rows), very strong, going up, and very unlikely to be chance (p <
    ↪ 0.0001).
  "hours" and "height": r = 0.06 (54 rows), very weak, going up, and could easily be chance (p = 0.667).
  "marks" and "height": r = 0.03 (54 rows), very weak, going up, and could easily be chance (p = 0.826).
Note: these p-values are not adjusted for testing 3 pairs. To adjust them: results = table.
    ↪ correlation_between("hours", "marks", "height", show="each pair"), then results.
    ↪ add_adjusted_p_value_column(from_column="p-value").
```

Example 8

More ways to sum up: percentiles, the most common value, and measures of shape.

```
import simplydata as sdata
marks = sdata.table(sdata.example("marks"))
marks.percentile_of("marks", 90, for_each="class")
marks.summary_of("marks", show=["median", "skewness", "coefficient of variation"], for_each="
    ↪ class")
school = sdata.table(sdata.example("school"))
school.most_common_of("days_absent")
```

What each line does

1. Load simplydata.
2. A table of the marks.
3. The 90th percentile: 90 % of the marks are below it (86.3 in class A).
4. Skewness says if the values lean to one side (below 0: a longer tail of low marks); the coefficient of variation is the standard deviation divided by the average.
5. A table of pupils.
6. The most common number of days absent (the mode): 2.

What it prints

```
The 90th percentile of "marks" for each "class": A 86.3, B 78.2, C 73.2.
Summary of "marks" for each "class":
  A: median 74, skewness -0.33, coefficient of variation 0.19
  B: median 68.5, skewness -0.59, coefficient of variation 0.23
  C: median 57, skewness 0.06, coefficient of variation 0.24
The most common value of "days_absent" is "2": 44 of 180 filled cells (24.4 %).
```

Good to know

- Correlation is not cause. Children who study more might also sleep more, or like the subject more; r only says that two columns go up and down together, not which one makes the other change.
- More summaries: `median_of`, `total_of`, `smallest_of`, `biggest_of`, `standard_deviation_of` and `count_rows`. Each one can take `for_each=`.
- A chart never changes its table, and a table step never changes a chart you already made. So one cleaned table can feed many charts.
- `table.add_rows_from(other)` puts the rows of another table under this one. `turn_rows_into_columns` does the opposite of `turn_columns_into_rows`.
- Lesson 13 teaches numbers, and lesson 14 the statistical tests: is a difference real, or could it be chance?
- Every summary skips empty cells and says how many it skipped. `summary_of` can also show `"variance"`, `"standard error"`, `"confidence interval"`, `"range"` and `"interquartile range"`.

Try it yourself

1. Make the fruit totals `for_each="shop"` instead. Which shop sold more?
2. Work out the average temperature `for_each="city"` in the weather table and draw it as bars.

3. Is there a link between height and marks? Use `correlation_between` to find out.
4. Make a cross table of the average mark `for_each="subject" across="class"`. Which class is best in Science?

Worked solutions: page 273, or type `sdata.lesson(12, solutions=True)`.

Words in this lesson: `table.average_of()`, `table.biggest_of()`, `table.correlation_between()`, `table.total_of()`, `table.chart()`, `sdata.side_by_side()`, `table.join_with()`, `table.turn_columns_into_rows()`, `table.add_rows_from()`, `table.summary_of()`, `table.percent_of()`, `table.count_rows()`, `table.count_different_values()`, `table.percentile_of()`, `table.most_common_of()`.

LESSON 13

Numbers: lists and grids of numbers

In this lesson you will learn

- how to make a list of numbers, and a grid of them
- two ways to do maths: symbols make new numbers, words change the numbers you have
- how to keep some numbers, sum them up, and draw them

Example 1

Numbers are for when you have just numbers, not a table with named columns.

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
print(nums)
bigger = nums * 2
print(bigger)
nums.add(1)
nums.undo()
nums.keep_numbers(above=10)
nums.average()
nums.standard_deviation()
nums.steps()
```

What each line does

1 Load simplydata.
2 Make a list of five numbers and call it nums.
3 Look at them. The first line says how many there are.
4 Maths with symbols makes NEW numbers: every number times 2. nums itself does not change.
5 24, 30, 18, 40 and 28.
6 Maths with words CHANGES the numbers you have: add 1 to each one.
7 Like a table, numbers can take back every word.
8 Keep only the numbers above 10. The same conditions as `keep_rows`: above, below, between ...
9 The average of what is left, said in a sentence.
10 How spread out they are (divide by n - 1, as in pandas and R).
11 List the steps you did.

What it prints

```
simplydata numbers: 5 numbers
[12, 15,  9, 20, 14]
simplydata numbers: 5 numbers
[24, 30, 18, 40, 28]
Undone: add 1
Kept 4 of 5 numbers.
The average of the numbers is 15.25 (4 numbers).
The standard deviation of the numbers is 3.4 (4 numbers).
1. keep numbers above 10
```

Example 2

Counting, random numbers and a chart.

```
import simplydata as sdata
months = sdata.numbers(1, to=12)
temps = sdata.numbers([4, 6, 9, 13, 17, 20, 22, 21, 17, 12, 8, 5])
chart = sdata.chart()
chart.lines(x=months, y=temps, dots=True)
chart.save("lesson13_line.png")
heights = sdata.random_numbers(how_many=200, average=150, standard_deviation=8, seed=1)
heights.median()
heights.percentile(90)
chart = heights.chart()
chart.histogram(x="value", bins=12)
chart.save("lesson13_heights.png")
```

What each line does

1. Load simplydata.
2. Count from 1 to 12. The end is included.
3. A temperature for each month.
4. Start an empty chart: the numbers go straight into the chart type.
5. Draw the temperatures as a line, month by month.
6. Save it.
7. 200 made-up heights that bunch around 150 in a bell shape (a normal distribution). `seed=1` gives the same numbers every time, so others can repeat your work.
8. The middle height.
9. The height that 90 % of them are below.
10. Start a chart from the numbers. Their column is called value.
11. Count how many heights fall in each range.
12. Save it. Can you see the bell shape?

What it prints

```
The median of the numbers is 150.02 (200 numbers).
The 90th percentile of the numbers is 157.2 (200 numbers).
```

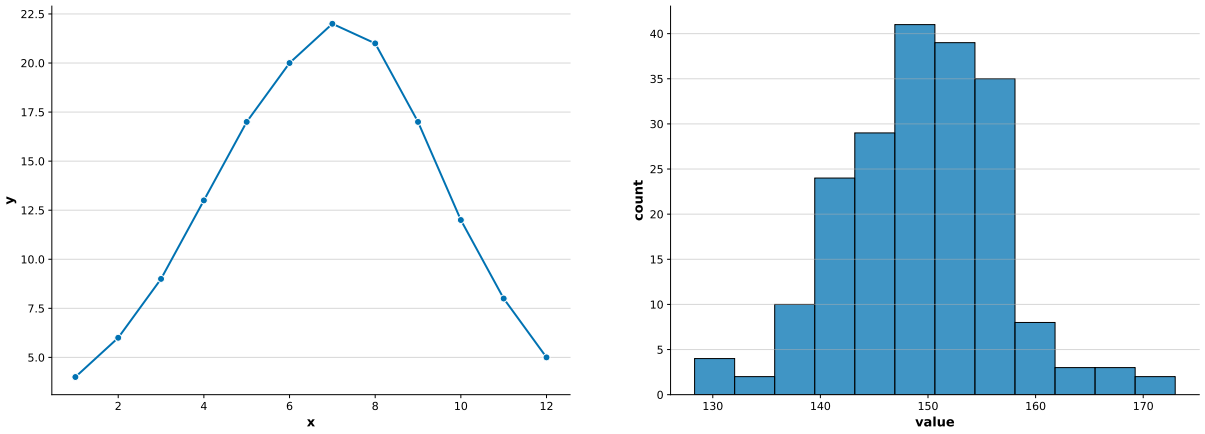


Figure 13.1: Lesson 13, example 2: `lesson13_line.png` and `lesson13_heights.png`.

Example 3

A grid holds numbers in rows and columns. Positions count from 1.

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
print(grid.number_at(row=1, column=2))
grid.total(for_each="row")
grid.average(for_each="column")
grid.swap_rows_and_columns()
print(grid)
chart = grid.chart()
chart.heatmap(numbers=True)
chart.save("lesson13_grid.png")
```

What each line does

1. Load simplydata.
2. A grid with 2 rows and 3 columns: one list for each row.
3. The number in row 1, column 2: 5.
4. The total of each row: 10 and 18.
5. The average of each column.
6. Rows become columns (in maths: the transpose).
7. Now 3 rows and 2 columns.
8. Start a chart from the grid.
9. One coloured square for each number, with the number written in.
10. Save it.

What it prints

```
5
The total of each row: 10, 18.
The average of each column: 5.5, 5.5, 3.
simplydata number grid: 3 rows, 2 columns
[[3, 8],
```

```
[5, 6],
[2, 4]]
```

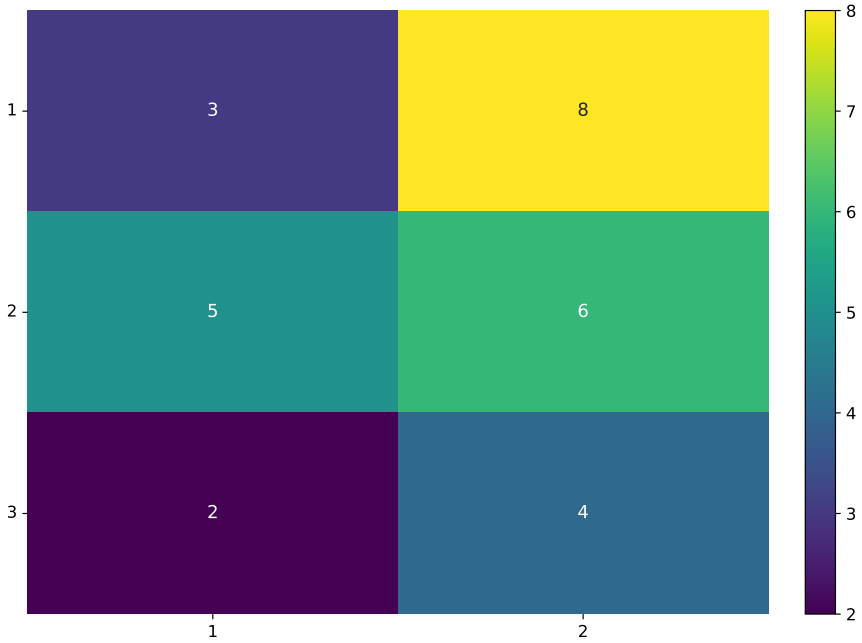


Figure 13.2: Lesson 13, example 3: `lesson13_grid.png`.

Good to know

- The words follow the table words, with numbers in place of rows: `keep_first_numbers`, `remove_empty_numbers`, `fill_empty_numbers`, `show_first_numbers`.
- Summaries have no _of, because there is no column to name: `nums.average()`, not `table.average_of("marks")`.
- A sum with no answer, such as dividing by 0 or the square root of a negative number, gives an empty value and a note, never a crash.
- `nums.table()` turns numbers into a table, and `nums.as_numpy()` gives a plain NumPy array if another Python tool needs one.

Try it yourself

1. Make the numbers 1 to 10, then square them in two ways: with ** 2 and with `power(2)`.
2. Make 6 dice throws with `sdata.random_numbers(how_many=6, between=[1, 6], whole_numbers=True, seed=1)`. What is their total?
3. Make a grid of 3 rows and 4 columns filled with 0: `sdata.number_grid(rows=3, columns=4, filled_with=0)`.

Worked solutions: page 274, or type `sdata.lesson(13, solutions=True)`.

Words in this lesson: `sdata.numbers()`, `sdata.random_numbers()`, `sdata.number_grid()`, `nums.add()`, `nums.undo()`, `nums.keep_numbers()`, `nums.average()`, `nums.standard_deviation()`, `nums.median()`, `nums.percentile()`, `nums.chart()`, `grid.number_at()`, `grid.swap_rows_and_columns()`, `nums.steps()`.

LESSON 14

Statistical tests: real difference, or chance?

In this lesson you will learn

- what a statistical test asks, and what a p-value means
- how to compare the averages of groups, and which groups differ
- how to compare counts, and how to fit a line or a formula

Example 1

Groups almost never have exactly the same average. A test asks: is the difference bigger than chance alone would often give?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.compare_averages("marks", for_each="class")
table.compare_each_pair("marks", for_each="class")
table.compare_averages("marks", for_each="subject", based_on="ranks")
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. Do the three classes really differ in their average mark? The answer comes in words. The p-value is the chance of a difference this big if the classes were really the same: below 0.05 we call it unlikely to be chance. With three or more groups this test is also called Welch's ANOVA; with two groups, a t-test.
4. Which classes differ? Each pair gets its own t-test (Welch's). Testing many pairs makes a false alarm more likely, so the p-values are adjusted (Holm's method).
5. `based_on="ranks"` puts the marks in order and compares the places instead (a Kruskal-Wallis test, or Mann-Whitney for two groups). It is safer when groups are small and the values are not bell-shaped.

What it prints

```
The average "marks" is 71.8 in A, 64.3 in B and 58.6 in C. Differences this big between the groups are
    ↪ unlikely to be chance (p = 0.028). Omega-squared = 0.0967: the groups explain about 10 % of the
    ↪ differences in "marks".
Test: Welch's one-way ANOVA (F = 3.99, df = 2 and 34.0), the usual test for three or more averages; it does
    ↪ not assume the groups have the same spread. To see which groups differ: table.compare_each_pair("
    ↪ marks", for_each="class").
Each pair of "class" groups compared on "marks" with Welch's t-test; the p-values are adjusted for 3 pairs
    ↪ with Holm's method, so the chance of any false alarm stays at 5 %.
  A vs B: the average of A is 7.56 higher; could easily be chance (Holm-adjusted p = 0.245).
  A vs C: the average of A is 13.2 higher; unlikely to be chance (Holm-adjusted p = 0.023).
  B vs C: the average of B is 5.67 higher; could easily be chance (Holm-adjusted p = 0.250).
The median "marks" is 62.0 in English, 64.5 in Maths and 70.0 in Science. Differences this big between the
    ↪ groups could easily be chance (p = 0.398); this does not show that there is no difference. Epsilon-
    ↪ squared = 0.0348: the groups explain about 3 % of the differences in the ranks.
```

```
Test: Kruskal-Wallis test (H = 1.84, df = 2), which compares ranks rather than averages, so it does not need
    ↪  bell-shaped values. To see which groups differ: table.compare_each_pair("marks", for_each="subject
    ↪ ", based_on="ranks").
```

Example 2

Counts can be compared too: does how often something happens differ between groups?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_group_column("result", from_column="marks", cut_at=[60], names=["below 60", "60 or
    ↪ more"])
table.compare_counts("result", for_each="class")
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. A new column that says if each mark is below 60 or not.
4. Do the classes differ in how many marks are 60 or more? This counts the rows for each class and result and uses the chi-square test. Cramer's V says how strong the link is, from 0 (none) to 1.

What it prints

```
Comparing how often each "result" turns up in each "class" (2 x 3 table, 54 rows): the differences could
    ↪ easily be chance (p = 0.122); this does not show that there is no difference. Cramer's V = 0.279 (0
    ↪  means no link, 1 a complete link).
Test: chi-square test of independence (chi-square = 4.21, df = 2), the usual test for counts.
```

Example 3

A fit finds the line or formula that follows the data best, and says how sure it is.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.fit_line(x="hours", y="marks")
growth = sdata.table(sdata.example("growth"))
growth.keep_rows("country", equal_to="Applia")
growth.fit_formula("population = a * exp(b * (year - 2000))")
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. How much do the marks go up for each extra hour of study? This is linear regression by least squares. R squared says how much of the differences in marks the line explains.
4. A second table, with the growth data.
5. Keep one country.
6. Fit a formula you write yourself. Names that are columns (population, year) are data; the others (a and b) are found by simplydata. Here b is the growth rate each year. This is nonlinear least squares.

What it prints

```
"marks" goes up by 5.20 for each 1 more "hours" (95 % CI 4.24 to 6.17); this slope is very unlikely to be
    ↪ chance (p < 0.0001). R squared = 0.69: "hours" explains about 69 % of the differences in "marks".
Line: marks = 5.20 * hours + 37.2, by least squares over 54 rows. A link does not show that one thing causes
    ↪  the other.
Kept 13 of 52 rows.
Fitted "population = a * exp(b * (year - 2000))" to 13 rows: a = 12.0 (95 % CI 11.8 to 12.2), b = 0.0297 (95
    ↪  % CI 0.0289 to 0.0306). Residual standard error 0.179 on 11 degrees of freedom; R squared = 1.00 (
    ↪ only a rough guide for a curve).
Values found: a, b. Data: year. Method: nonlinear least squares (Levenberg-Marquardt), as R nls(); intervals
    ↪  are estimate +/- t x standard error (R confint() on nls profiles the likelihood and can differ a
    ↪ little).
```

Example 4

Before a test, look at the shape of the values and at values far from the rest. After many tests, adjust the p-values.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.compare_shape_to("marks", "normal", for_each="class")
table.show_rows_with_outliers("marks", for_each="class")
pairs = table.correlation_between("hours", "marks", "height", show="each pair")
pairs.add_adjusted_p_value_column(method="holm")
pairs.show()
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. Do the marks of each class look bell-shaped (normal)? The Shapiro-Wilk test answers for each class. Tests on averages work best with bell-shaped values, or with many values.
4. The rows whose mark is far from the rest of its class, by Tukey's rule (the one box plots use): here one mark of 33 in class B. The table does not change.
5. Test three pairs of columns at once: a table with one row per pair and its p-value.
6. With many tests, one of them looks real by chance more often. Holm's adjustment keeps the chance of even one false alarm at 5 %.
7. The p-values and the adjusted ones, side by side.

What it prints

```
Do the values of "marks" look bell-shaped (normal) in each "class"? Shapiro-Wilk test (as R shapiro.test());
    ↪  a small p-value means too far from normal to be chance:
  A: could well be bell-shaped (W = 0.965, p = 0.700, 18 values)
  B: could well be bell-shaped (W = 0.954, p = 0.491, 18 values)
  C: could well be bell-shaped (W = 0.985, p = 0.985, 18 values)
A large p-value does not prove that the values are bell-shaped.
Outliers in "marks" by Tukey's rule: values more than 1.5 x IQR below the first quartile (Q1) or above the
    ↪ third quartile (Q3).
IQR, the interquartile range, is Q3 - Q1: the spread of the middle half of the values.
For each class:
  A: below 34.88 or above 107.88 (Q1 = 62.25, Q3 = 80.5); 0 rows found
  B: below 33.25 or above 97.25 (Q1 = 57.25, Q3 = 73.25); 1 row found
  C: below 14.88 or above 103.88 (Q1 = 48.25, Q3 = 70.5); 0 rows found
Found 1 row with an outlier, of 54 rows.
    name class subject  hours  marks  height
25  Ivan     B   Maths    0.5     33     147
```

```
The numbers on the left are the rows' places in the whole table, so table.remove_rows_by_position(25) would
    ↪ remove them.
Correlations between 3 columns (Pearson, 3 pairs; each pair uses the rows where both columns have a number).
    ↪  The strongest:
  "hours" and "marks": r = 0.83 (54 rows), very strong, going up, and very unlikely to be chance (p <
    ↪ 0.0001).
  "hours" and "height": r = 0.06 (54 rows), very weak, going up, and could easily be chance (p = 0.667).
  "marks" and "height": r = 0.03 (54 rows), very weak, going up, and could easily be chance (p = 0.826).
Note: these p-values are not adjusted for testing 3 pairs. To adjust them: results = table.
    ↪ correlation_between("hours", "marks", "height", show="each pair"), then results.
    ↪ add_adjusted_p_value_column(from_column="p-value").
Added "p-value (Holm)": 3 p-values from "p-value" adjusted with Holm's method (as R p.adjust(method = "holm
    ↪ ")): the chance of even one false alarm among all the tests stays at 5 %. Below 0.05: 1 of 3 after
    ↪ adjusting (1 before).
  column 1 column 2   method         r       p-value  rows               strength  p-value (Holm)
1    hours    marks  Pearson  0.831290  7.176542e-15    54  very strong, going up    2.152963e-14
2    hours   height  Pearson  0.059806  6.674978e-01    54    very weak, going up    1.000000e+00
3    marks   height  Pearson  0.030712  8.255157e-01    54    very weak, going up    1.000000e+00
```

Example 5

Some questions are about spread, or about counts compared with what you expect.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.compare_spreads("marks", for_each="class")
table.compare_each_pair("marks", for_each="class", test="tukey")
school = sdata.table(sdata.example("school"))
school.compare_counts_to("breakfast", expected={"yes": 0.6, "no": 0.4})
school.compare_percent_to("passed", value="yes", percent=50)
```

What each line does

1. Load simplydata.
2. Make a table of the marks.
3. Are the marks of one class more spread out? Levene's test says this could easily be chance (p = 0.91).
4. Tukey's test compares every pair at once: only A and C differ (p = 0.020).
5. A table of pupils.
6. Did 60 % eat breakfast, as a survey said? 115 of 180 is close enough (p = 0.29).
7. Did half the pupils pass? 43.3 % did; that could easily be chance (p = 0.086).

What it prints

```
The standard deviation of "marks" is 13.7 in A, 14.9 in B and 14.2 in C. Differences in spread this big
    ↪ between the groups could easily be chance (p = 0.911); this does not show that there is no
    ↪ difference.
Test: Levene's test centred on the medians, the Brown-Forsythe form (F = 0.0937, df = 2 and 51), as R car::
    ↪ leveneTest() and SciPy levene(center="median"); it copes well with values that are not bell-shaped.
Each pair of "class" groups compared on "marks" with Tukey's honest significant difference, as R TukeyHSD(
    ↪ aov(marks ~ class)): the p-values and 95 % confidence intervals hold for all 3 pairs at once, so
    ↪ the chance of any false alarm stays at 5 %. It assumes the groups have the same spread.
  A vs B: the average of A is 7.56 higher (95 % CI -3.93 to 19.0); could easily be chance (Tukey-adjusted p
    ↪ = 0.260).
  A vs C: the average of A is 13.2 higher (95 % CI 1.74 to 24.7); unlikely to be chance (Tukey-adjusted p =
    ↪ 0.020).
  B vs C: the average of B is 5.67 higher (95 % CI -5.82 to 17.1); could easily be chance (Tukey-adjusted p
    ↪ = 0.464).
```

```
Counts of "breakfast" (180 rows): yes 115 (expected 108), no 65 (expected 72.0). Differences this big from
    ↪ the expected counts could easily be chance (p = 0.287); this does not show that there is no
    ↪ difference. Cohen's w = 0.0794.
Test: chi-square goodness-of-fit test (chi-square = 1.13, df = 1), as R chisq.test(x, p = ...); the usual
    ↪ test of counts against expected shares.
"passed" is "yes" in 43.3 % of the rows (78 of 180; 95 % CI 36.0 % to 50.9 %), against 50 %. A difference
    ↪ this big could easily be chance (p = 0.086); this does not show that there is no difference. Cohen'
    ↪ s h = -0.134.
Test: exact binomial test, as R binom.test(78, 180, p = 0.5); the 95 % CI is the exact Clopper-Pearson
    ↪ interval.
```

Good to know

- The idea behind every test: imagine there is no real difference (this is called the null hypothesis). The p-value says how surprising your data would be then. A small p-value means: hard to explain by chance alone.
- A p-value does not say how big or important the difference is: the effect size does that (omega-squared, Cohen's d, Cramer's V ...). And a large p-value does not show that there is no difference. It only means these data cannot tell.
- A 95 % confidence interval is the range the true value is very likely in. A narrow one means the data pin the value down well; a wide one means you need more data.
- Each test gives back a one-row table with all its numbers, so you can save it: result = `table.compare_averages(...)`, then `result.save("test.csv")`.
- Also: `compare_before_after` for two measurements of the same thing (a paired t-test), and `compare_average_to` for one average against a number you choose.
- `remove_rows_with_outliers` removes such rows. Say in your report which rule removed how many rows, and never remove a value only because it spoils the result.
- `method="benjamini-hochberg"` adjusts for many tests more gently: it controls the share of false alarms among your findings (the false discovery rate).

Try it yourself

1. Compare the average marks `for_each="subject"`. Could the differences be chance?
2. Is the average temperature different between the cities of `sdata.example("weather")`?
3. Fit a straight line of `"forest"` against `"year"` in `sdata.example("growth")` for one country.
4. Remove the rows with outliers in `"marks"` `for_each="class"`, then compare the average marks `for_each="class"` again. Did the answer change?

Worked solutions: page 275, or type `sdata.lesson(14, solutions=True)`.

Words in this lesson: `table.compare_averages()`, `table.compare_each_pair()`, `table.compare_counts()`, `table.fit_line()`, `table.fit_formula()`, `table.add_group_column()`, `table.compare_shape_to()`, `table.show_rows_with_outliers()`, `table.add_adjusted_p_value_column()`, `table.remove_rows_with_outliers()`, `table.compare_spreads()`, `table.compare_counts_to()`, `table.compare_percent_to()`.

LESSON 15

A whole analysis: from a question to a figure

In this lesson you will learn

- how to put all the lessons together: open, look, clean, sum up, test, draw and save
- how to make a figure with panels A and B, ready for a report or a paper
- how `steps()` writes down what you did, in plain words
- how to hand your table to pandas or Polars, and back

Example 1

A whole analysis with one question: do children who study longer get better marks, and do the classes differ? One table feeds two charts, and the two charts become one figure.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.describe_columns()
table.remove_rows_with_empty_cells()
table.average_of("marks", for_each="class")
table.compare_averages("marks", for_each="class")
a = table.chart()
a.boxes(x="class", y="marks")
a.all_dots(x="class", y="marks")
a.label_y("Mark (out of 100)")
b = table.chart()
b.dots(x="hours", y="marks", colour="class")
b.fit_line(confidence_band=True)
b.label_x("Hours of study")
figure = sdata.grid(a, b, per_row=2, letters=True)
figure.title_top("Marks, classes and hours of study")
figure.style("paper")
figure.size(width=17, height=8, unit="cm")
figure.save("lesson15_figure.pdf")
figure.save("lesson15_figure.tif", resolution=600)
table.save("lesson15_clean.xlsx")
table.steps()
```

What each line does

1. Load simplydata.
2. Open the data. For your own file: `sdata.table("results.xlsx")`.
3. Look: what is in each column, and are any cells empty?
4. Clean: remove the rows with an empty cell. The practice table has none, so nothing is removed.
5. Sum up: the average mark of each class.
6. Test: could the differences between the classes be chance?
7. Chart A, from the cleaned table.

The marks of each class as boxes.
Every mark as a dot, on top of the boxes.
Name its y-axis.
Chart B, from the same table.
Hours of study against marks, one colour per class.
A straight line for each class, with its 95 % confidence band.
Name its x-axis.
Both charts on one page, with A and B at the top left.
A title for the whole page.
A quiet look that suits a paper.
The size a journal asks for: 17 cm wide (a full page width) and 8 cm high.
Save the figure as a PDF: sharp at any size.
And as a `.tif` picture with 600 dots per inch, as many journals ask.
Save the cleaned table that the figure was drawn from.
Every table step in plain words, in order: a good start for the methods part of your report.

What it prints

```
The table has 54 rows and 6 columns.

 column   kind  empty cells smallest biggest average different values
   name   text            0                                       18
  class   text            0                                        3
subject   text            0                                        3
  hours number            0      0.5     9.4    5.33
  marks number            0       31      98   64.91
 height number            0      141     162  150.39
Removed 0 of 54 rows. 54 rows left.
The average of "marks" for each "class": A 71.83, B 64.28, C 58.61.
The average "marks" is 71.8 in A, 64.3 in B and 58.6 in C. Differences this big between the groups are
    ↪ unlikely to be chance (p = 0.028). Omega-squared = 0.0967: the groups explain about 10 % of the
    ↪ differences in "marks".
Test: Welch's one-way ANOVA (F = 3.99, df = 2 and 34.0), the usual test for three or more averages; it does
    ↪ not assume the groups have the same spread. To see which groups differ: table.compare_each_pair("
    ↪ marks", for_each="class").
1. remove rows with an empty cell
```

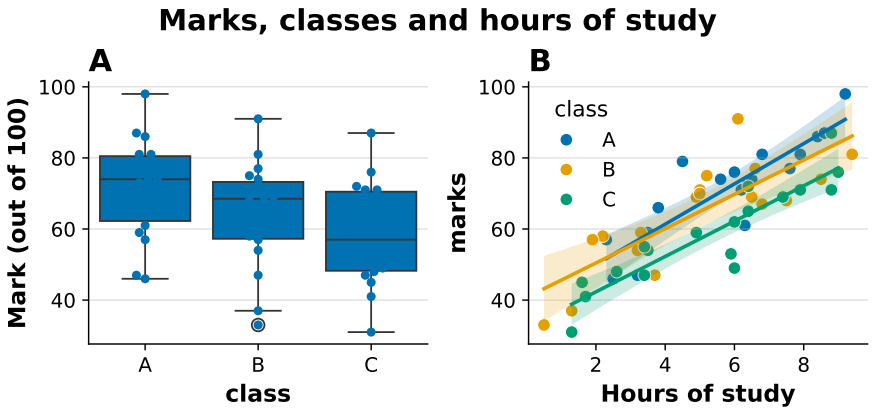


Figure 15.1: Lesson 15, example 1: `lesson15_figure.pdf`.

Example 2

Two questions about two tables give two charts. `sdata.grid` puts them on one page.

```
import simplydata as sdata
weather = sdata.table(sdata.example("weather"))
weather.keep_rows("city", equal_to="Rainbridge")
rain = weather.chart()
rain.bars(x="month", y="rain")
rain.line_at_y(100, label="100 mm")
growth = sdata.table(sdata.example("growth"))
forest = growth.chart()
forest.lines(x="year", y="forest", colour="country")
forest.label_y("Forest (% of the land)")
page = sdata.grid(rain, forest, per_row=2, letters=True)
page.save("lesson15_two_tables.png")
```

What each line does

1. Load simplydata.
2. Table 1: the weather.
3. Keep one city.
4. A chart of it, called rain.
5. Its rain through the year.
6. A line at 100 mm.
7. Table 2: four countries.
8. A chart of it, called forest.
9. Their forest over the years.
10. Name the y-axis.
11. Both charts on one page, with letters.
12. Save the page.

What it prints

```
Kept 12 of 36 rows.
```

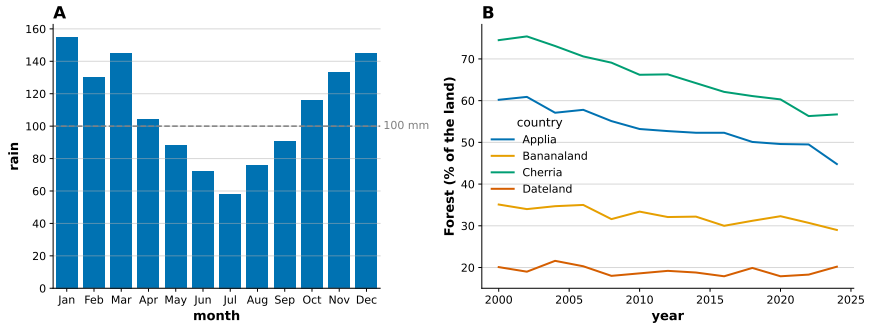


Figure 15.2: Lesson 15, example 2: `lesson15_two_tables.png`.

Example 3

A table goes to pandas or Polars and back whenever another Python tool needs it.

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.keep_rows("city", equal_to="Rainbridge")
df = table.as_pandas()
print(df["rain"].max())
```

```
again = sdata.table(df)
chart = sdata.chart(df)
chart.bars(x="month", y="rain")
chart.save("lesson15_rain.png")
```

What each line does

1 Load simplydata.
2 A table of the weather.
3 Keep one city.
4 A plain pandas DataFrame (`table.as_polars()` gives a Polars one).
5 Any pandas code works on it.
6 And back: `sdata.table` takes a pandas or Polars table you already have.
7 `sdata.chart` takes one too.
8 Draw it.
9 Save the chart.

What it prints

```
Kept 12 of 36 rows.
155
```

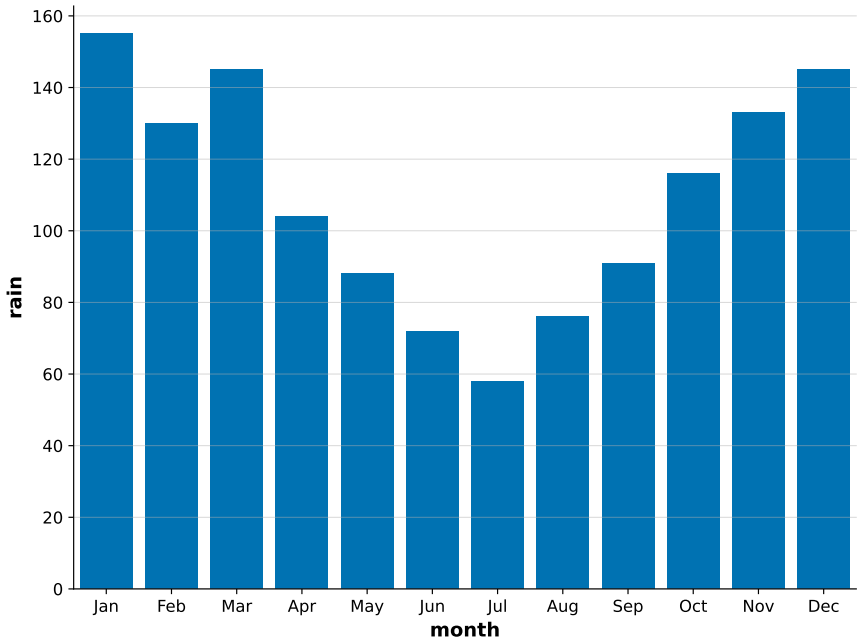


Figure 15.3: Lesson 15, example 3: `lesson15_rain.png`.

Good to know

- Every analysis follows the same plan: open, look, clean, sum up, test, draw, words, save. Lessons 1 to 14 taught each part; this lesson puts them together.
- The table and the charts are separate objects, as in pandas and matplotlib: one cleaned table can feed many charts, and each chart keeps the rows the table had when the chart was made.
- `table.save` saves the table, and `chart.save` or `page.save` the picture. Save both, so anyone can check the figure against its data.

- `table.steps()` lists the table steps. Together with the chart lines of your program, it says everything you did.
- simplydata words never need `as_pandas()`; it is only for other tools.

Try it yourself

1. Do a whole analysis of `sdata.example("weather")`: look at it, compare the average rain `for_each="city"`, draw the rain of each city as boxes, and save the chart and the table.
2. Make a second chart of the temperature, then put the two charts together with `sdata.side_by_side(..., letters=True)`.
3. Read the `steps()` of your table out loud. Would someone else understand what you did?

Worked solutions: page 277, or type `sdata.lesson(15, solutions=True)`.

Words in this lesson: `sdata.table()`, `table.describe_columns()`, `table.remove_rows_with_empty_cells()`, `table.compare_averages()`, `table.chart()`, `chart.boxes()`, `chart.all_dots()`, `chart.dots()`, `chart.fit_line()`, `sdata.grid()`, `chart.style()`, `table.save()`, `table.steps()`, `table.as_pandas()`, `sdata.chart()`.

LESSON 16

Models: explain one column from several others

In this lesson you will learn

- how to explain one column from several others at once (multiple regression)
- how to explain a yes/no column (logistic regression) and a column of counts (Poisson regression)
- how to allow for groups such as schools (a mixed model)
- how to use rows with empty cells without guessing once (multiple imputation)

Example 1

Marks depend on more than one thing. `explain()` looks at several columns together, so each effect is the effect with the other columns held fixed.

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.explain("marks", using=["hours", "sleep", "breakfast"])
table.explain("marks", using=["hours * breakfast"])
```

What each line does

1. Load simplydata.
2. Pupils in 12 schools: study, sleep, breakfast and marks.
3. Explain the marks from hours of study, hours of sleep and breakfast. Each extra hour of study goes with about 2.7 more marks, each extra hour of sleep with 2.3 more, and breakfast with 3.2 more, each with its 95 % confidence interval and p-value. This is a linear model (multiple regression) by least squares.
4. hours * breakfast also asks: does the effect of study depend on breakfast? That is an interaction. Here it could easily be chance (p = 0.31).

What it prints

```
Note: 19 rows with an empty cell in "marks" or "hours" or "sleep" or "breakfast" were skipped.
Explaining "marks" from "hours", "sleep" and "breakfast" (161 rows). Each effect below holds the other
    ↪ columns fixed:
  intercept: 14.7, the expected "marks" when every number column is 0 and every group column is at its
    ↪ baseline.
  hours: each 1 more "hours" goes with 2.69 more "marks" (95 % CI 2.15 to 3.23); very unlikely to be chance
    ↪ (p < 0.0001).
  sleep: each 1 more "sleep" goes with 2.29 more "marks" (95 % CI 0.876 to 3.70); unlikely to be chance (p =
    ↪  0.002).
  breakfast "yes" against "no": "marks" is 3.24 higher (95 % CI 0.493 to 5.99); unlikely to be chance (p =
    ↪ 0.021).
Method: linear model by least squares, as R lm(marks ~ hours + sleep + breakfast).
R squared = 0.430 (adjusted 0.419): the model explains about 43 % of the differences in "marks". The model
    ↪ as a whole (F = 39.4, df = 3 and 157): very unlikely to be chance (p < 0.0001).
A link does not show that one thing causes the other.
Explaining "marks" from "hours" and "breakfast" (180 rows). Each effect below holds the other columns fixed:
```

```
  intercept: 30.2, the expected "marks" when every number column is 0 and every group column is at its
     ↪ baseline.
  hours: each 1 more "hours" goes with 3.24 more "marks" (95 % CI 2.26 to 4.22); very unlikely to be chance
     ↪ (p < 0.0001).
  breakfast "yes" against "no": "marks" is 6.15 higher (95 % CI -0.175 to 12.5); could easily be chance (p =
     ↪  0.057).
  hours : breakfast (yes) (interaction): -0.592 (95 % CI -1.75 to 0.564), how much the effect of one changes
     ↪  with the other; could easily be chance (p = 0.313).
Method: linear model by least squares, as R lm(marks ~ hours * breakfast).
R squared = 0.407 (adjusted 0.397): the model explains about 41 % of the differences in "marks". The model
     ↪ as a whole (F = 40.2, df = 3 and 176): very unlikely to be chance (p < 0.0001).
A large p-value does not show that there is no effect, only that the data cannot tell. A link does not show
     ↪ that one thing causes the other.
```

Example 2

Not every column is a number to explain. A yes/no column needs the chance of yes, and a count needs a model for counts.

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.explain("passed", using=["hours", "breakfast"], outcome="yes or no")
table.explain("days_absent", using=["hours", "sleep"], outcome="counts")
table.add_predicted_column("marks", using=["hours", "breakfast"])
table.show_first_rows(5)
```

What each line does

1. Load simplydata.
2. The school table.
3. Did the pupil pass? `outcome="yes or no"` is logistic regression. Each extra hour of study multiplies the odds of passing by 1.66 (an odds ratio).
4. Days absent are counts (0, 1, 2 ...). `outcome="counts"` is Poisson regression; each extra hour of study multiplies the expected days absent by 0.88 (a rate ratio).
5. Add a column `"predicted marks"`: what the model expects for each pupil.
6. Look at the marks next to the predicted marks.

What it prints

```
Explaining "passed" from "hours" and "breakfast" (180 rows). Each odds ratio below holds the other columns
     ↪ fixed:
  intercept: 0.0454, the odds of yes when every number column is 0 and every group column is at its baseline
     ↪ .
  hours: each 1 more "hours" multiplies the odds of yes by 1.66 (95 % CI 1.40 to 1.98); very unlikely to be
     ↪ chance (p < 0.0001).
  breakfast "yes" against "no": the odds of yes is 1.52 times as high (95 % CI 0.747 to 3.07); could easily
     ↪ be chance (p = 0.250).
Method: logistic regression, as R glm(passed ~ hours + breakfast, family = binomial); counted as yes: "
     ↪ passed" = "yes". Wald 95 % intervals and z-tests, as R confint.default() and summary().
Residual deviance 199 on 177 df (with no columns: 246 on 179 df). The model as a whole (likelihood-ratio chi
     ↪ -square = 47.4, df = 2): very unlikely to be chance (p < 0.0001). AIC = 205.
A large p-value does not show that there is no effect, only that the data cannot tell. A link does not show
     ↪ that one thing causes the other.
Note: 19 rows with an empty cell in "days_absent" or "hours" or "sleep" were skipped.
Explaining "days_absent" from "hours" and "sleep" (161 rows). Each rate ratio below holds the other columns
     ↪ fixed:
  intercept: 14.3, the expected count when every number column is 0.
```

```
  hours: each 1 more "hours" multiplies the expected count by 0.876 (95 % CI 0.841 to 0.913); very unlikely
     ↪ to be chance (p < 0.0001).
  sleep: each 1 more "sleep" multiplies the expected count by 0.886 (95 % CI 0.803 to 0.977); unlikely to be
     ↪  chance (p = 0.015).
Method: Poisson regression, as R glm(days_absent ~ hours + sleep, family = poisson). Wald 95 % intervals and
     ↪  z-tests, as R confint.default() and summary().
Residual deviance 165 on 158 df (with no columns: 215 on 160 df). The model as a whole (likelihood-ratio chi
     ↪ -square = 49.9, df = 2): very unlikely to be chance (p < 0.0001). AIC = 603.
A link does not show that one thing causes the other.
Added "predicted marks": the fitted value of "marks" from "hours" and "breakfast" (least squares, 180 rows).
     school pupil  hours  ... passed  days_absent  predicted marks
1  School A  P001    4.3  ...     no            2        44.451206
2  School A  P002    3.4  ...     no            3        45.126707
3  School A  P003    4.8  ...    yes            3        49.068830
4  School A  P004    5.7  ...     no            2        48.393329
5  School A  P005   11.6  ...    yes            1        68.216285

[5 rows x 9 columns]
```

Example 3

Pupils in the same school are alike in ways the columns do not show. A mixed model allows for that, and multiple imputation uses the rows with empty cells.

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.explain("marks", using=["hours", "breakfast"], grouped_by="school")
table.explain("marks", using=["hours", "sleep", "breakfast"], empty_cells="multiple
    ↪ imputation")
```

What each line does

1. Load simplydata.
2. The school table.
3. A mixed model: each school gets its own starting level (a random intercept). About 39 % of the differences the columns do not explain lie between schools (the ICC).
4. 19 pupils did not say how long they slept. Instead of leaving them out, multiple imputation fills the empty cells 20 times, fits the model to each filled-in table and pools the 20 answers (Rubin's rules), so the extra uncertainty is counted.

What it prints

```
Explaining "marks" from "hours" and "breakfast" (180 rows). Each effect below holds the other columns fixed:
  intercept: 32.5, the expected "marks" when every number column is 0 and every group column is at its
     ↪ baseline.
  hours: each 1 more "hours" goes with 2.81 more "marks" (95 % CI 2.38 to 3.23); very unlikely to be chance
     ↪ (p < 0.0001).
  breakfast "yes" against "no": "marks" is 2.97 higher (95 % CI 0.818 to 5.13); unlikely to be chance (p =
     ↪ 0.007).
Method: linear mixed model with a random intercept for each "school", fitted by REML, as R lme4::lmer(marks
     ↪ ~ hours + breakfast + (1 | school)).
12 groups in "school". Variance between groups 30.1, within groups 46.8: about 39 % of the differences in "
     ↪ marks" that the columns do not explain lies between the groups (intraclass correlation, ICC =
     ↪ 0.392).
P-values: Wald z-tests, with Wald 95 % intervals (standard errors as R vcov(), intervals as R confint(method
     ↪  = "Wald")); R lmerTest gives Satterthwaite t-tests instead, which give slightly larger p-values
     ↪ when there are few groups.
A link does not show that one thing causes the other.
```

```
Explaining "marks" from "hours", "sleep" and "breakfast" (180 rows, 19 of them with empty cells filled in).
     ↪ Each effect below holds the other columns fixed:
  intercept: 14.7, the expected "marks" when every number column is 0 and every group column is at its
     ↪ baseline.
  hours: each 1 more "hours" goes with 2.74 more "marks" (95 % CI 2.23 to 3.25); very unlikely to be chance
     ↪ (p < 0.0001).
  sleep: each 1 more "sleep" goes with 2.27 more "marks" (95 % CI 0.901 to 3.64); unlikely to be chance (p =
     ↪  0.001).
  breakfast "yes" against "no": "marks" is 3.28 higher (95 % CI 0.698 to 5.87); unlikely to be chance (p =
     ↪ 0.013).
Method: multiple imputation by chained equations (predictive mean matching with 5 donors, seed=1): the empty
     ↪  cells were filled in 20 times, the model was fitted to each of the 20 filled-in tables, and the
     ↪ results were pooled with Rubin's rules (t-based intervals with Barnard-Rubin degrees of freedom),
     ↪ as R pool(with(mice(...), lm(marks ~ hours + sleep + breakfast))).
The filling-in adds uncertainty: up to 6 % of the variance of an effect comes from the differences between
     ↪ the filled-in tables. Tests for whole group columns are not pooled here.
A link does not show that one thing causes the other.
```

Good to know

- Each effect in `explain()` holds the other columns fixed: the effect of sleep is the effect for pupils who study the same number of hours and have the same breakfast.
- A text column such as breakfast is compared with its first group in A to Z order (here `"no"`); the answer says which group is the baseline.
- An odds ratio above 1 means higher odds of yes; a rate ratio above 1 means more counts. 1 means no difference.
- If counts vary much more than a Poisson model expects (overdispersion), simplydata says so and suggests `outcome="counts (overdispersed)"`; it never switches by itself.
- `grouped_by=` is for rows that come in groups: pupils in schools, measurements on the same animal, samples from the same site. Without it, the p-values are too small.
- Multiple imputation needs the empty cells to be explainable by the other columns (missing at random). Say in your report how many cells were empty and how they were handled.
- `grouped_by=` and multiple imputation need the statsmodels package: pip install `"simplydata[stats]"`.

Try it yourself

1. Explain `"marks"` using `"hours"` and `"school"` (school as a column of groups). Which schools differ from School A, and is school as a whole important?
2. Explain `"passed"` using `"hours"` and `"sleep"` with `outcome="yes or no"`.
3. Explain `"days_absent"` using `"hours"` and `"breakfast"` with `outcome="counts"`. Does breakfast matter?

Worked solutions: page 279, or type `sdata.lesson(16, solutions=True)`.

Words in this lesson: `table.explain()`, `table.add_predicted_column()`, `table.show_first_rows()`.

LESSON 17

Patterns across many columns

In this lesson you will learn

- how to sum up many columns in two new ones (principal components)
- how to measure how different rows are (Bray-Curtis and other distances)
- how to draw a map of the rows (NMDS) and test groups on all columns at once (PERMANOVA)
- how to find groups of similar rows (clustering)

Example 1

Ten plant counts per plot are too many to look at one by one. Principal components make a few new columns that hold most of the spread.

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.add_principal_components(how_many=2)
chart = table.chart()
chart.dots(x="PC1", y="PC2", colour="grazed")
chart.save("lesson17_pca.png")
```

What each line does

1. Load simplydata.
2. Plants counted in 30 plots, half of them grazed by sheep.
3. Principal component analysis (PCA): adds PC1 and PC2. PC1 holds 30 % of the spread, PC2 15 %. The printout says which plants weigh most in each.
4. Start a chart.
5. Each plot is a dot; grazed and ungrazed plots sit apart.
6. Save it.

What it prints

```
Principal components of 10 columns (clover, daisy, buttercup, dandelion, nettle, thistle, moss, fern, orchid
    ↪ , yarrow), each put on the same scale first (mean 0, standard deviation 1): PC1 29.9 % and PC2 15.2
    ↪  % of the spread (45.1 % together).
  PC1 weighs most on "thistle" (0.45), "orchid" (0.45) and "nettle" (0.41).
  PC2 weighs most on "buttercup" (0.54), "dandelion" (-0.51) and "clover" (0.47).
Method: principal component analysis by singular value decomposition, as R prcomp(scale. = TRUE). Added PC1
    ↪ and PC2 (the scores of each row). The sign of each component is arbitrary: flipping it changes
    ↪ nothing.
There are 10 components in all; table.principal_components_of() shows them all.
```

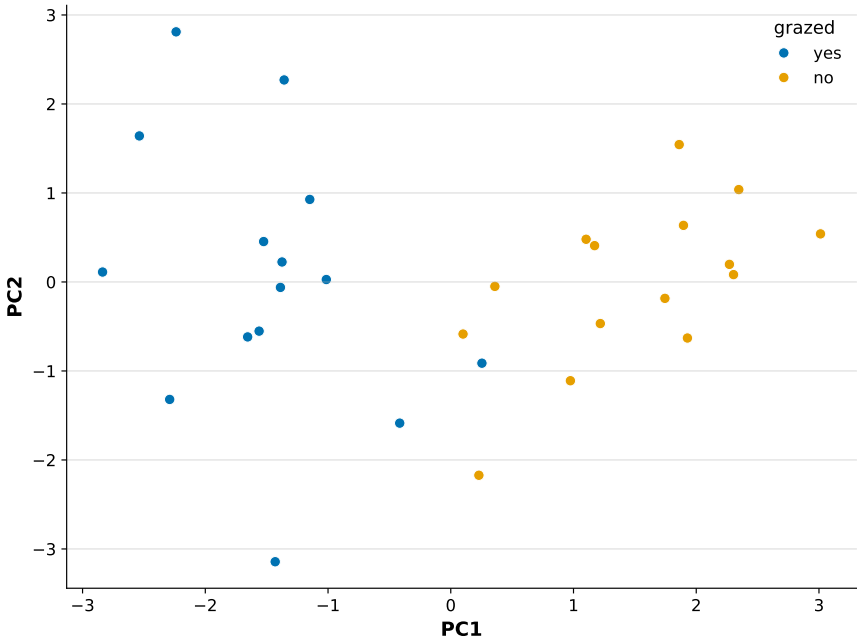


Figure 17.1: Lesson 17, example 1: `lesson17_pca.png`.

Example 2

Ecologists compare plots by how different their make-up is. Bray-Curtis is the usual distance for counts, and NMDS draws a map that keeps the order of the distances.

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
distances = table.distances_between_rows(measure="bray-curtis", names="plot")
distances.show_first_rows(3)
table.add_nmds(measure="bray-curtis")
table.compare_profiles(for_each="grazed")
chart = table.chart()
chart.dots(x="NMDS1", y="NMDS2", colour="grazed")
chart.save("lesson17_nmds.png")
```

What each line does

1. Load simplydata.
2. The meadow table.
3. A table of the Bray-Curtis distance between every pair of plots: 0 means the same plants in the same numbers, 1 means nothing in common.
4. Look at the first rows.

5 Non-metric multidimensional scaling (NMDS): adds NMDS1 and NMDS2. The stress (0.195) says how well the map keeps the distances: usable.
6 Do grazed and ungrazed plots differ in make-up? PERMANOVA tests all ten plants at once: grazing explains 25 % of the differences (p = 0.001).
7 Start a chart.
8 The NMDS map.
9 Save it.

What it prints

```
Bray-Curtis distances between 30 rows from 10 columns, as R vegan::vegdist(x, "bray"). 0 means the same; the
    ↪  bigger, the more different.
  plot       NG1       NG2  ...       RU3       RU4       RU5
1  NG1  0.000000  0.336493  ...  0.328947  0.474286  0.351351
2  NG2  0.336493  0.000000  ...  0.502347  0.601695  0.479675
3  NG3  0.280576  0.400000  ...  0.460993  0.597561  0.436782

[3 rows x 31 columns]
NMDS map of 30 rows from 10 columns (Bray-Curtis distances, 2 dimensions): stress = 0.195, usable, but do
    ↪ not read too much into small details (Clarke's rule of thumb: below 0.05 excellent, below 0.1 good,
    ↪  below 0.2 usable, above 0.2 hard to trust).
Method: non-metric multidimensional scaling (Kruskal's stress-1, SMACOF with monotone regression), lowest
    ↪ stress of 21 starts (classical scaling and 20 random, seed=1; the lowest was reached 6 times), as R
    ↪  vegan::metaMDS(x, distance = "bray", k = 2, autotransform = FALSE). Added NMDS1 and NMDS2; the
    ↪ axes have no units and their rotation and signs are arbitrary.
Note: left out NMDS1 and NMDS2 (made by an earlier pattern word); give columns= to use them.
The groups of "grazed" (no, yes) explain 24.6 % of the differences between the rows (R squared = 0.246; Bray
    ↪ -Curtis distances over 10 columns). A difference in make-up this big is unlikely to be chance (p =
    ↪ 0.001); pseudo-F = 9.14, from 999 permutations.
Test: PERMANOVA (permutational analysis of variance on distances), as R vegan::adonis2(distances ~ grazed,
    ↪ permutations = 999) with distances = vegan::vegdist(x, "bray").
Spread check: the average distance to the group centre is 0.240 in no and 0.278 in yes (as vegan::betadisper
    ↪  and permutest: F = 2.53, p = 0.108).
```

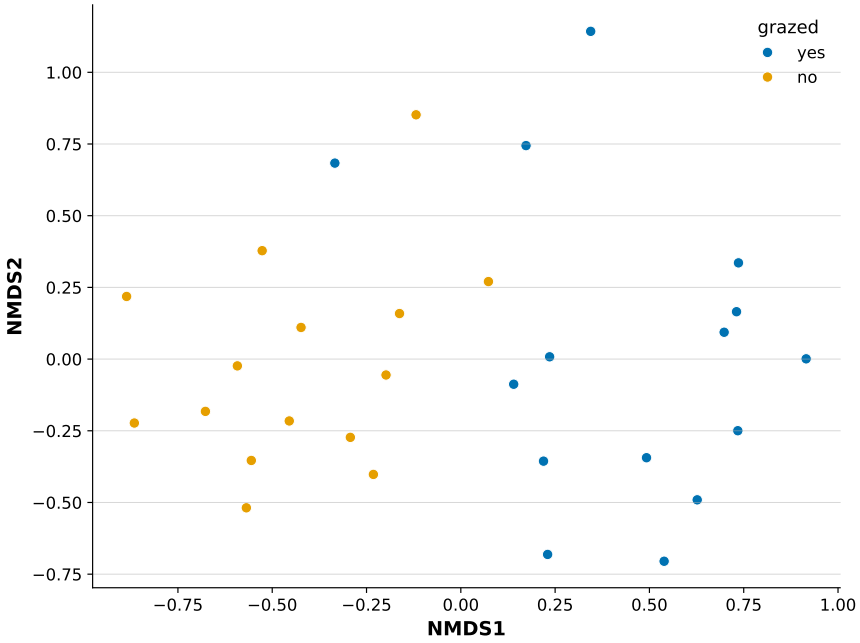


Figure 17.2: Lesson 17, example 2: `lesson17_nmds.png`.

Example 3

Clustering finds groups of similar rows without being told the groups.

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.add_clusters(how_many=2)
table.count_rows(for_each="cluster", across="grazed")
```

What each line does

1. Load simplydata.
2. The meadow table.
3. k-means clustering: adds a column `"cluster"` with 1 or 2.
4. Did the clusters find the grazing? Cluster 1 is 14 grazed plots, cluster 2 is the 15 ungrazed plots and one grazed one.

What it prints

```
Made 2 clusters of 30 rows from 10 columns (each put on the same scale first): sizes 14 and 16 (clusters 1
    ↪ and 2, numbered in the order they first appear). 24.4 % of the spread lies between the clusters (
    ↪ between-cluster / total sum of squares); the rest is within them.
Method: k-means with 25 random starts (seed=1); each start runs Lloyd's algorithm and then Hartigan's single
    ↪ -point moves, and the start with the smallest within-cluster sum of squares is kept (R kmeans(
    ↪ nstart = 25) uses Hartigan-Wong and reaches the same kind of solution). Added the column "cluster".
Rows for each "cluster" (rows) and each "grazed" (columns):
  1: no 0, yes 14
  2: no 15, yes 1
```

Good to know

- `same_scale=True` (the normal choice for PCA and clusters) puts every column on the same scale first, so a column with big numbers does not drown the others.
- The signs and the rotation of PC and NMDS axes are arbitrary; only the distances between the dots mean something.
- Clarke's rule for NMDS stress: below 0.05 excellent, below 0.1 good, below 0.2 usable, above 0.2 hard to trust. Try `dimensions=3` if the stress is high.
- PERMANOVA can mistake a difference in spread for a difference in make-up, so `compare_profiles` also checks the spread of each group and says if it differs.
- Clusters always appear, even in data without groups: check them against what you know.
- `method="hierarchical"` clusters step by step (Ward linkage) instead of k-means.

Try it yourself

1. Add principal components of `"hours"`, `"sleep"`, `"marks"` and `"days_absent"` in `sdata.example("school")`. Which columns weigh most in PC1?
2. Make an NMDS map of the meadow in 3 dimensions. Is the stress lower than in 2?
3. Do the three fields differ in make-up? Use `compare_profiles` with `for_each="field"`.
4. Cluster the meadow plots with `method="hierarchical"`. Do the clusters match the grazing?

Worked solutions: page 281, or type `sdata.lesson(17, solutions=True)`.

Words in this lesson: `table.add_principal_components()`, `table.principal_components_of()`, `table.add_clusters()`, `table.distances_between_rows()`, `table.add_nmds()`, `table.compare_profiles()`, `chart.dots()`.

LESSON 18

Survival, and planning a study

In this lesson you will learn

- how to draw survival curves (Kaplan-Meier) and compare them (log-rank test)
- how to explain the risk of an event from several columns (a Cox model)
- how many values a study needs, before you collect them (power analysis)

Example 1

In a trial, some patients are still alive when the follow-up ends. Survival words use their time too: they count as followed without the event (censored).

```
import simplydata as sdata
table = sdata.table(sdata.example("trial"))
table.survival_over_time(time="days", event="died", for_each="treatment")
chart = table.chart()
chart.survival_curves(time="days", event="died", colour="treatment", confidence_band=True,
    ↪ at_risk_table=True)
chart.save("lesson18_survival.png")
```

What each line does

1. Load simplydata.
2. 120 patients, a new medicine or usual care, followed for up to 3 years.
3. The Kaplan-Meier estimate: the share still alive at each time, and the median time (706 days with the new medicine, 545 with usual care).
4. Start a chart.
5. Survival curves with their 95 % bands, ticks where patients were censored, and the number still followed under the x-axis.
6. Save it.

What it prints

```
The event is "died" = "yes" (59 of 120 rows); the other rows count as followed without the event until their
    ↪  "days" (censored).
Median time until the event (by then half have had it), with its 95 % CI:
  new medicine: median "days" 706 (95 % CI 486 to not reached; 60 rows, 25 events).
  usual care: median "days" 545 (95 % CI 394 to not reached; 60 rows, 34 events).
Method: Kaplan-Meier estimate with 95 % confidence intervals on the log scale (Greenwood), as R survfit(Surv
    ↪ (days, died == "yes") ~ treatment).
```

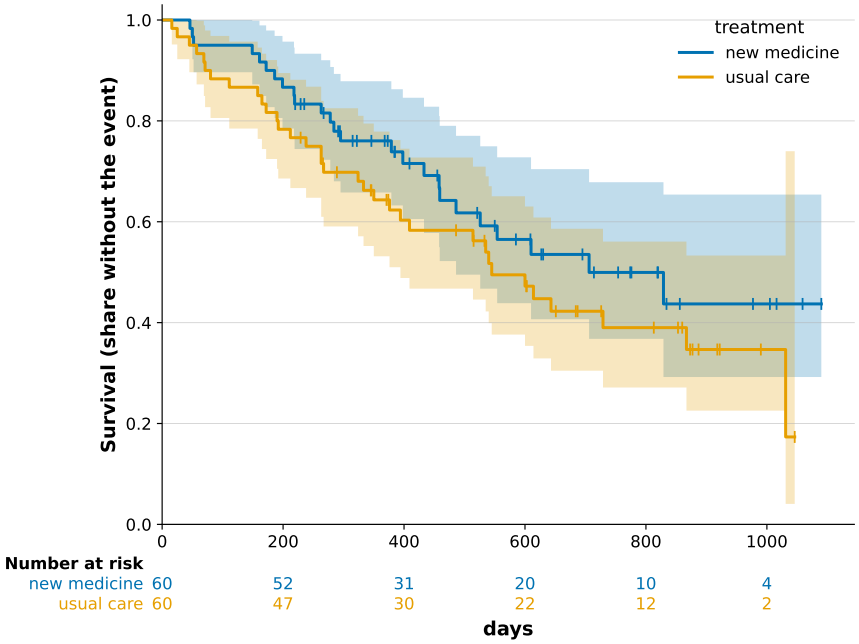


Figure 18.1: Lesson 18, example 1: `lesson18_survival.png`.

Example 2

Is the difference real? And what else changes the risk?

```
import simplydata as sdata
table = sdata.table(sdata.example("trial"))
table.compare_survival(time="days", event="died", for_each="treatment")
table.explain_survival(time="days", event="died", using=["treatment", "age"])
```

What each line does

1. Load simplydata.
2. The trial table.
3. The log-rank test compares the whole curves: here the difference could easily be chance (p = 0.22).
4. A Cox model: each year of age raises the risk of dying at any moment by about 5 % (hazard ratio 1.05), with the treatment held fixed.

What it prints

```
The event is "died" = "yes" (59 of 120 rows); the other rows count as followed without the event until their
    ↪  "days" (censored).
Events in each "treatment" group, observed vs expected if all groups were alike: new medicine 25 vs 29.7 and
    ↪  usual care 34 vs 29.3.
Differences this big in the time until the event could easily be chance (p = 0.216); this does not show that
    ↪  there is no difference.
Test: log-rank test (chi-square = 1.53, df = 1), as R survdiff(Surv(days, died == "yes") ~ treatment); it
    ↪ compares the whole curves, giving every event time the same weight.
The event is "died" = "yes" (59 of 120 rows); the other rows count as followed without the event until their
    ↪  "days" (censored).
Hazard ratios from 120 rows with 59 events (above 1: a higher risk of the event at any moment; below 1: a
    ↪ lower risk):
  "treatment" usual care vs new medicine: hazard ratio 1.33 (95 % CI 0.795 to 2.24); could easily be chance
    ↪ (p = 0.275).
  each 1 more "age": hazard ratio 1.05 (95 % CI 1.03 to 1.08); very unlikely to be chance (p < 0.0001).
The model as a whole (likelihood-ratio test, chi-square = 21.6, df = 2) is very unlikely to be chance (p <
    ↪ 0.0001).
Method: Cox proportional hazards model, Efron's method for tied times, as R coxph(Surv(days, died == "yes")
    ↪ ~ treatment + age).
```

Example 3

Was the trial big enough? Power analysis tells you how many values you need before you collect them.

```
import simplydata as sdata
sdata.how_many_needed(effect_size=0.5, power=0.8)
sdata.chance_to_find(effect_size=0.5, values_per_group=20)
```

What each line does

1. Load simplydata.
2. To find a medium difference between two averages (half a standard deviation) with an 80 % chance, you need 64 values in each group.
3. With only 20 values in each group, the chance to find it is just 34 %.

What it prints

```
To find a difference of 0.5 standard deviations (Cohen's d) with an 80 % chance, using a two-sided test at
    ↪ 0.05, you need 64 values in each group (128 in all). Method: two-sample t-test power, as R power.t.
    ↪ test().
Note: the calculation uses the usual equal-spread formula; compare_averages uses Welch's t-test, which is
    ↪ very close to it when the groups are the same size.
Cohen's rough guide calls this a medium effect (Cohen's d: 0.2 small, 0.5 medium, 0.8 large); it is only a
    ↪ hint, sizes from your own field are better.
With 20 values in each group (40 in all), a study has a 33.8 % chance (power = 0.338) to find a difference
    ↪ of 0.5 standard deviations (Cohen's d), using a two-sided test at 0.05. Method: two-sample t-test
    ↪ power, as R power.t.test().
Note: this is below the usual 80 %; how_many_needed() tells you how many values you need.
Note: the calculation uses the usual equal-spread formula; compare_averages uses Welch's t-test, which is
    ↪ very close to it when the groups are the same size.
Cohen's rough guide calls this a medium effect (Cohen's d: 0.2 small, 0.5 medium, 0.8 large); it is only a
    ↪ hint, sizes from your own field are better.
```

Good to know

- Never leave out the censored rows or count them as survivors: the survival words use

the time they were followed, which is what makes these methods right.

- A hazard ratio above 1 means a higher risk of the event at any moment; 1 means no difference. The Cox model assumes the ratio stays the same over time, and simplydata checks this and says so if it does not.
- A study that is too small often finds nothing even when there is something to find. Plan the size before collecting data, not after.
- `effect_size=` is Cohen's d for averages, f for 3 or more groups, w for counts and r for a correlation. You can also give `difference=` and `spread=` in your own units.

Try it yourself

1. Explain the risk of dying from `"age"` alone with `explain_survival`.
2. How many pairs does a before/after study need to find a small change (`effect_size=0.3`)?
3. What is the chance to find a medium difference (`effect_size=0.25`) between 3 groups with 30 values in each?

Worked solutions: page 282, or type `sdata.lesson(18, solutions=True)`.

Words in this lesson: `table.survival_over_time()`, `chart.survival_curves()`, `table.compare_survival()`, `table.explain_survival()`, `sdata.how_many_needed()`, `sdata.chance_to_find()`.

LESSON 19

Fitting curves, finding the best value and solving

In this lesson you will learn

- how to fit a straight line, a curve or any formula to your data, on a table and on a chart
- how to find where a formula is smallest or biggest
- how to solve an equation, also one made from a fitted formula

Example 1

Start with something you know: a straight line through the dots. `fit_line`, `fit_curve` and `fit_formula` are the same words on a table (the numbers) and on a chart (the picture).

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.fit_line(x="hours", y="marks")
table.fit_curve(x="hours", y="marks", degree=2)
chart = table.chart()
chart.dots(x="hours", y="marks")
chart.fit_line()
chart.fit_curve(degree=2)
chart.save("lesson19_line_and_curve.png")
```

What each line does

1 Load simplydata.
2 The marks table: hours of study and marks.
3 The straight line: about 5.2 marks more for each hour of study, and R squared 0.69.
4 A curve with one bend (hours and hours**2). R squared only goes from 0.69 to 0.70, so the bend adds little: the straight line is enough here.
5 Start a chart of the same table.
6 The dots.
7 The same straight line, drawn.
8 And the curve, to see how close they are.
9 Save it.

What it prints

```
"marks" goes up by 5.20 for each 1 more "hours" (95 % CI 4.24 to 6.17); this slope is very unlikely to be
    ↪ chance (p < 0.0001). R squared = 0.69: "hours" explains about 69 % of the differences in "marks".
Line: marks = 5.20 * hours + 37.2, by least squares over 54 rows. A link does not show that one thing causes
    ↪  the other.
Curve: marks = 31.0 + 8.26 * hours - 0.297 * hours**2 (each with its 95 % CI in the table); R squared =
    ↪ 0.70, adjusted 0.69.
Method: least squares on powers of hours, as R lm(marks ~ poly(hours, 2, raw = TRUE)).
```

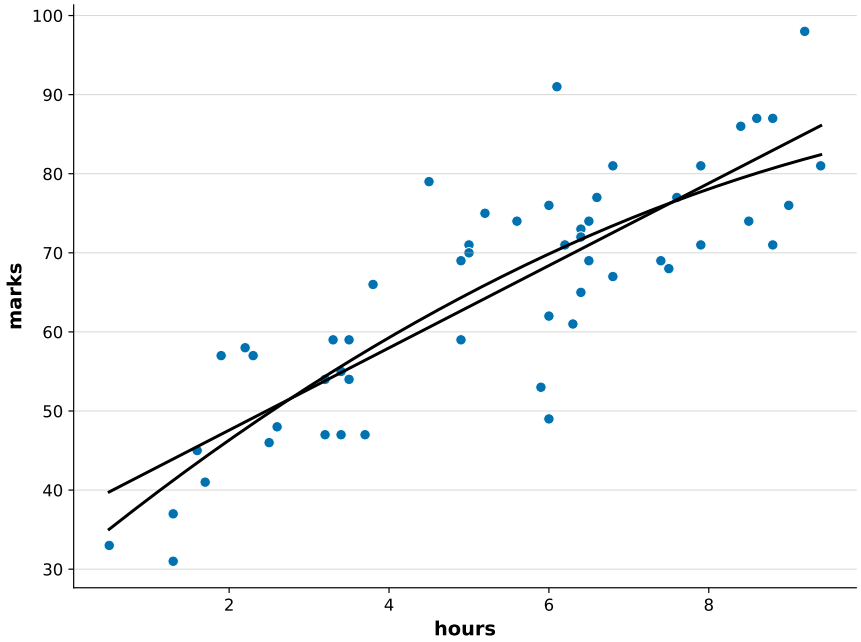


Figure 19.1: Lesson 19, example 1: `lesson19_line_and_curve.png`.

Example 2

A research question: how fast does each country grow? Growth by a fixed share each year is the formula population = a * `exp(b * (year - 2000))`. Names that are columns (population, year) are data; the other names (a, b) are the values to find.

```
import simplydata as sdata
table = sdata.table(sdata.example("growth"))
table.fit_formula("population = a * exp(b * (year - 2000))", for_each="country")
chart = table.chart()
chart.dots(x="year", y="population", colour="country")
chart.fit_formula("population = a * exp(b * (year - 2000))")
chart.save("lesson19_growth.png")
```

What each line does

1. Load simplydata.
2. Four countries, every two years.
3. One fit for each country, by nonlinear least squares (as R `nls()`). a is the population in 2000 and b the growth rate: Cherria grows fastest, b = 0.0443, about 4.5 % a year.
4. Start a chart.
5. The data, one colour per country.
6. The fitted curve of each country, with the same values as the table word.

7 Save it.

What it prints

```
Fitted "population = a * exp(b * (year - 2000))" for each "country":
  Applia (13 rows): a = 12.0 (95 % CI 11.8 to 12.2), b = 0.0297 (95 % CI 0.0289 to 0.0306). Residual
     ↪ standard error 0.179 on 11 degrees of freedom; R squared = 1.00 (only a rough guide for a curve).
  Bananaland (13 rows): a = 29.9 (95 % CI 29.5 to 30.3), b = 0.0122 (95 % CI 0.0114 to 0.0131). Residual
     ↪ standard error 0.371 on 11 degrees of freedom; R squared = 0.99 (only a rough guide for a curve).
  Cherria (13 rows): a = 4.99 (95 % CI 4.88 to 5.10), b = 0.0443 (95 % CI 0.0431 to 0.0456). Residual
     ↪ standard error 0.127 on 11 degrees of freedom; R squared = 1.00 (only a rough guide for a curve).
  Dateland (13 rows): a = 44.7 (95 % CI 44.3 to 45.1), b = 0.00406 (95 % CI 0.00342 to 0.00470). Residual
     ↪ standard error 0.368 on 11 degrees of freedom; R squared = 0.95 (only a rough guide for a curve).
Values found: a, b. Data: year. Method: nonlinear least squares (Levenberg-Marquardt), as R nls(); intervals
     ↪  are estimate +/- t x standard error (R confint() on nls profiles the likelihood and can differ a
     ↪ little).
Fitted "population = a * exp(b * (year - 2000))" for "Applia" to 13 rows: a = 12.0 (95 % CI 11.8 to 12.2), b
     ↪  = 0.0297 (95 % CI 0.0289 to 0.0306). Residual standard error 0.179 on 11 degrees of freedom.
Fitted "population = a * exp(b * (year - 2000))" for "Bananaland" to 13 rows: a = 29.9 (95 % CI 29.5 to
     ↪ 30.3), b = 0.0122 (95 % CI 0.0114 to 0.0131). Residual standard error 0.371 on 11 degrees of
     ↪ freedom.
Fitted "population = a * exp(b * (year - 2000))" for "Cherria" to 13 rows: a = 4.99 (95 % CI 4.88 to 5.10),
     ↪ b = 0.0443 (95 % CI 0.0431 to 0.0456). Residual standard error 0.127 on 11 degrees of freedom.
Fitted "population = a * exp(b * (year - 2000))" for "Dateland" to 13 rows: a = 44.7 (95 % CI 44.3 to 45.1),
     ↪  b = 0.00406 (95 % CI 0.00342 to 0.00470). Residual standard error 0.368 on 11 degrees of freedom.
Method: nonlinear least squares (Levenberg-Marquardt), as R nls(); intervals are estimate +/- t x standard
     ↪ error (R confint() on nls profiles the likelihood and can differ a little).
```

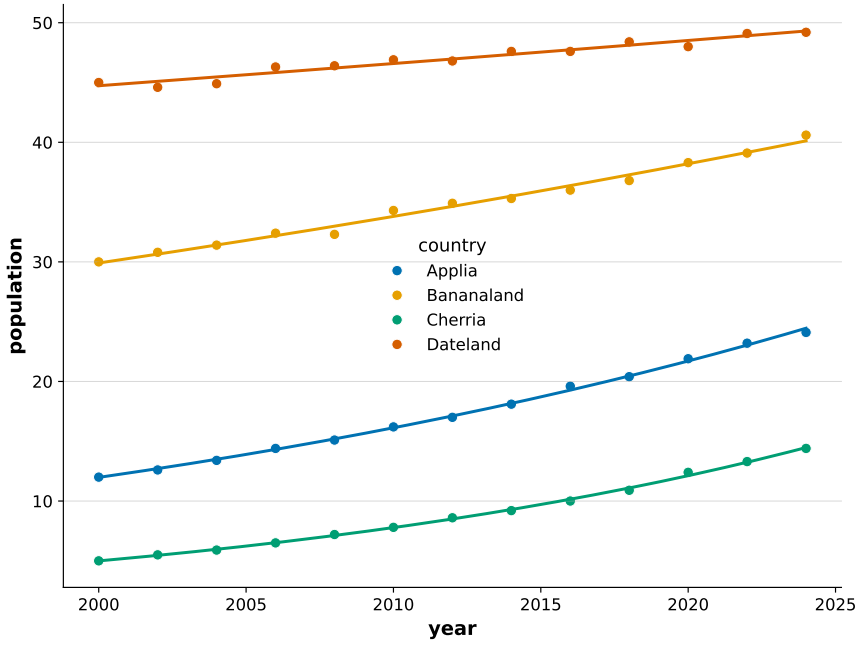


Figure 19.2: Lesson 19, example 2: `lesson19_growth.png`.

Example 3

Finding and solving: `sdata.find_smallest`, `sdata.find_biggest` and `sdata.solve` work on a formula. The names in it are what to find; `between=` says where to look.

```
import simplydata as sdata
sdata.find_smallest("x**2 - 4*x + 1", between=[0, 5])
sdata.find_biggest("3*x + 2*y", rules=["x + y <= 4", "x + 3*y <= 6", "x >= 0", "y >= 0"])
```

```
sdata.solve("x**3 - 2*x = 5", between=[0, 3])
table = sdata.table(sdata.example("growth"))
table.keep_rows("country", equal_to="Applia")
fit = table.fit_formula("population = a * exp(b * (year - 2000))")
sdata.solve("a * exp(b * (year - 2000)) = 20", between=[2000, 2050], known=fit)
```

What each line does

1. Load simplydata.
2. The smallest value is -3, at x = 2. The method and its R name are printed (R `optimize()`).
3. The biggest value under rules: 12, at x = 4 and y = 0. Everything is a straight line, so simplydata uses linear programming and says which rules are reached exactly.
4. The value of x that makes both sides equal: 2.09455. Every solution in the range is found.
5. Back to the growth table.
6. Keep one country.
7. Fit the growth formula and keep the result, a one-row table with a and b.
8. `known=fit` puts the fitted a and b into the equation: Applia reaches 20 million in about 2017 (year = 2017.24). A fitted curve answers questions, not only draws a line.

What it prints

```
The smallest value of x**2 - 4*x + 1 between 0 and 5 is -3, at x = 2.
Method: Brent's method on the range, as R optimize().
The biggest value of 3*x + 2*y under the 4 rules is 12, at x = 4, y = 0.
Method: linear programming (HiGHS), as R lpSolve::lp(), because the formula and every rule are straight-line
    ↪  (linear). Value of one more on the right of each rule that limits the answer (shadow price): x + y
    ↪  <= 4: 3; y >= 0: -1.
x + y <= 4 and y >= 0 are reached exactly; the other rules have room to spare.
x**3 - 2*x = 5 between 0 and 3 when x = 2.09455.
Method: Brent's method, as R uniroot(); every sign change on 1000 steps across the range was searched, so
    ↪ all solutions that cross were found (one where the curve only touches, like x**2 = 0, can be missed
    ↪ ).
Kept 13 of 52 rows.
Fitted "population = a * exp(b * (year - 2000))" to 13 rows: a = 12.0 (95 % CI 11.8 to 12.2), b = 0.0297 (95
    ↪  % CI 0.0289 to 0.0306). Residual standard error 0.179 on 11 degrees of freedom; R squared = 1.00 (
    ↪ only a rough guide for a curve).
Values found: a, b. Data: year. Method: nonlinear least squares (Levenberg-Marquardt), as R nls(); intervals
    ↪  are estimate +/- t x standard error (R confint() on nls profiles the likelihood and can differ a
    ↪ little).
Using a = 11.9771, b = 0.0297362 from known=.
a * exp(b * (year - 2000)) = 20 between 2000 and 2050 when year = 2017.24.
Method: Brent's method, as R uniroot(); every sign change on 1000 steps across the range was searched, so
    ↪ all solutions that cross were found (one where the curve only touches, like x**2 = 0, can be missed
    ↪ ).
```

Good to know

- Fitting is the same idea every time: a line, a curve or a formula is moved until it is as close to the points as it can be (least squares). The printed sentence says how sure each value is (its 95 % confidence interval).
- A curve with a high `degree=` can wiggle through every point and still say nothing true. Choose the simplest line or formula that follows the data, and a formula whose values mean something (here b is a growth rate).
- If a formula fit fails, give start values near the answer: `start={"a": 10, "b": 0.03}`.
- `find_smallest` with `between=` finds one lowest point; when the curve has several dips it

says so, and `search="whole range"` looks everywhere. solve with `between=` finds every crossing.

- Names from other tools point to these words: minimize, optimise and argmin to `find_smallest`, fsolve, brentq and uniroot to solve, `curve_fit` and nls to `fit_formula`, polyfit to `fit_curve`.

Try it yourself

1. Fit a curve of degree 3 to `"marks"` against `"hours"`. Does R squared go up much from the straight line?
2. With the weather table, keep the city Sunnyvale and fit `"temperature = a + b * cos(2 * pi * (month_number - c) / 12)"` with `start={"a": 15, "b": 8, "c": 7}`. In which month is it warmest (c)?
3. Try to remember without looking back: which word finds where a formula is biggest, and which word finds where the two sides of an equation are equal? Use them on `"x * (10 - x)"` between 0 and 10, and on `"x**2 = 2"` between 0 and 2.

Worked solutions: page 283, or type `sdata.lesson(19, solutions=True)`.

Words in this lesson: `table.fit_line()`, `table.fit_curve()`, `table.fit_formula()`, `chart.fit_line()`, `chart.fit_curve()`, `chart.fit_formula()`, `sdata.find_smallest()`, `sdata.find_biggest()`, `sdata.solve()`.

LESSON 20

Distributions, chances and values between points

In this lesson you will learn

- how to ask a distribution for chances, percentiles and cut-offs
- how to find which distribution fits your data, and check it
- how to estimate values between the points you measured

Example 1

A distribution says how likely each value is. Start with the bell curve (normal): marks with an average of 50 and a standard deviation of 10.

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.chance(above=80)
dist.chance(between=[40, 60])
dist.percentile(95)
dist.cut_off(above=0.025)
chart = dist.chart()
chart.save("lesson20_normal.png")
```

What each line does

1. Load simplydata.
2. Make the distribution. R's names (`mean=`, `sd=`) work too.
3. The chance of a mark above 80: 0.135 %. The R code that gives the same answer is printed too: `pnorm(80, ...)`.
4. 68.3 %: about two in three are within one standard deviation of the average.
5. 95 % of marks are below 66.4.
6. The cut-off with a 2.5 % chance above it: 69.6. Tests use such cut-offs (critical values).
7. A chart of the bell curve.
8. Save it.

What it prints

```
The chance of a value above 80 is 0.135 % (normal, average 50, standard deviation 10; R: pnorm(80, mean =
    ↪ 50, sd = 10, lower.tail = FALSE)).
The chance of a value between 40 and 60 is 68.3 % (normal, average 50, standard deviation 10; R: pnorm(60,
    ↪ mean = 50, sd = 10) - pnorm(40, mean = 50, sd = 10)).
95 % of the distribution is below 66.4 (normal, average 50, standard deviation 10; R: qnorm(0.95, mean = 50,
    ↪  sd = 10)).
The cut-off with a 2.5 % chance above it is 69.6 (normal, average 50, standard deviation 10; R: qnorm(0.025,
    ↪  mean = 50, sd = 10, lower.tail = FALSE)).
```

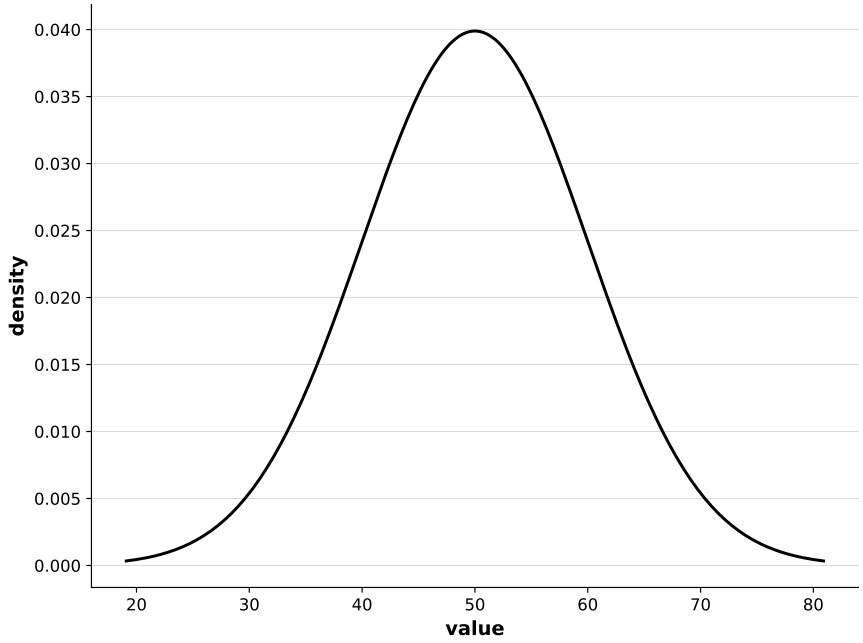


Figure 20.1: Lesson 20, example 1: `lesson20_normal.png`.

Example 2

A research question: pupils' days absent are counts (0, 1, 2 ...). Do they vary as a Poisson distribution expects, or more? The words that fit are `fit_distribution` and `compare_distributions`.

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.compare_distributions("days_absent", distributions=("poisson", "negative binomial"))
fit = table.fit_distribution("days_absent", "negative binomial")
fit.chance(at_least=10)
chart = table.chart()
chart.histogram(x="days_absent", show="density")
chart.fit_distribution("negative binomial")
chart.save("lesson20_absent.png")
```

What each line does

1. Load simplydata.
2. 180 pupils and their days absent.
3. Both are fitted by maximum likelihood and ranked by AIC: the negative binomial wins by 7, so the counts vary more than Poisson allows.
4. Fit it and keep the fitted distribution.
5. The chance a pupil misses 10 days or more: 0.66 %.

6 Start a chart.
7 The counts, as shares.
8 The fitted chance of each count, on top.
9 Save it.

What it prints

```
Best by AIC: negative binomial (AIC 744.4), then poisson (+7.0).
A difference below 2 is too small to choose between them. Method: maximum likelihood for each, ranked by AIC
    ↪ , as R fitdistrplus::gofstat().
Fitted a negative binomial distribution to "days_absent" (180 values): size = 8.79 (standard error 3.64),
    ↪ average = 3.09 (0.152). Log-likelihood -370.2, AIC 744.4.
Method: maximum likelihood, as R MASS::fitdistr(x, "negative binomial").
The chance of 10 or more is 0.663 % (negative binomial, size 8.79286, average 3.09444; R: pnbinom(9, size =
    ↪ 8.792862164, mu = 3.094444444, lower.tail = FALSE)).
Fitted a negative binomial distribution to "days_absent" (180 values): size = 8.79 (standard error 3.64),
    ↪ average = 3.09 (0.152). Log-likelihood -370.2, AIC 744.4.
Method: maximum likelihood, as R MASS::fitdistr(x, "negative binomial").
```

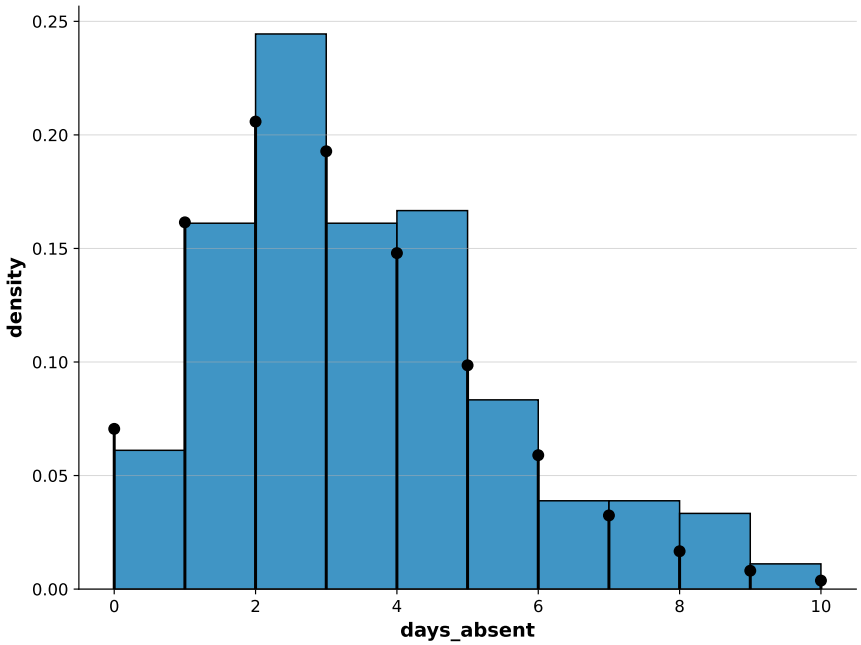


Figure 20.2: Lesson 20, example 2: `lesson20_absent.png`.

Example 3

Values between points: you measured every two years, but you need 2001. `values_at` joins the dots (straight lines, or a smooth curve) and reads off the value; `map_of` does it across a whole map.

```
import simplydata as sdata
table = sdata.table(sdata.example("growth"))
estimates = table.values_at("population", at={"year": [2001, 2011]}, for_each="country")
estimates.show()
island = sdata.table(sdata.example("island"))
heights = island.map_of("height", across=["east", "north"], steps=60, joined_by="smooth
    ↪ surface")
chart = heights.chart()
chart.contour(x="east", y="north", z="height")
chart.save("lesson20_map.png")
```

What each line does

1 Load simplydata.
2 Population every two years.
3 Straight lines between the known points (linear interpolation, as R `approx()`), for each country.
4 A new table with the estimates.
5 Heights measured on a 15 x 15 grid.
6 A finer 60 x 60 map, with a smooth surface through every point (a thin-plate spline).
7 A chart of the new map.
8 Its contour map.
9 Save it.

What it prints

```
Estimated "population" at 2 values of "year" for each of the 4 groups in "country", joined by straight lines
    ↪ between the known points (linear interpolation, as R approx()).
  year     country  population
1 2001      Applia       12.30
2 2011      Applia       16.60
3 2001  Bananaland       30.40
4 2011  Bananaland       34.60
5 2001     Cherria        5.25
6 2011     Cherria        8.20
7 2001    Dateland       44.80
8 2011    Dateland       46.85
Estimated "height" at 60 x 60 evenly spaced places across "east" and "north", joined by a smooth surface (a
    ↪ thin-plate spline, passing through every point).
```

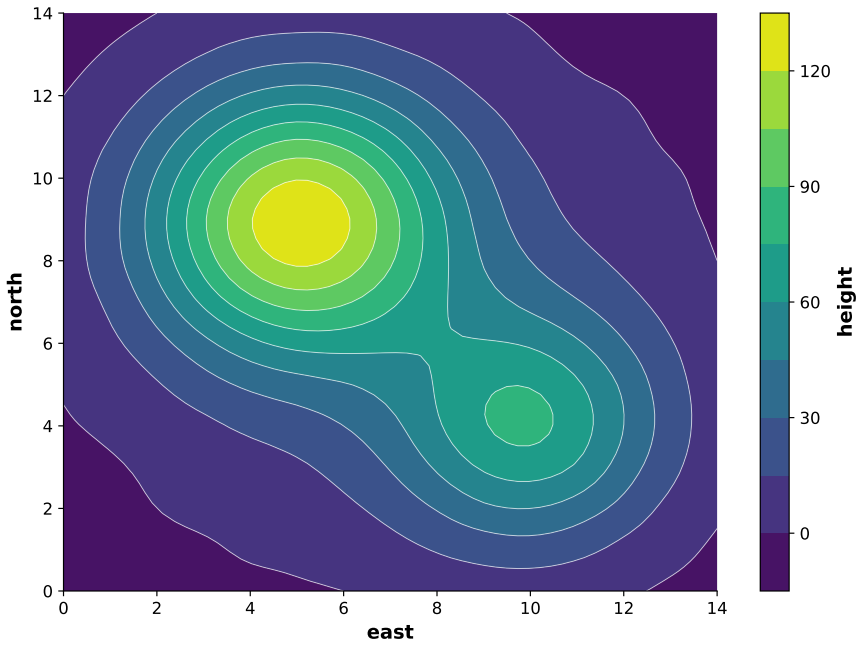


Figure 20.3: Lesson 20, example 3: `lesson20_map.png`.

Good to know

- `chance()` gives back a share from 0 to 1 and prints it as a percent. `above=` and `below=` leave the value itself out, `at_least=` and `at_most=` keep it in, as in `keep_rows`. For counts

that matters: `at_most=3` means 3 or fewer.

- `density_at()` is the height of the curve, not a chance: a chance is an area under the curve, so ask `chance(between=[...])`.
- A fitted distribution is only as good as its fit. Compare a few with `compare_distributions`, and test the shape with `table.compare_shape_to("column", "gamma")`.
- Values between points are estimates. Straight lines are the safe choice; a smooth curve can swing above or below the points (`joined_by="curve without overshoot"` never does). Outside the measured range the cells stay empty unless you ask for `outside=`.
- Names from other tools point to these words: pnorm and cdf to chance, qnorm and ppf to percentile, dnorm and pdf to `density_at`, interpolate and approx to `values_at`.

Try it yourself

1. Ten coins are thrown. With `sdata.distribution("binomial", tries=10, chance=0.5)`, what is the chance of 8 heads or more?
2. Which fits the daily `"sales"` of the shop table best: normal, lognormal, gamma or weibull? Then check the best one with `compare_shape_to`.
3. Try to remember without looking back: which word estimates a value between the points you measured? Use it to estimate the temperature of each city at `month_number` 6.5, joined by a smooth curve.

Worked solutions: page 285, or type `sdata.lesson(20, solutions=True)`.

Words in this lesson: `sdata.distribution()`, `dist.chance()`, `dist.percentile()`, `dist.cut_off()`, `dist.chart()`, `table.compare_distributions()`, `table.fit_distribution()`, `chart.fit_distribution()`, `table.values_at()`, `table.map_of()`, `chart.curve()`.

PART III

Dictionary of words

How to read the dictionary

Each entry gives the word as you type it, what it does (in the words of `sdata.help("word")`), the options you can put in its brackets and what they normally are, the library that does the work underneath, and an example you can run as it is. The same entries are in Python: `sdata.help("bars")`. The words come in the same groups as `sdata.words()`; the index of words at the end of the book lists them A to Z.

Chart words

Steps on a chart: `chart = sdata.chart(table)`, then `chart.bars(...)`. Words that start with `sdata.` are called on simplydata itself, and words that end in `=` are options inside a chart step.

Start, show and save

`sdata.chart()`

Start a chart. `sdata.chart(sdata.example("marks"))`

`sdata.chart(table)` a pandas or Polars table, like `sdata.example("marks")`
`sdata.chart("my_file.xlsx")` your own file (Excel, CSV, TSV, Parquet, JSON)
`sdata.chart("my_file.xlsx", sheet="2026")` one sheet of an Excel file
`sdata.chart()` no table: give lists to each step instead
`fast=True` always use the fast engine (Polars) to read the file

Options

`data=`
`sheet=`
`fast=` normally `False`

Underneath pandas reads the file (Polars for big files); the chart is drawn with matplotlib and seaborn

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.save("my_chart.png")
```

`sdata.example()`

A small made-up table to practise with. `sdata.example("marks")`

`sdata.example()` with no name lists every table and its columns.
The tables are: marks, weather, fruit, growth, shop, school, meadow, trial, island. `"shop"` has empty cells and a column of dates to practise with; `"school"` has empty cells too.

Options

`name=`

Underneath pandas (`read_csv` of a small file inside simplydata)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
sdata.example()
marks = sdata.example("marks")
print(marks)
```

chart.show()

Show the chart in a window. `chart.show()`

In Jupyter, VS Code and Spyder charts appear by themselves.

Options none: just write the empty brackets.

Good to know Not the same as the `show=` option inside bars and lines: for that, type `sdata.help("show=")`.

Underneath matplotlib (pyplot.show)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.show()
chart.save("marks.png")
```

chart.save()

Save the chart. `chart.save("marks.png")` or `chart.save("marks.tif", resolution=600)`

The end of the file name decides the format: `"marks.png"`, `"marks.pdf"`. `resolution=` is the sharpness in dots per inch (dpi); normally 300.

Options

`file_name`	needed
`quality=`	normally `"normal"`
`resolution=`	
`dpi=`	

Good to know File types: `.png` (a picture), `.pdf` and `.svg` (sharp at any size, good for printing), `.jpg`, `.tif`, and `.gif` for animations. `resolution=` is how sharp a picture is, in dots per inch (dpi): 300 is normal, journals often ask for 300 or 600 (`dpi=` means the same). `quality="high"` is the same as `resolution=600`. `.tif` files are compressed without losing anything. To choose the size of the file in cm, mm or pixels, use `chart.size()` first.

Underneath matplotlib (Figure.savefig); Pillow writes `.gif` animations

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.save("marks.png")
chart.save("marks.pdf")
```

```
chart.save("marks.tif", resolution=600)
```

Chart types

chart.bars()

Also written `chart.bar()`

Bars. `chart.bars(x="name", y="marks")` (with `sdata.example("marks")`)

`colour="class"` gives each group its own colour, or `colour="blue"` one colour. `show=` says what each bar stands for when a group has many rows: `"average"` (default), `"median"`, `"total"`, `"count"`, `"smallest"`, `"biggest"` or `"standard deviation"`. `error="standard deviation"`, `"standard error"` or `"confidence interval"` adds error bars (with `show="average"`); a line says which one was drawn. `stacked=True` puts groups on top of each other; `sideways=True` lays bars down. `transparency=0.5` makes them half see-through (0 is solid).

Options

`x=`	
`y=`	
`colour=`	
`show=`	normally `"average"`
`error=`	
`stacked=`	normally `False`
`sideways=`	normally `False`
`transparency=`	

Underneath seaborn (barplot; countplot for `show="count"`)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks", colour="class")
chart.rotate_labels_x()
chart.save("bars.png")
```

chart.lines()

Also written `chart.line()`

Lines. `chart.lines(x="month", y="temperature", colour="city")` (with `sdata.example("weather")`)

`colour="city"` draws one line per group. `dots=True` marks each point. `dashed=True` draws a dashed line. `show=` works as in bars (not `"count"`). `error="standard deviation"`, `"standard error"` or `"confidence interval"` adds a shaded band; a line says which one was drawn. `thickness=3` draws thicker lines (normal is 2, in points); `transparency=0.5` makes them half see-through (0 is solid). `joined_by=` says how the points are joined, as in `table.values_at()`: `"straight lines"` (normal), `"smooth curve"` (natural cubic spline), `"curve without overshoot"` (PCHIP), `"steps"` (each value stays until the next point), `"nearest"`; experts: `"akima"`, `"makima"`, `"not-a-knot"`, `"periodic"`. Curves need numbers on x and y.

Options

`x=`	
`y=`	
`colour=`	
`show=`	normally `"average"`
`error=`	
`dots=`	normally `False`
`dashed=`	normally `False`
`thickness=`	
`transparency=`	
`joined_by=`	normally `"straight lines"`

Underneath seaborn (lineplot); SciPy (interpolate) for `joined_by=` curves

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature", colour="city", dots=True)
chart.save("lines.png")
```

`chart.dots()`

Also written `chart.scatter()`

Dots (a scatter chart). `chart.dots(x="hours", y="marks")` (with `sdata.example("marks")`)

`colour="class"` colours dots by group. `size="height"` makes dots bigger for bigger values, or `size=10` makes all dots that big. `shape="subject"` gives each group a shape. `transparency=0.5` makes dots half see-through, so dots on top of each other show. With `colour=` a number column and `chart.colour_scale(...)`, dots get a smooth colour scale and a colour bar.

Options

`x=`	
`y=`	
`colour=`	
`size=`	
`shape=`	
`transparency=`	

Underneath seaborn (scatterplot)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks", colour="class", size="height")
chart.save("dots.png")
```

chart.area()

A line with the space under it coloured in.
`chart.area(x="year", y="population", colour="country")` (with `sdata.example("growth")`)

`colour="country"` gives each group its own area; `stacked=True` piles the groups on top of each other, so the top edge shows the total. `transparency=0.3` sets how see-through the coloured space is (0 is solid); `thickness=3` makes its top line thicker.

Options

`x=`	
`y=`	
`colour=`	
`stacked=`	normally `False`
`transparency=`	
`thickness=`	

Underneath matplotlib (`fill_between`, or stackplot with `stacked=True`)

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.area(x="year", y="population", colour="country", stacked=True)
chart.save("area.png")
```

chart.histogram()

Bars that count how many values fall in each range. `chart.histogram(x="marks", bins=10)` (with `sdata.example("marks")`)

`bins=` is how many bars (normally 10), or a list of where each bar starts and ends; `bin_width=5` makes every bar 5 wide instead. `show=` is what the height of a bar means: `"count"` (normal), `"percent"` (of the values) or `"density"` (the bars have an area of 1, to compare with a density curve). `colour="class"` draws one see-through histogram per group; with `show="percent"` or `"density"` each group is worked out on its own. `stacked=True` piles the groups on top of each other instead (then the whole stack makes 100 %). `steps=True` draws only the outline, as steps; `cumulative=True` adds up from the left, so each bar counts the values up to its right edge. `transparency=0.5` makes the bars half see-through; `thickness=` sets the outline width.

Options

`x=`	
`bins=`	
`colour=`	
`show=`	normally `"count"`
`stacked=`	normally `False`
`steps=`	normally `False`
`cumulative=`	normally `False`
`bin_width=`	
`transparency=`	
`thickness=`	

Underneath seaborn (histplot)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.histogram(x="marks", bins=10)
chart.save("histogram.png")
```

chart.boxes()

Also written chart.box()

Boxes that show the middle half of the values and the median.
chart.boxes(x="class", y="marks") (with sdata.example("marks"))

The line in the box is the median, the box holds the middle half of the values, the whiskers reach the rest, and lone dots are values far away from the others. transparency=0.5 makes the boxes half see-through; thickness= sets the width of their lines.

Options

x=
y=
colour=
transparency=
thickness=

Underneath seaborn (boxplot)

Example you can run (it uses the practice table marks)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.boxes(x="class", y="marks")
chart.save("boxes.png")
```

chart.violins()

Also written chart.violin()

Shapes that are wide where many values are. chart.violins(x="class", y="marks") (with sdata.example("marks"))

Each violin stops at the smallest and the largest value; the small box inside shows the middle half of the values. transparency=0.5 makes the violins half see-through; thickness= sets the width of their lines.

Options

x=
y=
colour=
transparency=
thickness=

Underneath seaborn (violinplot)

Example you can run (it uses the practice table marks)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.violins(x="subject", y="marks")
chart.save("violins.png")
```

chart.heatmap()

A grid of coloured squares, one colour for each number.

`chart.heatmap(x="month", y="city", value="rain", numbers=True)` (with `sdata.example("weather")`)

With no columns named, every number column of the table is used (rows are the table's rows): `chart.heatmap(numbers=True)`. `numbers=True` writes the number in each square. `cluster=True` puts similar rows and columns next to each other and draws a tree of how alike they are.

`chart.colour_scale(...)` chooses the colours. `transparency=0.3` lets grid lines show through.

Options

`x=`	
`y=`	
`value=`	
`numbers=`	normally `False`
`cluster=`	normally `False`
`transparency=`	

Underneath seaborn (heatmap); SciPy (scipy.cluster.hierarchy) for `cluster=True`

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.heatmap(x="month", y="city", value="rain", numbers=True)
chart.save("heatmap.png")
```

chart.pie()

A circle cut into slices, one slice per group. `chart.pie(labels="fruit", values="count")` (with `sdata.example("fruit")`)

Without `values=` each slice shows how many rows have that label. `transparency=0.3` makes the slices a little see-through (0 is solid).

Options

`labels=`	
`values=`	
`transparency=`	

Underneath matplotlib (Axes.pie)

Example you can run (it uses the practice table `fruit`)

```
import simplydata as sdata
fruit = sdata.example("fruit")
chart = sdata.chart(fruit)
chart.pie(labels="fruit", values="count")
```

```
chart.save("pie.png")
```

chart.polar()

Dots on a round chart: how far round, and how far out.

`chart.polar(angle="wind_direction", distance="wind_speed", colour="city")` (with `sdata.example("weather")`)

Angles are in degrees (a full turn is 360), going round from the right, the way maths measures angles. Use `degrees=False` if your angles are in radians. `transparency=0.5` makes them half see-through (0 is solid).

Options

`angle=`	
`distance=`	
`colour=`	
`degrees=`	normally `True`
`transparency=`	

Underneath matplotlib (polar axes)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.polar(angle="wind_direction", distance="wind_speed", colour="city")
chart.save("polar.png")
```

chart.contour()

A map where each colour band joins places with the same height.

`chart.contour(x="east", y="north", z="height")` (with `sdata.example("island")`)

`z=` is the height (or depth, or temperature). The points can be a full grid or scattered; scattered points are joined up with straight lines in between. `filled=False` draws only the lines, with their numbers written on them.

`chart.colour_scale(...)` chooses the colours (and steps= the bands). `transparency=0.3` lets what is under it show through; `thickness=` sets the width of the lines.

Options

`x=`	
`y=`	
`z=`	
`filled=`	normally `True`
`transparency=`	
`thickness=`	

Underneath matplotlib (contourf); SciPy (interpolate.griddata) for scattered points

Example you can run (it uses the practice table `island`)

```
import simplydata as sdata
island = sdata.example("island")
chart = sdata.chart(island)
```

```
chart.contour(x="east", y="north", z="height")
chart.save("contour.png")
```

chart.surface()

A 3D landscape: `z=` is how high the ground is.

`chart.surface(x="east", y="north", z="height")` (with `sdata.example("island")`)

The points can be a full grid or scattered; scattered points are joined up.

`chart.colour_scale(...)` chooses the colours. `transparency=0.3` makes it a little see-through.

Options

```
x=
y=
z=
transparency=
```

Underneath matplotlib (mplot3d `plot_surface`); SciPy (interpolate.griddata) for scattered points

Example you can run (it uses the practice table `island`)

```
import simplydata as sdata
island = sdata.example("island")
chart = sdata.chart(island)
chart.surface(x="east", y="north", z="height")
chart.save("surface.png")
```

chart.dots_3d()

Dots floating in a 3D box. `chart.dots_3d(x="hours", y="height", z="marks", colour="class")` (with `sdata.example("marks")`)

`colour="class"` gives each group its own colour; a number column with many values gets a colour scale (`chart.colour_scale(...)` chooses it). `transparency=0.5` makes dots half see-through (0 is solid).

Options

```
x=
y=
z=
colour=
transparency=
```

Underneath matplotlib (mplot3d scatter)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots_3d(x="hours", y="height", z="marks", colour="class")
chart.save("dots_3d.png")
```

chart.band()

A shaded band between a low and a high column, along x.

```
chart.band(x="month", between=["lowest", "highest"]) (with a table that has those columns)
```

`between=["low column", "high column"]` gives the bottom and the top edge at each x: a range of temperatures, a confidence band you worked out, or a smallest-to-biggest band around a line. Draw .lines() as well to show the middle. `colour="city"` draws one band per group (in the group's colour), or `colour="blue"` one colour. `label="Range"` names the band in the legend. Empty cells leave a gap. `transparency=` is how see-through the band is (normally 0.75; 0 is solid).

Options

```
x=
between=
colour=
label=
transparency=
```

Underneath matplotlib (`fill_between`)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.keep_rows("city", equal_to="Sunnyvale")
table.add_column("lowest", "temperature - 4")
table.add_column("highest", "temperature + 4")
chart = table.chart()
chart.band(x="month", between=["lowest", "highest"], label="4 degrees either side")
chart.lines(x="month", y="temperature")
chart.save("band.png")
```

chart.empty_cells()

A map of the empty cells: one row of the chart for each row of the table, one strip for each column, with empty cells coloured. `chart.empty_cells()` (with `sdata.example("shop")`)

Under each column it says how many of its cells are empty. `columns=["sales", "customers"]` shows only those columns. A cell is empty when it is missing or holds only spaces.

Options

```
columns=
```

Underneath pandas finds the empty cells (isna); matplotlib (imshow) draws the grid

Example you can run (it uses the practice table `shop`)

```
import simplydata as sdata
shop = sdata.example("shop")
chart = sdata.chart(shop)
chart.empty_cells()
chart.title_top("Where the empty cells are")
chart.save("empty_cells.png")
```

Statistics

show=

What each bar or point on a line stands for, when a group has many rows.

`show="average"` (the normal choice) adds up the values and divides by how many there are. `show="median"` is the middle value when they are put in order. `show="total"` adds them all up. `show="count"` counts the rows (bars only). `show="smallest"`, `"biggest"` and `"standard deviation"` work too. The names are the same as the table words: `average_of`, `median_of`, `total_of`, `smallest_of` ...

Works in `chart.bars()`, `chart.lines()`

Underneath seaborn (the estimator of barplot and lineplot)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks", show="median")
chart.save("median.png")
```

error=

Error bars on top of averages: how spread out, or how sure, the values are.

`error="standard deviation"` shows how far the values usually are from the average: how spread out they are. `error="standard error"` shows how sure we are about the average itself; it gets smaller as you collect more values. `error="confidence interval"` is the 95 % confidence interval: the range the true average is very likely to be in. Lines get a shaded band instead of bars. A line is printed saying which one was drawn, so your figure caption can say it too. The short forms `"sd"`, `"se"` and `"ci"` also work.

Works in `chart.bars()`, `chart.lines()`

Underneath seaborn (`errorbar="sd"` or `"se"`); the confidence interval from the t distribution (SciPy)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks", error="standard deviation")
chart.save("error.png")
```

chart.error_bars()

Error bars on dots or lines. `chart.error_bars("standard error")`

For every x value the bar shows the average of y and how sure or spread it is: `"standard deviation"` (n - 1): how spread out the values are around the average; `"standard error"`: how sure we are about the average itself; `"confidence interval"`: the 95 % range the true average is very likely in (t distribution). A line says which one was drawn. Empty cells are skipped. `column="error"` uses error sizes from your own column instead. With colour groups, bars at the same x move a little apart (with their lines and dots) so they do not hide each other; `dodge=False` keeps them on the true x.

Options

`kind=`	normally `"standard deviation"`
`column=`	
`dodge=`	normally `True`

Good to know Standard deviation or standard error? The standard deviation describes the values (how spread out they are). The standard error describes the average (how sure we are of it). Say in your caption which one you drew.

Underneath pandas (group averages) and matplotlib (errorbar); SciPy for the t-based confidence interval

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.lines(x="subject", y="marks", colour="class")
chart.error_bars("standard error")
chart.save("error_bars.png")
```

chart.fit_line()

A straight line that fits the dots best (linear regression). `chart.fit_line()`

The line is found by least squares: it makes the squared distances from the dots to the line as small as possible. `confidence_band=True` shades the 95 % confidence band: where the true line very likely is. `show_equation=True` writes the equation and R^2 on the chart. Each colour group gets its own line. `colour="red"` makes every line red. `thickness=3` draws a thicker line (normal is 2).

Options

`confidence_band=`	normally `False`
`show_equation=`	normally `False`
`colour=`	
`thickness=`	

Underneath SciPy (stats.linregress), drawn with matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
```

```
chart.fit_line(confidence_band=True, show_equation=True)
chart.save("fit_line.png")
```

chart.fit_curve()

A curve that fits the dots. `chart.fit_curve(degree=2)`

`degree=2` is a curve with one bend (x^2), `degree=3` can bend twice (x^3), and so on. `show_equation=True` writes the equation and R^2 on the chart. `thickness=3` draws a thicker curve (normal is 2).

Options

`degree=`	normally `2`
`show_equation=`	normally `False`
`colour=`	
`thickness=`	

Underneath NumPy (polyfit), drawn with matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.dots(x="month_number", y="temperature", colour="city")
chart.fit_curve(degree=2)
chart.save("fit_curve.png")
```

chart.fit_formula()

The curve of a formula fitted to the dots. `chart.fit_formula("population = a * exp(b * (year - 2000))")`

After dots, for example `chart.dots(x="year", y="population", colour="country")` (with `sdata.example("growth")`).

The same fit as `table.fit_formula` (nonlinear least squares, as R `nls()`): names that are columns are data, other names (a, b) are the values to find. `start=` and `between=` work as there. Each colour group gets its own curve, and the values found are printed (once). `show_equation=True` writes the formula with the values found on the chart. `colour="red"` makes every curve red; `thickness=3` draws thicker curves.

Options

`formula=`	
`start=`	
`between=`	
`show_equation=`	normally `False`
`colour=`	
`thickness=`	

Underneath SciPy (optimize.curve_fit, as `table.fit_formula`), drawn with matplotlib

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
```

```
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.dots(x="year", y="population", colour="country")
chart.fit_formula("population = a * exp(b * (year - 2000))")
chart.save("fit_formula.png")
```

chart.fit_distribution()

The best-fitting distribution's curve over a histogram or density chart. `chart.fit_distribution("normal")`

After a histogram or density, for example `chart.histogram(x="marks", show="density")` (with `sdata.example("marks")`).

The parameters are found by maximum likelihood, as `table.fit_distribution` (R MASS::`fitdistr()`), and printed (once). Each colour group gets its own curve. Over a histogram the curve is scaled to what the bars show (count, percent or density). `distribution=` is any name of `sdata.distribution`, such as `"gamma"` or `"lognormal"`. `colour="red"` makes every curve red; `thickness=3` draws thicker curves.

Options

`distribution=`	normally `"normal"`
`colour=`	
`thickness=`	

Underneath SciPy (scipy.stats, maximum likelihood as `table.fit_distribution`), drawn with matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.histogram(x="marks", show="density")
chart.fit_distribution("normal")
chart.save("fit_distribution.png")
```

chart.curve()

Draw a formula, or a distribution, as a curve. `chart.curve("y = 100 * exp(-0.3 * x)", between=[0, 10])`

The formula uses one name (x, or any other) and the functions of `table.add_column`; `between=[low, high]` says where along x (normally the x range of the chart).

`chart.curve(sdata.distribution("normal", average=50, standard_deviation=10))` draws a distribution: its density curve, or for whole-number distributions (binomial, poisson ...) the chance of each value. Over a histogram the curve is scaled to the bars (count, percent or density), with a note. `show="cumulative"` draws the chance of a value at or below x instead (on a cumulative histogram it is scaled the same way). `steps=` is how many points make the curve (normally 200); `dashed=True`, `colour="red"`, `thickness=3` and `label="model"` (shown in the legend) change how it looks.

Options

`formula_or_distribution=`	
`between=`	
`steps=`	normally `200`
`dashed=`	normally `False`
`colour=`	
`label=`	
`thickness=`	
`show=`	normally `"density"`

Underneath NumPy (the formula on a grid of x values) or scipy.stats (a distribution), drawn with matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.histogram(x="marks", show="density")
chart.curve(sdata.distribution("normal", average=60, standard_deviation=15), label="a bell
    ↪ curve")
chart.save("curve.png")
```

`chart.smooth_line()`

A smooth line that follows the dots (LOWESS). `chart.smooth_line()`

Good for seeing a trend that is not straight. Each colour group gets its own line. `thickness=3` draws a thicker line (normal is 2). `method="spline"` draws a smoothing spline instead (as `table.values_at(..., smoothness=)` and R `smooth.spline()`): `smoothness="auto"` chooses how smooth by generalised cross-validation; a number is R's `lambda=` (bigger is smoother).

Options

`colour=`	
`thickness=`	
`method=`	normally `"lowess"`
`smoothness=`	normally `"auto"`

Underneath NumPy (LOWESS, written out in simplydata as in R's `lowess()`) or SciPy (`make_smoothing_spline`, for `method="spline"`), drawn with matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.dots(x="month_number", y="rain", colour="city")
chart.smooth_line()
chart.save("smooth_line.png")
```

chart.density()

A smooth curve that is high where many values are. `chart.density(x="marks", colour="class")` (with `sdata.example("marks")`)

`colour="class"` draws one curve per group; each curve has an area of 1. `thickness=3` draws thicker curves; `transparency=0.5` makes them half see-through.

Options

```
x=
colour=
transparency=
thickness=
```

Underneath seaborn (kdeplot)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.density(x="marks", colour="class")
chart.save("density.png")
```

chart.count_bars()

Bars that count how many rows each group has. `chart.count_bars(x="fruit", colour="shop")` (with `sdata.example("fruit")`)

`colour="shop"` splits each bar by group; `sideways=True` lays the bars down. `transparency=0.5` makes them half see-through (0 is solid).

Options

```
x=
colour=
sideways=                    normally False
transparency=
```

Underneath seaborn (countplot)

Example you can run (it uses the practice table `fruit`)

```
import simplydata as sdata
fruit = sdata.example("fruit")
chart = sdata.chart(fruit)
chart.count_bars(x="fruit", colour="shop")
chart.save("count_bars.png")
```

chart.all_dots()

Every value as a dot, spread out so no dot hides another.
`chart.all_dots(x="class", y="marks")` (with `sdata.example("marks")`)

Good on top of .boxes() or .violins(), to show each value as well. `transparency=0.5` makes them half see-through (0 is solid).

Options

```
x=
y=
colour=
transparency=
```

Underneath seaborn (swarmplot; stripplot for groups of more than 100 dots)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.boxes(x="class", y="marks")
chart.all_dots(x="class", y="marks")
chart.save("all_dots.png")
```

chart.side_histograms()

Small histograms along the top and right edges of a dots chart. `chart.side_histograms()`

Options none: just write the empty brackets.

Underneath matplotlib (hist), with NumPy choosing the bins

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.side_histograms()
chart.save("side_histograms.png")
```

chart.qq_dots()

Dots that lie on a straight line when the values follow a normal (bell) curve.
`chart.qq_dots("marks")` (with `sdata.example("marks")`)

Each value, smallest to biggest, is drawn against where it would be if the values were normal (a normal Q-Q plot, scipy.stats.probplot), with the straight line that fits the dots best. Dots that bend away from the line at the ends mean long or short tails; a curve means the values lean to one side. Good for checking residuals. `for_each="class"` draws each group in its own colour, with its own line. SciPy places the dots at Filliben's positions; R's `qqnorm()` uses slightly different ones, and R's `qqline()` draws its line through the quartiles instead. `distribution="gamma"` (any name of

`sdata.distribution`) compares with that distribution instead, its parameters fitted to each group by maximum likelihood (printed once).

Options

`column=`
`for_each=`
`transparency=`
`thickness=`
`distribution=` normally `"normal"`

Underneath SciPy (stats.probplot, against a fitted scipy.stats distribution for `distribution=`) for the points and the line; matplotlib draws them

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.qq_dots("marks")
chart.save("qq_dots.png")
```

`chart.cumulative_share()`

A line that climbs from 0 to 1: the share of values at or below each x.

`chart.cumulative_share(x="marks", colour="class")` (with `sdata.example("marks")`)

Also called the empirical cumulative distribution (ECDF; seaborn's ecdfplot). It shows every value, needs no bins, and groups of any size compare directly: at marks = 50 a line at 0.3 means 30 % of that group scored 50 or less. `colour="class"` draws one line per group. `thickness=3` draws thicker lines.

Options

`x=`
`colour=`
`transparency=`
`thickness=`

Underneath seaborn (ecdfplot)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.cumulative_share(x="marks", colour="class")
chart.save("cumulative_share.png")
```

`sdata.pairs()`

Every number column against every other. `sdata.pairs(sdata.example("marks"), colour="class")`

Histograms on the diagonal (density curves when `colour=` makes groups), dots everywhere else. `colour="class"` colours each group. `columns=["hours", "marks"]` chooses the columns. Empty cells are skipped. Then use it like any chart: .title_top("..."), .save("pairs.png").

Options

`data`	needed
`colour=`	
`columns=`	

Underneath seaborn (scatterplot, histplot and kdeplot)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.pairs(marks, colour="class")
chart.title_top("Every number column against every other")
chart.save("pairs.png")
```

Lines and shading

`chart.line_at_x()`

A straight up-and-down line at one x value. `chart.line_at_x(2010, label="2010")`

`thickness=3` draws a thicker line (normal is 1.5).

Options

`value`	needed
`dashed=`	normally `True`
`colour=`	normally `"grey"`
`label=`	
`thickness=`	

Underneath matplotlib

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.lines(x="year", y="population", colour="country")
chart.line_at_x(2010, label="2010")
chart.save("line_at_x.png")
```

chart.line_at_y()

A straight side-to-side line at one y value. `chart.line_at_y(50, label="pass mark")` `thickness=3` draws a thicker line (normal is 1.5).

Options

`value`	needed
`dashed=`	normally `True`
`colour=`	normally `"grey"`
`label=`	
`thickness=`	

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.line_at_y(50, label="pass mark")
chart.rotate_labels_x()
chart.save("line_at_y.png")
```

chart.line_with_slope()

A straight line with a given slope, right across the chart. `chart.line_with_slope(1, through=(0, 0))` `chart.line_with_slope(1, label="1:1")` draws the 1:1 line (y = x) on a chart of observed against predicted values. `through=`(2, 10) makes the line pass through `x = 2, y = 10` (normally through `0, 0`); `intercept=5` instead sets where it crosses `x = 0`, so `y = intercept + slope * x`. `label=` names it in the legend. `thickness=3` draws a thicker line (normal is 1.5). Ordinary (not log) axes only.

Options

`slope`	needed
`through=`	
`intercept=`	
`dashed=`	normally `True`
`colour=`	normally `"grey"`
`label=`	
`thickness=`	

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="hours")
chart.line_with_slope(1, through=(0, 0))
chart.save("line_with_slope.png")
```

`chart.shade_between_x()`

Shade the band between two x values, behind the data. `chart.shade_between_x(6, 8)`

Options

`low`	needed
`high`	needed
`colour=`	normally `"grey"`

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month_number", y="temperature", colour="city")
chart.shade_between_x(6, 8)
chart.save("shade_between_x.png")
```

`chart.shade_between_y()`

Shade the band between two y values, behind the data. `chart.shade_between_y(40, 60)`

Options

`low`	needed
`high`	needed
`colour=`	normally `"grey"`

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.shade_between_y(40, 60)
chart.save("shade_between_y.png")
```

Words on the chart

`chart.title_top()`

A title above the chart. `chart.title_top("My class marks")`

Options

`text`	needed

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.title_top("Marks in each class")
chart.save("title_top.png")
```

chart.caption_bottom()

A small line of text under the whole chart. `chart.caption_bottom("Data from 2026")`

Options

`text`	needed

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.caption_bottom("Made-up data from sdata.example")
chart.save("caption_bottom.png")
```

chart.label_x()

The name under the x-axis. `chart.label_x("Student")`

Options

`text`	needed

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.label_x("Hours of study")
chart.save("label_x.png")
```

chart.label_y()

The name beside the y-axis. `chart.label_y("Marks")`

Options

`text`	needed

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.label_y("Marks (out of 100)")
chart.save("label_y.png")
```

chart.note()

A note on the chart at a place x, y. `chart.note("Biggest", x=2024, y=49)`

`arrow=True` puts the note a little away, with an arrow pointing at x, y. On bar charts x can be a name: `chart.note("Best", x="Ben", y=90)` on bars of names.

Options

`text`	needed
`x=`	
`y=`	
`arrow=`	normally `False`

Underneath matplotlib

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.lines(x="year", y="population", colour="country")
chart.note("Cherria grows fastest", x=2020, y=12, arrow=True)
chart.save("note.png")
```

chart.numbers_on_bars()

Write the value of each bar on it. `chart.numbers_on_bars()` · `decimals=1` shows 12.5

Options

`decimals=`	normally `0`

Underneath matplotlib

Example you can run (it uses the practice table `fruit`)

```
import simplydata as sdata
fruit = sdata.example("fruit")
chart = sdata.chart(fruit)
chart.bars(x="fruit", y="count", show="total")
chart.numbers_on_bars()
chart.save("numbers_on_bars.png")
```

chart.names_on_dots()

Write a name next to each dot. `chart.names_on_dots("name")` (with `chart.dots(x="hours", y="marks")` on `sdata.example("marks")`)

column is the column whose words are written (or a list: one name for each dot). Each name goes where it covers no other name or dot: right of its dot, else left, above, below or in a corner. A name with no free place is left out, and a line says how many; a bigger chart (`chart.size(...)`) makes room. `colour="grey"` writes the names in that colour.

Options

`column`	needed
`colour=`	

Underneath matplotlib (annotate), with simplydata choosing a free place for each name

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("subject", equal_to="Maths")
chart = table.chart()
chart.dots(x="hours", y="marks")
chart.names_on_dots("name")
chart.save("names_on_dots.png")
```

chart.font_size()

Make all the text bigger or smaller. `chart.font_size(14)` (normal is 12)

Options

`size`	needed

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.title_top("Big text")
chart.font_size(18)
chart.save("font_size.png")
```

chart.font_name()

The font for all the text on the chart. `chart.font_name("DejaVu Serif")`

Any font installed on your computer, such as `"Arial"` or `"Times New Roman"`, or a kind of font: `"serif"`, `"sans-serif"` (normal) or `"monospace"`. If the font is not installed, the message lists the installed fonts with a similar name.

Options

`name`	needed

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.font_name("DejaVu Serif")
chart.save("font_name.png")
```

Axes

chart.range_x()

From where to where the x-axis goes. `chart.range_x(0, 10)`

Use None to fix only one end: `chart.range_x(0, None)` starts at 0 and lets the top choose itself.

Options

`low=`	
`high=`	

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.range_x(0, 10)
chart.save("range_x.png")
```

chart.range_y()

From where to where the y-axis goes. `chart.range_y(0, 100)`

Use None to fix only one end: `chart.range_y(None, 100)` stops at 100.

Options

```
low=
high=
```

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.range_y(0, 100)
chart.save("range_y.png")
```

chart.log_scale_x()

A log scale on the x-axis: 1, 10, 100, 1000 are evenly spaced. `chart.log_scale_x()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.dots(x="population", y="forest", colour="country")
chart.log_scale_x()
chart.save("log_scale_x.png")
```

chart.log_scale_y()

A log scale on the y-axis: 1, 10, 100, 1000 are evenly spaced. `chart.log_scale_y()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.lines(x="year", y="population", colour="country")
chart.log_scale_y()
chart.save("log_scale_y.png")
```

chart.ticks_x()

Where the marks on the x-axis go. `chart.ticks_x(1, 4, 7, 10)`

`labels=["Jan", "Apr", "Jul", "Oct"]` writes your own words at those places. `every=5` puts a tick every 5 units; for dates `every="month"` (or `"year"`, `"week"`, `"day"`, `"hour"`, `"3 months"` ...). `how_many=5` asks for about 5 ticks at round numbers; `whole_numbers=True` keeps them to whole numbers, for counts.

Options

`values...`	as many as you like, separated by commas
`labels=`	
`every=`	
`how_many=`	
`whole_numbers=`	normally `False`

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month_number", y="rain", colour="city")
chart.ticks_x(1, 4, 7, 10, labels=["Jan", "Apr", "Jul", "Oct"])
chart.save("ticks_x.png")
```

chart.ticks_y()

Where the marks on the y-axis go. `chart.ticks_y(0, 50, 100)`

`labels=["low", "mid", "high"]` writes your own words at those places. `every=10` puts a tick every 10 units (`every="month"` for dates); `how_many=5` asks for about 5 ticks at round numbers; `whole_numbers=True` keeps them to whole numbers.

Options

`values...`	as many as you like, separated by commas
`labels=`	
`every=`	
`how_many=`	
`whole_numbers=`	normally `False`

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.ticks_y(0, 25, 50, 75)
chart.save("ticks_y.png")
```

chart.rotate_labels_x()

Turn the names under the x-axis so long ones do not bump into each other.

```
chart.rotate_labels_x() turns them 45 degrees; chart.rotate_labels_x(90) stands them up.
```

Options

`angle=`	normally `45`

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.rotate_labels_x(45)
chart.save("rotate_labels_x.png")
```

chart.rotate_labels_y()

Turn the names beside the y-axis. `chart.rotate_labels_y(30)`

Options

`angle=`	normally `45`

Underneath matplotlib

Example you can run (it uses the practice table `fruit`)

```
import simplydata as sdata
fruit = sdata.example("fruit")
chart = sdata.chart(fruit)
chart.bars(x="fruit", y="count", sideways=True)
chart.rotate_labels_y(30)
chart.save("rotate_labels_y.png")
```

chart.tick_format_x()

How the numbers or dates under the x-axis are written. `chart.tick_format_x("thousands")`

`"percent"` adds a % sign (`out_of=1` if your numbers are shares, so 0.25 shows as 25 %); `"thousands"` writes 12,500; `"scientific"` writes 1.2×10^4; `"plain"` never uses 1e4. `decimals=1` sets the number of decimals. For dates, write how 31 January 2026 should look: `"Jan 2026"`, `"31.01.2026"`, `"2026-01-31"`, `"31 Jan"` or `"13:45"` (or a code such as `"%d %b"`).

Options

`style=`	
`decimals=`	
`out_of=`	normally `100`

Underneath matplotlib

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.dots(x="population", y="forest")
chart.tick_format_x("thousands")
chart.save("tick_format_x.png")
```

chart.tick_format_y()

How the numbers beside the y-axis are written. `chart.tick_format_y("percent")`

`"percent"` adds a % sign (`out_of=1` if your numbers are shares, so 0.25 shows as 25 %); `"thousands"` writes 12,500; `"scientific"` writes 1.2×10^4; `"plain"` never uses 1e4. `decimals=1` sets the number of decimals. Dates work as in `tick_format_x`.

Options

`style=`	
`decimals=`	
`out_of=`	normally `100`

Underneath matplotlib

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.lines(x="year", y="forest", colour="country")
chart.tick_format_y("percent")
chart.save("tick_format_y.png")
```

chart.second_y()

A second y-axis on the right, for a column measured in other units.

```
chart.lines(x="month", y="temperature")
chart.second_y("rain", drawn_as="bars", label="Rain (mm)")
```

(with `sdata.example("weather")`)

`drawn_as=` can be `"lines"`, `"bars"` or `"dots"`. It shares the x-axis with the chart before it.

Options

`y`	needed
`drawn_as=`	normally `"lines"`
`label=`	
`colour=`	

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature")
```

```
chart.second_y("rain", drawn_as="bars", label="Rain (mm)")
chart.save("second_y.png")
```

chart.flip_x()

Turn the x-axis around, so numbers grow from right to left. `chart.flip_x()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.flip_x()
chart.save("flip_x.png")
```

chart.flip_y()

Turn the y-axis upside down, for example for depth going down. `chart.flip_y()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `island`)

```
import simplydata as sdata
island = sdata.example("island")
chart = sdata.chart(island)
chart.contour(x="east", y="north", z="height")
chart.flip_y()
chart.save("flip_y.png")
```

Colours and look

chart.colour()

Also written `chart.color()`

One colour for the whole chart. `chart.colour("blue")` or `chart.colour("#0072B2")`

Options

`name`	needed

Good to know Inside a chart step, `colour=` does more: `colour="class"` gives each class its own colour (and a legend), and `colour="red"` makes that one step red.

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.colour("purple")
chart.save("colour.png")
```

chart.colours()

Also written `chart.colors()`

Your own colours for the groups, in order. `chart.colours("blue", "orange", "green")`

Options

`names...`	as many as you like, separated by commas

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="subject", y="marks", colour="class")
chart.colours("orange", "teal", "purple")
chart.save("colours.png")
```

chart.colour_scale()

Also written `chart.color_scale()`

The colours for numbers on a heatmap, contour map, surface or dots coloured by a number column.

`chart.colour_scale("magma")` (with `chart.heatmap(x="month", y="city", value="rain")` on `sdata.example("weather")`)

A name: `"viridis"` (normal), `"cividis"`, `"magma"`, `"blues"`, `"greens"` ..., `"blue to red"` or `"purple to orange"` for numbers on both sides of a middle; these are all safe for colour-blind readers. Any matplotlib colour map name works too (`"Blues"`, `"turbo"`, `"rocket"`), and two or more colours make your own: `chart.colour_scale("white", "navy")`. `middle=0` puts the middle colour of a two-sided scale at 0 (or `middle=1` for ratios); the scale goes equally far on both sides, so a step of 1 is the same change of colour. `log=True` spreads the colours on a log scale (1, 10, 100 ... evenly), for counts that span many sizes. `steps=5` cuts the scale into 5 colours, or `steps=[0, 5, 10, 20]` sets where each colour starts. `between=[0, 30]` fixes the ends of the scale, so several charts can share it; values beyond them get the end colour (the colour bar shows an arrow).

Options

`names...`	as many as you like, separated by commas
`middle=`	
`log=`	normally `False`
`steps=`	
`between=`	

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.heatmap(x="month", y="city", value="rain")
chart.colour_scale("magma")
chart.save("colour_scale.png")
```

chart.style()

The overall look. `chart.style("school")`

`"paper"` (default), `"school"`, `"presentation"` or `"dark"`.

Options

`name`	needed

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="subject", y="marks", colour="class")
chart.style("school")
chart.save("style.png")
```

chart.size()

The size of the chart. `chart.size(width=17, height=12, unit="cm")`

`unit=` is `"inches"` (normal), `"cm"`, `"mm"` or `"pixels"`. A saved file is exactly this size. In pixels, the saved picture has exactly that many pixels at any `resolution=`.

Options

`width=`	normally `8`
`height=`	normally `6`
`unit=`	normally `"inches"`

Good to know `unit=` is `"inches"` (the normal choice), `"cm"`, `"mm"` or `"pixels"`. A saved file is exactly this size. With `"pixels"` the picture has exactly that many pixels whatever the `resolution=` in `chart.save()`. Text keeps its size, so a small chart for a paper (about 8 cm wide) usually also needs smaller text: `chart.font_size(8)`.

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="rain", colour="city")
chart.size(width=17, height=8, unit="cm")
chart.save("size.png")
```

chart.show_grid()

Grid lines across the chart in both directions. `chart.show_grid()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.show_grid()
chart.save("show_grid.png")
```

chart.hide_grid()

No grid lines. `chart.hide_grid()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.hide_grid()
chart.save("hide_grid.png")
```

chart.legend_top()

The legend above the chart, in one row. `chart.legend_top()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature", colour="city")
chart.legend_top()
chart.save("legend_top.png")
```

chart.legend_bottom()

The legend below the chart, in one row under the x-axis name. `chart.legend_bottom()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature", colour="city")
chart.legend_bottom()
chart.save("legend_bottom.png")
```

chart.legend_left()

The legend to the left of the chart. `chart.legend_left()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature", colour="city")
chart.legend_left()
chart.save("legend_left.png")
```

chart.legend_right()

The legend to the right of the chart, outside the plot. `chart.legend_right()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature", colour="city")
chart.legend_right()
chart.save("legend_right.png")
```

chart.hide_legend()

No legend. `chart.hide_legend()`

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature", colour="city")
chart.hide_legend()
chart.save("hide_legend.png")
```

chart.show_legend()

The legend in its usual place, for example after `hide_legend()`. `chart.show_legend()`

A legend only appears when `colour=` (or `shape=`) makes groups.

Options none: just write the empty brackets.

Underneath matplotlib

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="temperature", colour="city")
chart.hide_legend()
chart.show_legend()
chart.save("show_legend.png")
```

chart.label_legend()

The name above the legend. `chart.label_legend("Class")` (with `chart.dots(x="hours", y="marks", colour="class")` on `sdata.example("marks")`)

Without it the legend is named after the `colour=` column. `label_legend("")` leaves the name out.

Options

`text`	needed

Underneath matplotlib

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks", colour="class")
chart.label_legend("Class")
chart.save("label_legend.png")
```

`chart.label_colour_bar()`

Also written `chart.label_color_bar()`

The name beside the colour bar of a heatmap or contour. `chart.label_colour_bar("Height (m)")`

Options

`text`	needed

Underneath matplotlib

Example you can run (it uses the practice table `island`)

```
import simplydata as sdata
island = sdata.example("island")
chart = sdata.chart(island)
chart.contour(x="east", y="north", z="height")
chart.label_colour_bar("Height (m)")
chart.save("label_colour_bar.png")
```

Order of groups

`chart.order_x()`

Put the groups along the x-axis in your order. `chart.order_x("C", "B", "A")` (with `chart.bars(x="class", y="marks")` on `sdata.example("marks")`)

Groups you leave out come after the ones you name. `biggest_first=True` puts the biggest bar (or box, or line point) first, `biggest_first=False` the smallest; the size is what the chart shows (average, median, count ...). Month and weekday names (`"Jan"`, `"Monday"`) are put in calendar order without asking.

Options

`names...`	as many as you like, separated by commas
`biggest_first=`	

Underneath pandas (a stable sort of the rows, so every chart type, colour and panel follows the order)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.order_x("C", "B", "A")
chart.save("order_x.png")
```

chart.order_y()

Put the groups along the y-axis in your order, the first at the top.
`chart.order_y("C", "B", "A")` (with `chart.bars(x="class", y="marks", sideways=True)` on `sdata.example("marks")`)

Works like `order_x`, for sideways bars, sideways boxes and heatmap rows.

Options

`names...`	as many as you like, separated by commas
`biggest_first=`	

Underneath pandas (a stable sort of the rows, so every chart type, colour and panel follows the order)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks", sideways=True)
chart.order_y("C", "B", "A")
chart.save("order_y.png")
```

chart.order_legend()

Put the colour groups in your order: in the legend, the colours and side by side.
`chart.order_legend("Science", "Maths", "English")`
(with `chart.bars(x="class", y="marks", colour="subject")` on `sdata.example("marks")`)

Groups you leave out come after the ones you name. The slices of a pie follow it too. The first group gets the first colour.

Options

`names...`	as many as you like, separated by commas

Underneath pandas (a stable sort of the rows, so every chart type, colour and panel follows the order)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks", colour="subject")
chart.order_legend("Science", "Maths", "English")
chart.save("order_legend.png")
```

Several charts

chart.one_chart_per()

One small chart for each group. `chart.one_chart_per("class", per_row=3)` (with `sdata.example("marks")`)

`per_row=3` puts 3 small charts in each row. `down="class"`, `across="subject"` makes a table of charts: the classes go down (one row of charts each) and the subjects go across (one column of charts each). `same_x=False` / `same_y=False` lets each small chart choose its own axis range.

Options

`by=`	
`down=`	
`across=`	
`per_row=`	
`same_x=`	normally `True`
`same_y=`	normally `True`

Underneath matplotlib (a grid of subplots)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.one_chart_per("class", per_row=3)
chart.save("one_chart_per.png")
```

sdata.side_by_side()

Charts next to each other. `sdata.side_by_side(a, b).save("both.png")`

`same_y=True` gives them all the same y range, so heights can be compared. `letters=True` writes A, B, C ... at the top left of each chart, as in papers.

Options

`charts...`	as many as you like, separated by commas
`same_x=`	normally `False`
`same_y=`	normally `False`
`letters=`	normally `False`

Underneath matplotlib (subplots)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
a = sdata.chart(marks).bars(x="class", y="marks").title_top("Average")
b = sdata.chart(marks).boxes(x="class", y="marks").title_top("Spread")
page = sdata.side_by_side(a, b, same_y=True)
page.save("side_by_side.png")
```

sdata.one_above_other()

Charts on top of each other. `sdata.one_above_other(a, b).save("both.png")`

`same_x=True` gives them all the same x range, which suits time series. `letters=True` writes A, B, C ... at the top left of each chart, as in papers.

Options

`charts...`	as many as you like, separated by commas
`same_x=`	normally `False`
`same_y=`	normally `False`
`letters=`	normally `False`

Underneath matplotlib (subplots)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
weather = sdata.example("weather")
a = sdata.chart(weather).lines(x="month", y="temperature", colour="city")
b = sdata.chart(weather).bars(x="month", y="rain", colour="city")
page = sdata.one_above_other(a, b)
page.save("one_above_other.png")
```

sdata.grid()

Charts in rows and columns. `sdata.grid(a, b, c, d, per_row=2)`

`per_row=2` puts 2 charts in each row. `letters=True` writes A, B, C ... at the top left of each chart, as figures in papers do.

Options

`charts...`	as many as you like, separated by commas
`per_row=`	normally `2`
`same_x=`	normally `False`
`same_y=`	normally `False`
`letters=`	normally `False`

Underneath matplotlib (subplots)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
a = sdata.chart(marks).bars(x="class", y="marks")
b = sdata.chart(marks).histogram(x="marks")
c = sdata.chart(marks).dots(x="hours", y="marks")
d = sdata.chart(marks).boxes(x="subject", y="marks")
page = sdata.grid(a, b, c, d, per_row=2)
page.title_top("Four ways to look at the marks")
page.save("grid.png")
```

chart.zoom_box()

A small box showing part of the chart close up.

`chart.zoom_box(x_between=[2, 4], y_between=[40, 60])` (with `sdata.example("marks")`)
`x_between=` and `y_between=` are the part to show: [low, high] on each axis. `where="top right"` (default), `"top left"`, `"bottom right"` or `"bottom left"`.

Options

`x_between=`	
`y_between=`	
`where=`	normally `"top right"`

Underneath matplotlib (`inset_axes` and `indicate_inset_zoom`)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.zoom_box(x_between=[2, 4], y_between=[40, 60], where="top left")
chart.save("zoom_box.png")
```

chart.animate()

A moving chart with one picture for each value of a column.

`chart.animate(over="year")` (with `sdata.example("growth")`), then `chart.save("growth.gif")`
`keep=True` builds the chart up: each picture also keeps the values before it.

Options

`over`	needed
`seconds_per_frame=`	normally `0.8`
`keep=`	normally `False`

Underneath matplotlib (animation) and Pillow (the `.gif` file)

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.bars(x="country", y="population")
chart.range_y(0, 50)
chart.animate(over="year")
chart.save("growth.gif")
```

Table words

Steps on a table: `table = sdata.table(...)`, then `table.keep_rows(...)`. Table words work the same on pandas and Polars tables. Words that start with `sdata.` (finding and solving, planning a study) are called on simplydata itself, and words that start with `dist.` are steps on a distribution: `dist = sdata.distribution("normal")`.

Tables: start and look

`sdata.table()`

Start a table.

`sdata.table(sdata.example("marks"))` a practice table
`sdata.table("my_file.xlsx")` a file (Excel, CSV, TSV, Parquet, JSON; Word, SPSS, Stata)
`sdata.table("my_file.xlsx", sheet="2026")` one sheet of an Excel file; `sheet="all"` stacks every sheet
`sdata.table("data/*.csv")` several files stacked, with a `"file"` column (or a folder: `"data/"`)
`sdata.table("https://example.org/data.csv")` a file on the web
`fast=True` use the fast engine (Polars), for big data

Reading options, for files whose values are written differently:
`separator=";"` the character between values (`"tab"` for tabs)
`decimal_mark=","` 3,5 means three and a half
`thousands_mark="."` 1.000 means one thousand
`empty_values=["NA", "-"]` markers that mean `"no value"`
`skip_rows=2` skip lines above the column names
`encoding="latin-1"` for text files not saved as UTF-8
`table_number=2` which table in a Word file (`"all"` stacks them)

Options

`data=`	
`sheet=`	
`fast=`	normally `False`
`separator=`	
`decimal_mark=`	
`thousands_mark=`	
`empty_values=`	
`skip_rows=`	
`encoding=`	
`table_number=`	

Underneath pandas or Polars read the file; narwhals lets every table word run on both

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.show_first_rows()
```

table.show_first_rows()

Show the first rows, numbered from 1. `table.show_first_rows(5)`

Options

`rows=`	normally `5`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.show_first_rows(10)
```

table.show_last_rows()

Show the last rows, with their row numbers in the whole table. `table.show_last_rows(5)`

The last 3 rows of a table of 54 rows are numbered 52, 53 and 54, the numbers that `table.keep_rows_by_position(52, to=54)` takes.

Options

`rows=`	normally `5`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.show_last_rows(3)
```

table.show()

Show the table, every row and every column (or only the first `rows=`).

`table.show()` `table.show(rows=10)` (with `sdata.example("marks")`)
A table with more than 1,000 rows shows its first 1,000; choose another number with `rows=`. Rows are numbered from 1, the numbers `keep_rows_by_position()` takes.

Options

`rows=`	

Underneath pandas or Polars (printing the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("class", equal_to="A")
table.show(rows=10)
```

`table.column_names()`

The names of all columns. `table.column_names()`

Options none: just write the empty brackets.

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
print(table.column_names())
```

`table.describe_columns()`

Describe every column: its kind, empty cells, smallest, biggest and average.

`table.describe_columns()` (with `sdata.example("marks")`)

Options none: just write the empty brackets.

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.describe_columns()
```

`table.column()`

One column on its own: a column of numbers becomes `sdata.numbers`, any other a plain list.

`marks = table.column("marks")` (with `sdata.example("marks")`)
then `marks.average()`, `marks.chart()` … like any `sdata.numbers`. Empty cells become empty numbers. The table does not change.

Options

`name=`

Underneath NumPy (a column of numbers becomes `sdata.numbers`)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
marks = table.column("marks")
marks.average()
marks.percentile(90)
```

table.row()

One row as a plain dict of column: value, counting from 1. `table.row(3)`

`table.row(3)` (with `sdata.example("marks")`)
`table.row("last")` is the last row. Empty cells become None. The table does not change.
As pandas df.iloc[2].to_dict() and Polars `df.row(2, named=True)`.

Options

`position=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
print(table.row(3))
print(table.row("last"))
```

table.cell_at()

The value in one cell: a row counting from 1 and a column by name. `table.cell_at(row=3, column="marks")`

`table.cell_at(row=3, column="marks")` (with `sdata.example("marks")`)
`row="last"` is the last row. An empty cell gives None, with a note. The table does not change. As pandas df.at[2, `"marks"`] and Polars `df.item(2, "marks")`.

Options

`row=`
`column=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
print(table.cell_at(row=3, column="marks"))
print(table.cell_at(row="last", column="name"))
```

Choose columns

table.keep_columns()

Keep only these columns: by name (in the order you list them), a run of names, or by kind.

`table.keep_columns("name", "marks")` (with `sdata.example("marks")`)
`table.keep_columns("hours", to="height")` keeps a run of columns, both ends included; `kind="number"` (or `"text"`, `"date"`, `"true/false"`, the kinds `describe_columns()` prints) keeps every column of that kind; `starts_with=`, `ends_with=`, `contains=` (plain text, capital letters matter) and `pattern=` (a regular expression) choose by the column names. One way at a time; two steps one after the other keep the columns that meet both.

Options

`names...`	as many as you like, separated by commas
`to=`	
`kind=`	
`starts_with=`	
`ends_with=`	
`contains=`	
`pattern=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_columns("name", "marks")
table.show_first_rows()
table.start_again()
table.keep_columns("hours", to="height")
table.start_again()
table.keep_columns(kind="number")
table.show_first_rows()
```

table.remove_columns()

Remove these columns: by name, a run of names, or by kind.

`table.remove_columns("height")` (with `sdata.example("marks")`)
The same ways as `keep_columns`: `remove_columns("hours", to="marks")`, `remove_columns(kind="text")`, `remove_columns(starts_with="wind")`, `remove_columns(pattern="^wind_")`.

Options

`names...`	as many as you like, separated by commas
`to=`	
`kind=`	
`starts_with=`	
`ends_with=`	
`contains=`	
`pattern=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.remove_columns("height")
print(table.column_names())
table.remove_columns(kind="text")
print(table.column_names())
```

table.keep_columns_by_position()

Keep columns by their position, counting from 1 at the left; it says which columns they were.

`table.keep_columns_by_position(1, to=3)` (with `sdata.example("marks")`)
Both ends are included. `"last"` is the last column; `step=2` keeps every 2nd column; a list such as [3, 1] keeps those columns in that order. As pandas df.iloc[:, 0:3].

Options

```
positions=
to=
step=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_columns_by_position(1, to=3)
table.show_first_rows()
```

table.remove_columns_by_position()

Remove columns by their position, counting from 1 at the left; it says which columns they were.

`table.remove_columns_by_position(1)` (with `sdata.example("marks")`)
Handy for the `"Unnamed: 0"` column a CSV file often brings. Same positions as `keep_columns_by_position`.

Options

```
positions=
to=
step=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.remove_columns_by_position(1)
print(table.column_names())
```

`table.rename_column()`

Give a column a new name.

`table.rename_column("marks", to="score")` (with `sdata.example("marks")`)

Options

```
old=
to=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.rename_column("marks", to="score")
table.show_first_rows()
```

Choose rows

`table.keep_rows()`

Keep only the rows that match a condition.

`table.keep_rows("marks", above=50)` (with `sdata.example("marks")`)
`table.keep_rows("marks > 50 and class == 'A'")` a written condition

Plain words: above, below, `at_least`, `at_most`, `equal_to`, `not_equal_to`, `between=[low, high]` (both ends included), `one_of=[...]`, contains, `starts_with`, `ends_with` (capital letters matter), `is_empty=True`, and `after=` / `before=` for dates. Written conditions use == != < <= > >=, and, or, not, brackets, in [...], is empty; text and dates go in quotes. Empty cells only match `is_empty` / is empty.

Options

```
column=
above=
below=
at_least=
at_most=
equal_to=
not_equal_to=
between=
one_of=
contains=
starts_with=
ends_with=
is_empty=
after=
before=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
```

```
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks", above=50)
table.keep_rows("subject", one_of=["Maths", "Science"])
table.keep_rows("hours > 5 and class != 'C'")
table.show_first_rows()
```

table.remove_rows()

Remove the rows that match a condition (the opposite of `keep_rows`).

`table.remove_rows("subject", equal_to="English")` (with `sdata.example("marks")`)
`table.remove_rows("marks < 40 or hours < 2")` a written condition

Same conditions as `keep_rows`; `between=[low, high]` includes both ends. A row whose cell is empty is never removed by a comparison on that cell (use `is_empty=True`).

Options

```
column=
above=
below=
at_least=
at_most=
equal_to=
not_equal_to=
between=
one_of=
contains=
starts_with=
ends_with=
is_empty=
after=
before=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.remove_rows("subject", equal_to="English")
table.show_first_rows()
```

table.keep_first_rows()

Keep only the first rows.

`table.keep_first_rows(10)` (with `sdata.example("marks")`)

Options

`n=`	normally `5`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
```

```
table.keep_first_rows(10)
print(table)
```

table.keep_last_rows()

Keep only the last rows.

`table.keep_last_rows(10)` (with `sdata.example("marks")`)

Options

`n=` normally `5`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_last_rows(10)
print(table)
```

table.keep_rows_by_position()

Keep rows by their position, counting from 1: a run, every n-th row, or a list.

`table.keep_rows_by_position(10, to=20)` (with `sdata.example("marks")`)
Both ends are included, so rows 10 to 20 are 11 rows. `to="last"` goes to the end; `step=10` keeps every 10th row (1, 11, 21 ...); a list such as [1, 5, 9] keeps those rows in that order. The numbers are the ones `table.show()` prints, as pandas df.iloc[9:20].

Options

`positions=`
`to=`
`step=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.show_last_rows(3)
table.keep_rows_by_position(10, to=20)
table.keep_rows_by_position(1, step=2)
table.show()
```

table.remove_rows_by_position()

Remove rows by their position, counting from 1: a run, every n-th row, or a list.

`table.remove_rows_by_position([12, 40])` (with `sdata.example("marks")`)
The opposite of `keep_rows_by_position`, with the same positions: `remove_rows_by_position(1)` removes the first row (a second header read as data), (2, `step=2`) every 2nd row.

Options

`positions=`
`to=`
`step=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.remove_rows_by_position([12, 40])
table.remove_rows_by_position(1, step=2)
table.count_rows()
```

table.reverse_rows()

Turn the table upside down: the last row comes first. `table.reverse_rows()`

`table.reverse_rows()` (with `sdata.example("marks")`)
For an order by the values in a column use `table.sort_by("marks", biggest_first=True)`.

Options none: just write the empty brackets.

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.reverse_rows()
table.show_first_rows(3)
```

table.keep_random_rows()

Keep some rows picked at random; give a seed to get the same rows every time.

`table.keep_random_rows(10, seed=1)` (with `sdata.example("marks")`)

Options

`n=` normally `5`
`seed=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_random_rows(10, seed=1)
print(table)
```

table.remove_duplicate_rows()

Remove rows that repeat an earlier row, or that repeat it in the columns you name.

`table.remove_duplicate_rows(columns=["name"])` (with `sdata.example("marks")`)

Options

`columns=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.remove_duplicate_rows(columns=["name"])
print(table)
```

table.keep_biggest_rows()

Keep the n rows with the biggest values in a column (also for each group).

`table.keep_biggest_rows(3, by="marks", for_each="subject")` (with `sdata.example("marks")`)
The rows come biggest first. Exactly n rows are kept: when rows tie at the cut, the ones higher up in the table are kept (a note says so). Empty cells are never kept.

Options

`n=`
`by=`
`for_each=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_biggest_rows(3, by="marks", for_each="subject")
table.show()
```

table.keep_smallest_rows()

Keep the n rows with the smallest values in a column (also for each group).

`table.keep_smallest_rows(3, by="marks", for_each="subject")` (with `sdata.example("marks")`)
The rows come smallest first. Exactly n rows are kept: when rows tie at the cut, the ones higher up in the table are kept (a note says so). Empty cells are never kept.

Options

`n=`
`by=`
`for_each=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_smallest_rows(3, by="marks", for_each="subject")
table.show()
```

table.show_rows_with_outliers()

Show the rows whose value in a column is an outlier, and say why (the table does not change).

`table.show_rows_with_outliers("marks", for_each="subject")` (with `sdata.example("marks")`)
`rule="1.5 IQR"` (the default) is Tukey's rule, the one box plots use: below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR. Also `rule="3 IQR"` or `rule="3 standard deviations"`. Empty cells are never outliers. Each row is shown with its number in the whole table (counting from 1), so `remove_rows_by_position()` can follow.

Options

`column=`
`for_each=`
`rule=` normally `"1.5 IQR"`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.show_rows_with_outliers("marks", for_each="class")
```

table.remove_rows_with_outliers()

Remove the rows whose value in a column is an outlier, and say how many and why.

`table.remove_rows_with_outliers("marks", for_each="subject")` (with `sdata.example("marks")`)
`rule="1.5 IQR"` (the default) is Tukey's rule, the one box plots use: below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR. Also `rule="3 IQR"` or `rule="3 standard deviations"`. Rows with an empty cell stay. Look first with `show_rows_with_outliers()`, and say in your report which rule removed how many rows.

Options

`column=`
`for_each=`
`rule=` normally `"1.5 IQR"`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.remove_rows_with_outliers("marks", for_each="subject")
table.steps()
```

Empty cells

`table.count_empty_cells()`

Show how many empty cells each column has.

`table.count_empty_cells()` (with `sdata.example("shop")`)

Options none: just write the empty brackets.

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `shop`)

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.count_empty_cells()
```

`table.remove_rows_with_empty_cells()`

Remove rows with an empty cell (in any column, or only in the column you name).

`table.remove_rows_with_empty_cells("sales")` (with `sdata.example("shop")`)

Options

`column=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `shop`)

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.remove_rows_with_empty_cells("sales")
print(table)
```

table.fill_empty_cells()

Fill the empty cells of a column with a value, or with the average, median, value above or value below.

`table.fill_empty_cells("customers", with_the="median")` (with `sdata.example("shop")`)
`with_the="line between neighbours"` fills each gap on the straight line between the filled cells just before and just after it (linear interpolation). It follows the rows in table order, or `sorted_by=` a column of numbers or dates (then the line is spaced by those values, so a missing day counts as one day); `for_each=` fills each group on its own. Empty cells at the start or end have only one neighbour, so they stay empty.
`table.fill_empty_cells("sales", with_the="line between neighbours", sorted_by="date", for_each="shop")`
`with_the="curve between neighbours"` fills them on a smooth curve through the filled cells instead (a natural cubic spline, as R zoo::`na.spline(method = "natural")`), and `with_the="curve without overshoot"` on a smooth curve that never goes above or below its neighbours (PCHIP); both take `sorted_by=` and `for_each=` the same way.

Options

`column=`
`with_value=`
`with_the=`
`sorted_by=`
`for_each=`

Underneath narwhals; NumPy (interp) for the line between neighbours; SciPy (CubicSpline, PchipInterpolator) for the curves

Example you can run (it uses the practice table `shop`)

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.fill_empty_cells("customers", with_the="median")
table.fill_empty_cells("sales", with_the="line between neighbours", sorted_by="date",
    ↪ for_each="shop")
table.count_empty_cells()
```

table.add_is_empty_column()

Add a true/false column: True where a cell of `from_column` is empty.

Add it before filling empty cells, to keep track of which values were filled in.
`table.add_is_empty_column("sales_was_empty", from_column="sales")` (with `sdata.example("shop")`)

Options

`name=`
`from_column=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `shop`)

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.add_is_empty_column("sales_was_empty", from_column="sales")
```

```
table.fill_empty_cells("sales", with_the="line between neighbours", sorted_by="date",
    ↪ for_each="shop")
table.keep_rows("sales_was_empty")
table.show()
```

Fix columns

`table.make_numbers()`

Turn a column into numbers; cells that are not numbers become empty.

`table.make_numbers("marks")` (with `sdata.example("marks")`)
Understands 12.5, -3, 1e3, 1 200 and 1,200 (thousands) and 12,5 (decimal comma).

Options

`column=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.make_numbers("marks")
table.describe_columns()
```

`table.make_text()`

Turn a column into text.

`table.make_text("month_number")` (with `sdata.example("weather")`)

Options

`column=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.make_text("month_number")
table.describe_columns()
```

table.make_dates()

Turn a column into dates; cells that are not dates become empty.

`table.make_dates("year")` (with `sdata.example("growth")`)
Understands 2026-03-15, 15/03/2026, 15.03.2026, 15 March 2026, March 15, 2026, 2026-03 and 2026. 03/04/2026 is read day first, as 3 April (month first only when day first cannot work). `order=` says how the dates are written, so nothing is guessed: `"day month year"` (15/03/2026), `"month day year"` (03/15/2026) or `"year month day"` (2026/03/15).

```
table.make_dates("when", order="month day year")
```

Options

```
column=
order=
```

Underneath pandas (`to_datetime`) or Polars (str.strptime), trying the date formats in a fixed order

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
table = sdata.table(sdata.example("growth"))
table.make_dates("year")
table.show_first_rows()
```

table.make_lowercase()

Write all the text in a column in small letters.

`table.make_lowercase("name")` (with `sdata.example("marks")`)

Options

```
column=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.make_lowercase("name")
table.show_first_rows()
```

table.make_uppercase()

Write all the text in a column in capital letters.

`table.make_uppercase("name")` (with `sdata.example("marks")`)

Options

```
column=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.make_uppercase("name")
table.show_first_rows()
```

table.remove_extra_spaces()

Remove spaces at the start and end of the text, and double spaces inside it.

`table.remove_extra_spaces("name")` (with `sdata.example("marks")`)

Options

`column=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.remove_extra_spaces("name")
table.show_first_rows()
```

table.replace_values()

Replace a value (or a list of values) in a column; `with_value=None` makes those cells empty.

`table.replace_values("subject", "Maths", with_value="Math")` (with `sdata.example("marks")`)
`table.replace_values("class", {"A": "Class A", "B": "Class B"})` several at once, old: new
`table.replace_values("marks", above=90, with_value=90)` every value that meets a condition

With {old: new}, every cell is looked up once in the original column, so `{"A": "B", "B": "A"}` swaps A and B. A new value of None makes those cells empty. The conditions are the `keep_rows` ones, one at a time: above, below, `at_least`, `at_most`, `equal_to`, `not_equal_to`, between, `one_of`, contains, `starts_with`, `ends_with`, after, before. Empty cells never match (fill them with `fill_empty_cells`).

Options

`column=`
`old=`
`with_value=`
`above=`
`below=`
`at_least=`
`at_most=`
`equal_to=`
`not_equal_to=`
`between=`
`one_of=`
`contains=`
`starts_with=`
`ends_with=`
`after=`
`before=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.replace_values("subject", "Maths", with_value="Math")
table.count_each("subject")
table.replace_values("marks", above=90, with_value=90)
table.biggest_of("marks")
```

`table.round_to()`

Round numbers to this many decimal places: in one column, a list of columns, or every number column.

```
table.round_to("hours", 0) table.round_to(["hours", "marks"], 1) table.round_to(1)
```

(with `sdata.example("marks")`) A number exactly halfway goes to the even neighbour (2.5 becomes 2, 3.5 becomes 4), like R and NumPy; this stops a run of halves from always pushing totals up. `round_to(0)` gives whole numbers; a negative number of places rounds to tens (-1), hundreds (-2).

Options

```
column=
decimals=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.round_to("hours", 0)
table.show_first_rows()
```

`table.change_column()`

Change the values of a column in place, with the same ways as `add_column()`.

The column keeps its name and its place in the table; `undo()` brings the old values back.

`table.change_column("marks", "marks / 10")` (with `sdata.example("marks")`)
`table.change_column("marks", from_column="marks", plus=5)`
`table.change_column("marks", "marks - average(marks)", for_each="class")`

Options

```
column=
formula=
by_adding=
by_subtracting=
by_multiplying=
by_dividing=
by_joining_text=
separator=
by_running=
for_each=
from_column=
plus=
minus=
times=
divide_by=
value=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.change_column("marks", "marks / 10")
table.show_first_rows()
```

Sort and new columns

`table.sort_by()`

Put the rows in order of one or more columns; empty cells go last.

Numbers go smallest first and text goes A to Z. `biggest_first=True` turns this round: biggest number first, and text Z to A. Rows that tie keep the order they had.

`table.sort_by("marks", biggest_first=True)` (with `sdata.example("marks")`)

Options

`columns...`	as many as you like, separated by commas
`biggest_first=`	normally `False`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
```

```
table = sdata.table(sdata.example("marks"))
table.sort_by("marks", biggest_first=True)
table.show_first_rows()
```

table.add_column()

Add a new column, worked out from other columns in plain words or with a formula.

Choose one way:

`table.add_column("study_and_marks", by_adding=["hours", "marks"])`
`table.add_column("half", from_column="marks", divide_by=2)`
`table.add_column("school", value="Hill School")` same value in every row
`table.add_column("marks_per_hour", "marks / hours")` calculator style
`table.add_column("name_and_class", by_joining_text=["name", "class"], separator=" ")`

(all with `sdata.example("marks")`). `by_subtracting=["a", "b"]` means a - b and `by_dividing=["a", "b"]` means a / b, in the order written. Dividing by zero gives an empty cell. In a formula, a column name with spaces goes in backticks: `"‘max temp‘ - 2"`.

Days between two dates: `by_subtracting=["end", "start"]` with two date columns gives the number of whole days from start to end (a part of a day is not counted).

Summary words in a formula, `average()` `median()` `total()` `smallest()` `biggest()` `count()`, give one value for the whole column, or for each group with `for_each=`:

`table.add_column("share", "marks / total(marks) * 100", for_each="class")`

gives each mark as a percent of its class total (with `sdata.example("marks")`).

`by_running=` runs a function you wrote once for each row. This is ordinary Python: the function gets the row (row["marks"] is that row's mark; an empty cell is None) and gives back the new value. def `per_hour(row)`: return row["marks"] / row["hours"]

`table.add_column("per_hour", by_running=per_hour)` (with `sdata.example("marks")`)

Options

`name=`
`formula=`
`by_adding=`
`by_subtracting=`
`by_multiplying=`
`by_dividing=`
`by_joining_text=`
`separator=`
`by_running=`
`for_each=`
`from_column=`
`plus=`
`minus=`
`times=`
`divide_by=`
`value=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_column("study_and_marks", by_adding=["hours", "marks"])
```

```
table.add_column("marks_per_hour", "marks / hours")
table.show_first_rows()
```

table.add_group_column()

Add a column that puts each row into a group, like turning marks into grades.

Two ways. With numbers: `cut_at=` says where one group ends and the next begins; `names=` has one more name than `cut_at=` has numbers. A value exactly on a cut goes into the higher group: with `cut_at=[40, 60, 80]`, a mark of 60 is a `"B"`. Empty cells stay empty.

```
table.add_group_column("grade", from_column="marks", cut_at=[40, 60, 80], names=["D", "C", "B", "A"])
```

With rules: each rule is a condition and the group name for rows that meet it. The first rule a row meets wins; rows that meet no rule get `otherwise=` (or stay empty). An empty cell never meets a rule.

```
table.add_group_column("result", rules={"marks >= 80": "top", "marks >= 50": "pass"}, otherwise="fail")
```

(both with `sdata.example("marks")`). Rules can also be a list of pairs: `rules=[("marks >= 80", "top"), ("marks >= 50", "pass")]`.

Options

```
name=
from_column=
cut_at=
names=
rules=
otherwise=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_group_column("grade", from_column="marks", cut_at=[40, 60, 80], names=["D", "C", "B
    ↪ ", "A"])
table.count_each("grade")
```

table.add_rank_column()

Add a column with each row's place, where 1 is the best.

The biggest value gets place 1 (`biggest_first=False` gives 1 to the smallest). Rows that tie share the best place and the next place is skipped, as in sports tables: 1, 2, 2, 4. Empty cells get no place.

`table.add_rank_column("place", from_column="marks")` (with `sdata.example("marks")`)

Options

```
name=
from_column=
biggest_first=                    normally True
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_rank_column("place", from_column="marks")
table.sort_by("place")
table.show_first_rows()
```

Dates and text

`table.add_date_part_column()`

Add a column with one part of each date, such as its year, month or weekday.

`part=` can be `"year"`, `"quarter"` (1 to 4), `"month"` (1 to 12), `"month name"` (`"March"`), `"week"` (the ISO week number, 1 to 53), `"day"` (day of the month), `"weekday"` (1 is Monday ... 7 is Sunday, as in the ISO 8601 standard), `"weekday name"` (`"Monday"`), `"day of year"` (1 to 366), `"hour"` or `"minute"`. Empty dates give empty cells.

`table.add_date_part_column("weekday", from_column="date", part="weekday name")`
(with `sdata.example("shop")`: which weekday was each day?)

Options

`name=`
`from_column=`
`part=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `shop`)

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.add_date_part_column("weekday", from_column="date", part="weekday name")
table.average_of("sales", for_each="weekday")
```

`table.add_text_part_column()`

Add a column with one part of each text, such as its first letters or the text before a mark.

Choose one way:

`first_characters=3` the first 3 characters (letters, digits, spaces and signs)
`last_characters=3` the last 3 characters
`characters=[2, 4]` characters 2 to 4, counting from 1
`before=" "` the text before the first `" "`
`after=" "` the text after the first `" "`
`pattern=r"\d+"` the first piece that fits a regular expression; with brackets ( ) in it, only the part inside the first brackets Cells where the mark or pattern is not found give empty cells.

`table.add_text_part_column("initial", from_column="name", first_characters=1)` (with `sdata.example("marks")`)

Options

```
name=
from_column=
first_characters=
last_characters=
characters=
before=
after=
pattern=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_text_part_column("initial", from_column="name", first_characters=1)
table.show_first_rows()
```

`table.split_column()`

Split a text column into several columns at a separator, such as a comma or a space.

`into=` names the new columns; they take the place of the old column. The last new column keeps the rest of the text when there are more pieces. Spaces at the ends of each piece are removed, and missing pieces give empty cells.

`table.split_column("shop", separator=" ", into=["shop_kind", "shop_word"])` (with `sdata.example("fruit")`)

Options

```
column=
separator=                    normally " "
into=
```

Underneath pandas (str.split) or Polars (str.splitn), whichever holds the table

Example you can run (it uses the practice table `fruit`)

```
import simplydata as sdata
table = sdata.table(sdata.example("fruit"))
table.split_column("shop", separator=" ", into=["shop_kind", "shop_word"])
table.show_first_rows()
```

table.replace_text()

Replace a piece of text inside the cells of a text column, wherever it appears.

The old text is plain text and capitals matter. `with_value=""` removes it.
`table.replace_text("shop", " shop", with_value="")` (with `sdata.example("fruit")`)
`pattern=` takes a regular expression instead of plain text (a pattern language for text: \d means a digit, \s a space, + `"one or more"`):
`table.replace_text("shop", pattern=r"\s+", with_value="_")`

Options

`column=`
`old=`
`with_value=`
`pattern=`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `fruit`)

```
import simplydata as sdata
table = sdata.table(sdata.example("fruit"))
table.replace_text("shop", " shop", with_value="")
table.count_each("shop")
```

Row after row

table.add_rolling_column()

Add a rolling (moving) summary: for each row, the average of this row and the rows just before it.

`window=` is how many rows go into each summary (this row and the window - 1 rows before it). The first rows of each group are empty until there are enough rows, and a window with an empty cell in it gives an empty cell. `show=` can be `"average"`, `"total"`, `"median"`, `"smallest"`, `"biggest"` or `"standard deviation"`. `sorted_by=` works it out in the order of a column such as a date (the rows themselves keep their place).
`table.add_rolling_column("rain_3_months", from_column="rain", window=3, show="total", for_each="city", sorted_by="month_number")` (with `sdata.example("weather")`)

Options

`name=`
`from_column=`
`window=` normally `3`
`show=` normally `"average"`
`for_each=`
`sorted_by=`

Underneath narwhals (`rolling_mean`, `rolling_sum`, `rolling_std` with `over(order_by=)`); pandas or Polars rolling for median, smallest and biggest

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
```

```
table.add_rolling_column("rain_3_months", from_column="rain", window=3, show="total",
    ↪ for_each="city", sorted_by="month_number")
table.show_first_rows()
```

table.add_previous_value_column()

Add a column with the value from the row before (or `rows_back=` rows before).

The first row of each group has no row before it, so it gets an empty cell.

`table.add_previous_value_column("last_month", from_column="temperature", for_each="city", sorted_by="month_number")` (with `sdata.example("weather")`)

Options

`name=`	
`from_column=`	
`rows_back=`	normally `1`
`for_each=`	
`sorted_by=`	

Underneath narwhals (shift, `rolling_mean` and over with `order_by=`, so each group keeps its own rows)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.add_previous_value_column("last_month", from_column="temperature", for_each="city",
    ↪ sorted_by="month_number")
table.show_first_rows()
```

table.add_change_column()

Add a column with the change from the row before: this value minus the previous one.

`in_percent=True` gives the change as a percent of the previous value, (this - previous) / previous * 100; a previous value of 0 gives an empty cell. The first row of each group has nothing to compare with, so it is empty.

`table.add_change_column("growth", from_column="population", in_percent=True, for_each="country", sorted_by="year")` (with `sdata.example("growth")`)

Options

`name=`	
`from_column=`	
`in_percent=`	normally `False`
`rows_back=`	normally `1`
`for_each=`	
`sorted_by=`	

Underneath narwhals (shift, `rolling_mean` and over with `order_by=`, so each group keeps its own rows)

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
table = sdata.table(sdata.example("growth"))
```

```
table.add_change_column("growth", from_column="population", in_percent=True, for_each="
    ↪ country", sorted_by="year")
table.show_first_rows()
```

Summaries

table.count_rows()

How many rows the table has, or how many in each group.

`table.count_rows(for_each="class")` (with `sdata.example("marks")`)
`across=` gives a cross table (pandas crosstab), one column per value:
`table.count_rows(for_each="shop", across="fruit")` (with `sdata.example("fruit")`)
`in_percent=True` gives percents (0 to 100) instead: of all rows, or with `across=` of each row of the cross table, so every row adds up to 100 %. `every="month"` with a date column in `for_each=` counts the rows of each month.

Options

`for_each=`	
`across=`	
`every=`	
`in_percent=`	normally `False`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.count_rows()
table.count_rows(for_each="class")
```

table.count_each()

Each different value in a column, how many times it appears, and its percent.

Biggest count first. Empty cells count as a value of their own, so the percents are out of all rows. Percents are rounded to 1 decimal place.
`table.count_each("fruit")` (with `sdata.example("fruit")`)

Options

`column`	needed

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `fruit`)

```
import simplydata as sdata
table = sdata.table(sdata.example("fruit"))
table.count_each("fruit")
```

table.average_of()

The average (mean) of a column; empty cells are skipped.

`table.average_of("marks", for_each="class")` (with `sdata.example("marks")`)
`across=` gives a cross table, one column per subject:
`table.average_of("marks", for_each="class", across="subject")`
`every=` groups a date column in `for_each=` into `"hour"`, `"day"`, `"week"`, `"month"`, `"quarter"` or `"year"` periods:
`table.make_dates("year")`
`table.average_of("population", for_each="year", every="year")` (with `sdata.example("growth")`)

Options

`column`	needed
`for_each=`	
`every=`	
`across=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.average_of("marks")
by_class = table.average_of("marks", for_each="class")
print(by_class)
```

table.median_of()

The middle number of a column once it is sorted; empty cells are skipped.

`table.median_of("marks", for_each="class")` (with `sdata.example("marks")`)
`across=` and `every=` work as in `average_of`.

Options

`column`	needed
`for_each=`	
`every=`	
`across=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.median_of("marks", for_each="class")
```

table.total_of()

All the numbers in a column added up; empty cells are skipped.

`table.total_of("count", for_each="shop")` (with `sdata.example("fruit")`)
`table.total_of("count", for_each="shop", across="fruit")` a cross table
`every="month"` with a date column in `for_each=` gives the total of each month.

Options

`column`	needed
`for_each=`	
`every=`	
`across=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `fruit`)

```
import simplydata as sdata
table = sdata.table(sdata.example("fruit"))
table.total_of("count", for_each="shop")
```

table.smallest_of()

The smallest number in a column; empty cells are skipped.

`table.smallest_of("temperature", for_each="city")` (with `sdata.example("weather")`)
`across=` and `every=` work as in `average_of`.

Options

`column`	needed
`for_each=`	
`every=`	
`across=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.smallest_of("temperature", for_each="city")
```

table.biggest_of()

The biggest number in a column; empty cells are skipped.

`table.biggest_of("temperature", for_each="city")` (with `sdata.example("weather")`)
`across=` and `every=` work as in `average_of`.

Options

`column`	needed
`for_each=`	
`every=`	
`across=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.biggest_of("temperature", for_each="city")
```

table.standard_deviation_of()

How spread out the numbers are: the sample standard deviation (divide by n - 1).

It is the same as pandas .std() and R `sd()`. Empty cells are skipped.
`table.standard_deviation_of("marks", for_each="class")` (with `sdata.example("marks")`)
`across=` and `every=` work as in `average_of`.

Options

`column`	needed
`for_each=`	
`every=`	
`across=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.standard_deviation_of("marks", for_each="class")
```

table.summary_of()

Several measures of one or more columns at once, in one tidy table.

`table.summary_of("marks", for_each="class", show=["average", "standard deviation", "count"])`
(with `sdata.example("marks")`). Several columns: `table.summary_of(["marks", "hours"])`. `show=` takes any of: `"count"` (numbers, empty cells left out), `"average"`, `"median"`, `"total"`, `"smallest"`, `"biggest"`, `"standard deviation"` (sample, n - 1), `"variance"` (sample, n - 1), `"standard error"` (of the average), `"confidence interval"` (95 % t interval of the average, as R t.test), `"range"` (biggest minus smallest), `"interquartile range"` (upper quartile minus lower quartile, R type 7), `"empty cells"`, and these, with the definitions R uses: `"geometric mean"` (numbers above 0), `"skewness"` and `"kurtosis"` (the bias-corrected sample values G1 and G2, as SciPy skew/`kurtosis(bias=False)`, R e1071 type 2 and Excel; kurtosis is excess kurtosis, 0 for a bell shape), `"coefficient of variation"` (standard deviation / average, a ratio), `"median absolute deviation"` (as R `mad()`, times 1.4826), `"trimmed average"` (10 % cut from each end, as R `mean(trim = 0.1)`; `"trimmed average 20"` cuts 20 %), `"most common"` (the smallest if tied) and `"percentile 90"` (any percent, R quantile type 7). Without `show=`: count, average, standard deviation, smallest, median, biggest. A measure of your own also works: a function that gets one group's numbers (a NumPy array) and gives back one number, `show=["average", my_measure]`. The new table has one row for each column (and each group), so it can be saved or charted. `every=` groups a date column in `for_each=` by `"week"`, `"month"`, `"year"` ...

Options

`column=`
`show=`
`for_each=`
`every=`

Underneath narwhals (group summaries); SciPy (the t distribution) for the confidence interval

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.summary_of("marks", for_each="class", show=["average", "standard deviation", "count"])
```

`table.percent_of()`

What percent of the rows meet a condition, or each group's share.

`table.percent_of("marks", above=60, for_each="class")` (with `sdata.example("marks")`)
With a condition (the same words as `keep_rows`: above, below, `at_least`, `at_most`, `equal_to`, `not_equal_to`, between, `one_of`, contains, `starts_with`, `ends_with`, `is_empty`): the percent of rows that meet it, out of the rows with a value in the column, with its 95 % confidence interval (Clopper-Pearson exact interval, as R binom.test). Without a condition:
`table.percent_of(for_each="class")` each group's percent of all rows
`table.percent_of("count", for_each="shop")` each group's share of the total of a column (with `sdata.example("fruit")`) Percents go from 0 to 100. Without `for_each=` you get one number; with it a new table.

Options

`column=`
`for_each=`
`every=`
`above=`
`below=`
`at_least=`
`at_most=`
`equal_to=`
`not_equal_to=`
`between=`
`one_of=`
`contains=`
`starts_with=`
`ends_with=`
`is_empty=`

Underneath narwhals; SciPy (stats.binomtest, the Clopper-Pearson exact interval)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.percent_of("marks", above=60, for_each="class")
```

table.count_different_values()

How many different values a column has, or how many in each group.

`table.count_different_values("name", for_each="class")` (with `sdata.example("marks")`)
Empty cells are not counted (as pandas `nunique()`); a note says how many there were. Capital letters matter: `"Apple"` and `"apple"` are two values.

Options

```
column=
for_each=
every=
```

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.count_different_values("name", for_each="class")
```

table.percentile_of()

The value below which a given percent of the numbers lie; empty cells are skipped.

`table.percentile_of("marks", 90, for_each="class")` (with `sdata.example("marks")`)
`percentile_of("marks", 50)` is the median, 25 and 75 the quartiles. Between two numbers it uses the usual linear rule (R `quantile(type = 7)`, NumPy and pandas default). `across=` and `every=` work as in `average_of`.

Options

```
column=
percent=
for_each=
every=
across=
```

Underneath narwhals (quantile with linear interpolation, as R quantile type 7)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.percentile_of("marks", 90, for_each="class")
```

table.most_common_of()

The value that turns up most often (the mode), for numbers or text; empty cells are skipped.

`table.most_common_of("days_absent", for_each="school")` (with `sdata.example("school")`)
When values are tied, the smallest (first in order) is given and a note names the others. Without `for_each=` you get the value; with it a new table with, for each group, the most common value, how many rows have it, its percent of the group's filled cells, and how many values share the top count.

Options

`column=`
`for_each=`
`every=`

Underneath narwhals (value counts per group)

Example you can run (it uses the practice table `school`)

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.most_common_of("days_absent", for_each="school")
```

`table.correlation_between()`

How strongly columns of numbers go up or down together, from -1 to 1.

`table.correlation_between("hours", "marks")` (with `sdata.example("marks")`)
`method="pearson"` (a straight-line link, the usual one) or `"spearman"` (uses ranks, for any link that keeps going one way). Prints the strength in words and the p-value from SciPy. With two columns it gives back the number r; rows with an empty cell in either column are skipped. With three or more columns it gives back a correlation table (as pandas .corr()), and prints the strongest pairs:
`table.correlation_between("hours", "marks", "height")`
Each pair uses the rows where both columns have a number. `show="p-value"` gives the table of p-values instead, and `show="each pair"` one row per pair (r, p-value, rows), ready for `add_adjusted_p_value_column()`: testing many pairs makes a chance finding more likely. Strength words for the size of r: below 0.2 very weak, from 0.2 weak, from 0.4 medium, from 0.6 strong, from 0.8 very strong (after Evans 1996). p below 0.001: `"very unlikely to be chance"`; below 0.05: `"unlikely to be chance"`; otherwise `"could easily be chance"`. A correlation does not show that one thing causes the other.

Options

`columns...`	as many as you like, separated by commas
`method=`	normally `"pearson"`
`show=`	normally `"r"`

Underneath SciPy (stats.pearsonr or stats.spearmanr)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.correlation_between("hours", "marks")
```

Statistical tests

`table.compare_averages()`

Are the averages of the groups really different, or could it be chance?

`table.compare_averages("marks", for_each="class")` (with `sdata.example("marks")`)
Two groups: Welch's t-test (the R `t.test()` default). Three or more: Welch's one-way ANOVA (the R `oneway.test()` default). Neither needs the groups to have the same spread. `based_on="ranks"` uses ranks instead (Mann-Whitney U or Kruskal-Wallis): safer when groups are small and their values are not bell-shaped. `test=` chooses another test: `"welch"`, `"student"`, `"welch anova"`, `"anova"`, `"mann-whitney"` or `"kruskal-wallis"`. Prints the answer in words (with the test used and why) and gives back a one-row table of results. The effect size says how big the difference is: Hedges' g (the difference in standard deviations) for 2 groups, omega-squared (the share of the differences explained by the groups) for 3 or more. Each group is checked for a bell shape (Shapiro-Wilk) and a note says if it does not look like one; with 30 or more values that matters little (the central limit theorem). To see which groups differ, use `compare_each_pair()`.

Options

`column=`	
`for_each=`	
`based_on=`	normally `"averages"`
`test=`	

Underneath SciPy (stats.ttest_ind with `equal_var=False`; Welch's ANOVA written out with the F distribution; mannwhitneyu, kruskal)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
result = table.compare_averages("marks", for_each="class")
table.compare_averages("marks", for_each="class", based_on="ranks")
result.save("marks_test.csv")
```

`table.compare_each_pair()`

Which pairs of groups differ? Every pair is compared, with Holm's correction.

`table.compare_each_pair("marks", for_each="class")` (with `sdata.example("marks")`)
Each pair gets Welch's t-test (or Mann-Whitney U with `based_on="ranks"`). Testing many pairs makes a false alarm more likely, so the p-values are adjusted with Holm's method: the chance of at least one false alarm stays at 5 %. This is the same as R `pairwise.t.test(pool.sd = FALSE, p.adjust.method = "holm")`. Gives back a table with one row per pair. `test="tukey"` gives Tukey's honest significant difference instead (R `TukeyHSD(aov(marks ~ class))`, the Tukey-Kramer form when the groups differ in size): the textbook follow-up to a one-way ANOVA, with 95 % confidence intervals that hold for all pairs at once. It assumes the groups have the same spread; the default does not. `test="welch"` and `test="mann-whitney"` name the other two choices.

Options

`column=`	
`for_each=`	
`based_on=`	normally `"averages"`
`test=`	

Underneath SciPy (stats.ttest_ind or mannwhitneyu), with Holm's correction

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.compare_each_pair("marks", for_each="class")
```

table.compare_before_after()

Did the values change from one column to the other, row by row, more than chance?

`table.compare_before_after("Maths", "Science")` (with `sdata.example("marks")`)

For two measurements of the same thing, such as each child's mark before and after a course. Uses the change in each row (after minus before): a paired t-test, or the Wilcoxon signed-rank test with `based_on="ranks"`. Rows with an empty cell in either column are skipped. Effect size: Cohen's `d_z`, the average change in standard deviations of the changes. Gives back a one-row table of results.

Options

`before=`	
`after=`	
`based_on=`	normally `"averages"`

Underneath SciPy (stats.ttest_rel or stats.wilcoxon)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_columns("name", "subject", "marks")
table.turn_rows_into_columns(names_from="subject", values_from="marks")
table.compare_before_after("Maths", "Science")
```

table.compare_average_to()

Is the average of a column different from a number you choose, or could it be chance?

`table.compare_average_to("marks", 60)` (with `sdata.example("marks")`)

A one-sample t-test, or the Wilcoxon signed-rank test (on the median) with `based_on="ranks"`. Empty cells are skipped. Effect size: Cohen's d, the difference in standard deviations. Gives back a one-row table of results.

Options

`column=`	
`value=`	
`based_on=`	normally `"averages"`

Underneath SciPy (stats.ttest_1samp or stats.wilcoxon)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.compare_average_to("marks", 60)
```

table.compare_counts()

Does how often each value turns up differ between groups, more than chance?

`table.add_group_column("result", from_column="marks", cut_at=[60], names=["below 60", "60 or more"])`
`table.compare_counts("result", for_each="class")` (with `sdata.example("marks")`)
(Do the classes differ in how many children score 60 or more?) Counts the rows for each pair of values and uses the chi-square test of independence (as R `chisq.test()`: Yates' continuity correction for 2 x 2 tables only). If an expected count is below 5 a note says so; for a 2 x 2 table `test="fisher"` then gives Fisher's exact test. Empty cells are skipped. Effect size: Cramer's V, the strength of the link from 0 (none) to 1 (complete). Gives back a one-row table.

Options

`column=`
`for_each=`
`test=`

Underneath SciPy (stats.chi2_contingency, or `fisher_exact` with `test="fisher"`)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_group_column("result", from_column="marks", cut_at=[60], names=["below 60", "60 or more"])
table.compare_counts("result", for_each="class")
```

table.compare_shape_to()

Do the values look bell-shaped (normal), or are they too far from it to be chance?

`table.compare_shape_to("marks", "normal", for_each="class")` (with `sdata.example("marks")`)
Uses the Shapiro-Wilk test (as R `shapiro.test()`), for 3 to 5000 values in each group; empty cells are skipped. A small p-value says the shape is too far from a bell shape to be chance. A large p-value does not prove the values are normal: with few values the test can miss a different shape, and with many values (30 or more) it flags even tiny, harmless differences, while tests on averages are then usually fine anyway (the central limit theorem). Gives back a table with one row per group.

Any other smooth shape works too: `table.compare_shape_to("days", "gamma")` fits the distribution (as `table.fit_distribution`) and runs the Anderson-Darling test, with a p-value from 9999 simulated samples that are each fitted again (`seed=1`, printed).

Options

`column=`	
`shape=`	normally `"normal"`
`for_each=`	
`seed=`	normally `1`

Underneath SciPy (stats.shapiro for normal; stats.goodness_of_fit, Anderson-Darling with a Monte Carlo p-value, for other shapes)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.compare_shape_to("marks", "normal", for_each="class")
```

table.add_adjusted_p_value_column()

Adjust p-values for testing many things at once, in a new column.

`table = sdata.table(sdata.example("marks")).correlation_between("hours", "marks", "height", show="each pair")`
`table.add_adjusted_p_value_column("q", method="benjamini-hochberg")`
(table is now one row per pair of columns, with its p-value) The more tests you run, the more likely one of them looks real by chance; adjusted p-values allow for that. `method="holm"` (the default, as R `p.adjust()`): the chance of even one false alarm among all the tests stays at 5 %. `"bonferroni"`: the same promise, simpler and stricter. `"benjamini-hochberg"` (also `"bh"` or `"fdr"`): about 5 % of the findings below 0.05 are expected to be false alarms (the false discovery rate); it finds more with many tests. `"benjamini-yekutieli"` (`"by"`): a false discovery rate that is safe however the tests depend on each other. `from_column=` is the column of p-values (`"p-value"` in simplydata result tables). Without a name the new column is called, for example, `"p-value (Holm)"`. Empty cells stay empty and are not counted, as R `p.adjust()`.

Options

`name=`	
`from_column=`	normally `"p-value"`
`method=`	normally `"holm"`

Underneath NumPy (Holm, Bonferroni, Benjamini-Hochberg and Benjamini-Yekutieli, written out as in R `p.adjust()`)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table = table.correlation_between("hours", "marks", "height", show="each pair")
table.add_adjusted_p_value_column("q", method="benjamini-hochberg")
table.show()
```

table.compare_spreads()

Do the groups differ in how spread out their values are, more than chance?

`table.compare_spreads("marks", for_each="class")` (with `sdata.example("marks")`)
`test="levene"` (the default): Levene's test centred on each group's median, also called the Brown-Forsythe test. It is the default of R car::`leveneTest()` and SciPy `levene(center="median")`, and copes well with values that are not bell-shaped. `test="bartlett"`: Bartlett's test (R `bartlett.test()`), a little stronger when the values are bell-shaped but easily fooled when they are not. `test="fligner"`: the Fligner-Killeen test (R `fligner.test()`), based on ranks, the safest with outliers. Empty cells are skipped. Prints the standard deviation of each group and the answer in words; gives back a one-row table of results. `compare_averages()` uses Welch's test by default, which does not need equal spreads.

Options

`column=`
`for_each=`
`test=` normally `"levene"`

Underneath SciPy (stats.levene with `center="median"`, stats.bartlett, stats.fligner)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.compare_spreads("marks", for_each="class")
```

`table.compare_counts_to()`

Does how often each value turns up match the shares you expect, more than chance?

`table.compare_counts_to("breakfast", expected={"yes": 0.6, "no": 0.4})` (with `sdata.example("school")`)
The chi-square goodness-of-fit test (as R `chisq.test(x, p = ...)`). `expected=` gives the share of each value, adding up to 1 (or percents adding up to 100); for a ratio such as 9:3:3:1 write {`"a"`: 9/16, `"b"`: 3/16, ...}. Without `expected=` every value is expected equally often (as R `chisq.test(table(x))`). Empty cells are skipped. If an expected count is below 5 a note says so. Effect size: Cohen's w, how far the counts are from the expected shares (0.1 small, 0.3 medium, 0.5 large). For one value against one percent, `compare_percent_to()` gives an exact test. Gives back a one-row table with the count and expected count of each value.

Options

`column=`
`expected=`

Underneath SciPy (stats.chisquare)

Example you can run (it uses the practice table `school`)

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.compare_counts_to("breakfast", expected={"yes": 0.6, "no": 0.4})
```

`table.compare_percent_to()`

Is the percent of rows with one value different from a percent you choose, or could it be chance?

`table.compare_percent_to("passed", value="yes", percent=50)` (with `sdata.example("school")`)
The exact binomial test (as R `binom.test(x, n, p)`), with the exact Clopper-Pearson 95 % interval of the percent (as `percent_of()`). Percents go from 0 to 100. For a yes/no column (yes/no, True/False or 1/0) `value=` may be left out: `"yes"` (True, 1) is counted. Empty cells are skipped. Effect size: Cohen's h, the difference between the two shares on the arcsine scale (0.2 small, 0.5 medium, 0.8 large). Gives back a one-row table of results.

Options

`column=`
`value=`
`percent=`

Underneath SciPy (stats.binomtest, the Clopper-Pearson exact interval)

Example you can run (it uses the practice table `school`)

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.compare_percent_to("passed", value="yes", percent=50)
```

Fitting

`table.fit_line()`

Fit the straight line y = slope * x + intercept that best follows the points.

`table.fit_line(x="hours", y="marks")` (with `sdata.example("marks")`)
Ordinary least squares (the same as R `lm(y ~ x)`). Prints how much y goes up for each 1 more of x, with its 95 % confidence interval and p-value, and R squared: the share of the differences in y that the line explains. Rows with an empty cell in x or y are skipped. A link does not show that x causes y. Gives back a one-row table. The same line as `chart.fit_line()`; its old name `fit_straight_line` still works. For a curve use `fit_curve()`, for any formula `fit_formula()`.

Options

`x=`
`y=`

Underneath SciPy (stats.linregress)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.fit_line(x="hours", y="marks")
```

`table.fit_curve()`

Fit a curve y = b0 + b1 * x + b2 * x**2 ... that best follows the points.

`table.fit_curve(x="hours", y="marks", degree=2)` (with `sdata.example("marks")`)
`degree=2` is a curve with one bend, `degree=3` can bend twice, and so on (up to 10), the same curve as `chart.fit_curve(degree=)`. Least squares on the powers of x, as R `lm(y ~ poly(x, 2, raw = TRUE))`. Prints the curve and R squared; gives back one row per term (the intercept, x, x**2 ...) with its estimate, standard error, t, p-value and 95 % confidence interval. `weights="column"` counts some rows more (as R `lm(weights =)`); `for_each="column"` fits each group on its own. Rows with an empty cell are skipped.

Options

`x=`
`y=`
`degree=` normally `2`
`weights=`
`for_each=`

Underneath NumPy (least squares by QR on the powers of x)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.fit_curve(x="hours", y="marks", degree=2)
```

`table.fit_formula()`

Find the values in a formula that make it follow the data best.

`table.fit_formula("population = a * exp(b * (year - 2000))")` (with `sdata.example("growth")`)
Write the column to fit on the left of =. On the right, names that are columns are data; any other name (such as a, b or rate) is a value to find. Uses nonlinear least squares, as R `nls()` (Levenberg-Marquardt). Every value to find starts at 1; give better start values with `start={"a": 10, "b": 0.03}` if the fit fails. `between={"b": [0, None]}` keeps a value between limits (None: no limit), as R `nls(algorithm = "port")`; `search="whole range"` searches the whole range of `between=` before fitting (differential evolution, seed 1), for when start values are hard to guess. `weights="column"` counts some rows more (as R `nls(weights =)`). `for_each="column"` fits each group on its own. `method=` lets an expert choose `"levenberg-marquardt"`, `"trust region"` or `"dogbox"`. Prints each value with its 95 % confidence interval, the residual standard error and R squared (only a rough guide for a curve). Gives back one row (one per group) with each value, its standard error, t, p-value and 95 % CI, the residual standard error, degrees of freedom, R squared, AIC, rows and whether the fit converged. Residuals: `add_predicted_column` with the same formula, then `add_column("y - predicted y")`. Rows with an empty cell are skipped.

Options

`formula=`	
`start=`	
`between=`	
`weights=`	
`for_each=`	
`search=`	normally `"from start"`
`method=`	

Underneath SciPy (optimize.curve_fit; optimize.differential_evolution for `search="whole range"`)

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
table = sdata.table(sdata.example("growth"))
table.fit_formula("population = a * exp(b * (year - 2000))", for_each="country")
table.keep_rows("country", equal_to="Applia")
table.fit_formula("population = a * exp(b * (year - 2000))")
```

Finding and solving

sdata.find_smallest()

Find where a formula is smallest, and its value there.

`sdata.find_smallest("x**2 - 4*x + 1", between=[0, 5])` (the smallest value is -3, at x = 2)
The names in the formula (x, y, a, rate) are what to find. Give a range with `between=` ([0, 5] for one name; {"x": [0, 5], "y": [0, None]} for several, None = no limit) or starting guesses with `start=` (a number, or {"x": 1, "y": 2}). `rules=["x + y <= 4", "x >= 0"]` adds rules the answer must keep (=, <= or >=); `whole_numbers=True` asks for whole numbers (with a straight-line formula and rules). `search="whole range"` looks everywhere in `between=` instead of near the start. `known={"a": 2.1}` (or a one-row result table from `table.fit_formula`) fixes some names. The method is decided by what you give and printed (R `optimize()`, `optim()`, lpSolve::`lp()`; see the module notes); `method=` lets experts choose (`"nelder-mead"`, `"bfgs"`, `"l-bfgs-b"`, `"conjugate gradient"`, `"powell"`, `"trust region"`, `"slsqp"`, `"cobyla"`; with `search="whole range"`: `"differential evolution"`, `"dual annealing"`, `"shgo"`, `"basin hopping"`, `"direct"`). Gives back a one-row table: the value of each name, the smallest value, the method, whether the search settled, and for linear programs the value of one more on the right of each rule (shadow price, R `lp(compute.sens = TRUE)`$duals). `fast=True` gives the table on the fast engine (Polars).

Options

`formula=`	
`between=`	
`start=`	
`rules=`	
`whole_numbers=`	normally `False`
`search=`	normally `"from start"`
`method=`	
`known=`	
`fast=`	normally `False`

Underneath SciPy (optimize.minimize_scalar, minimize, linprog and milp with HiGHS, `differential_evolution` and other global searches)

Example you can run

```
import simplydata as sdata
sdata.find_smallest("x**2 - 4*x + 1", between=[0, 5])
sdata.find_smallest("(x - 1)**2 + (y - 2)**2", start={"x": 0, "y": 0})
```

sdata.find_biggest()

Find where a formula is biggest, and its value there.

```
sdata.find_biggest("3*x + 2*y", rules=["x + y <= 4", "x + 3*y <= 6", "x >= 0", "y >= 0"])
```
(12, at x = 4, y = 0)

The same options as `find_smallest()`: `between=`, `start=`, `rules=`, `whole_numbers=`, `search="whole range"`, `method=`, `known=` (a dict, or a one-row result table from `table.fit_formula`, so a fitted curve can be handed straight over). It works by finding the smallest value of minus the formula, and prints the biggest. Gives back a one-row table: each name, the biggest value, the method, whether it settled, and for linear programs the value of one more on the right of each rule (shadow price).

Options

`formula=`	
`between=`	
`start=`	
`rules=`	
`whole_numbers=`	normally `False`
`search=`	normally `"from start"`
`method=`	
`known=`	
`fast=`	normally `False`

Underneath SciPy (the same as `find_smallest`, on minus the formula)

Example you can run

```
import simplydata as sdata
sdata.find_biggest("x * (10 - x)", between=[0, 10])
sdata.find_biggest("3*x + 2*y", rules=["x + y <= 4", "x + 3*y <= 6", "x >= 0", "y >= 0"])
```

sdata.solve()

Find the values that make an equation true.

```
sdata.solve("x**3 - 2*x = 5", between=[0, 3])
```
(x = 2.09455)

One equation with one name: `between=[low, high]` finds every solution in the range where the two sides cross (Brent's method, R `uniroot()` on each sign change, as rootSolve::`uniroot.all()`); `start=` finds one solution by searching from the start (secant method); a polynomial (only + - * and whole powers) needs neither and gives all its solutions, complex ones too (R `polyroot()`). Several equations in a list:

```
sdata.solve(["x + y = 3", "x * y = 2"], start={"x": 0, "y": 1}) (Powell's hybrid method, as
```
R nleqslv::`nleqslv()`); straight-line equations are solved exactly (R `solve()`). `known=` fixes some names (a dict, or a one-row result table from `table.fit_formula`). Gives back a table with one row per solution: each name, `"left minus right"` at the solution (the check: close to 0) and the method. `fast=True` gives a Polars table.

Options

`equations=`	
`between=`	
`start=`	
`method=`	
`known=`	
`fast=`	normally `False`

Underneath SciPy (optimize.brentq, `root_scalar`, root) and NumPy (polynomial roots, linalg.solve)

Example you can run

```
import simplydata as sdata
sdata.solve("x**3 - 2*x = 5", between=[0, 3])
sdata.solve("x**2 - 2*x + 5 = 0")
sdata.solve(["x + y = 3", "x * y = 2"], start={"x": 0, "y": 1})
```

Values between points

`table.values_at()`

Estimate a column at places between the known points, by joining the dots.

`table.values_at("population", at={"year": [2001, 2003, 2011]}, for_each="country")` (with `sdata.example("growth")`)
`at=` names the `column(s)` that place the points and the places wanted. One column joins the points along a line; two or more (`at={"east": [1.5, 2.5], "north": [3, 4.5]}`, `sdata.example("island")`) join them across a surface: the lists are read as pairs, (1.5, 3) and (2.5, 4.5); `table.map_of()` gives every combination. `joined_by=` says how the points are joined (one column): `"straight lines"` (default, linear interpolation, R `approx()`), `"smooth curve"` (natural cubic spline, R `splinefun(method = "natural")`), `"curve without overshoot"` (PCHIP), `"steps"`, `"nearest"`; experts: `"akima"`, `"makima"`, `"not-a-knot"`, `"periodic"`. Two or more columns: `"straight lines"` (bilinear on a full grid, triangles otherwise), `"smooth surface"` (thin-plate spline), `"nearest"`; on a full grid also `"cubic"`, `"quintic"`, `"pchip"`. `smoothness="auto"` or a number gives a smoothing spline that passes near, not through, the points (R `smooth.spline()`; a number is R's `lambda=`); 0 passes through every point. `outside=` is what happens beyond the known points: `"empty"` (default), `"nearest end"` or `"extend"` (curves only; guesswork). `for_each="country"` joins each group on its own. Rows with an empty cell are skipped. Gives back a new table: the `at=` columns, the `for_each=` columns and the estimated column.

Options

`column=`	
`at=`	
`joined_by=`	normally `"straight lines"`
`smoothness=`	normally `0`
`for_each=`	
`outside=`	normally `"empty"`

Underneath NumPy (interp) and SciPy (interpolate: CubicSpline, PchipInterpolator, Akima1DInterpolator, `make_smoothing_spline`, griddata, RegularGridInterpolator, RBFInterpolator)

Example you can run (it uses the practice table `growth`)

```
import simplydata as sdata
table = sdata.table(sdata.example("growth"))
estimates = table.values_at("population", at={"year": [2001, 2003, 2011]}, for_each="country"
    ↪ )
estimates.show()
table.values_at("population", at={"year": [2001, 2003, 2011]}, joined_by="smooth curve",
    ↪ for_each="country")
```

`table.map_of()`

Estimate a column at evenly spaced places across one or more columns (a map), ready for a contour or heatmap.

`table.map_of("height", across=["east", "north"], steps=100)` (with `sdata.example("island")`)
The map runs from the smallest to the biggest known value of each `across=` column, with `steps=` places along each (a number, or one per column). The points are joined as in `table.values_at()` (same `joined_by=` and `smoothness=`). Places outside the outline of the known points stay empty, with a note. Gives back a new long table: one row per place, with the `across=` columns and the estimated column.

Options

`column=`	
`across=`	
`steps=`	normally `50`
`joined_by=`	normally `"straight lines"`
`smoothness=`	normally `0`

Underneath SciPy (interpolate: RegularGridInterpolator, griddata, RBFInterpolator)

Example you can run (it uses the practice table `island`)

```
import simplydata as sdata
table = sdata.table(sdata.example("island"))
heights = table.map_of("height", across=["east", "north"], steps=60, joined_by="smooth
    ↪ surface")
chart = heights.chart()
chart.contour(x="east", y="north", z="height")
chart.save("map_of.png")
```

Distributions

`sdata.distribution()`

Make a distribution to ask chances, percentiles and random numbers of.

`dist = sdata.distribution("normal", average=50, standard_deviation=10)`
then `dist.chance(above=80)`, `dist.percentile(95)`, `dist.random_numbers(how_many=100, seed=1)`. Names: normal, lognormal, exponential, gamma, beta, uniform, t, chi-square, F, weibull, cauchy, logistic, laplace, gumbel, pareto, triangular, half-normal, inverse gaussian, von mises; for counts binomial, bernoulli, poisson, negative binomial, geometric, hypergeometric, whole number uniform. Parameters are R's with plain names; R's names (`mean=`, `sd=`, `prob=`, `size=`, `df=`, ...) work too. Any other SciPy distribution by its SciPy name and SciPy's parameters: `sdata.distribution("burr12", c=2, d=3)`.

Options

`name=`	
`values...`	as many as you like, separated by commas

Underneath SciPy (scipy.stats frozen distributions)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
print(dist)
dist.chance(above=80)
coins = sdata.distribution("binomial", tries=10, chance=0.5)
coins.chance(at_most=3)
```

table.fit_distribution()

Fit a distribution to a column by maximum likelihood; gives back the fitted distribution.

`fit = table.fit_distribution("sales", "lognormal")` (with `sdata.example("shop")`) then `fit.chance(above=600)`, `fit.percentile(90)`, `fit.chart()`, and `fit.table()` for the result table (values, standard errors, log-likelihood, AIC, BIC, rows). As R MASS::`fitdistr(x, "gamma")` (fitdistrplus::fitdist for beta and others); standard errors from the observed information. Families without a location in R (gamma, weibull, lognormal, exponential, beta) keep it at 0, as R. `fixed={"shape": 2}` holds a parameter. With `for_each=` it gives back a result table, one row per group. Empty cells are skipped.

Options

`column=`	
`distribution=`	normally `"normal"`
`for_each=`	
`fixed=`	

Underneath SciPy (scipy.stats and optimize, maximum likelihood; closed forms where they exist)

Example you can run (it uses the practice table `shop`)

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
fit = table.fit_distribution("sales", "lognormal")
fit.chance(above=600)
fit.table()
```

table.compare_distributions()

Fit several distributions to a column and rank them by AIC (smaller is better).

`table.compare_distributions("sales")` (with `sdata.example("shop")`) `distributions=("normal", "lognormal", "gamma", "weibull")` by default. Each is fitted by maximum likelihood (as `fit_distribution`); the result table has one row per distribution with its values, log-likelihood, AIC, BIC, the AIC difference from the best, and the Anderson-Darling, Kolmogorov-Smirnov and Cramer-von Mises statistics, as R fitdistrplus::`gofstat()`. An AIC difference below 2 is too small to choose between them.

Options

`column=`	
`distributions=`	normally `('normal', 'lognormal', 'gamma', 'weibull')`
`for_each=`	

Underneath SciPy (scipy.stats, maximum likelihood; Anderson-Darling, Kolmogorov-Smirnov and Cramer-von Mises statistics)

Example you can run (it uses the practice table `shop`)

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.compare_distributions("sales")
```

`nums.fit_distribution()`

Fit a distribution to the numbers by maximum likelihood; gives back the fitted distribution.

`fit = nums.fit_distribution("gamma")` then `fit.chance(above=10)`, `fit.table()`
As R MASS::`fitdistr(x, "gamma")`; see `table.fit_distribution`. Empty values are skipped.

Options

`distribution=` normally `"normal"`
`fixed=`

Underneath SciPy (scipy.stats and optimize, as `table.fit_distribution`)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
fit = nums.fit_distribution("gamma")
fit.chance(above=18)
```

`dist.chance()`

The chance of a value in a range, as a share from 0 to 1 (printed as a percent).

`dist.chance(above=80)` `dist.chance(below=60)` `dist.chance(between=[40, 60])`
`at_most=` and `at_least=` include the number itself; `above=` and `below=` do not (as `keep_rows`). For counts, `dist.chance(3)` is the chance of exactly 3; `below=3` means 0, 1 or 2 and `at_most=3` means 3 or fewer (R `pbinom(3)`). One lower option (`above=`, `at_least=`) and one upper option (`below=`, `at_most=`) can go together. A list gives Numbers. R: `pnorm(q, lower.tail = )`, dbinom.

Options

`value=`
`above=`
`below=`
`at_most=`
`at_least=`
`between=`

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.chance(above=80)
dist.chance(between=[40, 60])
dice = sdata.distribution("whole number uniform", between=[1, 6])
dice.chance(6)
```

dist.density_at()

The height of the density curve at a value (continuous distributions).

`dist.density_at(60)` (R `dnorm(60, mean = 50, sd = 10)`). A list gives Numbers.
It is not a chance: a chance is an area under the curve (`dist.chance(between=[...])`).

Options

`value=`

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.density_at(60)
```

dist.percentile()

The value that this percent of the distribution is below: `percentile(95)`.

`dist.percentile(95)` (R `qnorm(0.95, mean = 50, sd = 10)`). `percentile(50)` is the median.
For counts: the smallest count with at least that percent at or below it (R qbinom). A list of percents gives Numbers.

Options

`percent=`

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.percentile(95)
dist.percentile([5, 50, 95])
```

dist.cut_off()

The value with this chance above it (or below it): critical values.

`dist.cut_off(above=0.025)` the value with a 2.5 % chance above it (R `qnorm(0.025, lower.tail = FALSE)`)
`dist.cut_off(below=0.05)` the value with a 5 % chance below it
Chances are shares from 0 to 1. A list gives Numbers.

Options

`above=`
`below=`

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.cut_off(above=0.025)
dist.cut_off(below=0.05)
```

dist.middle_range()

The range that holds the middle percent of the distribution, as Numbers [low, high].

`dist.middle_range(95)` (R `qnorm(c(0.025, 0.975), mean = 50, sd = 10)`)

Options

`percent=`	normally `95`

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.middle_range(95)
```

dist.average()

The average (mean) of the distribution. `dist.average()`

Options none: just write the empty brackets.

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.average()
```

dist.median()

The median: half of the distribution is below it. `dist.median()`

Options none: just write the empty brackets.

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.median()
```

dist.standard_deviation()

The standard deviation of the distribution. `dist.standard_deviation()`

Options none: just write the empty brackets.

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.standard_deviation()
```

dist.variance()

The variance (the standard deviation squared). `dist.variance()`

Options none: just write the empty brackets.

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.variance()
```

dist.summary()

A one-row table: average, median, standard deviation, variance, skewness, excess kurtosis, entropy and the lowest and highest possible values. `dist.summary()`

Options none: just write the empty brackets.

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
dist.summary()
```

dist.random_numbers()

Numbers drawn at random from the distribution; the same seed gives the same numbers.

`dist.random_numbers(how_many=100, seed=1)` (`rows=` and `columns=` give a grid)
R's `rnorm()` and friends use another random number generator, so their numbers differ.

Options

`how_many=`	
`seed=`	normally `1`
`rows=`	
`columns=`	

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
nums = dist.random_numbers(how_many=100, seed=1)
nums.average()
```

dist.chart()

A chart of the distribution: its density curve (or bars for counts).

`dist.chart()` or `dist.chart(show="cumulative")` for the chance of a value at or below x.

Options

`show=`	normally `"density"`
`between=`	

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
chart = dist.chart()
chart.title_top("Normal, average 50, standard deviation 10")
chart.save("distribution.png")
```

dist.table()

A one-row table: the distribution, its parameters and, for a fit, the standard errors, log-likelihood, AIC and BIC. `dist.table()`

Options none: just write the empty brackets.

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
result = dist.table()
result.show()
```

dist.parameters()

The parameters as a dictionary of plain names: {`"average"`: 50, `"standard_deviation"`: 10}. `dist.parameters()`

Options none: just write the empty brackets.

Underneath SciPy (scipy.stats frozen distributions: cdf, sf, ppf, isf, pdf, pmf, stats, rvs)

Example you can run

```
import simplydata as sdata
dist = sdata.distribution("normal", average=50, standard_deviation=10)
print(dist.parameters())
```

Models

table.explain()

Explain one column from several others with a model, and say how big each effect is.

`table.explain("marks", using=["hours", "breakfast"])` (with `sdata.example("school")`) `outcome="numbers"` (the default): linear model by least squares (R `lm()`). `outcome="yes or no"`: logistic regression, odds ratios (R `glm(family = binomial)`). `outcome="counts"`: Poisson regression, rate ratios (R `glm(family = poisson)`); `outcome="counts (overdispersed)"`: quasi-Poisson (R `glm(family = quasipoisson)`). A text column in `using=` is a group column: each group is compared with the first group in sorted order (the baseline). `"hours * breakfast"` adds both columns and their interaction (does the effect of hours depend on breakfast?). `grouped_by="school"`: a mixed model with its own starting level for each school (R lme4::`lmer()`), for rows that belong together. `empty_cells="multiple imputation"` fills in empty cells 20 times (R `mice()`) instead of leaving those rows out; `seed=` fixes the random filling. Prints each effect with its 95 % confidence interval and p-value in words, and how well the model fits; gives back a table with one row per term. A link does not show that one thing causes the other.

Options

`column=`	
`using=`	
`outcome=`	normally `"numbers"`
`grouped_by=`	
`empty_cells=`	normally `"leave out"`
`seed=`	normally `1`

Underneath NumPy (least squares, and iteratively reweighted least squares as R glm); statsmodels for `grouped_by=` (MixedLM) and multiple imputation (MICEData)

Example you can run (it uses the practice table `school`)

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.explain("marks", using=["hours", "sleep", "breakfast"])
table.explain("passed", using=["hours", "breakfast"], outcome="yes or no")
table.explain("marks", using=["hours", "breakfast"], grouped_by="school")
```

table.add_predicted_column()

Add a column with the value the model predicts for each row.

`table.add_predicted_column("marks", using=["hours", "breakfast"])` (with `sdata.example("school")`)
The same model as `explain()` with the same options: `outcome="numbers"` gives the fitted value (R `fitted(lm(...))`), `outcome="yes or no"` the chance of yes from 0 to 1 and `outcome="counts"` the expected count (R `fitted(glm(...))`). With `grouped_by=` the prediction includes each group's own

level (R `fitted(lmer(...))`). Rows with an empty cell in a column used stay empty. Undo with `table.undo()`.

A formula with one = sign is fitted as `table.fit_formula()` (with its options `start=`, `between=`, `weights=` and `for_each=`) and its value is added for every row, also rows whose left-hand column is empty (as R `predict(nls(...), newdata)`):

`table.add_predicted_column("population = a * exp(b * (year - 2000))", for_each="country")`

`using=` a `fit_formula()` result takes its values instead of fitting again.

Options

`column=`	
`using=`	
`outcome=`	normally `"numbers"`
`name=`	
`grouped_by=`	
`start=`	
`between=`	
`weights=`	
`for_each=`	

Underneath NumPy (the same model as `table.explain`); statsmodels for `grouped_by=`

Example you can run (it uses the practice table `school`)

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.add_predicted_column("marks", using=["hours", "breakfast"])
table.show_first_rows(5)
```

Patterns across columns

`table.add_principal_components()`

Add principal components: a few new columns that hold most of the spread of many columns.

`table.add_principal_components(how_many=2)` (with `sdata.example("meadow")`)

Principal component analysis turns many related columns into new columns PC1, PC2 ... that do not overlap: PC1 holds the most spread, PC2 the most of what is left, and so on. The new columns are the scores of each row. `columns=None` uses every number column; `same_scale=True` (the default, as R `prcomp(scale. = TRUE)`) first puts each column on the same scale (mean 0, standard deviation 1) so that big numbers do not dominate; `same_scale=False` keeps the columns as they are (use it when all columns are the same kind of measure on the same scale). Prints how much of the spread each component explains and the 3 columns with the largest weights in each. The sign of a component is arbitrary: flipping it changes nothing. Rows with an empty cell get empty PC cells. `table.principal_components_of()` gives all the weights as a table. Method: singular value decomposition, as R `prcomp()`.

Options

`columns=`	
`how_many=`	normally `2`
`same_scale=`	normally `True`

Underneath NumPy (linalg.svd)

Example you can run (it uses the practice table `meadow`)

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.add_principal_components(how_many=2)
chart = table.chart()
chart.dots(x="PC1", y="PC2", colour="grazed")
chart.save("meadow_pca.png")
```

`table.principal_components_of()`

A table of the principal components: how much spread each explains and the weight of each column.

`table.principal_components_of()` (with `sdata.example("meadow")`)
One row per component (PC1, PC2 ...) with its standard deviation, the share of the spread it explains, the cumulative share, and one column of weights per original column, `"weight of hours"` and so on (the loadings, R `prcomp()`$rotation). A large weight, plus or minus, means the column counts a lot in that component. The sign of a component is arbitrary. `same_scale=True` puts each column on the same scale first, as R `prcomp(scale. = TRUE)`.

Options

`columns=`
`same_scale=` normally `True`

Underneath NumPy (linalg.svd)

Example you can run (it uses the practice table `meadow`)

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
components = table.principal_components_of()
components.show()
```

`table.add_clusters()`

Sort the rows into groups of similar rows (clusters), in a new column.

`table.add_clusters(how_many=3)` (with `sdata.example("meadow")`)
`method="k-means"` (the default) finds `how_many` clusters whose rows are as close as possible to their cluster's average, from 25 random starts (as R `kmeans(nstart = 25)`); `seed=` makes the result the same every time. `method="hierarchical"` joins the closest rows step by step and cuts the tree into `how_many` clusters; `linkage="ward"` (the default, R `hclust(method = "ward.D2")`), `"average"`, `"complete"` or `"single"`. `same_scale=True` (the default) puts each column on the same scale first (R `scale()`). The clusters are numbered 1, 2, 3 in the order they first appear in the table. Prints the size of each cluster and the share of the spread that lies between clusters (R kmeans: `between_SS / total_SS`). Rows with an empty cell get an empty cluster cell.

Options

`columns=`	
`how_many=`	normally `3`
`method=`	normally `"k-means"`
`same_scale=`	normally `True`
`seed=`	normally `1`
`name=`	normally `"cluster"`
`linkage=`	normally `"ward"`

Underneath NumPy (k-means: Lloyd and Hartigan steps) or SciPy (cluster.hierarchy, for `method="hierarchical"`)

Example you can run (it uses the practice table `meadow`)

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.add_clusters(how_many=2)
table.count_rows(for_each="cluster", across="grazed")
```

`table.distances_between_rows()`

How different each row is from every other row: a square table of distances.

`table.distances_between_rows(names="plot")` (with `sdata.example("meadow")`)
`measure="bray-curtis"` (the default, as R vegan::`vegdist()`) is the usual measure for counts of species: 0 when two rows have the same counts, 1 when they share nothing. `"jaccard"` uses only present / absent (a count above 0 is present), as vegan::`vegdist(binary = TRUE)`: the share of the species found in either row that are found in only one. `"euclidean"` is the straight-line distance and `"manhattan"` the sum of the differences. `names=` is a column whose values label the rows and columns.

Options

`columns=`	
`measure=`	normally `"bray-curtis"`
`names=`	

Underneath SciPy (spatial.distance.pdist) and NumPy

Example you can run (it uses the practice table `meadow`)

```
import simplydata as sdata
```

```
table = sdata.table(sdata.example("meadow"))
distances = table.distances_between_rows(measure="bray-curtis", names="plot")
distances.show_first_rows(3)
```

table.add_nmds()

Draw the rows as points on a map where close points are similar rows (NMDS), in new columns.

`table.add_nmds()` (with `sdata.example("meadow")`)
Non-metric multidimensional scaling puts each row on a map with `dimensions=` axes (NMDS1, NMDS2) so that the order of the distances on the map follows the order of the distances between the rows (`measure="bray-curtis"`, as R vegan::metaMDS). It tries a classical-scaling start and `tries=` random starts (`seed=` makes them the same every time) and keeps the map with the lowest stress (Kruskal's stress-1: how badly the map fits). Clarke's (1993) rule of thumb: below 0.05 excellent, below 0.1 good, below 0.2 usable, above 0.2 hard to trust. The axes have no units; their rotation and signs are arbitrary. Rows with an empty cell get empty NMDS cells.

Options

`columns=`	
`measure=`	normally `"bray-curtis"`
`dimensions=`	normally `2`
`tries=`	normally `20`
`seed=`	normally `1`

Underneath NumPy (SMACOF with monotone regression; SciPy optimize.isotonic_regression when available) and SciPy (spatial.distance.pdist)

Example you can run (it uses the practice table `meadow`)

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.add_nmds(measure="bray-curtis")
chart = table.chart()
chart.dots(x="NMDS1", y="NMDS2", colour="grazed")
chart.save("meadow_nmds.png")
```

`table.compare_profiles()`

Do the groups differ in their make-up across many columns (PERMANOVA), or could it be chance?

`table.compare_profiles(for_each="grazed")` (with `sdata.example("meadow")`)
Compares whole profiles, such as the counts of all plant species in a plot, between the groups in `for_each=`. PERMANOVA, as R vegan::`adonis2(distances ~ group)`: R squared is the share of the differences between rows (`measure=` distances, Bray-Curtis by default) that the groups explain, and pseudo-F compares the differences between groups with those within groups. The p-value comes from shuffling the rows `permutations=` times (`seed=` makes it the same every time). It also checks whether the groups differ in spread (vegan::betadisper and permutest), because PERMANOVA can then mistake a difference in spread for a difference in make-up. Gives back a one-row table.

Options

`columns=`	
`for_each=`	
`measure=`	normally `"bray-curtis"`
`permutations=`	normally `999`
`seed=`	normally `1`

Underneath NumPy (PERMANOVA and the spread check, with permutations) and SciPy (spatial.distance.pdist)

Example you can run (it uses the practice table `meadow`)

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.compare_profiles(for_each="grazed")
```

Survival

`table.survival_over_time()`

The share still without the event over time (Kaplan-Meier), with the median time.

`table.survival_over_time(time="days", event="died", for_each="treatment")` (with `sdata.example("trial")`)
`time=` is the time until the event or the end of follow-up; `event=` says whether the event happened (yes/no, True/False or 1/0). `for_each=` gives one curve per group. Prints the median time (by when half have had the event) with its 95 % CI per group, or says it was not reached, as R `print(survfit(Surv(days, died == "yes") ~ treatment))`. Gives back a table with one row per event time: the group, time, at risk, events, censored (since the previous row), survival and its 95 % CI (log scale, R's default), as R `summary(survfit(...))`. Draw the curves with `chart.survival_curves(...)`.

Options

`time=`	
`event=`	
`for_each=`	

Underneath NumPy (Kaplan-Meier) and SciPy (stats, for the intervals)

Example you can run (it uses the practice table `trial`)

```
import simplydata as sdata
table = sdata.table(sdata.example("trial"))
table.survival_over_time(time="days", event="died", for_each="treatment")
```

table.compare_survival()

Do the groups differ in the time until the event, or could it be chance? (log-rank test)

`table.compare_survival(time="days", event="died", for_each="treatment")` (with `sdata.example("trial")`)
The log-rank test, as R `survdiff(Surv(days, died == "yes") ~ treatment)`: at each event time the events are shared out over the groups by how many rows are at risk in each, and the observed events are compared with these expected ones over the whole follow-up. Prints the answer in words and the observed vs expected events per group; gives back a one-row table of results.

Options

```
time=
event=
for_each=
```

Underneath NumPy (log-rank sums) and SciPy (stats.chi2)

Example you can run (it uses the practice table `trial`)

```
import simplydata as sdata
table = sdata.table(sdata.example("trial"))
table.compare_survival(time="days", event="died", for_each="treatment")
```

table.explain_survival()

Which columns go with a higher or lower risk of the event? (Cox model, hazard ratios)

`table.explain_survival(time="days", event="died", using=["treatment", "age"])` (with `sdata.example("trial")`)
The Cox proportional hazards model, as R `coxph(Surv(days, died == "yes") ~ treatment + age)`, with Efron's method for tied times. A hazard ratio above 1 means a higher risk of the event at any moment (2 = twice the risk), below 1 a lower risk. Numbers count per 1 more; text columns are groups compared with the first group (A to Z). Prints each hazard ratio with its 95 % CI and p-value, and the likelihood-ratio test of the whole model; a note says if a hazard ratio seems to change over time (the proportional-hazards check of R `cox.zph()`). Gives back a table with one row per term.

Options

```
time=
event=
using=
```

Underneath NumPy (Cox model: Newton-Raphson with Efron ties) and SciPy (stats)

Example you can run (it uses the practice table `trial`)

```
import simplydata as sdata
table = sdata.table(sdata.example("trial"))
table.explain_survival(time="days", event="died", using=["treatment", "age"])
```

`chart.survival_curves()`

The share still without the event over time (Kaplan-Meier curves).

`chart.survival_curves(time="days", event="died", colour="treatment")` (with `sdata.example("trial")`)

`time=` is the time until the event or the end of follow-up; `event=` says whether the event happened (yes/no, True/False or 1/0). `colour="treatment"` draws one curve per group, or `colour="blue"` one colour. Each curve starts at 1 and drops at each event; censored rows (followed without the event) are small ticks (`censor_marks=False` hides them). `confidence_band=True` shades the 95 % confidence interval. `at_risk_table=True` writes the number still at risk under the x-axis. `transparency=` (0 to 1) makes the curves see-through and `thickness=` sets their width in points. The numbers are those of `table.survival_over_time()` (as R `survfit()`).

Options

`time=`	
`event=`	
`colour=`	
`confidence_band=`	normally `False`
`censor_marks=`	normally `True`
`at_risk_table=`	normally `False`
`transparency=`	
`thickness=`	

Underneath NumPy (Kaplan-Meier), drawn with matplotlib

Example you can run (it uses the practice table `trial`)

```
import simplydata as sdata
trial = sdata.example("trial")
chart = sdata.chart(trial)
chart.survival_curves(time="days", event="died", colour="treatment", confidence_band=True)
chart.save("survival_curves.png")
```

Planning a study

`sdata.how_many_needed()`

How many values (in each group) does a study need to find a real difference?

`sdata.how_many_needed(effect_size=0.5)` (two groups, a medium difference: 64 in each)

`test=` is the simplydata test the study will use: `"compare_averages"` (2 groups: t-test; `groups=3` or more: ANOVA with Cohen's f), `"compare_before_after"` (paired, Cohen's d of the changes), `"compare_average_to"` (one sample), `"compare_counts"` (chi-square with Cohen's w; `groups=`(groups, kinds of outcome)) or `"correlation_between"` (r; the answer is pairs). The size of the difference: `effect_size=` (Cohen's d, f, w or r, the sentence says which), or `difference=` and `spread=` (the standard deviation) for the t-tests. `power=` is the chance you want to find it (0.8 is usual); `significance=` is the p-value limit (0.05); `sides=1` looks only in the direction you expect. The t-tests use the usual equal-spread formula (R `power.t.test()`), very close to Welch's t-test when

the groups are the same size; the others match R pwr. The number is rounded UP to a whole number; the exact value is in the result. Gives back a one-row table.

Options

`test=`	normally `"compare_averages"`
`effect_size=`	
`difference=`	
`spread=`	
`power=`	normally `0.8`
`significance=`	normally `0.05`
`groups=`	normally `2`
`sides=`	normally `2`

Underneath SciPy (stats.nct, ncf, ncx2 and norm, and optimize.brentq)

Example you can run

```
import simplydata as sdata
sdata.how_many_needed(effect_size=0.5, power=0.8)
sdata.how_many_needed(test="compare_averages", difference=5, spread=10)
sdata.how_many_needed(test="compare_averages", effect_size=0.25, groups=3)
```

`sdata.chance_to_find()`

What chance does a study have to find a real difference of a given size (its power)?

`sdata.chance_to_find(effect_size=0.5, values_per_group=30)` (two groups of 30: about 48 %)
The same options as `how_many_needed()`, with `values_per_group=` instead of `power=`: the number in each group (pairs for `compare_before_after` and `correlation_between`; all the values for `compare_average_to`). The t-tests use the usual equal-spread formula (R `power.t.test()`), very close to Welch's t-test when the groups are the same size; the others match R pwr. Gives back a one-row table.

Options

`test=`	normally `"compare_averages"`
`effect_size=`	
`difference=`	
`spread=`	
`values_per_group=`	
`significance=`	normally `0.05`
`groups=`	normally `2`
`sides=`	normally `2`

Underneath SciPy (stats.nct, ncf, ncx2 and norm)

Example you can run

```
import simplydata as sdata
sdata.chance_to_find(effect_size=0.5, values_per_group=20)
```

Combine and reshape

table.join_with()

Bring in the columns of another table, matching rows on a column such as a name.

`keep=` says which rows stay:
`"matching rows"` only rows found in both tables (the normal choice)
`"all rows from this table"` every row of this table; no match gives empty cells
`"all rows from both"` every row of both tables
other can be a simplydata table, a pandas or Polars table, or a file name. A column found in both tables (other than `using=`) gets `"_other"` added to the other table's copy.

`table.join_with({"name": ["Asha", "Ben"], "age": [12, 13]}, using="name")` (with `sdata.example("marks")`)

Options

`other=`	
`using=`	
`keep=`	normally `"matching rows"`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
ages = {"name": ["Asha", "Ben"], "age": [12, 13]}
table.join_with(ages, using="name", keep="all rows from this table")
table.show_first_rows()
```

table.add_rows_from()

Put the rows of another table under this one; columns are matched by their names.

`table.add_rows_from(sdata.example("marks"))` (with `sdata.example("marks")`: the class twice)

Options

`other=`	

Underneath pandas (concat) or Polars (concat), whichever holds the table

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_rows_from(sdata.example("marks"))
print(table)
```

`table.turn_columns_into_rows()`

Turn some columns into rows: each value gets its own row, with its old column name beside it.

This makes a `"long"` table, which most charts like. The new column `names_to=` holds the old column names and `values_to=` holds the values.

`table.turn_columns_into_rows(["temperature", "rain"])` (with `sdata.example("weather")`)

Options

`columns=`	
`names_to=`	normally `"column"`
`values_to=`	normally `"value"`

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.keep_columns("month", "city", "temperature", "rain")
table.turn_columns_into_rows(["temperature", "rain"])
table.show_first_rows()
```

`table.turn_rows_into_columns()`

Turn rows into columns: each different value in `names_from=` becomes a new column, filled from `values_from=`.

This makes a `"wide"` table, the opposite of `turn_columns_into_rows()`. All the other columns say which row is which, so rows only merge when those all agree.

`table.turn_rows_into_columns(names_from="city", values_from="temperature")` (with `sdata.example("weather")`)

Options

`names_from=`	
`values_from=`	

Underneath narwhals, which runs the step in pandas or Polars (whichever holds the table)

Example you can run (it uses the practice table `weather`)

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.turn_rows_into_columns(names_from="city", values_from="temperature")
table.show_first_rows()
```

Undo and finish

`table.undo()`

Take the last step back. `table.undo()`

Options none: just write the empty brackets.

Underneath simplydata itself: it keeps every version of the table

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks", above=50)
table.undo()
print(table)
```

`table.steps()`

List every step you did to this table, in order. `table.steps()`

Options none: just write the empty brackets.

Underneath simplydata itself: it keeps every version of the table

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks", above=50)
table.sort_by("marks")
table.steps()
```

`table.start_again()`

Go back to the table as it was at the start. `table.start_again()`

Options none: just write the empty brackets.

Underneath simplydata itself: it keeps every version of the table

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks", above=50)
table.remove_columns("height")
table.start_again()
print(table)
```

table.save()

Save the table; the end of the file name decides the type. `table.save("clean.xlsx")`

File types: `.xlsx` (Excel), `.csv`, `.tsv`, `.parquet` and `.json`.

Options

`file_name`	needed

Underneath pandas (`to_excel`, `to_csv` ...) or Polars, whichever holds the table

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks", above=50)
table.save("good_marks.xlsx")
table.save("good_marks.csv")
```

table.chart()

Start a chart from this table. chart = `table.chart()`

Options none: just write the empty brackets.

Underneath matplotlib and seaborn (the chart is drawn from a pandas copy of the table)

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("subject", equal_to="Maths")
chart = table.chart()
chart.bars(x="name", y="marks")
chart.rotate_labels_x()
chart.save("maths_marks.png")
```

table.as_pandas()

Give back the table as a plain pandas table, for other Python tools. df = `table.as_pandas()`

Only needed when another library wants pandas: simplydata words never need it.

Options none: just write the empty brackets.

Underneath pandas

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks", above=50)
df = table.as_pandas()
print(type(df))
```

`table.as_polars()`

Give back the table as a plain Polars table, for other Python tools. df = `table.as_polars()`

Needs Polars (pip install simplydata[fast]). simplydata words never need it.

Options none: just write the empty brackets.

Underneath Polars

Example you can run (it uses the practice table `marks`)

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks", above=50)
df = table.as_polars()
print(type(df))
```

Number words

Steps on numbers, `nums = sdata.numbers([12, 15, 9, 20, 14])`, and on grids of numbers, `grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])`.

Numbers: make and look

sdata.numbers()

Make a list of numbers.

`sdata.numbers([3, 5, 2, 8])` these numbers (or `sdata.numbers(3, 5, 2, 8)`)
`sdata.numbers(1, to=10)` 1, 2, 3 ... 10 (the end is included)
`sdata.numbers(0, to=100, step=25)` 0, 25, 50, 75, 100
`sdata.numbers(0, to=1, how_many=5)` 5 numbers evenly spaced: 0, 0.25, 0.5, 0.75, 1
`sdata.numbers(10, to=1)` counting down: 10, 9 ... 1
None or NaN in a list is an empty (missing) value.

Options

`values...`	as many as you like, separated by commas
`to=`	
`step=`	
`how_many=`	

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([3, 5, 2, 8])
print(nums)
print(sdata.numbers(1, to=10))
print(sdata.numbers(0, to=1, how_many=5))
```

sdata.random_numbers()

Make random numbers; give `seed=` to get the same numbers every time.

`sdata.random_numbers(how_many=10, between=[0, 1], seed=1)`
any number from 0 up to 1, each equally likely (a uniform distribution)
`sdata.random_numbers(how_many=10, between=[1, 6], whole_numbers=True, seed=1)`
whole numbers from 1 to 6, both ends included, like rolling a dice
`sdata.random_numbers(how_many=100, average=50, standard_deviation=10, seed=1)`
numbers that bunch around the average in a bell shape (a normal distribution); about 2 in 3 land within one standard deviation of the average `rows=` and `columns=` in place of `how_many=` give a grid.

Options

`how_many=`
`between=`
`average=`
`standard_deviation=`
`whole_numbers=` normally `False`
`seed=`
`rows=`
`columns=`

Underneath NumPy (random.default_rng)

Example you can run

```
import simplydata as sdata
dice = sdata.random_numbers(how_many=10, between=[1, 6], whole_numbers=True, seed=1)
dice.show()
heights = sdata.random_numbers(how_many=100, average=150, standard_deviation=8, seed=1)
heights.average()
```

`nums.show()`

Show every number. `nums.show()`

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.show()
```

`nums.show_first_numbers()`

Show the first numbers. `nums.show_first_numbers(5)`

Options

`how_many=` normally `5`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.show_first_numbers(3)
```

nums.show_last_numbers()

Show the last numbers. `nums.show_last_numbers(5)`

Options

`how_many=` normally `5`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.show_last_numbers(2)
```

nums.number_at()

The number at a position, counting from 1: `number_at(1)` is the first number.

```
nums.number_at(1)
```

Positions count from 1, the way people count and the way charts number the x axis. `number_at("last")` is the last number.

Options

`position=`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
print(nums.number_at(1))
print(nums.number_at("last"))
```

nums.count()

How many numbers there are (empty ones included). `nums.count()`

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
print(nums.count())
```

nums.count_empty_numbers()

How many values are empty (missing). `nums.count_empty_numbers()`

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, None, 9, None, 14])
print(nums.count_empty_numbers())
```

Numbers: maths in words

nums.add()

Add a number to every number (or add numbers one by one). `nums.add(5)`

For new numbers, write bigger = nums + 5 instead.

Options

`number=`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.add(5)
nums.show()
```

nums.subtract()

Take a number away from every number. `nums.subtract(2)`

For new numbers, write smaller = nums - 2 instead.

Options

`number=`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.subtract(2)
nums.show()
```

`nums.times()`

Multiply every number by a number. `nums.times(2)`

For new numbers, write double = nums * 2 instead.

Options

`number=`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.times(2)
nums.show()
```

`nums.divide_by()`

Divide every number by a number. `nums.divide_by(4)`

Dividing by 0 has no answer, so those values become empty and a note says how many. For new numbers, write quarter = nums / 4 instead.

Options

`number=`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.divide_by(4)
nums.show()
```

`nums.power()`

Raise every number to a power: `power(2)` squares, `power(3)` cubes. `nums.power(2)`

`power(0.5)` is the same as the square root. For new numbers, write squares = nums ** 2.

Options

`number=`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.power(2)
nums.show()
```

`nums.square_root()`

The square root of every number: 9 becomes 3. `nums.square_root()`

Negative numbers have no square root among ordinary numbers, so they become empty.

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([9, 16, 25])
nums.square_root()
nums.show()
```

`nums.absolute_value()`

Every number without its minus sign: -3 becomes 3. `nums.absolute_value()`

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([-3, 5, -2])
nums.absolute_value()
nums.show()
```

`nums.round_to()`

Round every number to this many decimal places. `nums.round_to(2)`

A number exactly halfway goes to the even neighbour (2.5 becomes 2, 3.5 becomes 4), like R and NumPy; this stops a run of halves from always pushing totals up. `round_to(0)` gives whole numbers; `round_to(-1)` rounds to tens.

Options

`decimals=`	normally `0`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([2.345, 3.14159, 7.07])
nums.round_to(1)
nums.show()
```

nums.log()

The logarithm of every number: natural (base e) unless you give `base=10` or `base=2`.

```
nums.log() nums.log(base=10) nums.log(base=2)
```

`log(base=10)` of 1000 is 3, because 10 x 10 x 10 = 1000. The log only exists for numbers above 0, so 0 and negative numbers become empty.

Options

`base=`	normally `"e"`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([1, 10, 100, 1000])
nums.log(base=10)
nums.show()
```

nums.exp()

e (about 2.718) to the power of every number; it undoes `log()`. `nums.exp()`

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([0, 1, 2])
nums.exp()
nums.show()
```

nums.running_total()

Each number becomes the total so far (the cumulative sum): 1, 2, 3 becomes 1, 3, 6.

```
nums.running_total()
```

Empty values stay empty and are skipped in the total, like pandas `cumsum()`.

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([1, 2, 3, 4])
nums.running_total()
nums.show()
```

Numbers: choose, sort and fill

nums.keep_numbers()

Keep only the numbers that match one condition. `nums.keep_numbers(above=50)`

Conditions (the same as `table.keep_rows`): above, below, `at_least`, `at_most`, `equal_to`, `not_equal_to`, `between=[low, high]` (both ends included), `one_of=[...]`, `is_empty=True`. Empty values only match `is_empty`.

Options

```
above=
below=
at_least=
at_most=
equal_to=
not_equal_to=
between=
one_of=
is_empty=
```

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.keep_numbers(above=10)
nums.show()
```

nums.remove_numbers()

Remove the numbers that match one condition (the opposite of `keep_numbers`).

```
nums.remove_numbers(below=0)
```

Same conditions as `keep_numbers`; `between=[low, high]` includes both ends.

Options

```
above=
below=
at_least=
at_most=
equal_to=
not_equal_to=
between=
one_of=
is_empty=
```

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.remove_numbers(below=10)
nums.show()
```

`nums.keep_first_numbers()`

Keep only the first numbers. `nums.keep_first_numbers(10)`

Options

`how_many=`	normally `5`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.keep_first_numbers(3)
nums.show()
```

`nums.keep_last_numbers()`

Keep only the last numbers. `nums.keep_last_numbers(10)`

Options

`how_many=`	normally `5`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.keep_last_numbers(3)
nums.show()
```

`nums.keep_random_numbers()`

Keep some numbers picked at random (they keep their order); a seed gives the same pick every time.

`nums.keep_random_numbers(10, seed=1)`

Options

`how_many=`	normally `5`
`seed=`	

Underneath NumPy (random.default_rng)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.keep_random_numbers(3, seed=1)
nums.show()
```

`nums.remove_duplicate_numbers()`

Remove numbers that repeat an earlier number, so each number is left once, in its first place.

```
nums.remove_duplicate_numbers()
```

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([3, 5, 3, 8, 5])
nums.remove_duplicate_numbers()
nums.show()
```

`nums.remove_empty_numbers()`

Remove the empty (missing) values. `nums.remove_empty_numbers()`

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, None, 9, 14])
nums.remove_empty_numbers()
nums.show()
```

`nums.keep_numbers_by_position()`

Keep numbers by their position, counting from 1: a run, every n-th number, or a list.

```
nums.keep_numbers_by_position(2, to=4)
```

Both ends are included. `to="last"` goes to the end; `step=2` keeps every 2nd number; a list such as [1, 4] keeps those numbers in that order. As NumPy a[1:4] (positions, not values: `keep_numbers(between=[2, 4])` keeps the values from 2 to 4).

Options

```
positions=
to=
step=
```

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.keep_numbers_by_position(2, to=4)
nums.show()
```

`nums.remove_numbers_by_position()`

Remove numbers by their position, counting from 1: a run, every n-th number, or a list.

```
nums.remove_numbers_by_position("last")
```

The opposite of `keep_numbers_by_position`, with the same positions. As NumPy `np.delete(a, -1)`.

Options

```
positions=
to=
step=
```

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.remove_numbers_by_position("last")
nums.show()
```

`nums.replace_numbers()`

Replace a number, or every number that meets one condition; `with_value=None` makes them empty.

```
nums.replace_numbers(-999, with_value=None)  a code for "missing" becomes empty
nums.replace_numbers(above=100, with_value=100)  every number above 100 becomes 100
```

The conditions are `keep_numbers`' (one at a time): above, below, `at_least`, `at_most`, `equal_to`, `not_equal_to`, between, `one_of`, `is_empty`. A grid keeps its rows and columns. As NumPy `np.where(a > 100, 100, a)` and pandas `s.mask(s > 100, 100)`.

Options

```
old=
with_value=
above=
below=
at_least=
at_most=
equal_to=
not_equal_to=
between=
one_of=
is_empty=
```

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, -999, 9, 20, 14])
nums.replace_numbers(-999, with_value=None)
nums.replace_numbers(above=14, with_value=14)
nums.show()
```

`nums.fill_empty_numbers()`

Fill the empty values with a number, or with the average, median, value above or value below.

`nums.fill_empty_numbers(with_value=0)` or `nums.fill_empty_numbers(with_the="average")` `with_the="line between neighbours"` fills each gap on the straight line between the values just before and just after it; empty values at the start or end stay empty.

Options

`with_value=`
`with_the=`

Underneath NumPy (interp for the line between neighbours)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, None, 9, None, 14])
nums.fill_empty_numbers(with_the="line between neighbours")
nums.show()
```

`nums.sort()`

Put the numbers in order, smallest first; empty values go last. `nums.sort(biggest_first=True)`

Options

`biggest_first=` normally `False`

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.sort(biggest_first=True)
nums.show()
```

`nums.reverse()`

Turn the order round: the last number comes first. `nums.reverse()`

Options none: just write the empty brackets.

Underneath NumPy

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.reverse()
nums.show()
```

Numbers: summaries

`nums.average()`

The average (mean) of the numbers; empty values are skipped.

```
nums.average()
```

Options none: just write the empty brackets.

Underneath NumPy (functions that skip empty values, such as nanmean)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.average()
```

`nums.median()`

The middle number once the numbers are sorted; empty values are skipped.

```
nums.median()
```

Options none: just write the empty brackets.

Underneath NumPy (functions that skip empty values, such as nanmean)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.median()
```

`nums.total()`

All the numbers added up; empty values are skipped.

```
nums.total()
```

Options none: just write the empty brackets.

Underneath NumPy (functions that skip empty values, such as nanmean)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.total()
```

nums.smallest()

The smallest number; empty values are skipped.

```
nums.smallest()
```

Options none: just write the empty brackets.

Underneath NumPy (functions that skip empty values, such as nanmean)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.smallest()
```

nums.biggest()

The biggest number; empty values are skipped.

```
nums.biggest()
```

Options none: just write the empty brackets.

Underneath NumPy (functions that skip empty values, such as nanmean)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.biggest()
```

nums.standard_deviation()

How spread out the numbers are: the sample standard deviation (divide by n - 1).

It is the same as pandas .std() and R `sd()`. Empty values are skipped.

```
nums.standard_deviation()
```

Options none: just write the empty brackets.

Underneath NumPy (functions that skip empty values, such as nanmean)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.standard_deviation()
```

nums.percentile()

The number that this percent of the numbers are below: `percentile(50)` is the median.

```
nums.percentile(90)
```

Uses NumPy's usual `"linear"` method, the same as pandas and R. Empty values are skipped.

Options

```
percent=
```

Underneath NumPy (nanpercentile, the usual `"linear"` method, as pandas and R)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.percentile(90)
```

nums.count_each()

Each different number, how many times it appears, and its percent, as a table.

Biggest count first. Empty values count as a value of their own, so the percents are out of all numbers. Percents are rounded to 1 decimal place.

```
nums.count_each()
```

Options none: just write the empty brackets.

Underneath NumPy (functions that skip empty values, such as nanmean)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([3, 5, 3, 8, 5, 3])
nums.count_each()
```

Grids of numbers

sdata.number_grid()

Make a grid of numbers in rows and columns.

`sdata.number_grid([[1, 2, 3], [4, 5, 6]])` from rows of numbers (2 rows, 3 columns)

`sdata.number_grid(rows=3, columns=4, filled_with=0)` a grid of 0s (filled_with=1 for 1s, or any number)

`sdata.number_grid([1, 2, 3, 4, 5, 6], columns=3)` a list cut into rows of 3

Options

```
values=
rows=
columns=
filled_with=                    normally 0
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[1, 2, 3], [4, 5, 6]])
print(grid)
print(sdata.number_grid(rows=2, columns=3, filled_with=0))
```

`nums.number_grid()`

Make a NEW grid from these numbers, filled row by row. grid = `nums.number_grid(columns=3)`

Give `rows=` or `columns=` (or both); the numbers must fill the grid exactly.

Options

```
rows=
columns=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([1, 2, 3, 4, 5, 6])
grid = nums.number_grid(columns=3)
print(grid)
```

`grid.count_rows()`

How many rows the grid has. `grid.count_rows()`

Options none: just write the empty brackets.

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
print(grid.count_rows())
```

`grid.count_columns()`

How many columns the grid has. `grid.count_columns()`

Options none: just write the empty brackets.

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
print(grid.count_columns())
```

`grid.shape()`

How many rows and columns, as (rows, columns). `grid.shape()`

Options none: just write the empty brackets.

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
print(grid.shape())
```

`grid.show_first_rows()`

Show the first rows. `grid.show_first_rows(5)`

Options

`rows=`	normally `5`

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
grid.show_first_rows(1)
```

`grid.show_last_rows()`

Show the last rows. `grid.show_last_rows(5)`

Options

`rows=`	normally `5`

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
grid.show_last_rows(1)
```

grid.number_at()

The number at a row and column, counting from 1. `grid.number_at(row=1, column=2)`

`"last"` is the last row or column: `grid.number_at(row="last", column="last")`.

Options

```
row=
column=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
print(grid.number_at(row=1, column=2))
```

grid.row()

One row of the grid as new numbers, counting from 1: `row(1)` is the top row. `grid.row(1)`

```
grid.row("last") is the bottom row.
```

Options

```
position=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
top = grid.row(1)
top.show()
```

grid.column()

One column of the grid as new numbers, counting from 1: `column(1)` is the left one. `grid.column(1)`

```
grid.column("last") is the right one.
```

Options

```
position=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
left = grid.column(1)
left.show()
```

grid.numbers()

All the numbers of the grid as a list, row by row (flatten). nums = `grid.numbers()`

Options none: just write the empty brackets.

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
nums = grid.numbers()
nums.show()
```

grid.keep_rows_by_position()

Keep rows of the grid by their position, counting from 1: a run, every n-th row, or a list.

```
grid.keep_rows_by_position(1, to=2)
```

Both ends are included; `to="last"` goes to the bottom; `step=2` keeps every 2nd row; a list such as [2, 1] keeps those rows in that order. A block is two steps:

```
grid.keep_rows_by_position(2, to=5).keep_columns_by_position(1, to=3), as NumPy g[1:5, 0:3].
```

Options

```
positions=
to=
step=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.numbers(1, to=24).number_grid(columns=6)
grid.keep_rows_by_position(2, to=3)
print(grid)
```

grid.keep_columns_by_position()

Keep columns of the grid by their position, counting from 1 at the left. `grid.keep_columns_by_position(2, to=3)`

The same positions as `keep_rows_by_position`: a run (`to=`), every n-th (`step=`) or a list. As NumPy g[:, 1:3].

Options

```
positions=
to=
step=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.numbers(1, to=24).number_grid(columns=6)
grid.keep_rows_by_position(2, to=3)
grid.keep_columns_by_position(1, to=3)
print(grid)
```

`grid.remove_rows_by_position()`

Remove rows of the grid by their position, counting from 1. `grid.remove_rows_by_position(1)`

The same positions as `keep_rows_by_position`. As NumPy `np.delete(g, 0, axis=0)`.

Options

```
positions=
to=
step=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.numbers(1, to=24).number_grid(columns=6)
grid.remove_rows_by_position(1)
print(grid)
```

`grid.remove_columns_by_position()`

Remove columns of the grid by their position, counting from 1 at the left. `grid.remove_columns_by_position(1)`

The same positions as `keep_columns_by_position`. As NumPy `np.delete(g, 0, axis=1)`.

Options

```
positions=
to=
step=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.numbers(1, to=24).number_grid(columns=6)
grid.remove_columns_by_position(1, step=2)
print(grid)
```

`grid.replace_numbers()`

Replace a number, or every number that meets one condition; the grid keeps its rows and columns.

```
grid.replace_numbers(below=4, with_value=0)
```

The conditions are `keep_numbers`' (one at a time): above, below, `at_least`, `at_most`, `equal_to`, `not_equal_to`, between, `one_of`, `is_empty`; `with_value=None` makes the numbers empty. As NumPy `np.where(g < 4, 0, g)` or $g[g < 4] = 0$.

Options

```
old=
with_value=
above=
below=
at_least=
at_most=
equal_to=
not_equal_to=
between=
one_of=
is_empty=
```

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
grid.replace_numbers(below=4, with_value=0)
print(grid)
```

`grid.swap_rows_and_columns()`

Turn rows into columns and columns into rows (the transpose in maths).

```
grid.swap_rows_and_columns()
```

A grid of 2 rows and 3 columns becomes 3 rows and 2 columns. Matrix multiplication and other linear algebra are planned for simplydata v0.7.

Options none: just write the empty brackets.

Underneath NumPy (a two-dimensional array)

Example you can run

```
import simplydata as sdata
grid = sdata.number_grid([[3, 5, 2], [8, 6, 4]])
grid.swap_rows_and_columns()
print(grid)
```

Numbers: undo and finish

nums.undo()

Take the last step back. `nums.undo()`

Options none: just write the empty brackets.

Underneath simplydata itself: it keeps every version of the numbers

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.add(5)
nums.undo()
nums.show()
```

nums.steps()

List every step you did, in order. `nums.steps()`

Options none: just write the empty brackets.

Underneath simplydata itself: it keeps every version of the numbers

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.add(5)
nums.keep_numbers(above=15)
nums.steps()
```

nums.start_again()

Go back to the numbers as they were at the start. `nums.start_again()`

Options none: just write the empty brackets.

Underneath simplydata itself: it keeps every version of the numbers

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.add(5)
nums.times(2)
nums.start_again()
nums.show()
```

nums.table()

Turn the numbers into a simplydata table. `nums.table(name="marks")`

Options

`name=`	normally `"value"`

Underneath pandas

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
table = nums.table(name="marks")
print(table)
```

nums.chart()

Start a chart from these numbers. chart = `nums.chart()`

Options none: just write the empty brackets.

Underneath matplotlib and seaborn, from a pandas table with one column called value

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
chart = nums.chart()
chart.histogram(x="value", bins=4)
chart.save("nums.png")
```

nums.save()

Save the numbers; the end of the file name decides the type. `nums.save("nums.csv")`

File types: `.csv`, `.tsv`, `.xlsx` (Excel), `.parquet` and `.json`, the same as tables.

Options

`file_name`	needed

Underneath pandas (`to_csv`, `to_excel` ...)

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
nums.save("nums.csv")
```

`nums.as_numpy()`

Give back the numbers as a plain NumPy array, for other Python tools. array = `nums.as_numpy()`

Options none: just write the empty brackets.

Underneath simplydata itself: it keeps every version of the numbers

Example you can run

```
import simplydata as sdata
nums = sdata.numbers([12, 15, 9, 20, 14])
array = nums.as_numpy()
print(type(array))
```

Help words

Words that teach simplydata.

Help

`sdata.words()`

Print every simplydata word with a short example. `sdata.words()`

For more about one word, type `sdata.help("bars")`.

Options none: just write the empty brackets.

Underneath plain Python: simplydata reads its own word list and docstrings

Example you can run

```
import simplydata as sdata
sdata.words()
```

`sdata.help()`

Explain one word: what it does, its options and an example you can run. `sdata.help("bars")`

```
sdata.help() on its own explains how to get help. Table words work too: sdata.help("keep_rows").
```

Options

```
word=
```

Underneath plain Python: simplydata reads its own word list and docstrings

Example you can run

```
import simplydata as sdata
sdata.help("bars")
sdata.help("show=")
sdata.help("keep_rows")
sdata.help("average")
```

`sdata.lesson()`

A lesson: an example to run, what each line does, and a task. `sdata.lesson(1)`

`sdata.lesson()` on its own lists all the lessons. `solutions=True` also shows a worked solution for each task: `sdata.lesson(3, solutions=True)`.

Options

`number=`
`solutions=` normally `False`

Underneath plain Python: simplydata reads its own word list and docstrings

Example you can run

```
import simplydata as sdata
sdata.lesson()
sdata.lesson(1)
```

PART IV

Data science ideas

Describing numbers

Before any test or model, a few numbers describe a column well: where its middle is, and how spread out it is. simplydata's summary words say each one in a sentence.

Average or median?

The **average** (formally the **arithmetic mean**, written $\bar{x}$) adds all the values and divides by how many there are:

$$\bar{x} = \frac{x_1 + x_2 + \cdots + x_n}{n}.$$

The **median** is the middle value when the values are put in order (with an even number of values, the average of the two in the middle). Half the values are below it and half above.

Both describe the "middle", but they react differently to values far from the rest (**outliers**). Five children earn pocket money of 2, 3, 3, 4 and 50 a week: the average is 12.4, though four of the five get 4 or less; the median is 3. One extreme value pulls the average towards it but hardly moves the median. So:

- Use the **average** when the values are roughly symmetric (a bell shape), and when you want to add things up (total = average × count).
- Use the **median** when the values are lopsided (**skewed**), such as incomes, house prices, waiting times or bacterial counts, or when there are outliers you cannot remove.
- When the two are far apart, say so: it tells the reader the values are skewed.

In simplydata: `table.average_of("marks")`, `table.median_of("marks")`, `nums.average()`, `nums.median()`, and on charts `show="average"` or `show="median"`. Lesson 5 draws both.

Spread: range, percentiles and standard deviation

Two classes can have the same average mark, one with every child near 65 and the other with marks from 20 to 100. The **spread** tells them apart.

- The **range** is the biggest value minus the smallest. It is easy to understand but depends on just two values, often the strangest ones.
- A **percentile** is the value that a given share of the values is below: `nums.percentile(90)` is the value that 90 % are below. The median is the 50th percentile. The middle half of the values lies between the 25th and 75th percentiles; that distance is the **interquartile range (IQR)**, and it is the box in a box plot (`chart.boxes`).
- The **standard deviation** (SD, written s) is about how far a typical value is from the average:

$$s = \sqrt{\frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n - 1}}.$$

  Its square, s^2, is the **variance**. Dividing by $n - 1$ instead of n (Bessel's correction) makes the sample variance an unbiased estimate of the variance of everything the sample came from;

simplydata always divides by $n - 1$, as pandas and R do (NumPy divides by n unless told otherwise).

The SD goes with the average and the IQR with the median. For all the marks of the practice table: “average 64.9 (SD 15.0)” or “median 68.5 (IQR 54.3 to 74.8)”.

The shape of the values: distributions

The way values are spread out is their **distribution**; a histogram (`chart.histogram`) or a density curve (`chart.density`) shows it. Many measurements, such as heights or measuring errors, follow a bell shape called the **normal** (or Gaussian) **distribution**. For a normal distribution about 68 % of the values lie within one SD of the average and about 95 % within two SD.

Not everything is bell-shaped. Counts and concentrations are often skewed to the right (many small values, a few large ones); a log scale (`chart.log_scale_y()`) often makes them look more symmetric. Look at the distribution before choosing what to report and which test to use.

How sure are we?

We almost never measure everything. We measure a **sample** (18 children, 40 water samples) and want to learn about the **population** it came from (all children of that age, the whole fjord). A number worked out from a sample, such as its average, is an **estimate**, and another sample would give a slightly different one. These ideas say how much.

Standard deviation or standard error?

They sound alike and are often mixed up, but they answer different questions.

	Standard deviation (SD)	**Standard error of the mean (SE, SEM)**
Describes	the *values*: how spread out they are	the *average*: how precisely the sample pins it down
Formula	s (above)	SE = $s/\sqrt{n}$
More data	stays about the same	gets smaller (4 times the data halves it)
Use it to	show the spread of individuals	compare averages, build confidence intervals
simplydata	`error="standard deviation"`	`error="standard error"`

Error bars of the SE are always shorter than those of the SD, and can look reassuringly small. That is why a figure caption must say which one was drawn; simplydata prints a line saying it, ready for your caption.

Confidence intervals

A **95 % confidence interval** (CI) is a range of plausible values for the true number in the population, worked out from the sample. For an average it is

$$\bar{x} \pm t_{0.975,\,n-1} \times \mathrm{SE},$$

where $t_{0.975,\,n-1}$ is a number from the t distribution: about 2 for most samples, a bit more for small ones (2.26 for $n = 10$, 2.09 for $n = 20$).

What "95 %" means, exactly: if you repeated the whole study many times and built an interval each time, about 95 out of 100 of those intervals would contain the true value. It does *not* mean that the true value moves around, nor that 95 % of the individual values lie inside (that would be about $\bar{x} \pm 2$ SD for a bell shape).

- A **narrow** interval means the data pin the value down well; a **wide** one means you need more data.
- An interval says more than a p-value: it gives the size of the effect *and* how sure you are of it. `table.fit_straight_line` prints the slope with its 95 % CI.
- On charts: `error="confidence interval"` on bars and lines, and `fit_line(confidence_band=True)` for a fitted line.

Why averages behave well: the central limit theorem

Even when the values themselves are skewed, the *average* of many of them is close to bell-shaped; this is the **central limit theorem**. It is why tests on averages (t-tests, ANOVA) usually work well with 30 or more values per group, even for values that are not bell-shaped. With small groups of skewed values, rank-based tests are safer (`based_on="ranks"`).

Testing: real difference, or chance?

The null hypothesis and the p-value

Two groups almost never have exactly the same average, even when nothing real separates them: samples differ by chance. A statistical test asks whether a difference is bigger than chance alone would often produce. It works like a court case, where the accused is innocent until proven guilty:

1. Assume there is **no real difference**. This assumption is the **null hypothesis** (H_0). The **alternative hypothesis** (H_1) is that there is one.
2. Work out how surprising your data would be if H_0 were true. The **p-value** is the probability of a difference at least as big as the one you saw, if there were really no difference.
3. A small p-value means the data are hard to explain by chance alone. By custom, below 0.05 is called **statistically significant**; that threshold is the **significance level** α.

simplydata says the p-value in words: below 0.001 "very unlikely to be chance", below 0.05 "unlikely to be chance", otherwise "could easily be chance".

Good to know

What a p-value is **not**:

- It is **not** the probability that H_0 is true, nor the probability that the result is a fluke.
- A small p-value does **not** say the difference is big or important. With thousands of values, tiny, meaningless differences become "significant". The *effect size* says how big it is.
- A large p-value does **not** show that there is no difference. It only says these data cannot tell. "Absence of evidence is not evidence of absence."
- 0.049 and 0.051 are practically the same evidence. Report the exact p-value, not just "$p < 0.05$".

Two kinds of mistake, and power

- A **false alarm** (**type I error**, a false positive): the test says there is a difference when there is none. With $\alpha = 0.05$ this happens in about 1 test in 20 when H_0 is true.
- A **missed difference** (**type II error**, a false negative): there is a real difference, but the test does not find it. Small samples miss differences often.

The **power** of a study is the chance of finding a difference of a given size if it is really there. More data, less noisy measurements and bigger true differences all raise power. A study with low power can only find large effects; plan the sample size before collecting data.

Many tests at once

Every test at $\alpha = 0.05$ carries a 5 % risk of a false alarm, and the risks add up: with 20 tests on pure noise, you expect about one "significant" result. Correcting for **multiple comparisons** keeps the chance of *any* false alarm at 5 %. `table.compare_each_pair` uses **Holm's method**, which is never less powerful than the simpler Bonferroni correction (multiply each p-value by the number of tests). Decide which comparisons you will make before you look at the data.

Effect sizes: how big is the difference?

The p-value answers "could it be chance?". The **effect size** answers "how big is it?", which is usually what matters. Report both, ideally with a confidence interval for the effect.

Effect size	What it says	Given by
Difference in averages	in the units of the data ("7.6 marks higher"): often the clearest	`compare_averages`, `compare_each_pair`
Cohen's d, Hedges' g	the difference in standard deviations; g corrects d for small samples. Rough guide: 0.2 small, 0.5 medium, 0.8 large	`compare_averages` (2 groups), `compare_average_to`, `compare_before_after` (d_z)
Omega-squared ω^2	the share of all the differences that the groups explain (0 to 1)	`compare_averages` (3+ groups)
Cramér's V	how strongly two categories are linked, 0 (none) to 1	`compare_counts`
Correlation r	how closely two columns go up and down together, −1 to 1	`correlation_between`
R^2	the share of the differences in y that a line or curve explains	`fit_straight_line`, `fit_formula`

The “small, medium, large” labels (from Jacob Cohen) are only a last resort: a “small” difference in survival can be important, and a “large” one in a lab assay may not be.

Links between columns

Correlation

A **correlation** measures how closely two columns of numbers go up and down together, from −1 (one always goes down when the other goes up) through 0 (no straight-line link) to 1.

- **Pearson’s** r (the usual one) measures a *straight-line* link. It is sensitive to outliers, and can be near 0 for a strong curved link (a U shape).
- **Spearman’s** ρ (rho) uses ranks: it measures any link that keeps going one way (always up, or always down), and is robust to outliers. `table.correlation_between("hours", "marks", method="spearman")`.
- r^2 is the share of the differences in one column that a straight line through the other explains. $r = 0.5$ explains only a quarter.

Always draw the dots (`chart.dots`) before trusting a correlation: very different patterns can have the same r (Anscombe’s quartet is the famous example).

Correlation is not cause

Ice cream sales and drownings go up together, but ice cream does not cause drowning: hot weather drives both. When two things go together there are several possible reasons:

1. A causes B.
2. B causes A (**reverse causation**): do people exercise because they are healthy, or are they healthy because they exercise?
3. A third thing causes both (a **confounder**): hot weather, age, wealth, or the size of a hospital.
4. Chance: test enough pairs of columns and some will line up by luck.

Only an **experiment** where you change A yourself, ideally by random assignment (a randomised controlled trial), shows cause directly. With observational data, say “is associated with”, not “causes”. simplydata’s link words remind you: “A link does not show that one thing causes the other.”

Fitting a line or a formula

Linear regression finds the straight line $y = a + b\,x$ that goes as close as possible to the dots, by making the sum of the squared vertical distances from the dots to the line as small as possible (**least squares**). The **slope** b says how much y changes for each 1 more of x; its p-value tests whether the slope could be 0, and its confidence interval gives a range of plausible slopes. R^2 says how much of the differences in y the line explains. `table.fit_straight_line(x="hours", y="marks")` prints all of these; `chart.fit_line()` draws the line.

When the relation is curved, such as growth that multiplies by the same share each year, write the formula yourself: `table.fit_formula("population = a * exp(b * (year - 2000))")` finds a and b by **nonlinear least squares**. A fitted line or curve describes the data you have; predicting far outside their range (**extrapolation**) is risky.

The shape of a table

Tidy data: long and wide tables

The same marks can be stored in two shapes:

Long (tidy)			**Wide**			
name	subject	marks	name	Maths	Science	English
Asha	Maths	61	Asha	61	66	77
Asha	Science	66	Ben	73	98	71
Asha	English	77				
Ben	Maths	73				
…						

A table is **tidy** (Hadley Wickham's term) when each **variable** is a column, each **observation** is a row, and each value has its own cell. The long table is tidy: it has one column for what was measured (`marks`) and columns that say which child and which subject.

- **Long tables** suit charts, groups and tests: `chart.bars(x="subject", y="marks")` and `compare_averages("marks", between="subject")` need the subject as a column.
- **Wide tables** suit sums across columns (a total per child) and paired tests (`compare_before_after("Maths", "Science")`), and are how people often type data into a spreadsheet.
- `table.turn_columns_into_rows` makes a wide table long (in pandas: `melt`; in R: `pivot_longer`); `table.turn_rows_into_columns` makes a long table wide (`pivot`).

Missing values and imputation

Real data has empty cells: a sample was lost, a question skipped. What to do depends on *why* they are empty:

- **Missing completely at random**: the reason has nothing to do with the data (a tube dropped). Leaving those rows out loses some power but does not bias the result.
- **Missing at random**: the reason depends on other columns you have (older people skip a question more often). Leaving rows out can bias the result; taking that other column into account helps.
- **Missing not at random**: the reason depends on the missing value itself (very ill patients drop out; concentrations below the detection limit). No simple fix removes the bias; say so in your report.

Removing rows with empty cells (`table.remove_rows_with_empty_cells()`) is simple and honest when few rows are affected. **Filling** them in is called **imputation**: `table.fill_empty_cells("marks", with_the="average")` (also `"median"`, `"value above"`, `"value below"`, or `with_value=0`). Filling with the average keeps the average the same, but makes the values look *less spread out* than they really are, so standard deviations, standard errors and p-values become too small. Filling with the value above or below suits a series in time order. Whatever you choose, count the empty cells first (`table.count_empty_cells()`) and say in your methods how you handled them.

Which test when?

First draw the data: a chart shows outliers, skew and odd values that a single p-value hides. Then answer four questions:

1. **What is the question?** A difference between groups, a change within the same rows, a link between two columns, or the shape of a relation?
2. **What kind of data?** Numbers (marks, heights, concentrations) or categories (pass or fail, species, site)?
3. **How many groups, and are they the same rows?** Two groups or more; independent groups (different children) or paired measurements (the same children twice)?
4. **Are the groups small and the values far from bell-shaped?** Then use the rank-based version: `based_on="ranks"`. simplydata checks each group for a bell shape (Shapiro–Wilk test) and adds a note when it matters.

Question	simplydata word	Test (the usual one)	With `based_on="ranks"`
Is one average different from a number you choose?	`compare_average_to("marks", 60)`	one-sample t-test	Wilcoxon signed-rank test
Do **two** groups differ in their average?	`compare_averages("marks", between="group")`	Welch's t-test	Mann–Whitney U test
Do **three or more** groups differ?	`compare_averages("marks", between="class")`	Welch's one-way ANOVA	Kruskal–Wallis test
Which groups differ?	`compare_each_pair("marks", between="class")`	pairwise Welch t-tests, Holm's correction	pairwise Mann–Whitney tests, Holm's correction
Did the values change, in the **same** rows measured twice?	`compare_before_after("before", "after")`	paired t-test	Wilcoxon signed-rank test
Do **counts** of categories differ between groups?	`compare_counts("result", between="class")`	chi-square test of independence	— (for a 2×2 table with small counts: `test="fisher"`, Fisher's exact test)
Do two columns of numbers go up and down together?	`correlation_between("hours", "marks")`	Pearson's r	`method="spearman"`: Spearman's ρ
How much does y change with x?	`fit_straight_line(x="hours", y="marks")`	linear regression (least squares)	—
Does the data follow a formula (growth, decay, saturation)?	`fit_formula("y = a * exp(b * x)")`	nonlinear least squares	—

Good to know

- **Welch, not Student.** For averages simplydata uses Welch's versions, which do not assume that the groups have the same spread. They are R's defaults and lose almost nothing when the spreads are equal. An expert can still choose: `test="student"` or `test="anova"`.
- **Paired or not?** If each row of one measurement belongs to a row of the other (the same child, the same site), the test must be paired: `compare_before_after`. Treating paired data as independent groups throws away information.
- **Counts, not percentages.** `compare_counts` needs the rows themselves (one row per observation), not percentages worked out by hand.
- **One test per question.** Decide the question and the test before looking at the results; trying several tests and keeping the best p-value makes false alarms much more likely.
- Each test gives back a one-row table with every number in it: `result = table.compare_averages(...)`, then `result.save("test.csv")`.

Writing it up

A good results sentence lets a reader check your work. Give:

1. the summary for each group (average and SD, or median and IQR) and the number of values n;
2. the effect size, ideally with its 95 % confidence interval;
3. the name of the test and the exact p-value;
4. in figure captions, what the bars or bands show (SD, SE or 95 % CI) and n.

For example: "Class A scored on average 13.2 marks more than class C (Welch's t-test with Holm's correction, $p = 0.023$; $n = 18$ per class)." simplydata's printed answers name each test, and `table.steps()` lists every change you made to the table, in order, for your methods.

PART V

Solutions

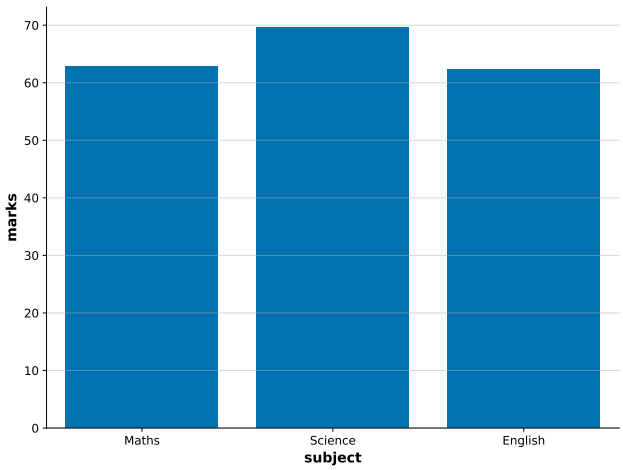
Solutions to the tasks

One worked solution for each “Try it yourself” task. There is often more than one good answer: if yours runs and answers the question, it is right. Each solution runs on its own and was run to make the chart and output shown. In Python, `sdata.lesson(3, solutions=True)` prints the solutions of lesson 3.

Lesson 1: Your first chart

Task 1. Change `x="name"` to `x="subject"`. What does each bar show now?

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="subject", y="marks")
chart.save("lesson1_subjects.png")
```

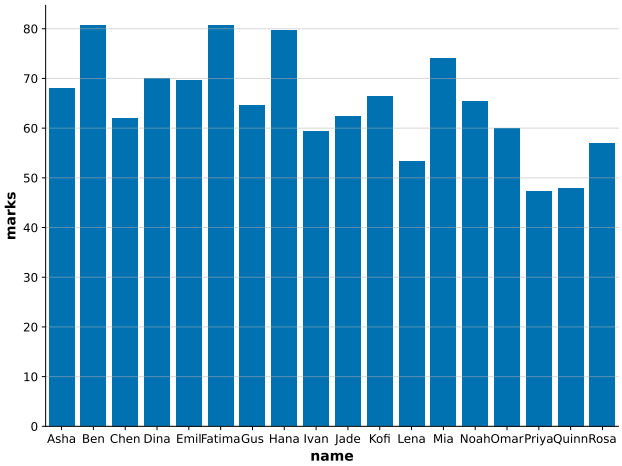


Lesson 1, task 1: `lesson1_subjects.png`.

Answer Each bar is now one subject, as tall as the average mark of all 18 children in that subject.

Task 2. Save the chart as `"my_first_chart.png"` and also as `"my_first_chart.pdf"`.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.save("my_first_chart.png")
chart.save("my_first_chart.pdf")
```

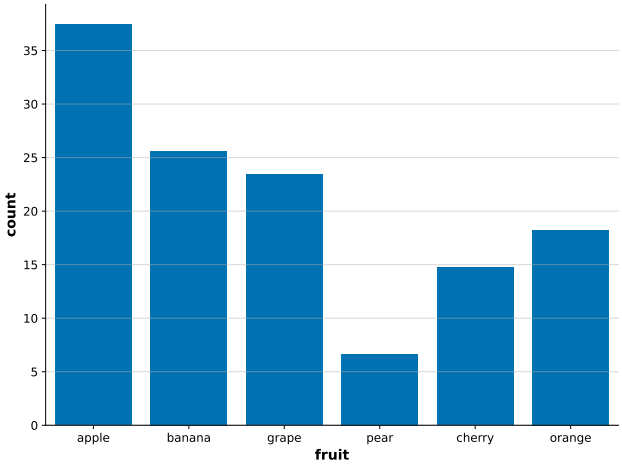


Lesson 1, task 2: `my_first_chart.png`.

Answer One chart can be saved many times; the end of the file name (`.png` or `.pdf`) decides the type.

Task 3. Make a chart of the `"fruit"` table: bars with `x="fruit"` and `y="count"`.

```
import simplydata as sdata
fruit = sdata.example("fruit")
chart = sdata.chart(fruit)
chart.bars(x="fruit", y="count")
chart.save("fruit.png")
```



Lesson 1, task 3: `fruit.png`.

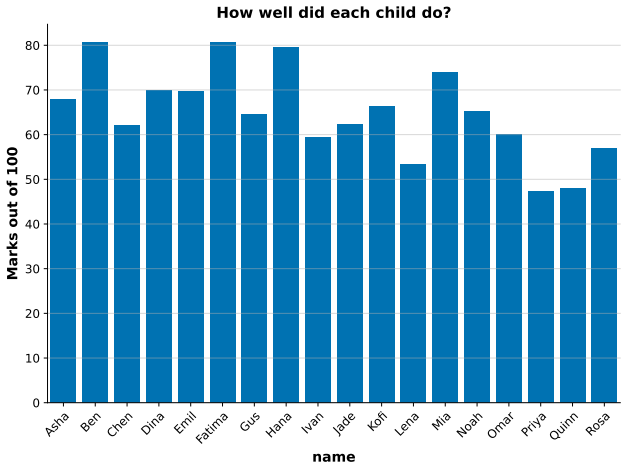
Answer One bar per fruit. Each fruit was sold several times, so a bar is the average count of one sale; `show="total"` would show how many were sold in the whole week.

Back to lesson 1

Lesson 2: Titles and labels

Task 1. Give the chart your own title, and label the y-axis `"Marks out of 100"`.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.title_top("How well did each child do?")
chart.label_y("Marks out of 100")
chart.rotate_labels_x(45)
chart.save("lesson2_my_title.png")
```

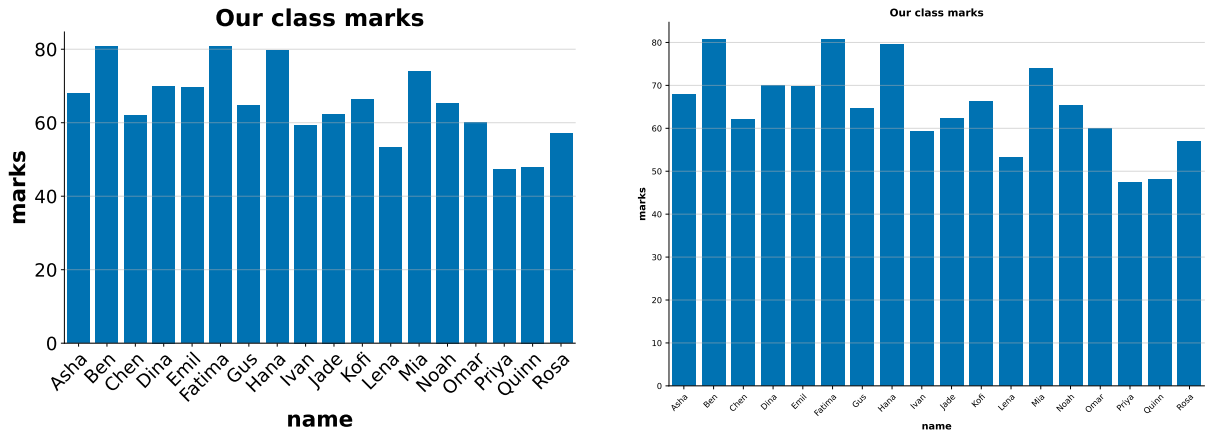


Lesson 2, task 1: `lesson2_my_title.png`.

Answer `title_top` writes any title you like, and `label_y` names the y-axis.

Task 2. Try `chart.font_size(20)`. Then try 9. Which is easier to read?

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.title_top("Our class marks")
chart.rotate_labels_x(45)
chart.font_size(20)
chart.save("lesson2_big_text.png")
chart.font_size(9)
chart.save("lesson2_small_text.png")
```

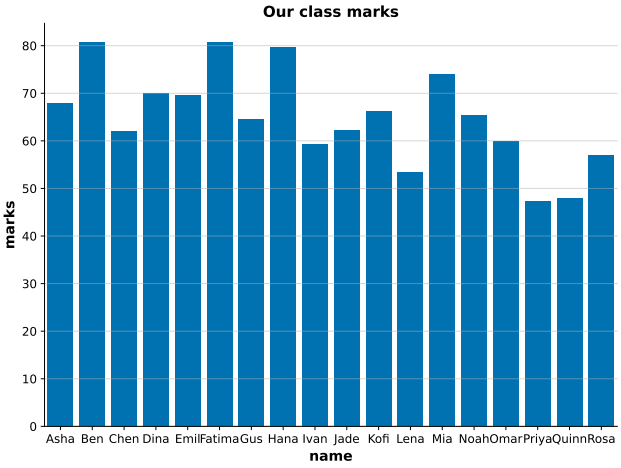


Lesson 2, task 2: `lesson2_big_text.png` and `lesson2_small_text.png`.

Answer The last `font_size` wins, so one chart gives both pictures. Size 20 is easy to read on a slide but crowds the names; 9 fits everything but is small. 12 to 14 suits most charts.

Task 3. Take away the line `chart.rotate_labels_x(45)`. What happens to the names?

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.title_top("Our class marks")
chart.save("lesson2_no_rotate.png")
```



Lesson 2, task 3: `lesson2_no_rotate.png`.

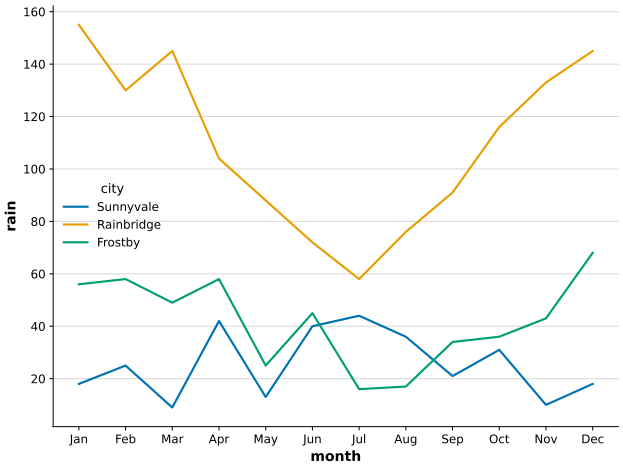
Answer Without `rotate_labels_x` the 18 names lie flat under the bars and run into each other.

Back to lesson 2

Lesson 3: Colours, groups and legends

Task 1. In the weather chart, add `colour="city"` to `chart.lines(...)`. How many lines do you get?

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.lines(x="month", y="rain", colour="city")
chart.save("lesson3_cities.png")
```

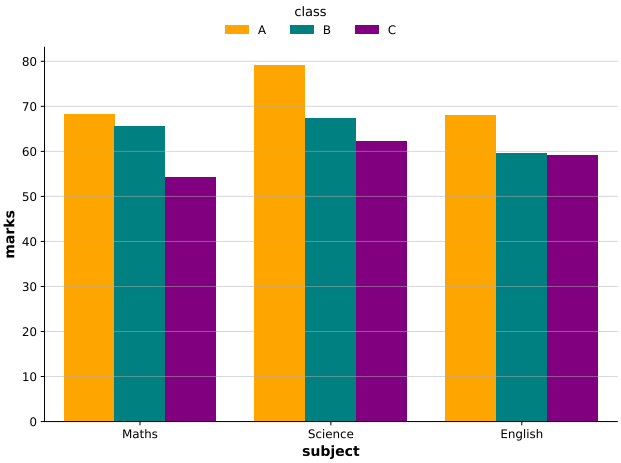


Lesson 3, task 1: `lesson3_cities.png`.

Answer Three lines, one for each city, and a legend that says which colour is which city.

Task 2. Move the legend to the top with `chart.legend_top()`.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="subject", y="marks", colour="class")
chart.colours("orange", "teal", "purple")
chart.legend_top()
chart.save("lesson3_legend_top.png")
```

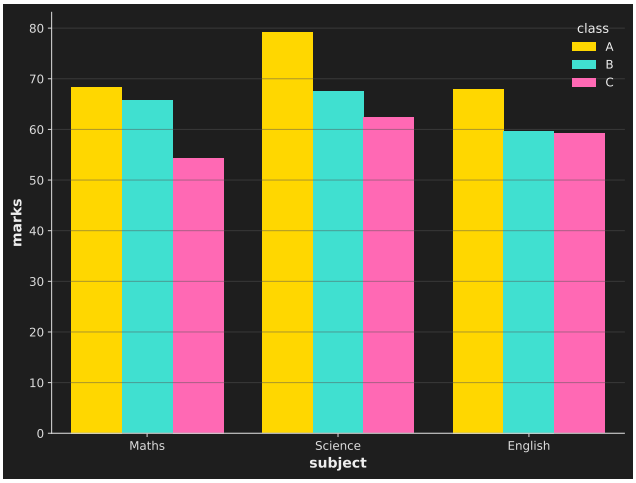


Lesson 3, task 2: `lesson3_legend_top.png`.

Answer `legend_top` puts the legend above the chart, in place of `legend_bottom`.

Task 3. Try `chart.style("dark")` and choose three colours that look good on it.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="subject", y="marks", colour="class")
chart.style("dark")
chart.colours("gold", "turquoise", "hotpink")
chart.save("lesson3_dark.png")
```



Lesson 3, task 3: `lesson3_dark.png`.

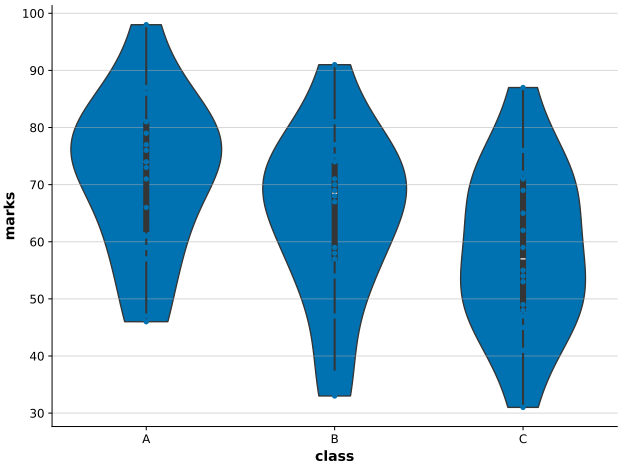
Answer Light, bright colours stand out on a dark background; dark colours such as navy would almost disappear.

Back to lesson 3

Lesson 4: Different chart types

Task 1. Draw `chart.violins(x="class", y="marks")` instead of boxes. What is different?

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.violins(x="class", y="marks")
chart.all_dots(x="class", y="marks")
chart.save("lesson4_violins.png")
```

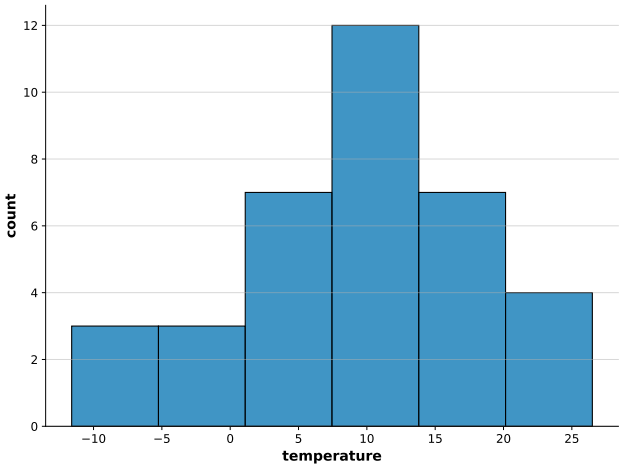


Lesson 4, task 1: `lesson4_violins.png`.

Answer A violin shows the whole shape of the marks: it is wide where many marks are and thin where few are. A box shows only the middle half and the median.

Task 2. Draw a histogram of the weather `"temperature"` column with `bins=6`.

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.histogram(x="temperature", bins=6)
chart.save("lesson4_temperature.png")
```

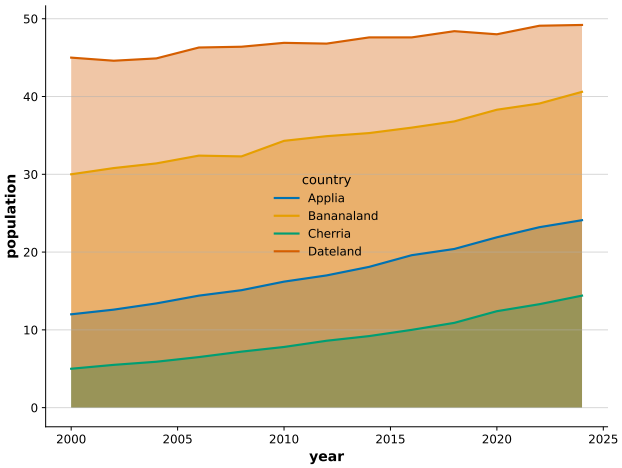


Lesson 4, task 2: `lesson4_temperature.png`.

Answer Six ranges of temperature, each bar as high as the number of months (of all three cities) that fall in it.

Task 3. Make a chart of `sdata.example("growth")` and draw `chart.area(x="year", y="population", colour="country")`.

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.area(x="year", y="population", colour="country")
chart.save("lesson4_area.png")
```



Lesson 4, task 3: `lesson4_area.png`.

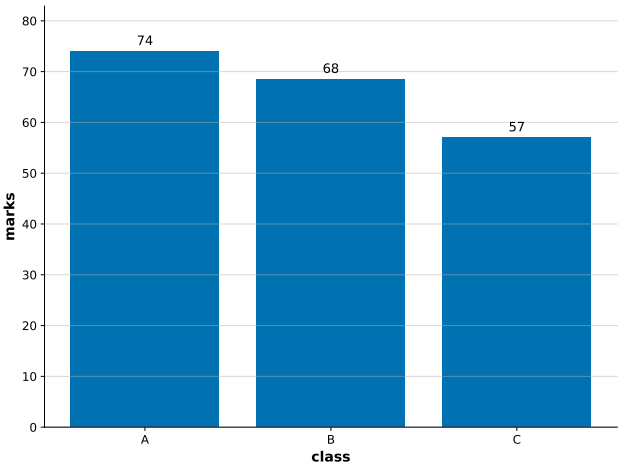
Answer An area chart is a line chart with the space under each line filled in.

Back to lesson 4

Lesson 5: Statistics: averages, error bars and fit lines

Task 1. Change `show="average"` to `show="median"`. Do the bars change much?

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks", show="median")
chart.numbers_on_bars()
chart.save("lesson5_median.png")
```



Lesson 5, task 1: `lesson5_median.png`.

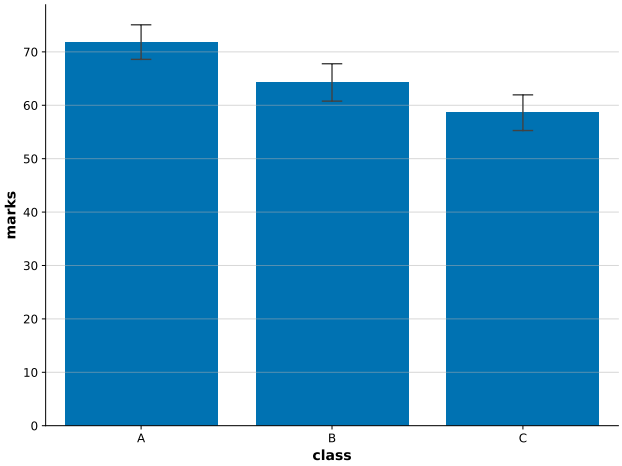
Answer Error bars go with averages, so `error=` comes out. The medians (A 74, B 68.5, C 57) are close to the averages (A 71.8, B 64.3, C 58.6): no single mark pulls them far.

Task 2. Change `error="standard deviation"` to `error="standard error"`. Are the error bars bigger or smaller? Why?

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks", show="average", error="standard error")
chart.save("lesson5_standard_error.png")
```

What it prints

```
Error bars: the standard error of the average (how sure we are about the average itself).
```

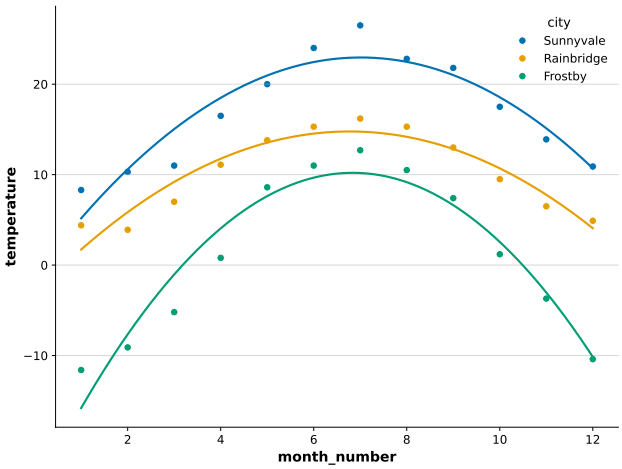


Lesson 5, task 2: `lesson5_standard_error.png`.

Answer Smaller: the standard error is the standard deviation divided by the square root of the number of values (here 18), so it is about a quarter of the standard deviation.

Task 3. Make a chart of the weather table, draw dots with `x="month_number"`, `y="temperature"` and add `chart.fit_curve(degree=2)`.

```
import simplydata as sdata
weather = sdata.example("weather")
chart = sdata.chart(weather)
chart.dots(x="month_number", y="temperature", colour="city")
chart.fit_curve(degree=2)
chart.save("lesson5_fit_curve.png")
```



Lesson 5, task 3: `lesson5_fit_curve.png`.

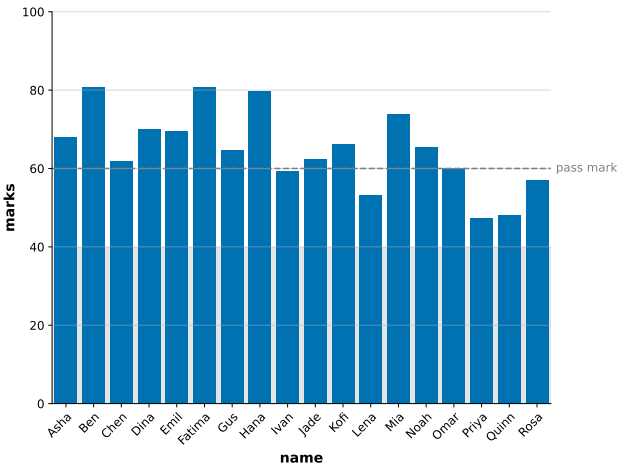
Answer Each city gets a curve with one bend: warm in the middle of the year, cold at both ends.

Back to lesson 5

Lesson 6: Axes and reference lines

Task 1. Move the pass mark to 60 and shade between 0 and 40 instead.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="name", y="marks")
chart.range_y(0, 100)
chart.line_at_y(60, label="pass mark")
chart.shade_between_y(0, 40)
chart.rotate_labels_x()
chart.save("lesson6_pass_60.png")
```

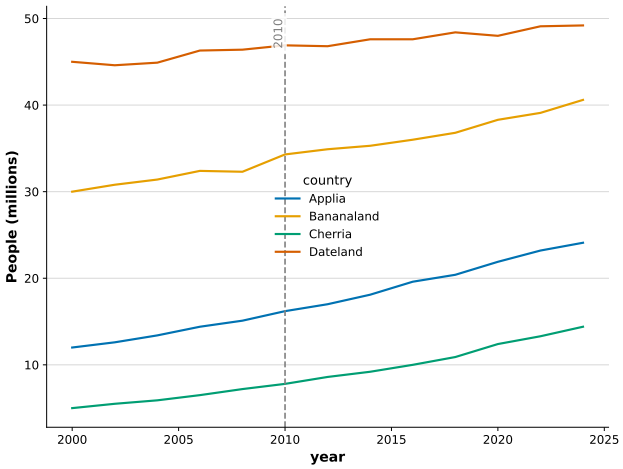


Lesson 6, task 1: `lesson6_pass_60.png`.

Answer `line_at_y(60)` moves the line; `shade_between_y(0, 40)` shades the band from 0 to 40.

Task 2. Take away `chart.log_scale_y()`. Which countries look different?

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.lines(x="year", y="population", colour="country")
chart.line_at_x(2010, label="2010")
chart.label_y("People (millions)")
chart.save("lesson6_no_log.png")
```

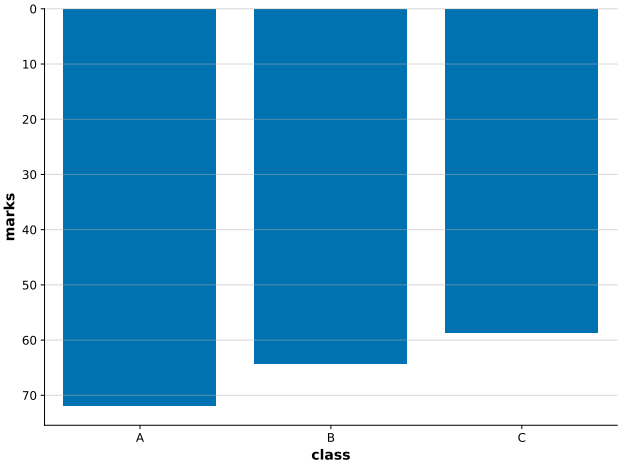


Lesson 6, task 2: `lesson6_no_log.png`.

Answer The lines now bend upwards instead of running straight, and the small countries (Cherria and Applia) sit low and look flat, although Cherria nearly triples.

Task 3. Try `chart.flip_y()` on any chart. What happens?

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.bars(x="class", y="marks")
chart.flip_y()
chart.save("lesson6_flip.png")
```



Lesson 6, task 3: `lesson6_flip.png`.

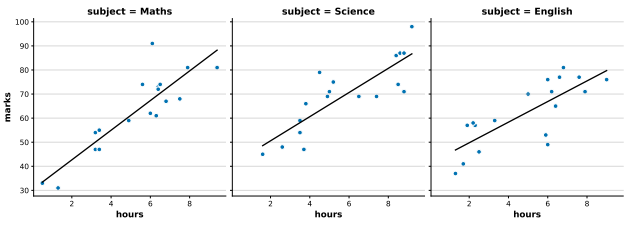
Answer The y-axis turns upside down: 0 is at the top and the bars hang down. Useful for depths, where bigger numbers are further down.

Back to lesson 6

Lesson 7: Several charts together

Task 1. Change `chart.one_chart_per("class")` to `chart.one_chart_per("subject")`.

```
import simplydata as sdata
marks = sdata.example("marks")
chart = sdata.chart(marks)
chart.dots(x="hours", y="marks")
chart.fit_line()
chart.one_chart_per("subject")
chart.save("lesson7_per_subject.png")
```

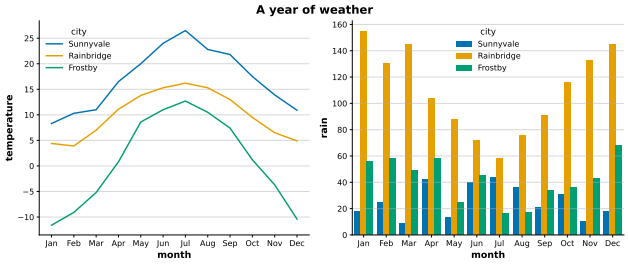


Lesson 7, task 1: `lesson7_per_subject.png`.

Answer One small chart for each subject (Maths, Science and English), each with its own fit line.

Task 2. Use `sdata.side_by_side(a, b)` instead of `sdata.one_above_other(a, b)` for the weather charts.

```
import simplydata as sdata
weather = sdata.example("weather")
a = sdata.chart(weather).lines(x="month", y="temperature", colour="city")
b = sdata.chart(weather).bars(x="month", y="rain", colour="city")
page = sdata.side_by_side(a, b)
page.title_top("A year of weather")
page.save("lesson7_side_by_side.png")
```

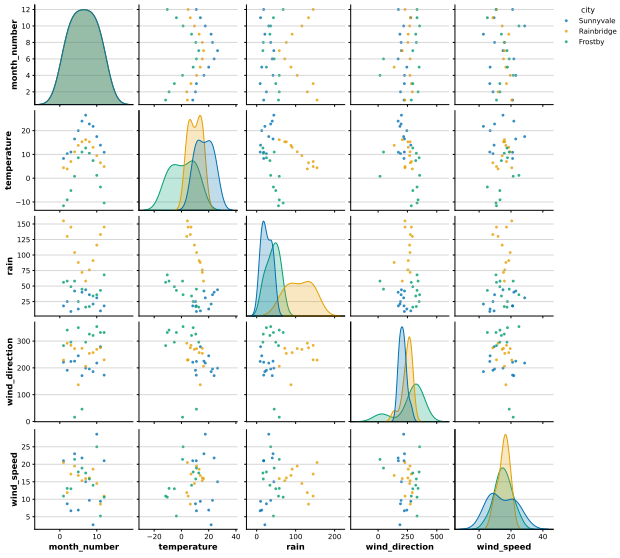


Lesson 7, task 2: `lesson7_side_by_side.png`.

Answer The same two charts, now next to each other instead of one above the other.

Task 3. Make `sdata.pairs` of `sdata.example("weather")` with `colour="city"`.

```
import simplydata as sdata
pairs = sdata.pairs(sdata.example("weather"), colour="city")
pairs.save("lesson7_weather_pairs.png")
```



Lesson 7, task 3: `lesson7_weather_pairs.png`.

Answer Every number column of the weather table against every other, one colour per city.

Back to lesson 7

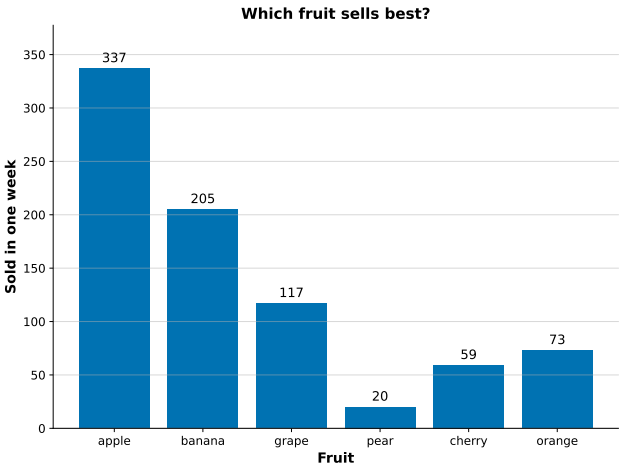
Lesson 8: Hard tricks, and your own program

Task 1. Write your own program, from an empty file, for a question you care about.

```
import simplydata as sdata
fruit = sdata.example("fruit")
print(fruit)
chart = sdata.chart(fruit)
chart.bars(x="fruit", y="count", show="total")
chart.numbers_on_bars()
chart.title_top("Which fruit sells best?")
chart.label_x("Fruit")
chart.label_y("Sold in one week")
chart.save("lesson8_fruit.png")
```

What it prints

```
    day          shop   fruit  count  price
0   Mon   Corner shop   apple     39   0.44
1   Mon  Market stall  banana     22   0.23
2   Mon  Market stall   grape     13   0.05
3   Mon  Market stall    pear      6   0.41
4   Tue   Corner shop   apple     39   0.44
5   Tue   Corner shop  cherry     15   0.11
6   Tue   Corner shop  orange     13   0.55
7   Tue  Market stall   apple     47   0.36
8   Tue  Market stall  banana     21   0.23
9   Wed   Corner shop   apple     19   0.44
10  Wed   Corner shop  banana     28   0.28
11  Wed   Corner shop  cherry     13   0.11
12  Wed   Corner shop  orange     23   0.55
13  Wed  Market stall   apple     28   0.36
14  Wed  Market stall  banana     18   0.23
15  Wed  Market stall   grape     29   0.05
16  Wed  Market stall  orange     19   0.45
17  Thu   Corner shop   apple     49   0.44
18  Thu   Corner shop  banana     39   0.28
19  Thu   Corner shop    pear      7   0.50
20  Thu  Market stall   apple     47   0.36
21  Thu  Market stall  banana     29   0.23
22  Thu  Market stall  cherry     15   0.09
23  Thu  Market stall   grape     28   0.05
24  Thu  Market stall  orange     18   0.45
25  Thu  Market stall    pear      7   0.41
26  Fri   Corner shop   apple     29   0.44
27  Fri   Corner shop  banana     23   0.28
28  Fri   Corner shop   grape     18   0.06
29  Fri  Market stall   apple     40   0.36
30  Fri  Market stall  banana     25   0.23
31  Fri  Market stall  cherry     16   0.09
32  Fri  Market stall   grape     29   0.05
```

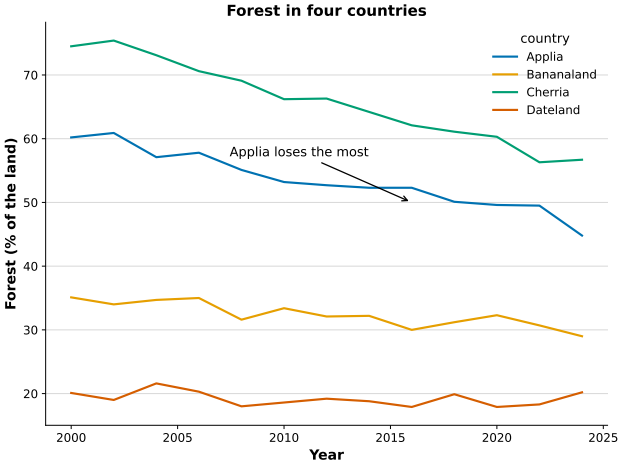


Lesson 8, task 1: `lesson8_fruit.png`.

Answer One possible question: which fruit sells best? Same plan as always: get the data, look, start, draw, words, save.

Task 2. Use any example table, or your own. Give it a title, axis names and a note.

```
import simplydata as sdata
growth = sdata.example("growth")
chart = sdata.chart(growth)
chart.lines(x="year", y="forest", colour="country")
chart.title_top("Forest in four countries")
chart.label_x("Year")
chart.label_y("Forest (% of the land)")
chart.note("Applia loses the most", x=2016, y=50, arrow=True)
chart.save("lesson8_forest.png")
```

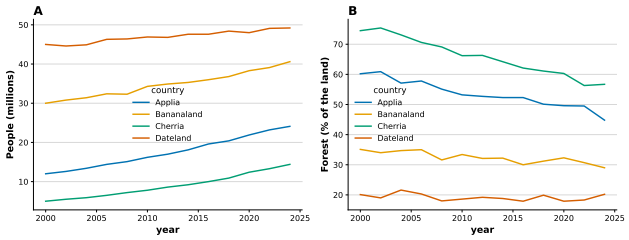


Lesson 8, task 2: `lesson8_forest.png`.

Answer Another table, with a title, both axis names and a note with an arrow.

Task 3. Make a second chart of the same table, put the two charts side by side and save the page as a PDF.

```
import simplydata as sdata
growth = sdata.example("growth")
people = sdata.chart(growth).lines(x="year", y="population", colour="country")
people.label_y("People (millions)")
forest = sdata.chart(growth).lines(x="year", y="forest", colour="country")
forest.label_y("Forest (% of the land)")
page = sdata.side_by_side(people, forest, letters=True)
page.save("lesson8_page.pdf")
```



Lesson 8, task 3: `lesson8_page.pdf`.

Answer Two charts of one table on one page, saved as a PDF because the name ends in `.pdf`.

Back to lesson 8

Lesson 9: Tables: open one and look at it

Task 1. Make a table of `sdata.example("weather")`. How many rows does it have?

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.count_rows()
```

What it prints

```
The table has 36 rows.
```

Answer 36 rows: 12 months for each of 3 cities.

Task 2. Describe its columns. Which column has the biggest numbers?

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.describe_columns()
```

What it prints

```
The table has 36 rows and 7 columns.

       column   kind  empty cells smallest biggest average different values
        month   text            0                                        12
 month_number number            0        1      12     6.5
         city   text            0                                         3
  temperature number            0    -11.6    26.5    9.35
```

```
          rain number            0        9     155   59.03
wind_direction number            0       16     354  244.42
    wind_speed number            0      2.7    28.6    15.1
```

Answer `wind_direction` has the biggest numbers (up to 354): it is an angle in degrees, from 0 to 360.

Task 3. Count each `"city"`. Does every city have the same number of rows?

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.count_each("city")
```

What it prints

```
How many rows have each "city" (3 different values): Frostby 12, Rainbridge 12, Sunnyvale 12.
```

Answer Yes: Frostby, Rainbridge and Sunnyvale have 12 rows each, one for every month.

Back to lesson 9

Lesson 10: Tables: choose rows and columns, and undo

Task 1. In the marks table, keep only class `"B"` and sort by `"hours"`. Who studied the longest?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("class", equal_to="B")
table.sort_by("hours", biggest_first=True)
table.show_first_rows(3)
```

What it prints

```
Kept 18 of 54 rows.
   name class  subject  hours  marks  height
1  Jade     B    Maths    9.4     81     146
2  Kofi     B  Science    8.5     74     146
3  Kofi     B    Maths    7.5     68     146
```

Answer Jade studied the longest: 9.4 hours, for Maths. `biggest_first=True` puts the longest on top.

Task 2. Undo the sorting, then sort by name instead.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("class", equal_to="B")
table.sort_by("hours", biggest_first=True)
table.undo()
table.sort_by("name")
table.show_first_rows()
```

What it prints

```
Kept 18 of 54 rows.
Undone: sort by hours (biggest first)
   name class  subject  hours  marks  height
1   Gus     B    Maths    6.8     67     151
2   Gus     B  Science    4.9     69     151
3   Gus     B  English    2.2     58     151
4  Hana     B    Maths    6.1     91     144
5  Hana     B  Science    5.0     71     144
```

Answer `undo()` takes back only the sorting; class B stays. Text is sorted from A to Z.

Task 3. Keep 5 random rows with `table.keep_random_rows(5, seed=1)` and save them as `"five.csv"`.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_random_rows(5, seed=1)
print(table)
table.save("five.csv")
```

What it prints

```
Kept 5 of 54 rows.
simplydata table: 5 rows, 6 columns
   name class  subject  hours  marks  height
1  Kofi     B    Maths    7.5     68     146
2  Asha     A  English    7.6     77     144
3  Omar     C  English    6.0     49     151
4  Kofi     B  English    1.9     57     146
5  Kofi     B  Science    8.5     74     146
```

Answer `seed=1` picks the same 5 rows every time, so someone else can repeat it.

Task 4. With one written condition, keep the rows with marks below 50 or hours above 8. How many rows are there?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows("marks < 50 or hours > 8")
table.count_rows()
```

What it prints

```
Kept 19 of 54 rows.
The table has 19 rows.
```

Answer 19 rows: or keeps a row when at least one of the two is true.

Task 5. From memory, without looking back: start again, keep rows 5 to 9, then remove the last row. How many rows are left, and whose name is in row 1 now?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_rows_by_position(5, to=9)
table.remove_rows_by_position("last")
table.count_rows()
```

```
print(table.cell_at(row=1, column="name"))
```

What it prints

```
Kept rows 5 to 9 (5 of 54 rows).
Removed row 5 (1 of 5 rows). 4 rows left.
The table has 4 rows.
Ben
```

Answer 4 rows: rows 5 to 9 are 5 rows (both ends count), and one is removed. Row 1 is now Ben's Science mark, which was row 5 before: positions are always of the table as it is now.

Back to lesson 10

Lesson 11: Tables: clean the data and add new columns

Task 1. In the first example, fill the empty marks with the median instead of the average.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.replace_values("marks", [61, 73], with_value=None)
table.count_empty_cells()
table.fill_empty_cells("marks", with_the="median")
table.count_empty_cells()
```

What it prints

```
Replaced 2 cells in marks.
Empty cells in each column:
  name     0
  class    0
  subject  0
  hours    0
  marks    2
  height   0
Total: 2 cells empty.
Filled 2 cells in marks.
Empty cells in each column:
  name     0
  class    0
  subject  0
  hours    0
  marks    0
  height   0
No empty cells.
```

Answer `with_the="median"` fills each empty mark with the middle mark of the others; afterwards no cell is empty.

Task 2. Add a column `"height_m"` with the height in metres: `from_column="height"`, `divide_by=100`.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.add_column("height_m", from_column="height", divide_by=100)
table.show_first_rows()
```

What it prints

```
   name class  subject  hours  marks  height  height_m
1  Asha     A     Maths    6.3     61     144      1.44
2  Asha     A  Science    3.8     66     144      1.44
3  Asha     A  English    7.6     77     144      1.44
4   Ben     A     Maths    6.4     73     152      1.52
5   Ben     A  Science    9.2     98     152      1.52
```

Answer The height in centimetres divided by 100: 144 becomes 1.44.

Task 3. In the last example, add a column `"best_two"` with the formula `"total - English"`.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.keep_columns("name", "class", "subject", "marks")
table.turn_rows_into_columns(names_from="subject", values_from="marks")
table.add_column("total", "Maths + Science + English")
table.add_column("best_two", "total - English")
table.show_first_rows()
```

What it prints

```
   name class  Maths  Science  English  total  best_two
1  Asha     A     61       66       77    204       127
2   Ben     A     73       98       71    242       171
3  Chen     A     81       59       46    186       140
4  Dina     A     74       79       57    210       153
5  Emil     A     47       86       76    209       133
```

Answer The formula can use a column made earlier (total); `best_two` is Maths + Science.

Task 4. In the shop example, add a column `"change"` with the change in sales from the day before, `for_each="shop"` and `sorted_by="date"` (`add_change_column`).

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.fill_empty_cells("sales", with_the="line between neighbours", sorted_by="date",
    ↪ for_each="shop")
table.add_change_column("change", from_column="sales", for_each="shop", sorted_by="date")
table.show(rows=5)
```

What it prints

```
Filled 8 cells in sales on the line between neighbours in order of date, for each shop on its own.
Note: 2 cells stayed empty: they are at the start or end of a group, with a filled neighbour on one side
    ↪ only.
Each "change" is the change in sales from the row before (separately for each shop), in order of date.
        date       shop  sales  customers  change
1 2026-01-01  Northgate  398.0       56.0     NaN
2 2026-01-02  Northgate  387.0       51.0   -11.0
3 2026-01-03  Northgate  605.0       88.0   218.0
4 2026-01-04  Northgate  592.0       83.0   -13.0
5 2026-01-05  Northgate  407.0       59.0  -185.0
(the first 5 rows of 180; table.show(rows=180) shows them all)
```

Answer Each change is today's sales minus yesterday's, in the same shop; the first day of each shop has nothing before it, so its change is empty.

Back to lesson 11

Lesson 12: Tables: summaries, and from a table to a chart

Task 1. Make the fruit totals `for_each="shop"` instead. Which shop sold more?

```
import simplydata as sdata
table = sdata.table(sdata.example("fruit"))
totals = table.total_of("count", for_each="shop")
print(totals)
```

What it prints

```
The total of "count" for each "shop": Corner shop 354, Market stall 457.
simplydata table: 2 rows, 2 columns
           shop  total of count
1   Corner shop             354
2  Market stall             457
```

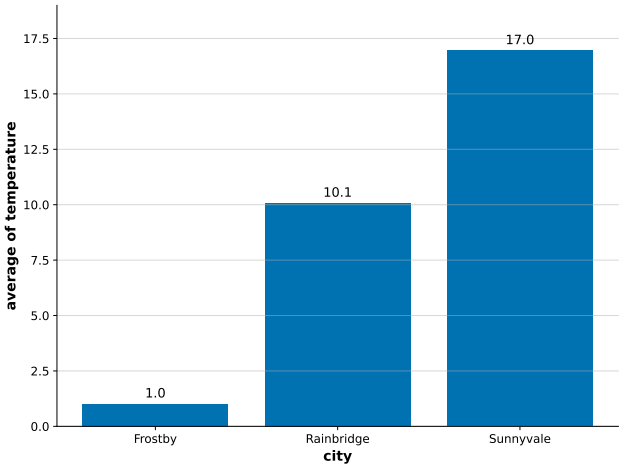
Answer The Market stall sold more: 457 pieces of fruit, against 354 at the Corner shop.

Task 2. Work out the average temperature `for_each="city"` in the weather table and draw it as bars.

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
averages = table.average_of("temperature", for_each="city")
chart = averages.chart()
chart.bars(x="city", y="average of temperature")
chart.numbers_on_bars(decimals=1)
chart.save("lesson12_temperature.png")
```

What it prints

```
The average of "temperature" for each "city": Frostby 1.02, Rainbridge 10.08, Sunnyvale 16.96.
```



Lesson 12, task 2: `lesson12_temperature.png`.

Answer Frostby 1.0, Rainbridge 10.1 and Sunnyvale 17.0 degrees. The summary table's column is called `"average of temperature"`.

Task 3. Is there a link between height and marks? Use `correlation_between` to find out.

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.correlation_between("height", "marks")
```

What it prints

```
The correlation between "height" and "marks" is r = 0.03 (Pearson, 54 rows): very weak, going up, and could
    ↪ easily be chance (p = 0.826).
```

Answer No: r = 0.03, very close to 0, and p = 0.83 says it could easily be chance.

Task 4. Make a cross table of the average mark `for_each="subject" across="class"`. Which class is best in Science?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.average_of("marks", for_each="subject", across="class")
```

What it prints

```
The average of "marks" for each "subject" (rows) and each "class" (columns): a new table with 3 rows and 3
    ↪ columns of numbers.
  English: A 68, B 59.67, C 59.17
  Maths: A 68.33, B 65.67, C 54.33
  Science: A 79.17, B 67.5, C 62.33
```

Answer Class A, with an average of 79.2 in Science, against 67.5 for B and 62.3 for C.

Back to lesson 12

Lesson 13: Numbers: lists and grids of numbers

Task 1. Make the numbers 1 to 10, then square them in two ways: with ** 2 and with `power(2)`.

```
import simplydata as sdata
nums = sdata.numbers(1, to=10)
squares = nums ** 2
print(squares)
nums.power(2)
nums.show()
```

What it prints

```
simplydata numbers: 10 numbers
[  1,   4,   9,  16,  25,  36,  49,  64,  81, 100]
[  1,   4,   9,  16,  25,  36,  49,  64,  81, 100]
```

Answer Both give 1, 4, 9 … 100. ** 2 makes new numbers and leaves nums alone; `power(2)` changes nums itself (and `undo()` can take it back).

Task 2. Make 6 dice throws with `sdata.random_numbers(how_many=6, between=[1, 6], whole_numbers=True, seed=1)`. What is their total?

```
import simplydata as sdata
dice = sdata.random_numbers(how_many=6, between=[1, 6], whole_numbers=True, seed=1)
dice.show()
dice.total()
```

What it prints

```
[3, 4, 5, 6, 1, 1]
The total of the numbers is 20 (6 numbers).
```

Answer With `seed=1` the throws are 3, 4, 5, 6, 1 and 1, which add up to 20.

Task 3. Make a grid of 3 rows and 4 columns filled with 0: `sdata.number_grid(rows=3, columns=4, filled_with=0)`.

```
import simplydata as sdata
grid = sdata.number_grid(rows=3, columns=4, filled_with=0)
print(grid)
```

What it prints

```
simplydata number grid: 3 rows, 4 columns
[[0, 0, 0, 0],
 [0, 0, 0, 0],
 [0, 0, 0, 0]]
```

Answer Three rows of four zeros: a grid to fill in later.

Back to lesson 13

Lesson 14: Statistical tests: real difference, or chance?

Task 1. Compare the average marks `for_each="subject"`. Could the differences be chance?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.compare_averages("marks", for_each="subject")
```

What it prints

```
The average "marks" is 62.3 in English, 62.8 in Maths and 69.7 in Science. Differences this big between the
    ↪ groups could easily be chance (p = 0.264); this does not show that there is no difference. Omega-
    ↪ squared = 0.0138: the groups explain about 1 % of the differences in "marks".
Test: Welch's one-way ANOVA (F = 1.39, df = 2 and 33.8), the usual test for three or more averages; it does
    ↪ not assume the groups have the same spread. To see which groups differ: table.compare_each_pair("
    ↪ marks", for_each="subject").
```

Answer Yes, they could: $p = 0.26$ is well above 0.05. That does not prove the subjects are the same; these data simply cannot tell.

Task 2. Is the average temperature different between the cities of `sdata.example("weather")`?

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.compare_averages("temperature", for_each="city")
table.compare_each_pair("temperature", for_each="city")
```

What it prints

```
The average "temperature" is 1.02 in Frostby, 10.1 in Rainbridge and 17.0 in Sunnyvale. Differences this big
    ↪  between the groups are very unlikely to be chance (p = 0.0002). Omega-squared = 0.465: the groups
    ↪ explain about 46 % of the differences in "temperature".
Test: Welch's one-way ANOVA (F = 13.0, df = 2 and 20.8), the usual test for three or more averages; it does
    ↪ not assume the groups have the same spread. To see which groups differ: table.compare_each_pair("
    ↪ temperature", for_each="city").
Each pair of "city" groups compared on "temperature" with Welch's t-test; the p-values are adjusted for 3
    ↪ pairs with Holm's method, so the chance of any false alarm stays at 5 %.
  Frostby vs Rainbridge: the average of Frostby is 9.06 lower; unlikely to be chance (Holm-adjusted p =
    ↪ 0.011).
  Frostby vs Sunnyvale: the average of Frostby is 15.9 lower; very unlikely to be chance (Holm-adjusted p =
    ↪ 0.0002).
  Rainbridge vs Sunnyvale: the average of Rainbridge is 6.88 lower; unlikely to be chance (Holm-adjusted p =
    ↪  0.011).
```

Answer Yes: p = 0.0002, very unlikely to be chance. `compare_each_pair` then shows which cities differ from each other.

Task 3. Fit a straight line of `"forest"` against `"year"` in `sdata.example("growth")` for one country.

```
import simplydata as sdata
table = sdata.table(sdata.example("growth"))
table.keep_rows("country", equal_to="Applia")
table.fit_line(x="year", y="forest")
```

What it prints

```
Kept 13 of 52 rows.
"forest" goes down by 0.574 for each 1 more "year" (95 % CI -0.672 to -0.477); this slope is very unlikely
    ↪ to be chance (p < 0.0001). R squared = 0.94: "year" explains about 94 % of the differences in "
    ↪ forest".
Line: forest = -0.574 * year + 1209, by least squares over 13 rows. A link does not show that one thing
    ↪ causes the other.
```

Answer In Applia the forest shrinks by about 0.57 percentage points a year (95 % CI 0.48 to 0.67), and the line explains 94 % of the differences.

Task 4. Remove the rows with outliers in `"marks"` `for_each="class"`, then compare the average marks `for_each="class"` again. Did the answer change?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.remove_rows_with_outliers("marks", for_each="class")
table.compare_averages("marks", for_each="class")
```

What it prints

```
Outliers in "marks" by Tukey's rule: values more than 1.5 x IQR below the first quartile (Q1) or above the
    ↪ third quartile (Q3).
IQR, the interquartile range, is Q3 - Q1: the spread of the middle half of the values.
For each class:
  A: below 34.88 or above 107.88 (Q1 = 62.25, Q3 = 80.5); 0 rows found
  B: below 33.25 or above 97.25 (Q1 = 57.25, Q3 = 73.25); 1 row found
  C: below 14.88 or above 103.88 (Q1 = 48.25, Q3 = 70.5); 0 rows found
Removed 1 of 54 rows. 53 rows left.
The average "marks" is 71.8 in A, 66.1 in B and 58.6 in C. Differences this big between the groups are
    ↪ unlikely to be chance (p = 0.028). Omega-squared = 0.109: the groups explain about 11 % of the
    ↪ differences in "marks".
Test: Welch's one-way ANOVA (F = 3.97, df = 2 and 33.3), the usual test for three or more averages; it does
    ↪ not assume the groups have the same spread. To see which groups differ: table.compare_each_pair("
    ↪ marks", for_each="class").
```

Answer Only one mark (33 in class B) is removed. Class B goes up from 64.3 to 66.1, but p stays 0.028: the answer does not change. Say in a report that you checked, and how.

Back to lesson 14

Lesson 15: A whole analysis: from a question to a figure

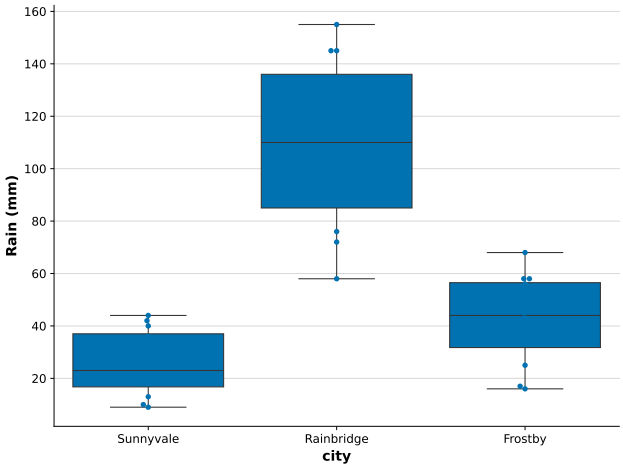
Task 1. Do a whole analysis of `sdata.example("weather")`: look at it, compare the average rain `for_each="city"`, draw the rain of each city as boxes, and save the chart and the table.

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.describe_columns()
table.remove_columns("wind_direction", "wind_speed")
table.average_of("rain", for_each="city")
table.compare_averages("rain", for_each="city")
chart = table.chart()
chart.boxes(x="city", y="rain")
chart.all_dots(x="city", y="rain")
chart.label_y("Rain (mm)")
chart.save("lesson15_rain_boxes.png")
table.save("lesson15_weather.xlsx")
```

What it prints

```
The table has 36 rows and 7 columns.

        column   kind  empty cells smallest biggest average different values
         month   text            0                                       12
  month_number number            0        1      12     6.5
          city   text            0                                        3
   temperature number            0    -11.6    26.5    9.35
          rain number            0        9     155   59.03
wind_direction number            0       16     354  244.42
    wind_speed number            0      2.7    28.6    15.1
The average of "rain" for each "city": Frostby 42.08, Rainbridge 109.42, Sunnyvale 25.58.
The average "rain" is 42.1 in Frostby, 109 in Rainbridge and 25.6 in Sunnyvale. Differences this big between
    ↪  the groups are very unlikely to be chance (p < 0.0001). Omega-squared = 0.720: the groups explain
    ↪ about 72 % of the differences in "rain".
Test: Welch's one-way ANOVA (F = 33.9, df = 2 and 20.2), the usual test for three or more averages; it does
    ↪ not assume the groups have the same spread. To see which groups differ: table.compare_each_pair("
    ↪ rain", for_each="city").
```

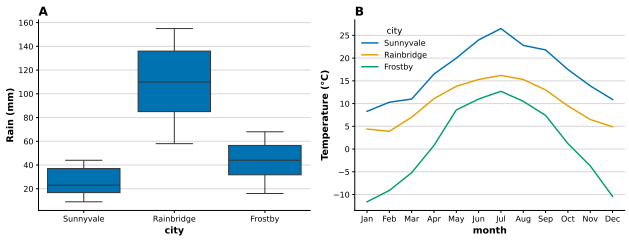


Lesson 15, task 1: `lesson15_rain_boxes.png`.

Answer Rainbridge is much wetter (109 mm a month on average) than Frostby (42) and Sunnyvale (26); $p < 0.0001$, so this is very unlikely to be chance.

Task 2. Make a second chart of the temperature, then put the two charts together with `sdata.side_by_side(..., letters=True)`.

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
rain = table.chart()
rain.boxes(x="city", y="rain")
rain.label_y("Rain (mm)")
warmth = table.chart()
warmth.lines(x="month", y="temperature", colour="city")
warmth.label_y("Temperature (°C)")
page = sdata.side_by_side(rain, warmth, letters=True)
page.save("lesson15_weather_page.png")
```



Lesson 15, task 2: `lesson15_weather_page.png`.

Answer Two charts of one table, side by side, with A and B so a report can point to each.

Task 3. Read the `steps()` of your table out loud. Would someone else understand what you did?

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.remove_columns("wind_direction", "wind_speed")
table.keep_rows("month_number", between=[6, 8])
table.sort_by("rain", biggest_first=True)
table.steps()
```

What it prints

```
Kept 9 of 36 rows.
1. remove columns wind_direction, wind_speed
2. keep rows where month_number is between 6 and 8
3. sort by rain (biggest first)
```

Answer `steps()` lists each change in plain words, in order; those lines are a first draft of the methods part of a report.

Back to lesson 15

Lesson 16: Models: explain one column from several others

Task 1. Explain `"marks"` using `"hours"` and `"school"` (school as a column of groups). Which schools differ from School A, and is school as a whole important?

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.explain("marks", using=["hours", "school"])
```

What it prints

```
Explaining "marks" from "hours" and "school" (180 rows). Each effect below holds the other columns fixed:
  intercept: 28.3, the expected "marks" when every number column is 0 and every group column is at its
    ↪ baseline.
  hours: each 1 more "hours" goes with 2.82 more "marks" (95 % CI 2.39 to 3.26); very unlikely to be chance
    ↪ (p < 0.0001).
  school "School B" against "School A": "marks" is 6.84 higher (95 % CI 1.82 to 11.9); unlikely to be chance
    ↪  (p = 0.008).
  school "School C" against "School A": "marks" is 5.89 lower (95 % CI -11.0 to -0.838); unlikely to be
    ↪ chance (p = 0.023).
  school "School D" against "School A": "marks" is 15.3 higher (95 % CI 10.3 to 20.4); very unlikely to be
    ↪ chance (p < 0.0001).
  school "School E" against "School A": "marks" is 12.5 higher (95 % CI 7.45 to 17.5); very unlikely to be
    ↪ chance (p < 0.0001).
  school "School F" against "School A": "marks" is 3.80 higher (95 % CI -1.22 to 8.83); could easily be
    ↪ chance (p = 0.137).
  school "School G" against "School A": "marks" is 4.58 higher (95 % CI -0.476 to 9.63); could easily be
    ↪ chance (p = 0.076).
  school "School H" against "School A": "marks" is 7.63 higher (95 % CI 2.58 to 12.7); unlikely to be chance
    ↪  (p = 0.003).
  school "School I" against "School A": "marks" is 3.83 higher (95 % CI -1.25 to 8.92); could easily be
    ↪ chance (p = 0.138).
  school "School J" against "School A": "marks" is 3.39 higher (95 % CI -1.63 to 8.41); could easily be
    ↪ chance (p = 0.184).
  school "School K" against "School A": "marks" is 11.0 higher (95 % CI 5.94 to 16.0); very unlikely to be
    ↪ chance (p < 0.0001).
  school "School L" against "School A": "marks" is 10.3 higher (95 % CI 5.24 to 15.3); very unlikely to be
    ↪ chance (p < 0.0001).
  school, all groups together: F test (F = 10.4, df = 11 and 167); very unlikely to be chance (p < 0.0001).
```

```
Method: linear model by least squares, as R lm(marks ~ hours + school).
R squared = 0.635 (adjusted 0.609): the model explains about 63 % of the differences in "marks". The model
    ↪ as a whole (F = 24.2, df = 12 and 167): very unlikely to be chance (p < 0.0001).
Note: the residuals (distances from the model) in this model don't look bell-shaped (Shapiro-Wilk p = 0.040)
    ↪ , but with 180 values the average is close to bell-shaped anyway (the central limit theorem), so
    ↪ this test is usually still fine.
A large p-value does not show that there is no effect, only that the data cannot tell. A link does not show
    ↪ that one thing causes the other.
```

Answer School as a whole matters a lot ($F = 10.4$, $p < 0.0001$); School D scores about 15 marks more than School A for the same hours of study. `grouped_by=` would instead treat the schools as a sample of many possible schools.

Task 2. Explain `"passed"` using `"hours"` and `"sleep"` with `outcome="yes or no"`.

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.explain("passed", using=["hours", "sleep"], outcome="yes or no")
```

What it prints

```
Note: 19 rows with an empty cell in "passed" or "hours" or "sleep" were skipped.
Explaining "passed" from "hours" and "sleep" (161 rows). Each odds ratio below holds the other columns fixed
    ↪ :
  intercept: 0.000233, the odds of yes when every number column is 0.
  hours: each 1 more "hours" multiplies the odds of yes by 1.69 (95 % CI 1.39 to 2.05); very unlikely to be
    ↪ chance (p < 0.0001).
  sleep: each 1 more "sleep" multiplies the odds of yes by 1.98 (95 % CI 1.29 to 3.04); unlikely to be
    ↪ chance (p = 0.002).
Method: logistic regression, as R glm(passed ~ hours + sleep, family = binomial); counted as yes: "passed" =
    ↪  "yes". Wald 95 % intervals and z-tests, as R confint.default() and summary().
Residual deviance 170 on 158 df (with no columns: 221 on 160 df). The model as a whole (likelihood-ratio chi
    ↪ -square = 51.5, df = 2): very unlikely to be chance (p < 0.0001). AIC = 176.
A link does not show that one thing causes the other.
```

Answer Each extra hour of study multiplies the odds of passing by 1.69 and each extra hour of sleep by 1.98; both are unlikely to be chance.

Task 3. Explain `"days_absent"` using `"hours"` and `"breakfast"` with `outcome="counts"`. Does breakfast matter?

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.explain("days_absent", using=["hours", "breakfast"], outcome="counts")
```

What it prints

```
Explaining "days_absent" from "hours" and "breakfast" (180 rows). Each rate ratio below holds the other
    ↪ columns fixed:
  intercept: 6.42, the expected count when every number column is 0 and every group column is at its
    ↪ baseline.
  hours: each 1 more "hours" multiplies the expected count by 0.861 (95 % CI 0.828 to 0.895); very unlikely
    ↪ to be chance (p < 0.0001).
  breakfast "yes" against "no": the expected count is 0.936 times as high (95 % CI 0.788 to 1.11); could
    ↪ easily be chance (p = 0.448).
Method: Poisson regression, as R glm(days_absent ~ hours + breakfast, family = poisson). Wald 95 % intervals
    ↪  and z-tests, as R confint.default() and summary().
Residual deviance 193 on 177 df (with no columns: 255 on 179 df). The model as a whole (likelihood-ratio chi
    ↪ -square = 62.4, df = 2): very unlikely to be chance (p < 0.0001). AIC = 693.
```

```
A large p-value does not show that there is no effect, only that the data cannot tell. A link does not show
    ↪ that one thing causes the other.
```

Answer Breakfast could easily be chance (rate ratio 0.94, p = 0.45); study hours matter (rate ratio 0.86 for each hour).

Back to lesson 16

Lesson 17: Patterns across many columns

Task 1. Add principal components of `"hours"`, `"sleep"`, `"marks"` and `"days_absent"` in `sdata.example("school")`. Which columns weigh most in PC1?

```
import simplydata as sdata
table = sdata.table(sdata.example("school"))
table.add_principal_components(columns=["hours", "sleep", "marks", "days_absent"])
```

What it prints

```
Note: 19 rows with an empty cell in "hours" or "sleep" or "marks" or "days_absent" were skipped.
Principal components of 4 columns (hours, sleep, marks, days_absent), each put on the same scale first (mean
    ↪  0, standard deviation 1): PC1 49.7 % and PC2 23.7 % of the spread (73.4 % together).
  PC1 weighs most on "hours" (0.60), "marks" (0.57) and "days_absent" (-0.49).
  PC2 weighs most on "sleep" (0.93), "hours" (-0.35) and "marks" (-0.08).
Method: principal component analysis by singular value decomposition, as R prcomp(scale. = TRUE). Added PC1
    ↪ and PC2 (the scores of each row). The sign of each component is arbitrary: flipping it changes
    ↪ nothing.
There are 4 components in all; table.principal_components_of() shows them all.
```

Answer PC1 (50 % of the spread) is mostly hours, marks and, the other way, days absent: pupils who study more get better marks and miss fewer days. PC2 is mostly sleep.

Task 2. Make an NMDS map of the meadow in 3 dimensions. Is the stress lower than in 2?

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.add_nmds(measure="bray-curtis", dimensions=3)
```

What it prints

```
NMDS map of 30 rows from 10 columns (Bray-Curtis distances, 3 dimensions): stress = 0.127, usable, but do
    ↪ not read too much into small details (Clarke's rule of thumb: below 0.05 excellent, below 0.1 good,
    ↪  below 0.2 usable, above 0.2 hard to trust).
Method: non-metric multidimensional scaling (Kruskal's stress-1, SMACOF with monotone regression), lowest
    ↪ stress of 21 starts (classical scaling and 20 random, seed=1; the lowest was reached 21 times), as
    ↪ R vegan::metaMDS(x, distance = "bray", k = 3, autotransform = FALSE). Added NMDS1, NMDS2 and NMDS3;
    ↪  the axes have no units and their rotation and signs are arbitrary.
```

Answer Yes: the stress drops from 0.195 to 0.127. That is still in the usable band (good is below 0.1), but the map keeps the distances clearly better.

Task 3. Do the three fields differ in make-up? Use `compare_profiles` with `for_each="field"`.

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.compare_profiles(for_each="field")
```

What it prints

```
The groups of "field" (North field, River field, South field) explain 11.8 % of the differences between the
    ↪ rows (R squared = 0.118; Bray-Curtis distances over 10 columns). A difference in make-up this big
    ↪ is unlikely to be chance (p = 0.037); pseudo-F = 1.81, from 999 permutations.
Test: PERMANOVA (permutational analysis of variance on distances), as R vegan::adonis2(distances ~ field,
    ↪ permutations = 999) with distances = vegan::vegdist(x, "bray").
Spread check: the average distance to the group centre is 0.287 in North field, 0.266 in River field and
    ↪ 0.296 in South field (as vegan::betadisper and permutest: F = 0.625, p = 0.543).
```

Answer The fields explain 12 % of the differences (p = 0.037), much less than grazing, and they do not differ in spread (p = 0.54).

Task 4. Cluster the meadow plots with `method="hierarchical"`. Do the clusters match the grazing?

```
import simplydata as sdata
table = sdata.table(sdata.example("meadow"))
table.add_clusters(how_many=2, method="hierarchical")
table.count_rows(for_each="cluster", across="grazed")
```

What it prints

```
Made 2 clusters of 30 rows from 10 columns (each put on the same scale first): sizes 15 and 15 (clusters 1
    ↪ and 2, numbered in the order they first appear). 24.3 % of the spread lies between the clusters (
    ↪ between-cluster / total sum of squares); the rest is within them.
Method: hierarchical clustering on Euclidean distances with ward linkage, as R hclust(dist(x), method = "
    ↪ ward.D2") and cutree(k = 2). Added the column "cluster".
Rows for each "cluster" (rows) and each "grazed" (columns):
  1: no 0, yes 15
  2: no 15, yes 0
```

Answer Exactly: cluster 1 is the 15 grazed plots and cluster 2 the 15 ungrazed ones.

Back to lesson 17

Lesson 18: Survival, and planning a study

Task 1. Explain the risk of dying from `"age"` alone with `explain_survival`.

```
import simplydata as sdata
table = sdata.table(sdata.example("trial"))
table.explain_survival(time="days", event="died", using="age")
```

What it prints

```
The event is "died" = "yes" (59 of 120 rows); the other rows count as followed without the event until their
    ↪  "days" (censored).
Hazard ratios from 120 rows with 59 events (above 1: a higher risk of the event at any moment; below 1: a
    ↪ lower risk):
  each 1 more "age": hazard ratio 1.05 (95 % CI 1.03 to 1.08); very unlikely to be chance (p < 0.0001).
The model as a whole (likelihood-ratio test, chi-square = 20.4, df = 1) is very unlikely to be chance (p <
    ↪ 0.0001).
```

```
Method: Cox proportional hazards model, Efron's method for tied times, as R coxph(Surv(days, died == "yes")
    ↪ ~ age).
```

Answer Each year of age multiplies the risk by 1.05 (95 % CI 1.03 to 1.08), p < 0.0001.

Task 2. How many pairs does a before/after study need to find a small change (`effect_size=0.3`)?

```
import simplydata as sdata
sdata.how_many_needed(test="compare_before_after", effect_size=0.3)
```

What it prints

```
To find an average change of 0.3 standard deviations of the changes (Cohen's d) with an 80 % chance, using a
    ↪  two-sided test at 0.05, you need 90 pairs (a before and an after value each). Method: paired t-
    ↪ test power, as R power.t.test(type = "paired").
Cohen's rough guide calls this a small effect (Cohen's d: 0.2 small, 0.5 medium, 0.8 large); it is only a
    ↪ hint, sizes from your own field are better.
```

Answer 90 pairs, each with a before and an after value.

Task 3. What is the chance to find a medium difference (`effect_size=0.25`) between 3 groups with 30 values in each?

```
import simplydata as sdata
sdata.chance_to_find(test="compare_averages", effect_size=0.25, groups=3, values_per_group
    ↪ =30)
```

What it prints

```
With 30 values in each group (90 in all), a study has a 54 % chance (power = 0.54) to find differences
    ↪ between the 3 group averages of Cohen's f = 0.25 (the spread of the group averages, in standard
    ↪ deviations), using the F test at 0.05. Method: one-way ANOVA power for groups of the same size, as
    ↪ R pwr::pwr.anova.test().
Note: this is below the usual 80 %; how_many_needed() tells you how many values you need.
Note: the calculation uses the usual equal-spread formula; compare_averages uses Welch's ANOVA, which is
    ↪ very close to it when the groups are the same size.
Cohen's rough guide calls this a medium effect (Cohen's f: 0.1 small, 0.25 medium, 0.4 large); it is only a
    ↪ hint, sizes from your own field are better.
```

Answer Only 54 %: too small. `how_many_needed` with `groups=3` tells you how many to aim for.

Back to lesson 18

Lesson 19: Fitting curves, finding the best value and solving

Task 1. Fit a curve of degree 3 to `"marks"` against `"hours"`. Does R squared go up much from the straight line?

```
import simplydata as sdata
table = sdata.table(sdata.example("marks"))
table.fit_curve(x="hours", y="marks", degree=3)
```

What it prints

```
Curve: marks = 22.7 + 15.5 * hours - 1.92 * hours**2 + 0.105 * hours**3 (each with its 95 % CI in the table)
    ↪ ; R squared = 0.71, adjusted 0.69.
Method: least squares on powers of hours, as R lm(marks ~ poly(hours, 3, raw = TRUE)).
```

Answer R squared is 0.71, against 0.69 for the straight line: the extra bends add very little.

Task 2. With the weather table, keep the city Sunnyvale and fit `"temperature = a + b * cos(2 * pi * (month_number - c) / 12)"` with `start={"a": 15, "b": 8, "c": 7}`. In which month is it warmest (c)?

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
table.keep_rows("city", equal_to="Sunnyvale")
table.fit_formula("temperature = a + b * cos(2 * pi * (month_number - c) / 12)",
                  start={"a": 15, "b": 8, "c": 7})
```

What it prints

```
Kept 12 of 36 rows.
Fitted "temperature = a + b * cos(2 * pi * (month_number - c) / 12)" to 12 rows: a = 17.0 (95 % CI 16.4 to
    ↪ 17.6), b = 8.18 (95 % CI 7.32 to 9.03), c = 7.19 (95 % CI 6.99 to 7.39). Residual standard error
    ↪ 0.930 on 9 degrees of freedom; R squared = 0.98 (only a rough guide for a curve).
Values found: a, b, c. Data: month_number. Method: nonlinear least squares (Levenberg-Marquardt), as R nls()
    ↪ ; intervals are estimate +/- t x standard error (R confint() on nls profiles the likelihood and can
    ↪  differ a little).
```

Answer c = 7.19: the warmest time is in July; a = 17.0 is the average and b = 8.18 how far it swings.

Task 3. Try to remember without looking back: which word finds where a formula is biggest, and which word finds where the two sides of an equation are equal? Use them on `"x * (10 - x)"` between 0 and 10, and on `"x**2 = 2"` between 0 and 2.

```
import simplydata as sdata
sdata.find_biggest("x * (10 - x)", between=[0, 10])
sdata.solve("x**2 = 2", between=[0, 2])
```

What it prints

```
The biggest value of x * (10 - x) between 0 and 10 is 25, at x = 5.
Method: Brent's method on the range, as R optimize().
x**2 = 2 between 0 and 2 when x = 1.41421.
Method: Brent's method, as R uniroot(); every sign change on 1000 steps across the range was searched, so
    ↪ all solutions that cross were found (one where the curve only touches, like x**2 = 0, can be missed
    ↪ ).
```

Answer `find_biggest` gives 25 at x = 5, and solve gives x = 1.41421, the square root of 2.

Back to lesson 19

Lesson 20: Distributions, chances and values between points

Task 1. Ten coins are thrown. With `sdata.distribution("binomial", tries=10, chance=0.5)`, what is the chance of 8 heads or more?

```
import simplydata as sdata
coins = sdata.distribution("binomial", tries=10, chance=0.5)
coins.chance(at_least=8)
```

What it prints

```
The chance of 8 or more is 5.47 % (binomial, 10 tries, chance 0.5; R: pbinom(7, size = 10, prob = 0.5, lower
    ↪ .tail = FALSE)).
```

Answer 5.47 %: 8, 9 or 10 heads happen about once in 18 tries.

Task 2. Which fits the daily `"sales"` of the shop table best: normal, lognormal, gamma or weibull? Then check the best one with `compare_shape_to`.

```
import simplydata as sdata
table = sdata.table(sdata.example("shop"))
table.compare_distributions("sales")
table.compare_shape_to("sales", "lognormal")
```

What it prints

```
Note: 10 rows with an empty cell in "sales" were skipped.
Best by AIC: lognormal (AIC 2111.1), then gamma (+3.1), normal (+21.0), weibull (+28.0).
A difference below 2 is too small to choose between them. Method: maximum likelihood for each, ranked by AIC
    ↪ , as R fitdistrplus::gofstat().
Note: 10 rows with an empty cell in "sales" were skipped.
The values of "sales" could well follow a lognormal distribution (Anderson-Darling A = 0.283, p = 0.652, 170
    ↪  values; fitted log average 6.08, log standard deviation 0.273). This does not prove that they do.
Test: Anderson-Darling test of a fitted lognormal shape, with a Monte Carlo p-value from 9999 simulated
    ↪ samples that are each fitted again (SciPy goodness_of_fit, seed=1); the statistic is R goftest::ad.
    ↪ test(x, "plnorm", ...) with the fitted values.
```

Answer Lognormal has the smallest AIC, and its shape could well fit (p = 0.65).

Task 3. Try to remember without looking back: which word estimates a value between the points you measured? Use it to estimate the temperature of each city at `month_number` 6.5, joined by a smooth curve.

```
import simplydata as sdata
table = sdata.table(sdata.example("weather"))
estimates = table.values_at("temperature", at={"month_number": [6.5]},
                            joined_by="smooth curve", for_each="city")
estimates.show()
```

What it prints

```
Estimated "temperature" at 1 value of "month_number" for each of the 3 groups in "city", joined by a smooth
    ↪ curve through every known point (a natural cubic spline, as R splinefun(method = "natural")).
  month_number        city  temperature
1          6.5     Frostby    12.093466
2          6.5  Rainbridge    15.900152
3          6.5   Sunnyvale    25.966898
```

> **Answer** `values_at` joins the months with a smooth curve and reads off each city at 6.5.

Back to lesson 20

Index of words

Every word and option, A to Z, with the page of its dictionary entry. Other spellings of a word (such as `chart.bar()` for `chart.bars()`) point to the same entry.

nums.absolute_value() 219
nums.add() 217
table.add_adjusted_p_value_column() 186
table.add_change_column() 175
table.add_clusters() 203
table.add_column() 170
table.add_date_part_column() 172
table.add_group_column() 171
table.add_is_empty_column() 164
table.add_nmds() 204
table.add_predicted_column() 200
table.add_previous_value_column() 175
table.add_principal_components() 201
table.add_rank_column() 171
table.add_rolling_column() 174
table.add_rows_from() 209
table.add_text_part_column() 172
chart.all_dots() 127
chart.animate() 150
chart.area() 115
nums.as_numpy() 237
table.as_pandas() 212
table.as_polars() 213
dist.average() 197
nums.average() 226
table.average_of() 177
chart.band() 120
chart.bar() 113
chart.bars() 113
nums.biggest() 227
table.biggest_of() 178
chart.box() 116
chart.boxes() 116
chart.caption_bottom() 132
table.cell_at() 154
dist.chance() 195
sdata.chance_to_find() 208
table.change_column() 169
dist.chart() 199
nums.chart() 236
sdata.chart() 111
table.chart() 212
chart.color() 140
chart.color_scale() 141
chart.colors() 141
chart.colour() 140
chart.colour_scale() 141
chart.colours() 141
grid.column() 231
table.column() 153
table.column_names() 153
table.compare_average_to() 184
table.compare_averages() 183
table.compare_before_after() 184
table.compare_counts() 185
table.compare_counts_to() 187
table.compare_distributions() 194
table.compare_each_pair() 183
table.compare_percent_to() 187
table.compare_profiles() 205
table.compare_shape_to() 185
table.compare_spreads() 186
table.compare_survival() 206
chart.contour() 118
table.correlation_between() 182
nums.count() 216
chart.count_bars() 126
grid.count_columns() 229
table.count_different_values() 181
nums.count_each() 228
table.count_each() 176
table.count_empty_cells() 163
nums.count_empty_numbers() 217
grid.count_rows() 229
table.count_rows() 176
chart.cumulative_share() 128
chart.curve() 124
dist.cut_off() 196
chart.density() 126
dist.density_at() 196
table.describe_columns() 153
table.distances_between_rows() 203
sdata.distribution() 193
nums.divide_by() 218
chart.dots() 114
chart.dots_3d() 119

chart.empty_cells() 120
error= 121
chart.error_bars() 122
sdata.example() 111
nums.exp() 220
table.explain() 200
table.explain_survival() 206
table.fill_empty_cells() 164
nums.fill_empty_numbers() 225
sdata.find_biggest() 191
sdata.find_smallest() 190
chart.fit_curve() 123
table.fit_curve() 188
chart.fit_distribution() 124
nums.fit_distribution() 195
table.fit_distribution() 194
chart.fit_formula() 123
table.fit_formula() 189
chart.fit_line() 122
table.fit_line() 188
chart.flip_x() 140
chart.flip_y() 140
chart.font_name() 135
chart.font_size() 134
sdata.grid() 149
chart.heatmap() 117
sdata.help() 238
chart.hide_grid() 143
chart.hide_legend() 145
chart.histogram() 115
sdata.how_many_needed() 207
table.join_with() 209
table.keep_biggest_rows() 161
table.keep_columns() 155
grid.keep_columns_by_position() 232
table.keep_columns_by_position() 156
nums.keep_first_numbers() 222
table.keep_first_rows() 158
nums.keep_last_numbers() 222
table.keep_last_rows() 159
nums.keep_numbers() 221
nums.keep_numbers_by_position() 223
nums.keep_random_numbers() 222
table.keep_random_rows() 160
table.keep_rows() 157
grid.keep_rows_by_position() 232
table.keep_rows_by_position() 159
table.keep_smallest_rows() 162
chart.label_color_bar() 146
chart.label_colour_bar() 146
chart.label_legend() 145
chart.label_x() 132
chart.label_y() 132
chart.legend_bottom() 144
chart.legend_left() 144
chart.legend_right() 144
chart.legend_top() 143
sdata.lesson() 239
chart.line() 113
chart.line_at_x() 129
chart.line_at_y() 130
chart.line_with_slope() 130
chart.lines() 113
nums.log() 220
chart.log_scale_x() 136
chart.log_scale_y() 136
table.make_dates() 166
table.make_lowercase() 166
table.make_numbers() 165
table.make_text() 165
table.make_uppercase() 166
table.map_of() 193
dist.median() 197
nums.median() 226
table.median_of() 177
dist.middle_range() 197
table.most_common_of() 181
chart.names_on_dots() 134
chart.note() 133
grid.number_at() 231
nums.number_at() 216
nums.number_grid() 229
sdata.number_grid() 228
grid.numbers() 232
sdata.numbers() 214
chart.numbers_on_bars() 133
sdata.one_above_other() 149
chart.one_chart_per() 148
chart.order_legend() 147
chart.order_x() 146
chart.order_y() 147
sdata.pairs() 129
dist.parameters() 199
table.percent_of() 180
dist.percentile() 196
nums.percentile() 228
table.percentile_of() 181
chart.pie() 117
chart.polar() 118
nums.power() 218
table.principal_components_of() 202
chart.qq_dots() 127
dist.random_numbers() 198
sdata.random_numbers() 214
chart.range_x() 135
chart.range_y() 136

table.remove_columns() 155
grid.remove_columns_by_position() 233
table.remove_columns_by_position() 156
nums.remove_duplicate_numbers() 223
table.remove_duplicate_rows() 161
nums.remove_empty_numbers() 223
table.remove_extra_spaces() 167
nums.remove_numbers() 221
nums.remove_numbers_by_position() 224
table.remove_rows() 158
grid.remove_rows_by_position() 233
table.remove_rows_by_position() 160
table.remove_rows_with_empty_cells() 163
table.remove_rows_with_outliers() 162
table.rename_column() 157
grid.replace_numbers() 234
nums.replace_numbers() 224
table.replace_text() 174
table.replace_values() 167
nums.reverse() 225
table.reverse_rows() 160
chart.rotate_labels_x() 138
chart.rotate_labels_y() 138
nums.round_to() 219
table.round_to() 168
grid.row() 231
table.row() 154
nums.running_total() 220
chart.save() 112
nums.save() 236
table.save() 212
chart.scatter() 114
chart.second_y() 139
chart.shade_between_x() 131
chart.shade_between_y() 131
grid.shape() 230
chart.show() 112
nums.show() 215
show= 121
table.show() 152
nums.show_first_numbers() 215
grid.show_first_rows() 230
table.show_first_rows() 152
chart.show_grid() 143
nums.show_last_numbers() 216
grid.show_last_rows() 230
table.show_last_rows() 152
chart.show_legend() 145
table.show_rows_with_outliers() 162
sdata.side_by_side() 148
chart.side_histograms() 127
chart.size() 142
nums.smallest() 227
table.smallest_of() 178
chart.smooth_line() 125
sdata.solve() 191
nums.sort() 225
table.sort_by() 169
table.split_column() 173
nums.square_root() 219
dist.standard_deviation() 198
nums.standard_deviation() 227
table.standard_deviation_of() 179
nums.start_again() 235
table.start_again() 211
nums.steps() 235
table.steps() 211
chart.style() 142
nums.subtract() 217
dist.summary() 198
table.summary_of() 179
chart.surface() 119
chart.survival_curves() 207
table.survival_over_time() 205
grid.swap_rows_and_columns() 234
dist.table() 199
nums.table() 236
sdata.table() 151
chart.tick_format_x() 138
chart.tick_format_y() 139
chart.ticks_x() 137
chart.ticks_y() 137
nums.times() 218
chart.title_top() 131
nums.total() 226
table.total_of() 178
table.turn_columns_into_rows() 210
table.turn_rows_into_columns() 210
nums.undo() 235
table.undo() 211
table.values_at() 192
dist.variance() 198
chart.violin() 116
chart.violins() 116
sdata.words() 238
chart.zoom_box() 150